Patrones de diseño en Java con Spring Boot

Los 23 patrones GoF, en Java 21 y Spring Boot 4.1, construyendo la tienda ShopLab

Fábrica de Cursos
8 de octubre de 2026

Índice general

Cómo usar este libro **vii**

I Punto de partida **1**

1. Java 21 moderno, SOLID y el esqueleto de ShopLab **2**

1.1. Qué es un patrón y cómo lo defenderás en una code review 2

1.2. Records y constructores compactos 3

1.2.1. BigDecimal para el dinero 4

1.2.2. Money, el tipo del dinero en ShopLab 4

1.3. Interfaces selladas y switch con patrones 5

1.4. SOLID con ejemplos de ShopLab 7

1.4.1. SRP: un motivo para cambiar 7

1.4.2. OCP: ampliar sin editar 8

1.4.3. LSP: el subtipo no sorprende 9

1.4.4. ISP: interfaces a la medida 9

1.4.5. DIP: depende de una abstracción 10

1.5. Generar el proyecto con Initializr y primer arranque 10

1.5.1. El build.gradle 11

1.5.2. El wrapper de Gradle 12

1.5.3. Primer arranque 13

1.6. En el proyecto: H1 14

1.6.1. El producto y su tipo 14

1.6.2. Un tipo sellado y JSON 15

1.6.3. El repositorio en memoria 16

1.7. Ejercicios 17

1.8. Resumen 21

Soluciones de los ejercicios 21

II Patrones creacionales **38**

2. Singleton y el contenedor de Spring **39**

2.1. El olor: estado compartido y `new` por todas partes 39

2.2. Singleton en Java puro 40

2.2.1. Inicialización perezosa . . . 40
2.2.2. El idioma del holder . . . 43
2.2.3. Singleton con `enum` . . . 43
2.3. Las críticas: estado global y acoplamiento . . . 44
2.4. Y en Spring: el contenedor como Singleton . . . 45
2.4.1. Declarar un bean con `@Configuration` y `@Bean` . . . 45
2.4.2. Pedir un bean al contexto: `ApplicationContext` y `getBean` . . . 47
2.4.3. El ámbito `prototype` . . . 48
2.5. Cuándo sí y cuándo no . . . 50
2.6. En el proyecto: H2a . . . 50
2.7. Ejercicios . . . 51
2.8. Resumen . . . 56
Soluciones de los ejercicios . . . 57

3. Factory Method y Abstract Factory 66
3.1. El olor: `new` y `switch` por tipo repartidos . . . 66
3.2. Factory Method en Java puro . . . 67
3.2.1. La estructura clásica . . . 68
3.2.2. La variante parametrizada . . . 69
3.3. Y en Spring: `@Bean` como método de fábrica . . . 70
3.3.1. Varios beans del mismo tipo . . . 71
3.4. El olor: objetos que deben ser coherentes entre sí . . . 73
3.5. Abstract Factory en Java puro . . . 74
3.6. Y en Spring: las familias como beans . . . 76
3.7. Factory Method, Abstract Factory y fábricas estáticas . . . 77
3.8. Primeras pruebas con JUnit . . . 78
3.8.1. Anatomía de una prueba . . . 80
3.9. En el proyecto: H2b . . . 81
3.10. Ejercicios . . . 82
3.11. Resumen . . . 90
Soluciones de los ejercicios . . . 90

4. Builder y Prototype, con Lombok 119
4.1. El olor: constructor telescópico y objetos a medio construir . . . 119
4.2. Builder en Java puro . . . 120
4.3. Instalación de Lombok . . . 121
4.4. Las anotaciones de Lombok que usaremos . . . 122
4.5. Builder con Lombok, validación y límites frente a un `record` . . . 122
4.6. El olor: copiar un objeto a mano . . . 124

4.7. Prototype: constructor de copia, copia profunda y `Cloneable` . . . 124
4.8. Y en Spring: ámbito `prototype` y traducción de excepciones . . . 125
4.9. En el proyecto: Hito 3 . . . 126
4.10. Ejercicios . . . 126
4.11. Resumen . . . 129
Soluciones de los ejercicios . . . 129

III Patrones estructurales 145

5. Adapter, Facade y Bridge 146
5.1. El olor: el dominio lleno de llamadas a clientes ajenos . . . 146
5.2. Adapter en Java puro . . . 147
5.3. Y en Spring: adaptadores como beans y selección por `@Qualifier` . . . 148
5.4. El olor: el controlador conoce siete servicios . . . 149
5.5. Facade en Java puro y en Spring . . . 149
5.6. El olor: explosión de subclases (mensaje por canal) . . . 150
5.7. Bridge en Java puro y en Spring . . . 150
5.8. Comparación: Adapter frente a Facade . . . 152
5.9. En el proyecto: Hito 4 . . . 153
5.10. Ejercicios . . . 153
5.11. Resumen . . . 156
Soluciones de los ejercicios . . . 156

6. Composite y Decorator 173
6.1. El olor: tratar distinto un producto, una categoría y un pack . . . 173
6.2. Composite en Java puro con `sealed` . . . 174
6.3. Y en Spring: el árbol como servicio y recorrido desde un controlador . . . 177
6.4. El olor: recargos que se acumulan con subclases . . . 180
6.5. Decorator en Java puro: composición y orden . . . 180
6.6. Y en Spring: decorar un bean con `@Primary` y `@Bean` . . . 182
6.7. Decorator frente a herencia y composición simple . . . 183
6.8. En el proyecto: Hito 5a . . . 183
6.9. Ejercicios . . . 183
6.10. Resumen . . . 186
Soluciones de los ejercicios . . . 186

7. Proxy y Flyweight 206
7.1. El olor: consultas repetidas y código transversal . . . 206
7.2. Proxy en Java puro . . . 207
7.3. Proxy dinámico del JDK . . . 209

7.4. Instalar el starter de AOP . . . 210
7.5. Y en Spring: `@Aspect`, `@Around` y pointcuts . . . 210
7.6. El olor: miles de objetos idénticos . . . 212
7.7. Flyweight: compartir lo que no cambia . . . 212
7.8. Y en Spring: la fábrica como bean singleton . . . 213
7.9. En el proyecto: Hito 5b . . . 213
7.10. Ejercicios . . . 213
7.11. Resumen . . . 217
Soluciones de los ejercicios . . . 218

IV Patrones de comportamiento y cierre 236

8. Strategy y Template Method 237
8.1. El olor: un `if/else` por cada regla . . . 237
8.2. Strategy en Java puro . . . 238
8.3. Y en Spring: todas las estrategias como beans en una `List` . . . 239
8.4. El olor: procesos casi iguales . . . 240
8.5. Template Method en Java puro . . . 240
8.6. Y en Spring: `JdbcTemplate` y callbacks . . . 242
8.7. Strategy frente a Template Method . . . 242
8.8. En el proyecto: Hito 6 . . . 242
8.9. Ejercicios . . . 243
8.10. Resumen . . . 245
Soluciones de los ejercicios . . . 246

9. State, Command y Memento 264
9.1. El olor: banderas y transiciones ilegales . . . 264
9.2. State en Java puro con `sealed` y `switch` . . . 265
9.3. Y en Spring: la máquina de estados como bean . . . 266
9.4. El olor: acciones que no se pueden deshacer . . . 267
9.5. Command con deshacer en Java puro . . . 267
9.6. Y en Spring: el historial como bean . . . 268
9.7. El olor: restaurar un estado anterior . . . 269
9.8. Memento con records y su relación con Command . . . 269
9.9. Y en Spring: el almacén de instantáneas como bean . . . 270
9.10. En el proyecto: Hito 7 . . . 270
9.11. Ejercicios . . . 271
9.12. Resumen . . . 274
Soluciones de los ejercicios . . . 274

10. Observer y Mediator **299**
10.1. El olor: el servicio de pedidos avisa a mano . . . 299
10.2. Observer en Java puro . . . 300
10.3. Y en Spring: eventos, `@EventListener` y orden . . . 301
10.4. El olor: colaboradores que se llaman entre sí . . . 302
10.5. Mediator en Java puro . . . 302
10.6. Y en Spring: el mediador como bean . . . 304
10.7. Mediator frente a Facade y Observer . . . 304
10.8. En el proyecto: H8a . . . 304
10.9. Ejercicios . . . 306
10.10Resumen . . . 309
Soluciones de los ejercicios . . . 309

11. Chain of Responsibility e Iterator **330**
11.1. El olor: validaciones en un método de 80 líneas . . . 330
11.2. Chain of Responsibility en Java puro . . . 331
11.3. Y en Spring: `List<T>` ordenada con `@Order` . . . 333
11.4. El olor: recorrer un árbol conociendo su estructura . . . 333
11.5. Iterator, `Iterable` y el bucle `for-each` . . . 334
11.6. Y en Spring: el catálogo expone `Iterable` . . . 336
11.7. Cadena frente a lista de reglas . . . 336
11.8. En el proyecto: H8b . . . 336
11.9. Ejercicios . . . 337
11.10Resumen . . . 339
Soluciones de los ejercicios . . . 340

12. Visitor e Interpreter **361**
12.1. El olor: `instanceof` en cadena para generar informes . . . 361
12.2. Visitor clásico y doble despacho . . . 362
12.3. Visitor con `sealed` y `switch` . . . 364
12.4. Y en Spring: visitantes como beans seleccionados por nombre . . . 365
12.5. El olor: reglas de negocio escritas en `if` . . . 365
12.6. Interpreter: gramática, AST y analizador mínimo . . . 366
12.7. Y en Spring: `PromoService` con reglas de la configuración o de la petición . . . 371
12.8. Cuándo no usar Visitor ni Interpreter . . . 371
12.9. En el proyecto: H9 . . . 371
12.10Ejercicios . . . 372
12.11Resumen . . . 375
Soluciones de los ejercicios . . . 375

13. Elegir y defender el patrón **394**

13.1. Del olor al patrón: una checklist . . . 394
13.2. Patrones que se parecen . . . 395
13.3. Sobrediseño: cuándo no usarlos . . . 396
13.4. Cómo argumentar en una code review . . . 396
13.5. En el proyecto: H10 . . . 396
13.6. Ejercicios . . . 397
13.7. Resumen . . . 401
Soluciones de los ejercicios . . . 401

Referencias **413**

Cómo usar este libro

A quién va dirigido

Este libro es para ti si ya programas en Java y usas Spring Boot en proyectos reales: conoces los *records*, las lambdas y los *streams*, y has trabajado con configuración, perfiles, JPA y algo de AOP. No necesitas conocer ningún patrón de diseño: el libro explica cada uno antes de usarlo. Tampoco necesitas dominar SOLID ni el Java más reciente, porque el capítulo 1 los repasa.

Se da por hecho que ya tienes instalados y funcionando JDK 21, Gradle, Git y NeoVim con soporte para Java (por ejemplo, jdtls). El libro no explica esas instalaciones. Trabajas en Linux, y todas las órdenes del libro son válidas en cualquier distribución.

Qué vas a aprender

Al terminar sabrás escribir código de producción más mantenible, elegir el patrón adecuado para un problema y defender tu decisión en una *code review*. El libro cubre los 23 patrones del libro de la «banda de los cuatro» (*Gang of Four*, GoF), todos con la misma profundidad. Cada patrón tiene su sección y sus ejercicios, y se enseña en dos pasos: primero a mano, en Java puro, y después cómo lo resuelve Spring (`@Bean`, inyección de dependencias, AOP).

Cada patrón se razona con el mismo método: partes de un *olor* (un problema concreto en el código), aplicas el patrón y justificas por qué lo resuelve y cuándo no conviene usarlo. Ese razonamiento es lo que te sirve en una revisión de código.

El libro tiene trece capítulos, agrupados en cuatro partes:

- **Parte I, Punto de partida** (capítulo 1): Java 21 moderno, SOLID y el esqueleto de la tienda.
- **Parte II, Patrones creacionales** (capítulos 2 a 4): Singleton, Factory Method, Abstract Factory, Builder y Prototype.
- **Parte III, Patrones estructurales** (capítulos 5 a 7): Adapter, Facade, Bridge, Composite, Decorator, Proxy y Flyweight.
- **Parte IV, Patrones de comportamiento y cierre** (capítulos 8 a 13): Strategy, Template Method, State, Command, Memento, Observer, Mediator, Chain of Responsibility, Iterator, Visitor e Interpreter, y un capítulo final para elegir y defender el patrón.

Calcula unas 35 horas en total, entre 2 y 3 por capítulo. Los capítulos 5 y 9, que tratan tres patrones cada uno, son los más largos.

El proyecto del curso

A lo largo del libro construyes **ShopLab**, una tienda online como API REST con Java 21, Spring Boot 4.1.x, Gradle y, desde el capítulo 4, Lombok. Empieza con tres endpoints (`GET /products`, `GET /products/{id}` y `POST /products`) y, capítulo a capítulo, gana pedidos,

pagos, descuentos, notificaciones, informes y promociones. Al final tienes una tienda completa en la que cada pieza responde a un patrón y sabes explicar por qué.

Todo es local y sencillo, para que la dificultad esté en los patrones y no en el entorno: los repositorios son clases en memoria detrás de interfaces `XxxRepository`, y las pasarelas de pago, los transportistas y los avisos son clases simuladas. No hay base de datos, ni Docker, ni servicios externos, ni seguridad. Cada capítulo avanza un hito del proyecto y su ejercicio de tipo *proyecto* parte del código que dejó el capítulo anterior. Los otros ejercicios usan clases propias o una copia del proyecto, para que el estado de ShopLab no dependa de ellos.

Los ejercicios

Cada capítulo termina con ejercicios. Todos se pueden resolver con lo que has visto hasta ese punto del libro: si un ejercicio necesita una herramienta o una dependencia, el libro te habrá explicado antes cómo instalarla y usarla. Cada ejercicio indica su dificultad:

Dificultad	Qué significa
★☆☆☆☆	Reproducir: aplicar lo que acabas de ver siguiendo los pasos que se indican.
★★☆☆☆	Aplicar con guía: usar un concepto nuevo en un caso parecido, con pistas.
★★★☆☆	Aplicar sin guía: resolver un problema nuevo combinando lo visto en el capítulo.
★★★★☆	Integrar: combinar conceptos de varios capítulos y tomar decisiones de diseño.
★★★★★	Diseñar: problema abierto con varias soluciones válidas; se da una de referencia y criterios para valorar la tuya.

y su tipo:

Tipo	En qué consiste
Guiado	Pasos numerados que te llevan de la mano.
Práctica	Resolver un problema concreto con lo aprendido.
Proyecto	Avanzar el proyecto del curso un hito más.
Diseño	Decidir cómo estructurar algo y justificarlo.
Depuración	Encontrar y corregir el fallo de un código dado.
Lectura de código	Leer código y responder preguntas sobre él.
Test	Escribir o completar pruebas automáticas.

Las soluciones completas están al final de cada capítulo, en la sección *Soluciones de los ejercicios*. Cada enunciado enlaza con su solución y cada solución con su enunciado. Intenta resolver el ejercicio antes de mirarla: la solución es más útil cuando la comparas con la tuya.

Los ejercicios de tipo *test* son pocos a propósito: este no es un libro sobre pruebas, sino sobre diseño.

Cómo trabajar con el código

El código del libro es código real, con sus `import` y sus nombres, pensado para compilar tal cual. Se escribe en inglés (clases, métodos y variables), mientras que el texto está en español. Para compilar y arrancar ShopLab usas el *wrapper* de Gradle del proyecto, desde su carpeta:

Compilar y arrancar

```
./gradlew build
./gradlew bootRun
```

Para probar la API usas únicamente `curl` desde la terminal, siempre con `-i` (para ver el código de estado y las cabeceras) y, cuando envías un cuerpo JSON, con `-H "Content-Type: application/json"`. No hace falta ningún cliente gráfico ni archivos de peticiones.

Qué necesitas instalar para empezar

Para el capítulo 1 solo necesitas lo que ya tienes (JDK 21, Gradle y NeoVim con Java) y `curl`, que suele venir con la distribución. Comprueba las versiones:

Comprobar el entorno

```
java -version
gradle --version
curl --version
```

`java -version` debe mostrar una versión 21. El proyecto de ShopLab lo generas en el capítulo 1 con Spring Initializr, que también te da el *wrapper* `./gradlew`. El resto de herramientas y dependencias (por ejemplo, Lombok en el capítulo 4 o el soporte de AOP en el capítulo 7) se instalan en el capítulo donde se usan por primera vez.

Referencias

El libro se apoya en libros, artículos y documentación que puedes consultar para profundizar. Aparecen citados entre corchetes, como [1], y la lista completa está al final, en *Referencias*.

Parte I

Punto de partida

CAPÍTULO 1

Java 21 moderno, SOLID y el esqueleto de ShopLab

Objetivos del capítulo

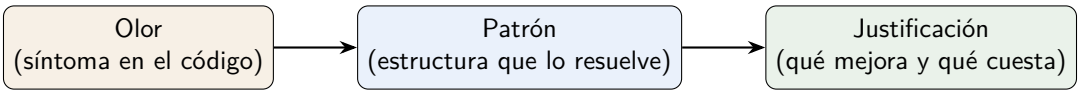
- Explicar qué es un patrón de diseño y razonarlo con el método olor, patrón y justificación.
- Leer y escribir *records*, interfaces selladas y `switch` con patrones en Java 21.
- Identificar qué principio SOLID rompe un fragmento de código y cómo se arregla.
- Generar ShopLab con Spring Initializr y dejarlo arrancando con tres endpoints.

Casi todo el libro se apoya en dos cosas: un Java moderno que te deje escribir modelos pequeños e inmutables, y unos principios de diseño que te permitan decir *por qué* un código es difícil de cambiar. Este capítulo repasa las dos, antes del primer patrón, y deja listo el proyecto que irás ampliando hasta el final: ShopLab, una tienda online como API REST. Es un capítulo de repaso: si ya dominas algo, ve directo a los ejercicios y vuelve al texto cuando lo necesites.

1.1 Qué es un patrón y cómo lo defenderás en una code review

Un *patrón de diseño* es una solución con nombre a un problema que se repite en un contexto concreto. La idea viene de la arquitectura de edificios, y Beck y Cunningham fueron de los primeros en llevarla a la programación orientada a objetos [1]. El catálogo que usaremos como referencia lo publicaron Gamma, Helm, Johnson y Vlissides, la «banda de los cuatro» (*Gang of Four*, GoF) [7]. Cada patrón de ese catálogo se describe con cuatro elementos: un nombre, el problema que resuelve, la solución y sus consecuencias.

Los dos últimos elementos son los que más se olvidan. Un patrón no es gratis: añade clases, indirecciones y vocabulario nuevo que el equipo tiene que conocer. Por eso el libro no te pide aprender estructuras de memoria, sino saber *defender* una decisión. Para eso seguimos un método que tomamos del enfoque de refactorización hacia patrones de Kerievsky [11]: no se parte del patrón, se parte del código.



Cada patrón del libro se razona con esa misma ficha:

1. **Olor.** Un síntoma concreto en ShopLab: una cadena de `if` por tipo, un constructor con ocho parámetros, una clase que sabe demasiado.
2. **Patrón.** Qué estructura aplicas, primero a mano en Java puro y después cómo la resuelve Spring.
3. **Justificación.** Qué principio de diseño mejora, qué coste añade y en qué caso *no* lo usarías.

Así suena en una revisión de código. Fíjate en que nombra el síntoma, el principio y el coste, y que admite el caso en el que no merece la pena:

> *«Propongo extraer un `ProductExporter`. Hoy, añadir un formato obliga a editar `ExportService` (olor: cadena de `if` por formato). Con una interfaz y una clase por formato, el servicio no cambia al añadir otro (principio abierto/cerrado). Coste: una interfaz y una clase más por formato. Si solo vamos a tener CSV y JSON, lo dejaría como está.»*

Un patrón sin olor es sobrediseño

Si no puedes señalar el síntoma que el patrón arregla, no lo uses. Aplicar un patrón «porque queda profesional» es la forma más rápida de que un código sencillo se vuelva difícil de leer.

1.2 Records y constructores compactos

Un `record` es una clase para datos inmutables. Declaras sus componentes y el compilador genera los campos privados y finales, el constructor, un accesor por componente, `equals`, `hashCode` y `toString` [16]. (Para un repaso general del lenguaje, *Core Java* [10] cubre Java 21.) Este ejemplo guarda las medidas de una caja:

Dimensions.java

```
package shoplab.ejemplos;

public record Dimensions(int widthCm, int heightCm, int depthCm) {

    public Dimensions {
        if (widthCm <= 0 || heightCm <= 0 || depthCm <= 0) {
            throw new IllegalArgumentException("dimensions must be positive");
        }
    }

    public static Dimensions cube(int sideCm) {
        return new Dimensions(sideCm, sideCm, sideCm);
    }

    public int volumeCm3() {
        return widthCm * heightCm * depthCm;
    }
}
```

Fíjate en tres cosas:

- El *constructor compacto* (`public Dimensions { ... }`) no lleva lista de parámetros. Se ejecuta antes de asignar los campos, así que es el sitio para validar. Si quieres normalizar un valor, reasigna el parámetro (`width = Math.abs(width);`); no se escribe `this.width = ...`.
- Los accesores se llaman como el componente: `box.widthCm()`, no `getWidthCm()`.
- Un record puede tener métodos de instancia y estáticos, e implementar interfaces. No puede extender otra clase ni añadir campos de instancia.

```
Dimensions box = new Dimensions(30, 20, 10);
System.out.println(box);
// Dimensions[widthCm=30, heightCm=20, depthCm=10]
System.out.println(box.volumeCm3());                    // 6000
System.out.println(box.equals(new Dimensions(30, 20, 10)));   // true
```

La igualdad es por valor: dos records con los mismos componentes son iguales.

Inmutable solo a medias

Un record no se puede modificar, pero sus componentes sí si son mutables: un record con una `List` deja que otros cambien la lista. En los capítulos siguientes copiaremos las colecciones con `List.copyOf(...)` dentro del constructor compacto.

1.2.1 BigDecimal para el dinero

Los precios no se guardan en `double`. Los decimales binarios no representan exactamente números como 0,1:

```
System.out.println(0.1 + 0.2);                    // 0.30000000000000004
```

`java.math.BigDecimal` guarda un número decimal exacto. Lo que necesitas de él:

- Créalo desde un `String`: `new BigDecimal("19.99")`. Con `new BigDecimal(0.1)` arrastras el error del `double`.
- Es inmutable. `add`, `subtract` y `multiply` devuelven un número nuevo.
- Multiplicar suma decimales. `setScale(2, RoundingMode.HALF_UP)` deja dos decimales y redondea la mitad hacia arriba.
- `signum()` devuelve -1, 0 o 1 según el signo.
- `equals` compara también la escala: 2,0 y 2,00 no son `equals`, aunque `compareTo` dé 0.

```
BigDecimal price = new BigDecimal("19.99");
BigDecimal withVat = price.multiply(new BigDecimal("1.21"));
System.out.println(withVat);                              // 24.1879
System.out.println(withVat.setScale(2, RoundingMode.HALF_UP));   // 24.19
System.out.println(new BigDecimal("-0.50").signum());       // -1
```

1.2.2 Money, el tipo del dinero en ShopLab

Pasar un `BigDecimal` suelto por todo el proyecto invita a errores: sumar euros con dólares, olvidar el redondeo, aceptar un precio negativo. Por eso ShopLab tendrá un record `Money(BigDecimal amount, String currency)` con estas reglas, que construirás en el ejercicio 1.1:

- El importe no es negativo y la moneda no está en blanco; si no, el constructor lanza `IllegalArgumentException`.
- El importe se normaliza a dos decimales (simplificación del curso: todas las monedas llevan dos decimales).

- `plus` y `minus` exigen la misma moneda. `minus` lanza `IllegalArgumentException` si el resultado sería negativo.
- `times(int)` y `times(BigDecimal)` multiplican; `zero(String)` crea un importe cero en una moneda.

Los capítulos siguientes usan estas operaciones: el IVA, los recargos y los descuentos son multiplicaciones y restas de `Money`.

1.3 Interfaces selladas y switch con patrones

Una interfaz *sellada* limita quién puede implementarla. Lo declaras con `sealed` y la lista `permits` [16]. Cada subtipo permitido debe ser `final`, `sealed` o `non-sealed`; los records son `final` de forma implícita. Con eso, el compilador conoce *todas* las variantes. Este ejemplo modela los cupones de descuento de una tienda:

Coupon.java

```
1  package shoplab.ejemplos;
2
3  import shoplab.Money;
4
5  public sealed interface Coupon
6          permits Coupon.PercentOff, Coupon.AmountOff, Coupon.FreeShipping {
7
8      record PercentOff(int percent) implements Coupon {
9      }
10
11     record AmountOff(Money amount) implements Coupon {
12     }
13
14     record FreeShipping() implements Coupon {
15     }
16 }
```

Antes de Java 21 tendrías que recorrer las variantes con `instanceof` y casts. Un `switch` con patrones [2] lo hace de una vez: cada `case` comprueba el tipo y puede *deconstruir* el record, es decir, sacar sus componentes en variables.

CouponText.java

```
1  package shoplab.ejemplos;
2
3  import java.math.BigDecimal;
4  import shoplab.Money;
5
6  public final class CouponText {
7
8      private CouponText() {
9      }
10
11     public static String describe(Coupon coupon) {
12         return switch (coupon) {
13             case Coupon.PercentOff(int percent) when percent >= 100 ->
14                     "Gratis";
```

```
            case Coupon.PercentOff(int percent) -> percent + " % de descuento";
            case Coupon.AmountOff(Money(BigDecimal value, String currency)) ->
                    value + " " + currency + " de descuento";
            case Coupon.FreeShipping() -> "Envío gratis";
        };
    }
}
```

Qué ocurre aquí:

- El primer `case` es un *record pattern*: si el cupón es un `PercentOff`, declara la variable `percent` con su componente.
- `when percent >= 100` es una *guarda*: el `case` solo se cumple si la condición es cierta.
- `Coupon.AmountOff(Money(BigDecimal value, String currency))` anida un patrón dentro de otro: abre el `AmountOff` y, dentro, el `Money`.
- El `switch` es una expresión: devuelve el valor de la rama elegida.
- No hay `default`. Como `Coupon` es sellada y las cuatro ramas cubren todas las variantes, el `switch` es *exhaustivo*.

Lo valioso de la exhaustividad es lo que pasa cuando el código cambia. Si añades una cuarta variante a `permits`, este `switch` deja de compilar y el compilador te señala todos los sitios que debes revisar. Un `default` lo evitaría, y con él perderías ese aviso.

Los patrones de tipo funcionan también con tipos que no son sellados, pero entonces sí necesitas `default`:

```
Object value = 42;
String text = switch (value) {
    case Integer number when number > 40 -> "big " + number;
    case Integer number -> "int " + number;
    case String word -> "text " + word;
    default -> "other";
};
System.out.println(text);       // big 42
```

Dos reglas más del compilador:

- **El orden importa.** Una rama con guarda va antes de la misma rama sin guarda. Si pones primero `case Integer number`, la guarda de debajo nunca se alcanzaría y el compilador lo rechaza (la primera rama «domina» a la segunda).
- Un `switch` sobre un valor `null` lanza `NullPointerException`, salvo que añadas `case null`.

Sin patrones sin nombre

Java 21 también describe patrones y variables sin nombre (el guion bajo `_`), pero en esta versión son una función en vista previa y exigen `-enable-preview` [16]. En este libro nunca los usamos: cuando una variable no se usa, escribe un nombre real (`case Coupon.FreeShipping shipping`) o usa un patrón de tipo.

1.4 SOLID con ejemplos de ShopLab

SOLID es el acrónimo de cinco principios de diseño orientado a objetos que Martin reunió y explicó junto a varios patrones [14]. Son criterios para juzgar un diseño, no reglas mecánicas. Los usarás en cada ficha olor, patrón y justificación.

Principio	Idea	Síntoma en el código
SRP, responsabilidad única	Una clase, un motivo para cambiar.	Una clase que tocas por razones sin relación.
OCP, abierto/cerrado	Se amplía añadiendo código, no editando el existente.	Una cadena de `if` por tipo que crece con cada caso.
LSP, sustitución de Liskov	Un subtipo puede usarse donde se espera su tipo base.	Un subtipo que lanza «no soportado».
ISP, segregación de interfaces	Interfaces pequeñas, hechas para quien las usa.	Clases obligadas a implementar métodos que no usan.
DIP, inversión de dependencias	Depender de abstracciones, no de clases concretas.	`new` de una clase concreta dentro de la lógica.

1.4.1 SRP: un motivo para cambiar

`StockReport` hace tres cosas: decide qué productos tienen poco stock, los convierte a CSV y los guarda en un archivo.

```
class StockReport {

    List<Product> lowStock(List<Product> products) {
        List<Product> result = new ArrayList<>();
        for (Product product : products) {
            if (product.stock() < 5) {
                result.add(product);
            }
        }
        return result;
    }

    String toCsv(List<Product> products) {
        StringBuilder csv = new StringBuilder("id,name,stock\n");
        for (Product product : products) {
            csv.append(product.id()).append(',')
               .append(product.name()).append(',')
               .append(product.stock()).append('\n');
        }
        return csv.toString();
    }

    void saveTo(Path path, String csv) throws IOException {
        Files.writeString(path, csv);
    }
}
```

Tiene tres motivos para cambiar: el umbral de «poco stock», el formato y el destino. Un cambio en el formato obliga a tocar una clase que también contiene la regla de negocio. La solución es

una clase por motivo: una para la regla, otra para el formato y otra para el destino.

1.4.2 OCP: ampliar sin editar

`ExportService` elige el formato con una cadena de `if`:

```
class ExportService {

    String export(String format, List<Product> products) {
        if (format.equals("csv")) {
            StringBuilder csv = new StringBuilder("id,name\n");
            for (Product product : products) {
                csv.append(product.id()).append(',')
                   .append(product.name()).append('\n');
            }
            return csv.toString();
        } else if (format.equals("json")) {
            StringBuilder json = new StringBuilder("[");
            for (int i = 0; i < products.size(); i++) {
                if (i > 0) {
                    json.append(',');
                }
                json.append("{\"id\":\"")
                    .append(products.get(i).id()).append("\"}");
            }
            return json.append(']').toString();
        }
        throw new IllegalArgumentException("unknown format: " + format);
    }
}
```

Para soportar XML hay que editar `export`, y cada edición puede romper los formatos que ya funcionaban. Si defines una interfaz y una clase por formato, el servicio queda cerrado a la modificación y abierto a la extensión:

```
interface ProductExporter {

    String format();

    String export(List<Product> products);
}

class ExportServiceOpen {

    private final List<ProductExporter> exporters;

    ExportServiceOpen(List<ProductExporter> exporters) {
        this.exporters = exporters;
    }

    String export(String format, List<Product> products) {
        for (ProductExporter exporter : exporters) {
            if (exporter.format().equals(format)) {
                return exporter.export(products);
```

```
            }
        }
        throw new IllegalArgumentException("unknown format: " + format);
    }
}
```

Los exportadores concretos (`CsvExporter`, `JsonExporter`) implementan `ProductExporter`. Añadir XML es escribir un `XmlExporter` y pasarlo en la lista; `ExportServiceOpen` no cambia. Este olor, la cadena de `if` por tipo, reaparece en el capítulo 8.

1.4.3 LSP: el subtipo no sorprende

```
interface Refundable {

    void refund(Money amount);
}

class CashPayment implements Refundable {

    @Override
    public void refund(Money amount) {
        throw new UnsupportedOperationException("cash cannot be refunded");
    }
}
```

El código que recibe una `List<Refundable>` y llama a `refund` sobre cada elemento espera que funcione. `CashPayment` promete lo que no cumple y rompe a quien lo usa: no es sustituible. Un subtipo respeta el contrato de su tipo base si no endurece las condiciones de entrada, no debilita lo que promete y no lanza excepciones nuevas que el contrato no anuncia. La salida suele ser que `CashPayment` no implemente `Refundable`: ese es el motivo de la regla siguiente.

1.4.4 ISP: interfaces a la medida

```
interface FatProductRepository {

    Product save(Product product);
    Optional<Product> findById(String id);
    List<Product> findAll();
    void delete(String id);
    String exportCsv();
    void importCsv(String csv);
}
```

Un controlador que solo consulta productos queda atado a `delete` e `importCsv`, y cualquier implementación debe tenerlos. Es mejor partir la interfaz según quien la usa:

```
interface ProductReader {

    Optional<Product> findById(String id);
```

```
    List<Product> findAll();
}

interface ProductWriter {

    Product save(Product product);
    void delete(String id);
}
```

1.4.5 DIP: depende de una abstracción

```
class RigidNotifier {

    void orderPlaced(String email) {
        SmtpClient smtp = new SmtpClient("smtp.shop.local");
        smtp.send(email, "Order received", "Thank you");
    }
}
```

`RigidNotifier` crea con `new` una clase concreta. No puedes cambiar el transporte (SMS, otro servidor) sin editarla, y no puedes ejecutarla sin un servidor de correo. Con DIP, la clase depende de una interfaz que recibe por el constructor:

```
interface Mailer {

    void send(String to, String subject, String body);
}

class OrderNotifier {

    private final Mailer mailer;

    OrderNotifier(Mailer mailer) {
        this.mailer = mailer;
    }

    void orderPlaced(String email) {
        mailer.send(email, "Order received", "Thank you");
    }
}
```

Esta forma de pasar las dependencias por el constructor es la que Spring automatiza: lo verás en la sección siguiente.

1.5 Generar el proyecto con Initializr y primer arranque

ShopLab es un proyecto Spring Boot con Gradle. En vez de crear a mano la estructura, usas *Spring Initializr*, un servicio web que genera un proyecto base con la versión de Boot y las dependencias que le pides. Tienes ya instalados JDK 21, Gradle y NeoVim con jdtls, y no los repetimos aquí.

☞ Generar ShopLab con Spring Initializr

Initializr se puede usar desde la web (https://start.spring.io) o desde la terminal. Aquí usamos la terminal, para no salir de NeoVim. Primero comprueba que tienes `curl`, la herramienta con la que también probarás la API:

```
$ curl --version
```

Si imprime una versión, sigue. Genera el proyecto en una carpeta nueva:

```
$ curl https://start.spring.io/starter.zip -o shoplab.zip \
    -d type=gradle-project -d language=java \
    -d bootVersion=4.1.1 -d javaVersion=21 \
    -d groupId=shoplab -d artifactId=shoplab \
    -d name=ShopLab -d packageName=shoplab \
    -d dependencies=web
$ unzip shoplab.zip -d shoplab
$ cd shoplab
```

Los parámetros fijan Gradle con DSL Groovy (`gradle-project`), Spring Boot 4.1.1, Java 21, el paquete base `shoplab` y la dependencia `web` (en Initializr se llama «Spring Web»). Con la web obtienes lo mismo: elige Gradle - Groovy, Java, Spring Boot 4.1.1, Java 21, paquete `shoplab` y la dependencia Spring Web.

Esto es lo que genera:

```
shoplab/
|-- build.gradle
|-- settings.gradle
|-- gradlew
|-- gradlew.bat
|-- gradle/wrapper/
|   |-- gradle-wrapper.jar
|   `-- gradle-wrapper.properties
`-- src/
    |-- main/
    |   |-- java/shoplab/ShopLabApplication.java
    |   `-- resources/application.properties
    `-- test/java/shoplab/ShopLabApplicationTests.java
```

(También aparecen `HELP.md`, `.gitignore` y `.gitattributes`, que no usaremos, y las carpetas vacías `static` y `templates`.)

1.5.1 El build.gradle

`build.gradle`

```
plugins {
    id 'java'
    id 'org.springframework.boot' version '4.1.1'
    id 'io.spring.dependency-management' version '1.1.7'
}
```

```
6
7  group = 'shoplab'
8  version = '0.0.1-SNAPSHOT'
9
10 java {
11     toolchain {
12         languageVersion = JavaLanguageVersion.of(21)
13     }
14 }
15
16 repositories {
17     mavenCentral()
18 }
19
20 dependencies {
21     implementation 'org.springframework.boot:spring-boot-starter-webmvc'
22     testImplementation 'org.springframework.boot:spring-boot-starter-webmvc-
       ↪ test'
23     testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
24 }
25
26 tasks.named('test') {
27     useJUnitPlatform()
28 }
```

- El plugin `org.springframework.boot` añade las tareas `bootRun` (arrancar) y `bootJar` (empaquetar). Su versión, `4.1.1`, es la de Spring Boot.
- `io.spring.dependency-management` hace que no escribas versiones en las dependencias de Spring: las fija Boot.
- El *toolchain* pide Java 21, aunque tu sistema tenga otro JDK por defecto.
- `spring-boot-starter-webmvc` es el *starter* de aplicaciones web con Spring MVC y un Tomcat embebido. En Boot 4 los starters están más repartidos que en Boot 3, y el que antes se llamaba `spring-boot-starter-web` para MVC es ahora este [22]. Si copias un `build.gradle` de un proyecto antiguo, revisa los nombres.
- Las dos líneas de test las genera Initializr y las explicamos en el capítulo 3, donde escribirás tus primeras pruebas. No las toques.

1.5.2 El wrapper de Gradle

El script `./gradlew` es el *wrapper*: descarga la versión de Gradle que indica el archivo `gradle-wrapper.properties`, de la carpeta `gradle/wrapper`, y la usa. Así no depende del Gradle que tengas instalado. Con el Initializr de octubre de 2026, cuando se escribió el libro, ese archivo apunta a Gradle 9.7.1; si el tuyo indica otra versión, es normal. El plugin de Boot 4.1.1 exige Gradle 8.14 o posterior, o la línea 9 [24]. La primera ejecución descarga Gradle (necesita conexión) y tarda más.

1.5.3 Primer arranque

```
$ ./gradlew bootRun
...
Tomcat started on port 8080 (http) with context path '/'
Started ShopLabApplication in 0.465 seconds (process running for 0.563)
```

El servidor queda en primer plano. Abre otra terminal y pregúntale por la raíz. Todavía no hay ningún endpoint, así que responde `404`:

```
$ curl -i -H "Content-Type: application/json" http://localhost:8080/
HTTP/1.1 404
Content-Type: application/json
...
{"timestamp":"...","status":404,"error":"Not Found","path":"/"}
```

Usaremos siempre `curl` con dos opciones: `-i`, que muestra la línea de estado y las cabeceras (necesarias para ver el `201` o el `404`), y `-H "Content-Type: application/json"`, que indica que el cuerpo es JSON. Cuando mandas un cuerpo con `-d '...'`, curl hace un `POST`. Para detener el servidor, pulsa `Ctrl+C` en la terminal donde corre `bootRun`.

Otras tres órdenes que usarás:

- `./gradlew compileJava` compila sin arrancar nada.
- `./gradlew build` compila, ejecuta las pruebas y empaqueta.
- Para ejecutar una clase con `main` que no usa Spring (los ejercicios de este capítulo), compila y lanza la clase con `java`:

```
$ ./gradlew compileJava
$ java -cp build/classes/java/main shoplab.ejercicios.MoneyCheck
```

Para abrir el proyecto en NeoVim, ejecuta `nvim .` desde la carpeta `shoplab`. jdtls reconoce `build.gradle` y `settings.gradle` como un proyecto Gradle; la primera importación tarda un rato.

Recordatorio: Spring MVC e inyección por constructor

Ya conoces estas piezas; las repasamos porque el resto del libro las usa a diario.

- `@RestController`: la clase atiende peticiones HTTP, y lo que devuelven sus métodos se convierte en JSON (con Jackson 3 en Boot 4, que serializa un record por los nombres de sus componentes).
- `@RequestMapping("/products")` fija la ruta base. `@GetMapping` y `@PostMapping` enlazan un método con un verbo HTTP.
- `@PathVariable` toma un trozo de la ruta (`/{id}`); `@RequestBody` convierte el JSON de la petición en un objeto.
- `ResponseEntity<T>` te deja elegir el estado y las cabeceras: `ResponseEntity.ok(x)` o `ResponseEntity.notFound().build()`. `@ResponseStatus(HttpStatus.CREATED)` fija un estado fijo, útil para el `201` de un `POST`.

- `@Component` marca una clase como *bean*: Spring la crea y la guarda en su contenedor. `@Service` y `@Repository` son variantes con nombre más expresivo.
- *Inyección por constructor*: si una clase gestionada por Spring tiene un único constructor, Spring le pasa los beans que pide, sin `@Autowired`. Es la forma recomendada [24], y es DIP en acción: tu clase declara qué necesita y no cómo se crea.
- Spring solo encuentra tus clases si están en el paquete de `ShopLabApplication` o en uno de sus subpaquetes. Aquí, `shoplab`.

Un ejemplo mínimo que no añadas al proyecto, solo para ver todas las piezas juntas (es la misma idea del tutorial oficial [29]):

```
record Greeting(String message) {
}

@RestController
public class GreetingController {

    @GetMapping("/greeting/{name}")
    public Greeting greet(@PathVariable String name) {
        return new Greeting("Hello, " + name);
    }
}
```

1.6 En el proyecto: H1

Hito 1: el esqueleto de ShopLab

Al terminar el capítulo, ShopLab arranca y expone:

- `GET /products`: la lista de productos (vacía al principio).
- `GET /products/{id}`: un producto, o `404` si no existe.
- `POST /products`: crea un producto y responde `201`. El `id` lo envía el cliente en el cuerpo.

Las clases son `ShopLabApplication`, `Money`, `ProductKind`, `Product`, `ProductRepository`, `InMemoryProductRepository` y `ProductController`. La construyes en el ejercicio 1.6.

Antes de ponerte con él, tres decisiones de diseño.

1.6.1 El producto y su tipo

Un producto tiene identificador, nombre, precio (un `Money`), tipo, existencias y categoría. El tipo es la interfaz sellada que construirás en el ejercicio 1.3: un producto es físico (con peso), digital o un servicio (con duración).

Product.java

```
package shoplab;

import java.util.Objects;
```

```

public record Product(
        String id,
        String name,
        Money price,
        ProductKind kind,
        int stock,
        String category) {

    public Product {
        Objects.requireNonNull(price, "price");
        Objects.requireNonNull(kind, "kind");
        if (id == null || id.isBlank()) {
            throw new IllegalArgumentException("id must not be blank");
        }
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("name must not be blank");
        }
        if (category == null || category.isBlank()) {
            throw new IllegalArgumentException("category must not be blank");
        }
        if (stock < 0) {
            throw new IllegalArgumentException(
                    "stock must not be negative: " + stock);
        }
    }
}
```

Para validar, el constructor compacto lanza `IllegalArgumentException`. Si el cliente manda un cuerpo que no cumple las reglas, Spring responde `400` (más adelante, en el capítulo 4, lo traduciremos a un error propio).

1.6.2 Un tipo sellado y JSON

JSON no sabe qué es una interfaz sellada. Cuando llegue `"kind":{"weightGrams":800}`, Jackson no puede adivinar si es un `Physical`, un `Digital` o un `Service`. Hay que enseñárselo con dos anotaciones sobre `ProductKind`: `@JsonTypeInfo` añade un campo `type` al JSON y `@JsonSubTypes` asocia cada valor de `type` con un record. Estas anotaciones siguen en el paquete `com.fasterxml.jackson.annotation`, aunque el resto de Jackson 3 vive en `tools.jackson`.

ProductKind.java

```
package shoplab;

import com.fasterxml.jackson.annotation.JsonSubTypes;
import com.fasterxml.jackson.annotation.JsonTypeInfo;

@JsonTypeInfo(use = JsonTypeInfo.Id.NAME, property = "type")
@JsonSubTypes({
        @JsonSubTypes.Type(value = ProductKind.Physical.class,
                name = "physical"),
        @JsonSubTypes.Type(value = ProductKind.Digital.class,
                name = "digital"),
        @JsonSubTypes.Type(value = ProductKind.Service.class,
                name = "service")
```

```
})
public sealed interface ProductKind
        permits ProductKind.Physical, ProductKind.Digital,
                ProductKind.Service {

    record Physical(int weightGrams) implements ProductKind {
    }

    record Digital() implements ProductKind {
    }

    record Service(int durationMinutes) implements ProductKind {
    }
}
```

Un producto físico viaja como `"kind":{"type":"physical","weightGrams":800}` y uno digital como `"kind":{"type":"digital"}`. Si el `type` no existe, Jackson rechaza la petición con un 400.

1.6.3 El repositorio en memoria

La aplicación guarda los productos tras una interfaz. Hoy la implementación usa un mapa en memoria; el servicio que la use depende solo de la interfaz (DIP), y cambiarla por una base de datos sería escribir otra clase.

ProductRepository.java

```
package shoplab;

import java.util.List;
import java.util.Optional;

public interface ProductRepository {

    Product save(Product product);

    Optional<Product> findById(String id);

    List<Product> findAll();
}
```

La implementación en memoria guarda un `ConcurrentHashMap<String, Product>` con el `id` como clave. Es un mapa seguro para varios hilos, y lo necesitas porque Tomcat atiende cada petición en un hilo distinto y el repositorio es un bean que comparten todas. `save` guarda el producto (si el `id` ya existía, lo reemplaza), `findById` devuelve un `Optional` y `findAll` devuelve una copia de los valores con `List.copyOf`. Sin orden garantizado.

1.7 Ejercicios

Ejercicio 1.1: Money, tu primer record ★☆☆☆☆

Crea el record `Money` del proyecto. Partes del proyecto `shoplab` que generaste en la sección anterior.

1. Crea `src/main/java/shoplab/Money.java` en el paquete `shoplab` e importa `java.math.BigDecimal`, `java.math.RoundingMode` y `java.util.Objects`.
2. Declara `public record Money(BigDecimal amount, String currency)`.
3. Escribe el constructor compacto: comprueba con `Objects.requireNonNull` que ninguno es nulo, lanza `IllegalArgumentException` si `amount.signum() < 0` o si la moneda está en blanco, y termina con `amount = amount.setScale(2, RoundingMode.HALF_UP);`.
4. Añade el método estático `zero(String currency)`.
5. Añade `plus(Money)` y `minus(Money)`. Ambos lanzan `IllegalArgumentException` si las monedas son distintas; `minus` lanza la misma excepción si el resultado sería negativo.
6. Añade `times(int)` y `times(BigDecimal)`.
7. Crea `src/main/java/shoplab/ejercicios/MoneyCheck.java` con este código, compila con `./gradlew compileJava` y ejecútalo.

MoneyCheck.java

```java
package shoplab.ejercicios;

import java.math.BigDecimal;
import shoplab.Money;

public class MoneyCheck {

    public static void main(String[] args) {
        Money price = new Money(new BigDecimal("19.99"), "EUR");
        System.out.println(price);
        System.out.println(price.plus(price));
        System.out.println(price.times(3));
        System.out.println(price.times(new BigDecimal("0.21")));
        System.out.println(price.minus(price));
        System.out.println(Money.zero("EUR").equals(price.minus(price)));
        try {
            price.minus(price.times(2));
        } catch (IllegalArgumentException e) {
            System.out.println("error: " + e.getClass().getSimpleName());
        }
        try {
            price.plus(Money.zero("USD"));
        } catch (IllegalArgumentException e) {
            System.out.println("error: " + e.getClass().getSimpleName());
        }
    }
}
```

Está bien si la salida es exactamente esta:

```
Money[amount=19.99, currency=EUR]
Money[amount=39.98, currency=EUR]
Money[amount=59.97, currency=EUR]
Money[amount=4.20, currency=EUR]
Money[amount=0.00, currency=EUR]
true
error: IllegalArgumentException
error: IllegalArgumentException
```

Guiado ▪ solución en la pág. 22

Ejercicio 1.2: Lee un switch con patrones ★☆☆☆☆

Lee este código y predice, sin ejecutarlo, qué imprime cada una de las cuatro llamadas. Después responde: ¿por qué el `switch` no necesita un `default`?

```java
public sealed interface Fee permits Fee.Flat, Fee.PerItem, Fee.None {

    record Flat(int cents) implements Fee {
    }

    record PerItem(int cents, int items) implements Fee {
    }

    record None() implements Fee {
    }
}

static int total(Fee fee) {
    return switch (fee) {
        case Fee.Flat(int cents) -> cents;
        case Fee.PerItem(int cents, int items) when items > 10 -> cents *
    ↪ 10;
        case Fee.PerItem(int cents, int items) -> cents * items;
        case Fee.None() -> 0;
    };
}

System.out.println(total(new Fee.Flat(300)));
System.out.println(total(new Fee.PerItem(50, 4)));
System.out.println(total(new Fee.PerItem(50, 12)));
System.out.println(total(new Fee.None()));
```

Lectura de código ▪ solución en la pág. 24

Ejercicio 1.3: ProductKind y el coste de envío ★★☆☆☆

Modela el tipo de producto de ShopLab y calcula su coste de envío con un `switch` exhaustivo.

- En `src/main/java/shoplab/ProductKind.java`, define la interfaz sellada `ProductKind` con tres records anidados: `Physical(int weightGrams)`, `Digital()` y `Service(int durationMinutes)`. Todavía sin anotaciones de Jackson (las añades en

el ejercicio 1.6).

- En `shoplab.ejercicios.ShippingCosts`, escribe el método estático `Money shippingCost(ProductKind kind)`: un físico de más de 5000 gramos cuesta 9,90 EUR; cualquier otro físico, 4,90 EUR; un digital o un servicio, 0 EUR. Usa un `switch` sin `default` y sin `_`.
- Añade un `main` que imprima el coste de `new ProductKind.Physical(800)`, `Physical(8000)`, `Digital()` y `Service(60)`.

Pista: *para un caso en el que no usas ningún componente, escribe un patrón de tipo con nombre:* `case ProductKind.Digital digital ->`.
Está bien si imprime 4,90, 9,90, 0,00 y 0,00 euros. Además, comprueba el compilador: añade temporalmente una cuarta variante a `permits`, mira el error y quítala.

Práctica ▪ solución en la pág. 25

Ejercicio 1.4: Diagnóstico SOLID de un servicio ★★☆☆☆

Lee este `OrderService`. Nombra los principios SOLID que rompe y justifica cada uno en una o dos frases, señalando las líneas o construcciones que lo delatan. Indica también si LSP o ISP se ven afectados. (`OrderRequest`, `OrderItem`, `SqlOrderDao` y `SmtpClient` son clases concretas que existen; no hace falta ver su código.)

```
1  public class OrderService {
2
3      public void placeOrder(OrderRequest request) {
4          if (request.items().isEmpty()) {
5              throw new IllegalArgumentException("empty order");
6          }
7          BigDecimal total = BigDecimal.ZERO;
8          for (OrderItem item : request.items()) {
9              total = total.add(item.unitPrice()
10                     .multiply(BigDecimal.valueOf(item.quantity())));
11         }
12         if (request.customerType().equals("VIP")) {
13             total = total.multiply(new BigDecimal("0.80"));
14         } else if (request.customerType().equals("PREMIUM")) {
15             total = total.multiply(new BigDecimal("0.90"));
16         }
17         SqlOrderDao dao = new SqlOrderDao();
18         dao.insert(request, total);
19         SmtpClient smtp = new SmtpClient("smtp.shop.local");
20         smtp.send(request.customerEmail(), "Pedido recibido",
21                 "Total: " + total);
22     }
23 }
```

Lectura de código ▪ solución en la pág. 26

Ejercicio 1.5: Refactoriza un descuento con OCP ★★★☆☆

Este servicio calcula el precio final según el tipo de cliente. Añadir un tipo (por ejemplo `STUDENT`, con un 15 %) obliga a editarlo.

```java
public class PricingService {

    public Money finalPrice(String customerType, Money base) {
        if (customerType.equals("REGULAR")) {
            return base;
        } else if (customerType.equals("PREMIUM")) {
            return base.minus(base.times(new BigDecimal("0.10")));
        } else if (customerType.equals("VIP")) {
            return base.minus(base.times(new BigDecimal("0.20")));
        }
        throw new IllegalArgumentException(
                "unknown customer type: " + customerType);
    }
}
```

Refactorízalo en el paquete `shoplab.ejercicios.ocp` para que añadir un tipo de cliente no obligue a modificar el servicio. Escribe también un `main` que construya el servicio y calcule el precio de 100,00 EUR para `REGULAR`, `PREMIUM`, `VIP` y `STUDENT`.
Está bien si:

- el servicio no contiene ninguna condición por tipo de cliente;
- `STUDENT` se añade con una clase nueva y una línea al construir la lista, sin tocar el servicio;
- los precios son 100,00, 90,00, 80,00 y 85,00 EUR, y un tipo desconocido lanza `IllegalArgumentException`.

Pista: *define una interfaz con dos métodos, uno que diga si aplica a un tipo de cliente y otro que devuelva la tasa de descuento, y que el servicio reciba una lista de ellas.*

Práctica ▪ solución en la pág. 26

Ejercicio 1.6: Hito 1: ShopLab arranca ★★★☆☆

Deja ShopLab funcionando con `GET /products`, `GET /products/{id}` y `POST /products`. Sigue estos cinco pasos.

1. **Generar.** Si no lo hiciste en la sección anterior, genera el proyecto con Initializr. Comprueba que `./gradlew bootRun` arranca y detenlo.
2. **Dominio.** En el paquete `shoplab` deja `Money` (ejercicio 1.1), `ProductKind` con las anotaciones de Jackson de la sección anterior (parte del ejercicio 1.3) y `Product`.
3. **Repositorio.** Crea la interfaz `ProductRepository` y la clase `InMemoryProductRepository` con un `ConcurrentHashMap`, anotada con `@Component`.
4. **Controlador.** Crea `ProductController` con inyección por constructor del repositorio y las tres rutas. `GET /products/{id}` devuelve `404` si no existe; `POST /products` responde `201` con el producto creado.
5. **curl.** Arranca con `./gradlew bootRun` y, con `curl -i -H "Content-Type: application/json"`, crea un producto físico (`p1`) y uno digital (`p2`), consulta la lista, consulta `p1`, consulta un id que no existe (`zz`) y envía un producto con un `type` desconocido.

Está bien si:

- los dos `POST` responden `HTTP/1.1 201` y devuelven el producto con el precio con dos decimales;
- `GET /products` responde `200` con los dos productos;
- `GET /products/p1` responde `200`, y `GET /products/zz` responde `404`;
- el producto con un `type` desconocido responde `400`.

Pista: *para el 404, `findById` devuelve un `Optional<Product>`: conviértelo con `map(ResponseEntity::ok)` y `orElseGet(...)`.*

Proyecto ▪ solución en la pág. 30

1.8 Resumen

Resumen

- Un patrón es una solución con nombre a un problema recurrente. En el libro lo razonas con la ficha olor, patrón y justificación, y siempre dices qué cuesta.
- Un `record` es un tipo de datos inmutable con constructor compacto para validar; el dinero se guarda en `BigDecimal`, y `Money` agrupa importe y moneda.
- Una interfaz `sealed` fija sus variantes y un `switch` con patrones (de tipo, de record, con guarda `when`) es exhaustivo sin `default`. No usamos patrones ni variables sin nombre (`_`), porque son vista previa en Java 21.
- SOLID da el vocabulario para justificar un diseño: SRP, OCP, LSP, ISP y DIP.
- Initializr genera el proyecto; `./gradlew bootRun` lo arranca y `curl -i -H "Content-Type: application/json"` lo prueba.
- ShopLab (H1) tiene `Money`, `Product` con `ProductKind`, un repositorio en memoria y tres endpoints.

Soluciones de los ejercicios

Solución del ejercicio 1.1 enunciado en la pág. 17

Money.java

```java
package shoplab;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.Objects;

public record Money(BigDecimal amount, String currency) {

    public Money {
        Objects.requireNonNull(amount, "amount");
        Objects.requireNonNull(currency, "currency");
        if (amount.signum() < 0) {
            throw new IllegalArgumentException(
                    "amount must not be negative: " + amount);
        }
        if (currency.isBlank()) {
            throw new IllegalArgumentException("currency must not be blank");
        }
        amount = amount.setScale(2, RoundingMode.HALF_UP);
    }

    public static Money zero(String currency) {
        return new Money(BigDecimal.ZERO, currency);
    }

    public Money plus(Money other) {
        requireSameCurrency(other);
        return new Money(amount.add(other.amount), currency);
    }

    public Money minus(Money other) {
        requireSameCurrency(other);
        return new Money(amount.subtract(other.amount), currency);
    }

    public Money times(int quantity) {
        BigDecimal factor = BigDecimal.valueOf(quantity);
        return new Money(amount.multiply(factor), currency);
    }

    public Money times(BigDecimal factor) {
        return new Money(amount.multiply(factor), currency);
    }

    private void requireSameCurrency(Money other) {
        if (!currency.equals(other.currency)) {
            throw new IllegalArgumentException(
                    "different currencies: " + currency
                            + " and " + other.currency);
        }
    }
}
```

El constructor compacto concentra las reglas: todo `Money` que existe cumple «importe no negativo, moneda con texto y dos decimales». Por eso `minus` no necesita su propia comprobación del resultado: al construir el `Money` con un importe negativo, el constructor lanza la excepción. Normalizar la escala hace que `Money.zero("EUR")` sea `equals` a `19.99 - 19.99`, algo que con `BigDecimal` sin normalizar fallaría.
Errores típicos: escribir `this.amount = ...` en el constructor compacto (no compila); crear el importe con `new BigDecimal(0.1)` en vez de con un `String`; olvidar que `times(BigDecimal)` puede producir más de dos decimales (aquí los recorta el constructor); comparar dos `BigDecimal` con `equals` sin fijar la escala.

Solución del ejercicio 1.2

enunciado en la pág. 18

Las salidas son:

```
300
200
500
0
```

- `Flat(300)` devuelve su único componente: 300.
- `PerItem(50, 4)`: no cumple `items > 10`, así que cae en la segunda rama, $50 \times 4 = 200$.
- `PerItem(50, 12)`: cumple la guarda, así que la tarifa tiene un tope de diez unidades, $50 \times 10 = 500$.
- `None()` devuelve 0.

No hace falta `default` porque `Fee` es sellada y sus tres variantes (`Flat`, `PerItem` y `None`) están cubiertas: la rama `PerItem` sin guarda cubre todos los `PerItem`, con o sin la guarda de arriba. Si añadieras una cuarta variante a `permits`, el `switch` dejaría de compilar con el aviso de que no cubre todos los casos.
Error típico: intercambiar el orden de las dos ramas `PerItem`. La rama sin guarda dominaría a la que tiene guarda y el compilador lo rechazaría.

Solución del ejercicio 1.3

enunciado en la pág. 18

ProductKind.java

```java
package shoplab;

public sealed interface ProductKind
        permits ProductKind.Physical, ProductKind.Digital,
                ProductKind.Service {

    record Physical(int weightGrams) implements ProductKind {
    }

    record Digital() implements ProductKind {
    }

    record Service(int durationMinutes) implements ProductKind {
    }
}
```

ShippingCosts.java

```java
package shoplab.ejercicios;

import java.math.BigDecimal;
import shoplab.Money;
import shoplab.ProductKind;

public final class ShippingCosts {

    private ShippingCosts() {
    }

    public static Money shippingCost(ProductKind kind) {
        return switch (kind) {
            case ProductKind.Physical(int grams) when grams > 5000 ->
                    new Money(new BigDecimal("9.90"), "EUR");
            case ProductKind.Physical physical ->
                    new Money(new BigDecimal("4.90"), "EUR");
            case ProductKind.Digital digital -> Money.zero("EUR");
            case ProductKind.Service service -> Money.zero("EUR");
        };
    }

    public static void main(String[] args) {
        System.out.println(shippingCost(new ProductKind.Physical(800)));
        System.out.println(shippingCost(new ProductKind.Physical(8000)));
        System.out.println(shippingCost(new ProductKind.Digital()));
        System.out.println(shippingCost(new ProductKind.Service(60)));
    }
}
```

Salida: `Money[amount=4.90, currency=EUR]`, `9.90`, `0.00` y `0.00`.
El `switch` no tiene `default`: la rama `Physical` sin guarda cubre todos los físicos, y los otros dos casos cubren las otras variantes. Los patrones de tipo con nombre (`physical`,

`digital`, `service`) evitan los patrones sin nombre, que son vista previa en Java 21. Si añades una cuarta variante a `permits` (por ejemplo `Subscription`), el compilador marca el `switch` como no exhaustivo; un `default` lo habría ocultado.
Errores típicos: poner la rama con guarda después de la rama sin guarda (error de dominación); añadir un `default -> Money.zero("EUR")`, que compila pero elimina el aviso del compilador cuando aparezcan variantes nuevas.

Solución del ejercicio 1.4

enunciado en la pág. 19

Principios que rompe:

- **SRP.** `placeOrder` valida el pedido, calcula el total, aplica el descuento, lo guarda y envía el correo: cuatro motivos para cambiar (reglas de validación, cálculo, persistencia y notificación). Cambiar el texto del correo obliga a tocar la clase que también guarda en la base de datos.
- **OCP.** La cadena `if (... "VIP") ... else if (... "PREMIUM")` obliga a editar el método para cada tipo de cliente nuevo.
- **DIP.** `new SqlOrderDao()` y `new SmtpClient("smtp.shop.local")` atan la lógica de negocio a clases concretas: no se puede cambiar el almacenamiento ni el canal, y no se puede ejecutar sin base de datos ni servidor de correo.

LSP e ISP no se ven afectados. No hay jerarquía de tipos ni interfaces, así que no hay subtipos que sustituir ni interfaces demasiado anchas. Que no se rompan no significa que el diseño sea bueno: es lo que el código no permite evaluar.
Cómo se arreglaría: una clase que valida, otra que calcula el total (con una interfaz por tipo de cliente, como en el ejercicio 1.5), una interfaz `OrderRepository` con una implementación SQL y una interfaz `Mailer` con una implementación SMTP, y un `OrderService` que reciba esas piezas por el constructor y solo coordine.
Error típico: decir que rompe LSP porque «usa cadenas de `if`». Las cadenas de `if` son un síntoma de OCP; LSP trata de subtipos que no cumplen el contrato de su tipo base.

Solución del ejercicio 1.5

enunciado en la pág. 19

CustomerDiscount.java

```java
package shoplab.ejercicios.ocp;

import java.math.BigDecimal;

public interface CustomerDiscount {

    boolean appliesTo(String customerType);

    BigDecimal rate();
}
```

RegularDiscount.java

```java
package shoplab.ejercicios.ocp;

import java.math.BigDecimal;

public class RegularDiscount implements CustomerDiscount {

    @Override
    public boolean appliesTo(String customerType) {
        return customerType.equals("REGULAR");
    }

    @Override
    public BigDecimal rate() {
        return BigDecimal.ZERO;
    }
}
```

PremiumDiscount.java

```java
package shoplab.ejercicios.ocp;

import java.math.BigDecimal;

public class PremiumDiscount implements CustomerDiscount {

    @Override
    public boolean appliesTo(String customerType) {
        return customerType.equals("PREMIUM");
    }

    @Override
    public BigDecimal rate() {
        return new BigDecimal("0.10");
    }
}
```

VipDiscount.java

```java
package shoplab.ejercicios.ocp;

import java.math.BigDecimal;

public class VipDiscount implements CustomerDiscount {

    @Override
    public boolean appliesTo(String customerType) {
        return customerType.equals("VIP");
    }

    @Override
    public BigDecimal rate() {
        return new BigDecimal("0.20");
    }
}
```

StudentDiscount.java

```java
package shoplab.ejercicios.ocp;

import java.math.BigDecimal;

public class StudentDiscount implements CustomerDiscount {

    @Override
    public boolean appliesTo(String customerType) {
        return customerType.equals("STUDENT");
    }

    @Override
    public BigDecimal rate() {
        return new BigDecimal("0.15");
    }
}
```

OpenPricingService.java

```java
package shoplab.ejercicios.ocp;

import java.util.List;
import shoplab.Money;

public class OpenPricingService {

    private final List<CustomerDiscount> discounts;

    public OpenPricingService(List<CustomerDiscount> discounts) {
        this.discounts = List.copyOf(discounts);
    }

    public Money finalPrice(String customerType, Money base) {
        for (CustomerDiscount discount : discounts) {
            if (discount.appliesTo(customerType)) {
                return base.minus(base.times(discount.rate()));
            }
        }
        throw new IllegalArgumentException(
                "unknown customer type: " + customerType);
    }
}
```

PricingMain.java

```java
package shoplab.ejercicios.ocp;

import java.math.BigDecimal;
import java.util.List;
import shoplab.Money;

public class PricingMain {

    public static void main(String[] args) {
        OpenPricingService service = new OpenPricingService(List.of(
                new RegularDiscount(),
                new PremiumDiscount(),
                new VipDiscount(),
                new StudentDiscount()));
        Money base = new Money(new BigDecimal("100.00"), "EUR");
        System.out.println(service.finalPrice("REGULAR", base));
        System.out.println(service.finalPrice("PREMIUM", base));
        System.out.println(service.finalPrice("VIP", base));
        System.out.println(service.finalPrice("STUDENT", base));
        try {
            service.finalPrice("GHOST", base);
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

La salida es `100.00`, `90.00`, `80.00` y `85.00` EUR, y la última llamada imprime `unknown customer type: GHOST`. El servicio recorre las reglas y usa la primera que aplica; añadir `STUDENT` fue escribir una clase y una línea en el `main`, sin editar `OpenPricingService`. Quien construye la lista (aquí `main`; más adelante, Spring) es quien decide qué reglas existen.

Este olor, una familia de reglas intercambiables detrás de una interfaz, es el que resuelve el patrón Strategy del capítulo 8. Aquí lo hemos aplicado sin ponerle nombre.

Errores típicos: dejar la cadena de `if` dentro del servicio y mover solo las tasas; pasar a la regla el `Money` en vez de la tasa y duplicar el cálculo en cada clase; olvidar el caso de un tipo desconocido, que devolvería `null` o el precio sin descuento sin avisar.

Solución del ejercicio 1.6

enunciado en la pág. 20

Estado final del proyecto (los archivos nuevos o modificados de este capítulo):

```
shoplab/
|-- build.gradle                 (generado por Initializr, sin cambios)
|-- settings.gradle
|-- gradlew, gradlew.bat, gradle/wrapper/
`-- src/
    |-- main/
    |   |-- java/shoplab/
    |   |   |-- ShopLabApplication.java
    |   |   |-- Money.java
    |   |   |-- ProductKind.java
    |   |   |-- Product.java
    |   |   |-- ProductRepository.java
    |   |   |-- InMemoryProductRepository.java
    |   |   `-- ProductController.java
    |   `-- resources/application.properties
    `-- test/java/shoplab/ShopLabApplicationTests.java
```

ShopLabApplication.java

```java
package shoplab;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class ShopLabApplication {

    public static void main(String[] args) {
        SpringApplication.run(ShopLabApplication.class, args);
    }
}
```

Money.java

```java
package shoplab;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.Objects;

public record Money(BigDecimal amount, String currency) {

    public Money {
        Objects.requireNonNull(amount, "amount");
        Objects.requireNonNull(currency, "currency");
        if (amount.signum() < 0) {
            throw new IllegalArgumentException(
                    "amount must not be negative: " + amount);
        }
        if (currency.isBlank()) {
            throw new IllegalArgumentException("currency must not be blank")
    ↪ ;
        }
        amount = amount.setScale(2, RoundingMode.HALF_UP);
    }

    public static Money zero(String currency) {
        return new Money(BigDecimal.ZERO, currency);
    }

    public Money plus(Money other) {
        requireSameCurrency(other);
        return new Money(amount.add(other.amount), currency);
    }

    public Money minus(Money other) {
        requireSameCurrency(other);
        return new Money(amount.subtract(other.amount), currency);
    }

    public Money times(int quantity) {
        BigDecimal factor = BigDecimal.valueOf(quantity);
        return new Money(amount.multiply(factor), currency);
    }

    public Money times(BigDecimal factor) {
        return new Money(amount.multiply(factor), currency);
    }

    private void requireSameCurrency(Money other) {
        if (!currency.equals(other.currency)) {
            throw new IllegalArgumentException(
                    "different currencies: " + currency
                            + " and " + other.currency);
        }
    }
}
```

ProductKind.java

```java
package shoplab;

import com.fasterxml.jackson.annotation.JsonSubTypes;
import com.fasterxml.jackson.annotation.JsonTypeInfo;

@JsonTypeInfo(use = JsonTypeInfo.Id.NAME, property = "type")
@JsonSubTypes({
        @JsonSubTypes.Type(value = ProductKind.Physical.class,
                name = "physical"),
        @JsonSubTypes.Type(value = ProductKind.Digital.class,
                name = "digital"),
        @JsonSubTypes.Type(value = ProductKind.Service.class,
                name = "service")
})
public sealed interface ProductKind
        permits ProductKind.Physical, ProductKind.Digital,
                ProductKind.Service {

    record Physical(int weightGrams) implements ProductKind {
    }

    record Digital() implements ProductKind {
    }

    record Service(int durationMinutes) implements ProductKind {
    }
}
```

Product.java

```java
package shoplab;

import java.util.Objects;

public record Product(
        String id,
        String name,
        Money price,
        ProductKind kind,
        int stock,
        String category) {

    public Product {
        Objects.requireNonNull(price, "price");
        Objects.requireNonNull(kind, "kind");
        if (id == null || id.isBlank()) {
            throw new IllegalArgumentException("id must not be blank");
        }
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("name must not be blank");
        }
        if (category == null || category.isBlank()) {
            throw new IllegalArgumentException("category must not be blank")
    ↪ ;
        }
        if (stock < 0) {
            throw new IllegalArgumentException(
                    "stock must not be negative: " + stock);
        }
    }
}
```

ProductRepository.java

```java
package shoplab;

import java.util.List;
import java.util.Optional;

public interface ProductRepository {

    Product save(Product product);

    Optional<Product> findById(String id);

    List<Product> findAll();
}
```

InMemoryProductRepository.java

```java
package shoplab;

import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;
import org.springframework.stereotype.Component;

@Component
public class InMemoryProductRepository implements ProductRepository {

    private final Map<String, Product> products = new ConcurrentHashMap<>();

    @Override
    public Product save(Product product) {
        products.put(product.id(), product);
        return product;
    }

    @Override
    public Optional<Product> findById(String id) {
        return Optional.ofNullable(products.get(id));
    }

    @Override
    public List<Product> findAll() {
        return List.copyOf(products.values());
    }
}
```

ProductController.java

```java
package shoplab;

import java.util.List;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/products")
public class ProductController {

    private final ProductRepository repository;

    public ProductController(ProductRepository repository) {
        this.repository = repository;
    }

    @GetMapping
    public List<Product> list() {
        return repository.findAll();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Product> get(@PathVariable String id) {
        return repository.findById(id)
                .map(ResponseEntity::ok)
                .orElseGet(() -> ResponseEntity.notFound().build());
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Product create(@RequestBody Product product) {
        return repository.save(product);
    }
}
```

Con el servidor arrancado (`./gradlew bootRun`), las pruebas con curl y sus resultados (de cada respuesta se muestran la línea de estado y el cuerpo):

```
$ curl -i -H "Content-Type: application/json" \
    http://localhost:8080/products
HTTP/1.1 200
Content-Type: application/json
[]

$ curl -i -H "Content-Type: application/json" \
    -d '{"id":"p1","name":"Teclado",
         "price":{"amount":49.9,"currency":"EUR"},
         "kind":{"type":"physical","weightGrams":800},
         "stock":12,"category":"hardware"}' \
    http://localhost:8080/products
HTTP/1.1 201
Content-Type: application/json
{"id":"p1","name":"Teclado","price":{"amount":49.90,"currency":"EUR"},
 "kind":{"type":"physical","weightGrams":800},"stock":12,
 "category":"hardware"}

$ curl -i -H "Content-Type: application/json" \
    -d '{"id":"p2","name":"Curso",
         "price":{"amount":"19.99","currency":"EUR"},
         "kind":{"type":"digital"},"stock":0,"category":"books"}' \
    http://localhost:8080/products
HTTP/1.1 201
{"id":"p2","name":"Curso","price":{"amount":19.99,"currency":"EUR"},
 "kind":{"type":"digital"},"stock":0,"category":"books"}

$ curl -i -H "Content-Type: application/json" \
    http://localhost:8080/products/p1
HTTP/1.1 200
(el mismo JSON que devolvió el POST de p1)

$ curl -i -H "Content-Type: application/json" \
    http://localhost:8080/products/zz
HTTP/1.1 404

$ curl -i -H "Content-Type: application/json" \
    -d '{"id":"p3","name":"X","price":{"amount":1,"currency":"EUR"},
         "kind":{"type":"cosa"},"stock":0,"category":"books"}' \
    http://localhost:8080/products
HTTP/1.1 400
```

Tras crear los dos productos, `GET /products` devuelve los dos en una lista (sin orden garantizado, porque un `ConcurrentHashMap` no ordena).

Todo funciona porque cada pieza tiene una sola tarea: el record valida, el repositorio guarda, el controlador traduce HTTP. El controlador depende de la interfaz `ProductRepository` y Spring le inyecta la única implementación, marcada con `@Component`. `ProductKind` lleva las anotaciones de Jackson porque, sin ellas, no se podría deserializar el JSON a una interfaz. Este es el estado del proyecto del que parte el capítulo siguiente.

Errores típicos: poner las clases fuera del paquete `shoplab` (Spring no las encuentra y el controlador no responde); olvidar `-H "Content-Type: application/json"` en el `POST` (responde `415`); enviar un importe negativo (el constructor de `Money` lo rechaza y la respuesta es `400`); usar `HashMap` en vez de `ConcurrentHashMap`, que funciona en pruebas manuales y

falla con peticiones simultáneas.

Parte II

Patrones creacionales

CAPÍTULO 2

Singleton y el contenedor de Spring

Objetivos del capítulo

- Reconocer el olor que justifica un Singleton y escribir uno correcto en Java puro (holder y `enum`).
- Explicar por qué una inicialización perezosa sin sincronizar puede crear dos instancias.
- Defender cuándo un Singleton es una mala idea y qué ofrece Spring en su lugar.
- Declarar beans con `@Configuration`, `@Bean` y `@Value`, y elegir entre los ámbitos `singleton` y `prototype`.

Hay objetos de los que solo debe existir uno: la configuración de la tienda, un generador de números de recibo, un registro de tiendas. Si cada clase hace su propio `new`, cada una tendrá su copia y los datos dejarán de coincidir. El patrón Singleton resuelve eso, pero es también el patrón más criticado del catálogo. En este capítulo lo escribes a mano, ves por qué conviene evitarlo y descubres que Spring ya te da lo bueno del patrón sin sus defectos: el contenedor de beans.

2.1 El olor: estado compartido y `new` por todas partes

Imagina que, en ShopLab, cada controlador crea su propio repositorio:

```
public class ProductController {
    private final ProductRepository repository = new InMemoryProductRepository
    ↪ ();
    // ...
}

public class OrderController {
    private final ProductRepository repository = new InMemoryProductRepository
    ↪ ();
    // ...
}
```

Cada `new` crea un `ConcurrentHashMap` distinto. Un producto que guardas por `POST /products` no lo ve `OrderController`, porque mira en su propio mapa. El síntoma es sutil: no hay ningún error, solo datos que «desaparecen». El olor tiene dos partes:

- **Estado que debería ser único** y está duplicado.
- **`new` repartido por el código**: cada clase decide cómo se crea su dependencia, y cambiarlo exige editar todas.

En el capítulo 1 ya arreglaste la segunda parte con DIP: los controladores reciben el repositorio por constructor. Queda la primera: que de verdad sea *el mismo* objeto para todos. Esa garantía

es lo que busca el Singleton.

2.2 Singleton en Java puro

Gamma y sus coautores definen el propósito del patrón así: garantizar que una clase tenga una única instancia y dar un punto de acceso a ella [7]. Para conseguirlo hacen falta tres piezas:

1. un constructor `private`, para que nadie pueda hacer `new`;
2. un campo estático que guarda la única instancia;
3. un método estático (`getInstance()`) que la devuelve.

La diferencia entre las variantes está en *cuándo se crea* la instancia y en si funciona bien con varios hilos. Los ejemplos de esta sección y de la siguiente son un generador de números de recibo; ponlos en el paquete `shoplab.ejemplos` solo para probarlos, no forman parte del proyecto.

2.2.1 Inicialización perezosa

La versión «de libro» crea la instancia la primera vez que alguien la pide:

LazyReceiptNumbers.java

```
1  package shoplab.ejemplos;
2
3  import java.util.concurrent.atomic.AtomicInteger;
4
5  public class LazyReceiptNumbers {
6
7      private static LazyReceiptNumbers instance;
8
9      private final AtomicInteger last = new AtomicInteger();
10
11     private LazyReceiptNumbers() {
12     }
13
14     public static LazyReceiptNumbers getInstance() {
15         if (instance == null) {
16             instance = new LazyReceiptNumbers();
17         }
18         return instance;
19     }
20
21     public int next() {
22         return last.incrementAndGet();
23     }
24 }
```

Con un solo hilo funciona. Con varios, no: dos hilos pueden ver `instance == null` a la vez y crear una instancia cada uno. Para entender por qué, hace falta un recordatorio de concurrencia.

Recordatorio: hilos, `synchronized` y `volatile`

Ya has trabajado con hilos, así que va solo lo que necesitas en este capítulo. Tomcat atiende cada petición HTTP en un hilo distinto, y todos comparten los beans de la aplicación; por

eso importa.

- Un `Thread` ejecuta un `Runnable` en paralelo: `new Thread(tarea)`, `start()` para lanzarlo y `join()` para esperar a que termine.
- Una *condición de carrera* aparece cuando el resultado depende del orden en que se intercalan los hilos. `n++` no es una operación atómica: lee, suma y escribe, y dos hilos pueden leer el mismo valor y perder una suma. Lo mismo pasa con «comprobar y después crear».
- `synchronized` marca un método o un bloque (`synchronized (lock) { ... }`) con un cerrojo: solo un hilo a la vez puede estar dentro con el mismo cerrojo.
- `volatile` en un campo garantiza que lo que escribe un hilo lo ven los demás (visibilidad) y que las lecturas y escrituras no se reordenan alrededor de él. No convierte `n++` en atómico.
- Para contadores y mapas compartidos, el JDK trae clases ya seguras: `AtomicInteger` y `ConcurrentHashMap` [17].

Este programa lo muestra. El resultado sin `synchronized` cambia en cada ejecución y casi siempre es menor que 200 000:

CounterRace.java

```
package shoplab.ejemplos;

public class CounterRace {

    private static final Object LOCK = new Object();
    private static int plain = 0;
    private static int guarded = 0;

    public static void main(String[] args) throws InterruptedException {
        Runnable task = () -> {
            for (int i = 0; i < 100_000; i++) {
                plain++;
                synchronized (LOCK) {
                    guarded++;
                }
            }
        };
        Thread first = new Thread(task);
        Thread second = new Thread(task);
        first.start();
        second.start();
        first.join();
        second.join();
        System.out.println("sin synchronized: " + plain);
        System.out.println("con synchronized: " + guarded);
    }
}
```

Ya puedes arreglar la versión perezosa. La primera solución es sincronizar el método:

SyncReceiptNumbers.java

```java
package shoplab.ejemplos;

import java.util.concurrent.atomic.AtomicInteger;

public class SyncReceiptNumbers {

    private static SyncReceiptNumbers instance;

    private final AtomicInteger last = new AtomicInteger();

    private SyncReceiptNumbers() {
    }

    public static synchronized SyncReceiptNumbers getInstance() {
        if (instance == null) {
            instance = new SyncReceiptNumbers();
        }
        return instance;
    }

    public int next() {
        return last.incrementAndGet();
    }
}
```

Es correcto, pero bloquea en *cada* llamada aunque la instancia ya exista. La variante de doble comprobación (*double-checked locking*) evita el cerrojo una vez creada la instancia; exige `volatile`, porque sin él otro hilo podría ver una referencia a un objeto a medio construir:

DclReceiptNumbers.java

```java
package shoplab.ejemplos;

import java.util.concurrent.atomic.AtomicInteger;

public class DclReceiptNumbers {

    private static volatile DclReceiptNumbers instance;

    private final AtomicInteger last = new AtomicInteger();

    private DclReceiptNumbers() {
    }

    public static DclReceiptNumbers getInstance() {
        DclReceiptNumbers local = instance;
        if (local == null) {
            synchronized (DclReceiptNumbers.class) {
                local = instance;
                if (local == null) {
                    local = new DclReceiptNumbers();
                    instance = local;
                }
            }
```

```
        }
        return local;
    }

    public int next() {
        return last.incrementAndGet();
    }
}
```

Funciona, pero es mucho código delicado para un problema que el lenguaje resuelve mejor.

2.2.2 El idioma del holder

La JVM inicializa una clase una sola vez y de forma segura para hilos, y solo cuando se usa por primera vez. El *holder* aprovecha eso: la instancia vive en una clase interna estática que no se carga hasta que alguien llama a `getInstance()`.

ReceiptNumbers.java

```
package shoplab.ejemplos;

import java.util.concurrent.atomic.AtomicInteger;

public final class ReceiptNumbers {

    private final AtomicInteger last = new AtomicInteger();

    private ReceiptNumbers() {
    }

    private static final class Holder {
        private static final ReceiptNumbers INSTANCE = new ReceiptNumbers();
    }

    public static ReceiptNumbers getInstance() {
        return Holder.INSTANCE;
    }

    public int next() {
        return last.incrementAndGet();
    }
}
```

Es perezoso, seguro para hilos y no necesita `synchronized` ni `volatile`. Para la mayoría de los casos, es la versión clásica que debes usar.

2.2.3 Singleton con `enum`

Bloch recomienda un `enum` de un solo elemento como la mejor forma de implementar un Singleton en Java [3]: es corto, la JVM garantiza una instancia y resiste dos trampas de las otras variantes (la deserialización y la reflexión pueden crear una segunda instancia de una clase normal, pero no de un `enum`).

ReceiptSequence.java

```java
package shoplab.ejemplos;

import java.util.concurrent.atomic.AtomicInteger;

public enum ReceiptSequence {
    INSTANCE;

    private final AtomicInteger last = new AtomicInteger();

    public int next() {
        return last.incrementAndGet();
    }
}
```

Dos limitaciones: un `enum` no puede extender otra clase y su creación no es perezosa en el sentido estricto (se crea al cargar el `enum`). Para este uso no importan.

Este programa comprueba las dos variantes:

SingletonDemo.java

```java
package shoplab.ejemplos;

public class SingletonDemo {

    public static void main(String[] args) {
        ReceiptNumbers a = ReceiptNumbers.getInstance();
        ReceiptNumbers b = ReceiptNumbers.getInstance();
        System.out.println("holder, misma instancia: " + (a == b));
        System.out.println("holder: " + a.next() + " " + b.next());
        System.out.println("enum: " + ReceiptSequence.INSTANCE.next()
                + " " + ReceiptSequence.INSTANCE.next());
    }
}
```

```
holder, misma instancia: true
holder: 1 2
enum: 1 2
```

El `AtomicInteger` no es un adorno

El Singleton comparte su estado entre todos los hilos. Si `next()` usara un `int` con `++`, tendrías la carrera del recordatorio aunque la instancia fuera única. Un Singleton con estado mutable debe ser seguro para hilos.

2.3 Las críticas: estado global y acoplamiento

El Singleton es útil, pero tiene fama de ser un patrón que se aplica de más. La revisión sistemática de Zhang y Budgen sobre los estudios empíricos de patrones señala que Singleton, aun siendo útil, puede tener aspectos negativos [30]. Los problemas habituales son estos:

- **Estado global.** Cualquier clase puede llamar a `getInstance()` y modificar su estado. La dependencia no aparece en el constructor ni en ninguna firma: está escondida en el cuerpo de los métodos.
- **Acoplamiento a una clase concreta.** Quien llama a `ReceiptNumbers.getInstance()` depende de `ReceiptNumbers`, no de una abstracción. Es lo contrario de DIP: no puedes cambiar la implementación sin editar a los clientes.
- **Difícil de sustituir en pruebas.** Como no se inyecta, no puedes dar al código bajo prueba una versión falsa ni reiniciar el estado entre pruebas: el contador sigue donde lo dejó la prueba anterior.
- **Hace dos cosas.** Controla su propia unicidad *y* hace su trabajo (numerar recibos). Rompe SRP.

Fíjate en que la mayoría de estas críticas no son contra «tener una sola instancia», sino contra *cómo se garantiza*: con un acceso estático global. La buena noticia es que se puede conseguir lo primero sin lo segundo.

2.4 Y en Spring: el contenedor como Singleton

Spring Framework gestiona los objetos de tu aplicación en un contenedor, el `ApplicationContext`. Los objetos que crea y gestiona el contenedor se llaman *beans* [27]. Tú describes cómo se construyen y el contenedor los crea, los conecta entre sí y se los entrega a quien los pide, que es la idea de la inyección de dependencias que Fowler describió [4].

Por defecto, un bean tiene el ámbito `singleton`: el contenedor crea *una* instancia y se la da a todos los que la piden. Es el Singleton de siempre, pero sin constructor privado, sin `getInstance()` y sin estado global: la clase es una clase normal y es el contenedor quien garantiza la unicidad. Ya lo usaste en el capítulo 1: `InMemoryProductRepository` lleva `@Component`, y Spring le pasa la misma instancia a todo lo que pide un `ProductRepository`.

«Singleton» de Spring no es «una instancia por clase»

El ámbito singleton de Spring significa una instancia *por bean* dentro de un contenedor, no una por clase [27]. Si defines dos beans con distinto nombre de la misma clase, tendrás dos instancias. Y como los beans singleton los comparten todos los hilos de Tomcat, deben ser sin estado o seguros para hilos, por lo que viste en el recordatorio de concurrencia.

2.4.1 Declarar un bean con `@Configuration` y `@Bean`

`@Component` sirve para clases tuyas. Para crear un bean a mano (porque necesita valores que le pasas tú, o porque es una clase de una biblioteca), se escribe una clase con `@Configuration` y un método con `@Bean`: el método devuelve el objeto y Spring lo registra con el nombre del método.

ReceiptConfig.java

```
1 package shoplab.ejemplos;
2
3 import org.springframework.context.annotation.Bean;
4 import org.springframework.context.annotation.Configuration;
5
6 @Configuration
```

```
7  public class ReceiptConfig {
8
9      @Bean
10     public ReceiptCounter receiptCounter() {
11         return new ReceiptCounter();
12     }
13 }
```

ReceiptCounter.java

```
1  package shoplab.ejemplos;
2
3  import java.util.concurrent.atomic.AtomicInteger;
4
5  public class ReceiptCounter {
6
7      private final AtomicInteger last = new AtomicInteger();
8
9      public int next() {
10         return last.incrementAndGet();
11     }
12 }
```

`ReceiptCounter` es una clase corriente: constructor público, sin anotaciones. Ya no hay nada que proteja la unicidad en el código de la clase; la protege el contenedor. Cualquier clase que pida un `ReceiptCounter` en su constructor recibirá el mismo. (Si alguien hiciera `new ReceiptCounter()` por su cuenta, obtendría otro objeto ajeno al contenedor: el truco es no hacerlo.)

Recordatorio: `application.properties`

El archivo `src/main/resources/application.properties` contiene la configuración de la aplicación. Boot lo lee al arrancar [24].

- Cada línea es `clave=valor`; las líneas que empiezan por `#` son comentarios. Todo son cadenas: Spring las convierte al tipo que pide quien las recibe (`int`, `boolean`, etc.).
- Boot tiene propiedades propias (`server.port`, `spring.application.name`, que ya viste en el proyecto). Las tuyas puedes llamarlas como quieras; en ShopLab todas empiezan por `shop.`, en minúsculas y con guiones (`shop.store-name`).
- Para leer una propiedad, un parámetro o campo lleva `@Value`: describe la clave como `@Value("${shop.store-name}")`. Si la clave no existe, la aplicación *no arranca*. Puedes dar un valor por defecto tras dos puntos: `${shop.store-name:ShopLab}`.

Este `@Value` es el de Spring (paquete `org.springframework.beans.factory.annotation`).

Con esto ya puedes crear un bean cuyos valores vienen de la configuración. Por ejemplo, un bean con el prefijo de los recibos:

ReceiptFormat.java

```
1  package shoplab.ejemplos;
2
3  public record ReceiptFormat(String prefix) {
4
```

```
    public String format(int number) {
        return prefix + "-" + number;
    }
}
```

ReceiptFormatConfig.java

```
package shoplab.ejemplos;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class ReceiptFormatConfig {

    @Bean
    public ReceiptFormat receiptFormat(
            @Value("${shop.receipt-prefix:R}") String prefix) {
        return new ReceiptFormat(prefix);
    }
}
```

El valor por defecto `R` se usa si `shop.receipt-prefix` no está en `application.properties`; si lo añades (`shop.receipt-prefix=TICKET`), manda el archivo. El proyecto de este capítulo (sección 2.6) hace lo mismo sin valores por defecto.

2.4.2 Pedir un bean al contexto: `ApplicationContext` y `getBean`

Normalmente no pides los beans: te llegan por constructor. Pero para ver qué instancia entrega el contenedor, puedes pedírsela tú: `ApplicationContext` es un bean más, así que también se inyecta por constructor, y `getBean(Clase.class)` devuelve el bean de ese tipo. Para ejecutar código al arrancar usamos un `ApplicationRunner`, una interfaz con un único método `run(ApplicationArguments args)` que Boot llama cuando el contexto está listo. Cualquier bean que la implemente se ejecuta al arrancar.

SingletonCheck.java

```
package shoplab.ejemplos;

import org.springframework.boot.ApplicationArguments;
import org.springframework.boot.ApplicationRunner;
import org.springframework.context.ApplicationContext;
import org.springframework.stereotype.Component;

@Component
public class SingletonCheck implements ApplicationRunner {

    private final ApplicationContext context;

    public SingletonCheck(ApplicationContext context) {
        this.context = context;
    }

```

```
    @Override
    public void run(ApplicationArguments args) {
        ReceiptCounter first = context.getBean(ReceiptCounter.class);
        ReceiptCounter second = context.getBean(ReceiptCounter.class);
        System.out.println("[c02] singleton, misma instancia: "
                + (first == second));
        System.out.println("[c02] "
                + context.getBean(ReceiptFormat.class).format(first.next()));
    }
}
```

Si arrancas ShopLab (`./gradlew bootRun`) con estas clases en el proyecto, la consola muestra, entre los mensajes de Boot:

c02

```
 singleton, misma instancia: true
[c02] R-1
```

Solo para probar

`getBean` desde el código de la aplicación es el *Service Locator* que Fowler contrapone a la inyección [4]: la clase va a buscar sus dependencias en vez de recibirlas. Úsalo solo para experimentar, como aquí. En el código real, inyecta por constructor.

2.4.3 El ámbito `prototype`

A veces no quieres compartir: cada cliente necesita su propia instancia. El ámbito `prototype` hace que el contenedor cree un objeto *nuevo* cada vez que alguien lo pide [27]. Se declara con `@Scope("prototype")`. Este bean, que genera un identificador corto distinto en cada instancia, sirve de ejemplo:

ShortId.java

```
package shoplab.ejemplos;

import java.util.UUID;
import org.springframework.context.annotation.Scope;
import org.springframework.stereotype.Component;

@Component
@Scope("prototype")
public class ShortId {

    private final String value = UUID.randomUUID().toString().substring(0, 8);

    public String value() {
        return value;
    }
}
```

Con el mismo `ApplicationRunner` comparas dos peticiones al contexto:

PrototypeCheck.java

```java
package shoplab.ejemplos;

import org.springframework.boot.ApplicationArguments;
import org.springframework.boot.ApplicationRunner;
import org.springframework.context.ApplicationContext;
import org.springframework.stereotype.Component;

@Component
public class PrototypeCheck implements ApplicationRunner {

    private final ApplicationContext context;

    public PrototypeCheck(ApplicationContext context) {
        this.context = context;
    }

    @Override
    public void run(ApplicationArguments args) {
        ShortId first = context.getBean(ShortId.class);
        ShortId second = context.getBean(ShortId.class);
        System.out.println("[c02] prototype, misma instancia: "
                + (first == second));
    }
}
```

c02

```
prototype, misma instancia: false
```

Un prototype no es lo contrario de un Singleton, sino otra decisión: «una instancia compartida» frente a «una por uso». Dos cosas que debes saber:

- Spring crea el prototype y se lo entrega a quien lo pide, pero no lo gestiona después: no llama a los métodos de destrucción. Si necesita liberar recursos, es cosa tuya.
- **Trampa:** si un bean singleton recibe un prototype por constructor, el constructor se ejecuta una sola vez, así que el singleton se queda con *la misma* instancia para siempre. El «nuevo cada vez» se pierde.

La salida de la trampa es pedir el prototype *en el momento de usarlo* con un `ObjectProvider<T>`: un objeto que inyectas en lugar del bean y al que llamas `getObject()` cada vez que quieres una instancia nueva.

IdIssuer.java

```java
package shoplab.ejemplos;

import org.springframework.beans.factory.ObjectProvider;
import org.springframework.stereotype.Component;

@Component
public class IdIssuer {

    private final ObjectProvider<ShortId> ids;
```

```
10
11      public IdIssuer(ObjectProvider<ShortId> ids) {
12          this.ids = ids;
13      }
14
15      public String issue() {
16          return ids.getObject().value();
17      }
18  }
```

`IdIssuer` es un singleton, pero cada llamada a `issue()` obtiene un `ShortId` nuevo. Si hubieras inyectado `ShortId` directamente, todas las llamadas devolverían el mismo identificador. Lo comprobarás tú en el ejercicio 2.4.

2.5 Cuándo sí y cuándo no

Elige con estas reglas:

- **Un recurso único de verdad** (configuración, un registro, un generador de números, un reloj, un pool): una sola instancia. En Spring, un bean singleton.
- **Singleton con `getInstance()`**: casi nunca en una aplicación Spring. El contenedor da la unicidad sin estado global y sin acoplar a la clase concreta. Ese Singleton «a mano» es útil fuera de Spring (una biblioteca, una utilidad sin contenedor) y para entender el patrón.
- **Estado por cliente o por operación**: no es un singleton. Un objeto nuevo por uso (`new`, o prototype con `ObjectProvider` si debe crearlo Spring).
- **Si el singleton tiene estado mutable**: hazlo seguro para hilos (`AtomicInteger`, `ConcurrentHashMap`, `synchronized`) o, mejor, déjalo sin estado.

Resumido en la ficha del libro: el *olor* es estado compartido duplicado; el *patrón* es una única instancia, mejor gestionada por el contenedor que por un `getInstance()`; la *justificación* es que sacas la unicidad de la clase (SRP) y de los clientes (DIP) a cambio de depender del contenedor. Para el catálogo de referencia del patrón, consulta también Refactoring.Guru [20] y su tratamiento más visual en *Head First Design Patterns* [5].

2.6 En el proyecto: H2a

Hito 2a: la configuración de la tienda

Partes de ShopLab tal como lo dejó el hito 1. Al terminar el capítulo, la tienda tiene unos ajustes globales (nombre, moneda por defecto y máximo de artículos por pedido) que vienen de `application.properties`, viven en un único bean y se consultan con `GET /settings`. Las clases nuevas son `ShopSettings`, `ShopConfig` y `SettingsController`. La construyes en el ejercicio 2.5.

Los ajustes de la tienda son un caso claro de «único de verdad»: si dos clases tuvieran copias distintas, una aplicaría un máximo y otra otro. El diseño tiene tres piezas.

1. `ShopSettings`, un record inmutable con los tres valores. Valida en el constructor compacto, como `Product`.

2. `ShopConfig`, una clase `@Configuration` con un `@Bean` que lee las propiedades con `@Value` y construye el record. Spring crea esa instancia una vez.
3. `application.properties` con las tres claves `shop.*`.

Cualquier clase que necesite los ajustes los pide por constructor (`ShopSettings settings`): en los capítulos siguientes lo harán las fábricas y los controladores de pedidos.

2.7 Ejercicios

Ejercicio 2.1: ShopRegistry con el idioma del holder ★☆☆☆☆

Escribe un Singleton `ShopRegistry` con el idioma del holder. Es un registro de identificadores de tienda: guarda un conjunto de identificadores y no admite repetidos. Trabaja en el paquete `shoplab.ejercicios` de tu proyecto.

1. Crea `src/main/java/shoplab/ejercicios/ShopRegistry.java`. La clase es `public final class ShopRegistry`.
2. Añade un campo de instancia `private final Set<String> storeIds` creado con `ConcurrentHashMap.newKeySet()` (un conjunto seguro para hilos).
3. Declara un constructor privado sin parámetros y vacío.
4. Declara dentro una clase interna `private static final class Holder` con un campo `private static final ShopRegistry INSTANCE = new ShopRegistry();`.
5. Añade el método estático `public static ShopRegistry getInstance()`, que devuelve `Holder.INSTANCE`.
6. Añade `public boolean register(String storeId)`, que añade el identificador y devuelve `true` si no estaba, y `public int count()`, con el número de identificadores.
7. Crea la clase de comprobación con este código, compila con `./gradlew compileJava` y ejecútala con `java -cp build/classes/java/main shoplab.ejercicios.ShopRegistryCheck`.

ShopRegistryCheck.java

```
package shoplab.ejercicios;

public class ShopRegistryCheck {

    public static void main(String[] args) {
        ShopRegistry first = ShopRegistry.getInstance();
        ShopRegistry second = ShopRegistry.getInstance();
        System.out.println(first == second);
        System.out.println(first.register("madrid"));
        System.out.println(second.register("madrid"));
        System.out.println(second.register("lisboa"));
        System.out.println(first.count());
    }
}
```

Está bien si la salida es exactamente esta (`madrid` se registra una vez aunque lo intentes por dos referencias, porque ambas son el mismo objeto):

```
true
true
false
true
2
```

Guiado ▪ solución en la pág. 57

Ejercicio 2.2: Dos instancias donde solo debía haber una ★★☆☆☆

Esta clase es un Singleton perezoso sin sincronizar. Su constructor tarda en ejecutarse (la pausa de 50 ms representa leer un archivo grande), lo que hace visible el fallo. Crea los dos archivos en `shoplab.ejercicios`.

PromoCodeBook.java

```java
package shoplab.ejercicios;

import java.util.concurrent.atomic.AtomicInteger;

public class PromoCodeBook {

    static final AtomicInteger CREATED = new AtomicInteger();

    private static PromoCodeBook instance;

    private PromoCodeBook() {
        CREATED.incrementAndGet();
        try {
            Thread.sleep(50);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    public static PromoCodeBook getInstance() {
        if (instance == null) {
            instance = new PromoCodeBook();
        }
        return instance;
    }

    public boolean isValid(String code) {
        return code.startsWith("PROMO-");
    }
}
```

RaceCheck.java

```java
package shoplab.ejercicios;

import java.util.ArrayList;
import java.util.List;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;

public class RaceCheck {

    public static void main(String[] args) throws InterruptedException {
        Set<PromoCodeBook> seen = ConcurrentHashMap.newKeySet();
        List<Thread> threads = new ArrayList<>();
        for (int i = 0; i < 8; i++) {
            Thread thread = new Thread(
                    () -> seen.add(PromoCodeBook.getInstance()));
            threads.add(thread);
            thread.start();
        }
        for (Thread thread : threads) {
            thread.join();
        }
        System.out.println("instancias distintas: " + seen.size());
        System.out.println("constructor ejecutado: "
                + PromoCodeBook.CREATED.get() + " veces");
    }
}
```

1. Compila y ejecuta `RaceCheck`. Debería imprimir más de una instancia.
2. Explica en una frase qué ocurre entre `if (instance == null)` y la asignación.
3. Corrige `PromoCodeBook` sin tocar `RaceCheck`.

Está bien si, tras la corrección, `RaceCheck` imprime `instancias distintas: 1` y `constructor ejecutado: 1 veces`.

Pista: *no hace falta* `synchronized`*: cambia dónde vive la instancia para que la inicialice la JVM.*

Depuración ▪ solución en la pág. 58

Ejercicio 2.3: Un bean, dos consumidores ★★☆☆☆

Demuestra que el contenedor entrega la misma instancia de un bean a dos clases distintas. Hazlo **en una copia del proyecto** (`cp -r shoplab shoplab-c02`), porque añade beans que no son del hito. Usa el paquete `shoplab.ejercicios`.

1. Crea `TicketCounter`, una clase *sin anotaciones* con un `AtomicInteger` y un método `int next()`.
2. Crea `TicketConfig` con `@Configuration` y un `@Bean` que devuelva un `TicketCounter`.
3. Crea dos beans `@Component`, `TicketPrinter` y `TicketAudit`, que reciben el `TicketCounter` por constructor. `TicketPrinter` tiene `String print(String text)`, que devuelve `"ticket " + n + ": " + text`. `TicketAudit` tiene `String record(String action)`, que devuelve `"audit " + n + ": " + action`. Ambos ob-

tienen `n` con `next()`. Los dos exponen el contador con `TicketCounter counter()`.

4. Añade este `ApplicationRunner` y arranca con `./gradlew bootRun`.

TicketCheck.java

```java
package shoplab.ejercicios;

import org.springframework.boot.ApplicationArguments;
import org.springframework.boot.ApplicationRunner;
import org.springframework.stereotype.Component;

@Component
public class TicketCheck implements ApplicationRunner {

    private final TicketPrinter printer;
    private final TicketAudit audit;

    public TicketCheck(TicketPrinter printer, TicketAudit audit) {
        this.printer = printer;
        this.audit = audit;
    }

    @Override
    public void run(ApplicationArguments args) {
        System.out.println("[c02] " + printer.print("camiseta"));
        System.out.println("[c02] " + audit.record("devolucion"));
        System.out.println("[c02] " + printer.print("gorra"));
        System.out.println("[c02] misma instancia: "
                + (printer.counter() == audit.counter()));
        System.out.println("[c02] con new: "
                + (printer.counter() == new TicketCounter()));
    }
}
```

Está bien si la consola muestra estas cinco líneas (entre los mensajes de Boot) y la numeración sigue de un consumidor al otro:

c02

```
 ticket 1: camiseta
[c02] audit 2: devolucion
[c02] ticket 3: gorra
[c02] misma instancia: true
[c02] con new: false
```

Responde también: ¿qué garantiza que `TicketPrinter` y `TicketAudit` compartan contador, si `TicketCounter` tiene un constructor público?

Práctica ▪ solución en la pág. 59

Ejercicio 2.4: Singleton frente a prototype ★★★☆☆

Comprueba la diferencia entre los ámbitos y la trampa del prototype dentro de un singleton. Hazlo **en una copia del proyecto** (puedes reutilizar la del ejercicio anterior), con beans propios en `shoplab.ejercicios`.

Crea `Voucher`, un bean de ámbito `prototype` que guarda un código `"V" + n`, donde `n` sale de un `AtomicInteger` estático que se incrementa en cada construcción, y un método `String code()`. Crea `VoucherDesk`, un singleton con `String issue()`, que devuelve el código de su `Voucher`. Esta primera versión de `VoucherDesk` recibe el `Voucher` directamente por constructor. Este `ApplicationRunner` lo comprueba (`VoucherDesk` y `Voucher` deben estar anotados como beans):

VoucherCheck.java

```java
package shoplab.ejercicios;

import org.springframework.boot.ApplicationArguments;
import org.springframework.boot.ApplicationRunner;
import org.springframework.context.ApplicationContext;
import org.springframework.stereotype.Component;

@Component
public class VoucherCheck implements ApplicationRunner {

    private final ApplicationContext context;
    private final VoucherDesk desk;

    public VoucherCheck(ApplicationContext context, VoucherDesk desk) {
        this.context = context;
        this.desk = desk;
    }

    @Override
    public void run(ApplicationArguments args) {
        Voucher first = context.getBean(Voucher.class);
        Voucher second = context.getBean(Voucher.class);
        System.out.println("[c02] getBean: " + first.code() + " "
                + second.code() + ", distintos: " + (first != second));
        System.out.println("[c02] desk: " + desk.issue() + " "
                + desk.issue() + " " + desk.issue());
    }
}
```

1. Arranca con la primera versión. Debe imprimir `getBean: V2 V3, distintos: true` y `desk: V1 V1 V1`. Explica por qué el pupitre repite el código y por qué empieza en `V1`.
2. Cambia `VoucherDesk` para que cada `issue()` entregue un código nuevo, sin quitar el ámbito `prototype` de `Voucher`.

Está bien si, tras el cambio, la salida es `getBean: V1 V2, distintos: true` y `desk: V3 V4 V5`.

Pista: *en vez de recibir el `Voucher`, `VoucherDesk` puede recibir algo que fabrique uno cada vez que lo pides.*

Práctica ▪ solución en la pág. 61

Ejercicio 2.5: Hito 2a: ShopSettings y GET /settings ★★★☆☆

Parte del proyecto tal como lo dejó el hito 1 (sin ninguna clase de los ejercicios anteriores de este capítulo). Añade los ajustes de la tienda como un bean configurado desde `application.properties`.

1. **Propiedades.** En `src/main/resources/application.properties`, deja `spring.application.name=ShopLab` y añade tres propiedades: `shop.store-name=ShopLab`, `shop.default-currency=EUR` y `shop.max-items-per-order=20`.
2. **Record.** Crea en `shoplab` el record `ShopSettings(String storeName, String defaultCurrency, int maxItemsPerOrder)`. Su constructor compacto lanza `IllegalArgumentException` si un texto está en blanco o es nulo, o si `maxItemsPerOrder` no es positivo.
3. **Configuración.** Crea `ShopConfig`, con `@Configuration`, y un `@Bean shopSettings` cuyos parámetros se rellenan con `@Value`.
4. **Controlador.** Crea `SettingsController`, que recibe el `ShopSettings` por constructor y lo devuelve en `GET /settings`.
5. **curl.** Arranca con `./gradlew bootRun` y llama al endpoint con `curl -i -H "Content-Type: application/json" http://localhost:8080/settings`.

Está bien si:

- la respuesta es `HTTP/1.1 200` con el cuerpo `{"storeName":"ShopLab","defaultCurrency":"EUR","maxItemsPerOrder":20}`;
- cambiar `shop.max-items-per-order` en el archivo y reiniciar cambia el valor de la respuesta, sin tocar el código;
- los endpoints de `/products` siguen funcionando.

Pista: *el nombre de cada propiedad dentro de* `@Value("${...}")` *es el mismo que en el archivo.*

Proyecto ▪ solución en la pág. 63

2.8 Resumen

Resumen

- Singleton garantiza una sola instancia con un punto de acceso. El olor que lo justifica es estado compartido que acaba duplicado por `new` repartidos.
- En Java puro, usa el idioma del holder o un `enum`. La versión perezosa sin sincronizar tiene una condición de carrera; `synchronized` y `volatile` la evitan, pero son delicados.
- Un Singleton con `getInstance()` crea estado global y acopla a la clase concreta: es difícil de sustituir en pruebas y rompe DIP y SRP.
- En Spring, el ámbito por defecto es `singleton`: una instancia por bean, que el contenedor crea y entrega a quien la pide. Compartida por todos los hilos, debe ser segura para hilos.
- Un bean se declara con `@Component` o con `@Configuration` y `@Bean`; `@Value("${clave}")` lee `application.properties`.
- `@Scope("prototype")` crea una instancia nueva por petición; dentro de un singleton,

pídela con `ObjectProvider`.

- ShopLab (H2a) tiene `ShopSettings` configurado desde `application.properties` y expuesto en `GET /settings`.

Soluciones de los ejercicios

Solución del ejercicio 2.1 enunciado en la pág. 51

ShopRegistry.java

```java
package shoplab.ejercicios;

import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;

public final class ShopRegistry {

    private final Set<String> storeIds = ConcurrentHashMap.newKeySet();

    private ShopRegistry() {
    }

    private static final class Holder {
        private static final ShopRegistry INSTANCE = new ShopRegistry();
    }

    public static ShopRegistry getInstance() {
        return Holder.INSTANCE;
    }

    public boolean register(String storeId) {
        return storeIds.add(storeId);
    }

    public int count() {
        return storeIds.size();
    }
}
```

La clase `Holder` no se carga hasta la primera llamada a `getInstance()`. Al cargarla, la JVM ejecuta su inicializador estático una sola vez y de forma segura para hilos, y crea la instancia. El constructor es privado, así que no hay otra forma de obtener un `ShopRegistry`. Por eso `first == second` es `true` y el segundo `register("madrid")` devuelve `false`: el conjunto ya lo tenía.

Errores típicos: dejar el constructor público (cualquiera puede hacer `new`); guardar el conjunto en un campo `static` de `ShopRegistry` en vez de usar el de instancia (funciona, pero ya no es un objeto con estado propio); usar un `HashSet` en vez de un conjunto concurrente, que fallaría con peticiones simultáneas.

Solución del ejercicio 2.2

enunciado en la pág. 52

Al ejecutar `RaceCheck` con la clase original, la salida cambia de una ejecución a otra, pero casi siempre se parece a esto (a veces hay una instancia distinta menos que ejecuciones del constructor, porque un hilo puede devolver la que acaba de asignar otro):

```
instancias distintas: 8
constructor ejecutado: 8 veces
```

Qué pasa: el primer hilo ve `instance == null` y entra en el constructor, que tarda 50 ms. Mientras tanto `instance` sigue siendo `null`, y los otros siete hilos también pasan la comprobación. Cada uno ejecuta `new` y asigna su propia instancia; la última asignación gana, pero cada hilo ya devolvió la suya. Es la carrera «comprobar y después crear».
La corrección: mover la instancia a un holder, como en el ejercicio anterior.

PromoCodeBook.java

```
1  package shoplab.ejercicios;
2
3  import java.util.concurrent.atomic.AtomicInteger;
4
5  public class PromoCodeBook {
6
7      static final AtomicInteger CREATED = new AtomicInteger();
8
9      private PromoCodeBook() {
10         CREATED.incrementAndGet();
11         try {
12             Thread.sleep(50);
13         } catch (InterruptedException e) {
14             Thread.currentThread().interrupt();
15         }
16     }
17
18     private static final class Holder {
19         private static final PromoCodeBook INSTANCE = new PromoCodeBook();
20     }
21
22     public static PromoCodeBook getInstance() {
23         return Holder.INSTANCE;
24     }
25
26     public boolean isValid(String code) {
27         return code.startsWith("PROMO-");
28     }
29 }
```

Con el holder, la JVM inicializa `Holder` una vez y los demás hilos esperan a que termine; salen todos con la misma instancia:

```
instancias distintas: 1
constructor ejecutado: 1 veces
```

Alternativas válidas: declarar `getInstance()` como `synchronized` (correcto, pero bloquea en cada llamada), el `enum` de un solo elemento, o la doble comprobación con `volatile`. **Errores típicos:** añadir `synchronized` a *otro* método que no sea `getInstance()`; usar doble comprobación sin `volatile`, que parece funcionar y puede fallar de forma esporádica; hacer `Thread.sleep` más corto «para que no falle»: el fallo era la carrera, no el tiempo.

Solución del ejercicio 2.3

enunciado en la pág. 53

TicketCounter.java

```java
package shoplab.ejercicios;

import java.util.concurrent.atomic.AtomicInteger;

public class TicketCounter {

    private final AtomicInteger last = new AtomicInteger();

    public int next() {
        return last.incrementAndGet();
    }
}
```

TicketConfig.java

```java
package shoplab.ejercicios;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class TicketConfig {

    @Bean
    public TicketCounter ticketCounter() {
        return new TicketCounter();
    }
}
```

TicketPrinter.java

```java
package shoplab.ejercicios;

import org.springframework.stereotype.Component;

@Component
public class TicketPrinter {

    private final TicketCounter counter;

    public TicketPrinter(TicketCounter counter) {
        this.counter = counter;
    }

    public String print(String text) {
        return "ticket " + counter.next() + ": " + text;
    }

    public TicketCounter counter() {
        return counter;
    }
}
```

TicketAudit.java

```java
package shoplab.ejercicios;

import org.springframework.stereotype.Component;

@Component
public class TicketAudit {

    private final TicketCounter counter;

    public TicketAudit(TicketCounter counter) {
        this.counter = counter;
    }

    public String record(String action) {
        return "audit " + counter.next() + ": " + action;
    }

    public TicketCounter counter() {
        return counter;
    }
}
```

`TicketCheck` es el del enunciado. Spring crea `ticketCounter` una vez, al procesar `TicketConfig`, y lo pasa a los constructores de `TicketPrinter` y `TicketAudit`. Por eso la numeración sigue de uno a otro (1, 2, 3) y la comparación con `==` da `true`.
Respuesta a la pregunta: lo garantiza el contenedor, no la clase. `TicketCounter` no sabe nada de unicidad: tiene un constructor público y el `new TicketCounter()` de la última línea de `TicketCheck` crea un objeto distinto, ajeno al contenedor (de ahí el `false`). El

ámbito singleton es una propiedad de la *definición del bean*, no de la clase.
Errores típicos: olvidar `@Configuration` (el `@Bean` se ignora y la aplicación no arranca, porque falta un bean `TicketCounter`); poner también `@Component` en `TicketCounter`, que lo registraría dos veces (dos beans del mismo tipo y un error de ambigüedad); hacer `new TicketCounter()` dentro de `TicketPrinter`.

Solución del ejercicio 2.4

enunciado en la pág. 54

Voucher.java

```java
package shoplab.ejercicios;

import java.util.concurrent.atomic.AtomicInteger;
import org.springframework.context.annotation.Scope;
import org.springframework.stereotype.Component;

@Component
@Scope("prototype")
public class Voucher {

    private static final AtomicInteger CREATED = new AtomicInteger();

    private final String code = "V" + CREATED.incrementAndGet();

    public String code() {
        return code;
    }
}
```

Versión original de `VoucherDesk` (la trampa):

```java
@Component
public class VoucherDesk {

    private final Voucher voucher;

    public VoucherDesk(Voucher voucher) {
        this.voucher = voucher;
    }

    public String issue() {
        return voucher.code();
    }
}
```

Salida de la primera versión:

c02

```
 getBean: V2 V3, distintos: true
[c02] desk: V1 V1 V1
```

`getBean` entrega un `Voucher` nuevo por llamada, y por eso `V2` y `V3` son distintos. `VoucherDesk` es un singleton y Spring lo crea al arrancar, antes del `ApplicationRunner`:

en ese momento construye su único `Voucher` (`V1`) y se queda con él para siempre, de ahí `V1 V1 V1`.
Versión corregida, con `ObjectProvider`:

VoucherDesk.java

```
package shoplab.ejercicios;

import org.springframework.beans.factory.ObjectProvider;
import org.springframework.stereotype.Component;

@Component
public class VoucherDesk {

    private final ObjectProvider<Voucher> vouchers;

    public VoucherDesk(ObjectProvider<Voucher> vouchers) {
        this.vouchers = vouchers;
    }

    public String issue() {
        return vouchers.getObject().code();
    }
}
```

Ahora el pupitre ya no crea ningún `Voucher` al arrancar; cada `issue()` llama a `getObject()`, que pide una instancia nueva al contenedor:

c02

```
 getBean: V1 V2, distintos: true
[c02] desk: V3 V4 V5
```

`VoucherCheck` no cambia, pero lo repetimos entero:

VoucherCheck.java

```java
package shoplab.ejercicios;

import org.springframework.boot.ApplicationArguments;
import org.springframework.boot.ApplicationRunner;
import org.springframework.context.ApplicationContext;
import org.springframework.stereotype.Component;

@Component
public class VoucherCheck implements ApplicationRunner {

    private final ApplicationContext context;
    private final VoucherDesk desk;

    public VoucherCheck(ApplicationContext context, VoucherDesk desk) {
        this.context = context;
        this.desk = desk;
    }

    @Override
    public void run(ApplicationArguments args) {
        Voucher first = context.getBean(Voucher.class);
        Voucher second = context.getBean(Voucher.class);
        System.out.println("[c02] getBean: " + first.code() + " "
                + second.code() + ", distintos: " + (first != second));
        System.out.println("[c02] desk: " + desk.issue() + " "
                + desk.issue() + " " + desk.issue());
    }
}
```

Errores típicos: quitar `@Scope("prototype")` de `Voucher` para «arreglarlo» (entonces todo es singleton y no hay instancias nuevas, que es justo lo contrario de lo que se quería); inyectar `ApplicationContext` en el pupitre y llamar a `getBean` (funciona, pero es un Service Locator; `ObjectProvider` declara la dependencia en el constructor); olvidar el tipo genérico y escribir `ObjectProvider` sin `<Voucher>`.

Solución del ejercicio 2.5

enunciado en la pág. 56

Primero, `src/main/resources/application.properties`:

```properties
spring.application.name=ShopLab
shop.store-name=ShopLab
shop.default-currency=EUR
shop.max-items-per-order=20
```

ShopSettings.java

```java
package shoplab;

import java.util.Objects;

public record ShopSettings(
        String storeName,
        String defaultCurrency,
        int maxItemsPerOrder) {

    public ShopSettings {
        Objects.requireNonNull(storeName, "storeName");
        Objects.requireNonNull(defaultCurrency, "defaultCurrency");
        if (storeName.isBlank()) {
            throw new IllegalArgumentException("storeName must not be blank
    ↪ ");
        }
        if (defaultCurrency.isBlank()) {
            throw new IllegalArgumentException(
                    "defaultCurrency must not be blank");
        }
        if (maxItemsPerOrder <= 0) {
            throw new IllegalArgumentException(
                    "maxItemsPerOrder must be positive: " +
    ↪ maxItemsPerOrder);
        }
    }
}
```

ShopConfig.java

```java
package shoplab;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class ShopConfig {

    @Bean
    public ShopSettings shopSettings(
            @Value("${shop.store-name}") String storeName,
            @Value("${shop.default-currency}") String defaultCurrency,
            @Value("${shop.max-items-per-order}") int maxItemsPerOrder) {
        return new ShopSettings(storeName, defaultCurrency,
    ↪ maxItemsPerOrder);
    }
}
```

SettingsController.java

```java
package shoplab;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/settings")
public class SettingsController {

    private final ShopSettings settings;

    public SettingsController(ShopSettings settings) {
        this.settings = settings;
    }

    @GetMapping
    public ShopSettings get() {
        return settings;
    }
}
```

Con el servidor arrancado (`./gradlew bootRun`):

```
$ curl -i -H "Content-Type: application/json" \
    http://localhost:8080/settings
HTTP/1.1 200
Content-Type: application/json
{"storeName":"ShopLab","defaultCurrency":"EUR","maxItemsPerOrder":20}
```

Cómo funciona: al arrancar, Spring procesa `ShopConfig`, resuelve los tres `@Value` con `application.properties`, convierte el último a `int` y llama al método `shopSettings`. El resultado es un bean singleton. Al construir `SettingsController`, Spring ve que pide un `ShopSettings` y le pasa ese bean. Cualquier otra clase que lo pida recibirá la misma instancia. Si cambias `shop.max-items-per-order=50` y reinicias, la respuesta cambia sin recompilar.

Este es el estado del proyecto del que parte el capítulo siguiente: el del hito 1 más estas tres clases y las tres propiedades. No incluye nada de los ejercicios anteriores.

Errores típicos: escribir mal la clave (`shop.storeName` en el código y `shop.store-name` en el archivo): la aplicación no arranca con un error de «Could not resolve placeholder»; poner `ShopConfig` fuera del paquete `shoplab` (Spring no la encuentra); olvidar `@Configuration` (el `@Bean` se ignora y falta el bean); importar otro `Value` que no sea el de `org.springframework.beans.factory.annotation`.

CAPÍTULO 3

Factory Method y Abstract Factory

Objetivos del capítulo

- Reconocer el olor de los `new` y los `switch` por tipo repartidos por el código y sustituirlos por un Factory Method.
- Usar un método `@Bean` como método de fábrica y elegir entre varios beans del mismo tipo con `Map<String, T>`, `List<T>` y `@Qualifier`.
- Explicar cuándo hace falta una familia de objetos coherentes (Abstract Factory) y distinguirla de un Factory Method y de una fábrica estática.
- Escribir y ejecutar tus primeras pruebas con JUnit Jupiter.
- Añadir a ShopLab los procesadores de pago y las políticas regionales, y exponer `GET /products/{id}/quote`.

En el capítulo anterior el contenedor de Spring te dio instancias únicas. Ahora la pregunta es otra: quién decide *qué clase concreta* se instancia. Una tienda cobra con tarjeta o por transferencia, y calcula impuestos y envío según el país. Si cada clase que necesita un procesador de pago hace su propio `new CardProcessor()`, añadir un método de pago obliga a editar todas. Los patrones Factory Method y Abstract Factory sacan esa decisión a un solo sitio. Además, este capítulo es el primero en el que escribes pruebas automáticas, así que también aprenderás lo mínimo de JUnit.

3.1 El olor: `new` y `switch` por tipo repartidos

ShopLab va a cobrar pedidos, y cobrar significa elegir un procesador de pago según el método que escoja el cliente. Un primer intento, muy natural, es decidirlo donde se necesita. Estas clases todavía no existen en el proyecto; son el «antes»:

CheckoutService.java

```
public class CheckoutService {

    public PaymentResult pay(PaymentType type, Money amount) {
        PaymentProcessor processor;
        if (type == PaymentType.CARD) {
            processor = new CardProcessor();
        } else if (type == PaymentType.TRANSFER) {
            processor = new BankTransferProcessor();
        } else {
            throw new IllegalArgumentException("unsupported: " + type);
        }
        return processor.charge(amount);
    }
}
```

Semanas después, otra parte de la tienda necesita reintentar un cobro fallido, y alguien repite la decisión con otra forma:

RetryService.java

```
public class RetryService {

    public PaymentResult retry(PaymentType type, Money amount) {
        PaymentProcessor processor = switch (type) {
            case CARD -> new CardProcessor();
            default -> new BankTransferProcessor();
        };
        return processor.charge(amount);
    }
}
```

El código funciona, pero tiene un olor con tres partes:

- **El conocimiento de «qué clase para qué tipo» está duplicado.** Si mañana se añade `PaymentType.CASH`, hay que acordarse de editar `CheckoutService`, `RetryService` y cualquier otra clase que repita la cadena. Es justo lo que el principio abierto/cerrado (OCP) pide evitar.
- **Las dos clases dependen de las implementaciones concretas** (`CardProcessor`, `BankTransferProcessor`) y no solo de la interfaz `PaymentProcessor`. Rompe DIP.
- **El `default` esconde los olvidos.** En `RetryService`, un `CASH` nuevo se cobraría por transferencia sin que nadie se enterase.

Kerievsky llama a la salida de este olor «mover el conocimiento de creación a una fábrica» [11]: un único lugar sabe qué clase construir, y los demás piden un `PaymentProcessor` sin saber cuál es.

3.2 Factory Method en Java puro

Gamma y sus coautores definen el propósito de Factory Method así: definir una interfaz para crear un objeto, pero dejar que sean las subclases las que decidan qué clase instanciar [7]. Para verlo hacen falta primero las piezas del dominio de pagos, que usará todo el capítulo. Son pequeñas.

PaymentType.java

```
package shoplab;

public enum PaymentType {
    CARD,
    TRANSFER
}
```

PaymentResult.java

```
package shoplab;

public record PaymentResult(
```

```
        boolean success,
        String transactionId,
        String message) {
}
```

PaymentProcessor.java

```
package shoplab;

public interface PaymentProcessor {

    PaymentResult charge(Money amount);
}
```

CardProcessor.java

```
package shoplab;

import java.util.concurrent.atomic.AtomicInteger;

public class CardProcessor implements PaymentProcessor {

    private final AtomicInteger sequence = new AtomicInteger();

    @Override
    public PaymentResult charge(Money amount) {
        String transactionId = "card-" + sequence.incrementAndGet();
        String message = "charged " + amount.amount() + " "
                + amount.currency() + " by card";
        return new PaymentResult(true, transactionId, message);
    }
}
```

`PaymentProcessor` es el único puerto de pago de todo el curso. Los procesadores son *simulados*: no hay red ni pasarela real, solo un identificador de transacción y un mensaje. El contador es un `AtomicInteger` porque, como verás en Spring, un mismo procesador atenderá muchas peticiones a la vez. `BankTransferProcessor` es igual con otro prefijo (`transfer-`) y otro mensaje; lo escribes tú en el ejercicio 3.1.

3.2.1 La estructura clásica

En su forma original, Factory Method tiene un *creador* (una clase abstracta) que declara el método de fábrica y lo usa dentro de su lógica, y *creadores concretos* (subclases) que deciden qué producto devuelve ese método.

PaymentService.java

```
package shoplab.ejemplos;

import shoplab.Money;
import shoplab.PaymentProcessor;
import shoplab.PaymentResult;

```

```java
public abstract class PaymentService {

    public PaymentResult pay(Money amount) {
        PaymentProcessor processor = createProcessor();
        return processor.charge(amount);
    }

    protected abstract PaymentProcessor createProcessor();
}
```

CardPaymentService.java

```java
package shoplab.ejemplos;

import shoplab.CardProcessor;
import shoplab.PaymentProcessor;

public class CardPaymentService extends PaymentService {

    @Override
    protected PaymentProcessor createProcessor() {
        return new CardProcessor();
    }
}
```

Asocia los nombres del patrón con las clases:

Papel en el patrón	Clase del ejemplo
Producto	`PaymentProcessor`
Producto concreto	`CardProcessor` (y `BankTransferProcessor`)
Creador	`PaymentService`, con el método de fábrica `createProcessor()`
Creador concreto	`CardPaymentService` (y un `TransferPaymentService`)

`pay` no sabe qué procesador usa: llama a `createProcessor()`, que en cada subclase devuelve el suyo. Es una aplicación de Template Method (cap. 8) en la que el paso variable es la creación. Funciona bien cuando cada tipo de creador aporta además comportamiento propio. Aquí solo varía la clase que se crea, y una clase por método de pago es mucho andamiaje.

3.2.2 La variante parametrizada

Gamma describe también una variante habitual: el método de fábrica recibe un *parámetro* que identifica el producto que hay que crear [7]. Es la versión que más verás en proyectos reales y la que usará ShopLab:

PaymentProcessorFactory.java

```java
package shoplab;

import java.util.Objects;

public class PaymentProcessorFactory {

    public PaymentProcessor create(PaymentType type) {
```

```
        Objects.requireNonNull(type, "type");
        return switch (type) {
            case CARD -> new CardProcessor();
            case TRANSFER -> new BankTransferProcessor();
        };
    }
}
```

Dos detalles que vienen del capítulo 1. El `switch` es una *expresión* sobre un `enum` y no tiene `default`: el compilador exige que cubra todos los valores. Si añades `CASH` a `PaymentType`, esta fábrica deja de compilar hasta que decidas qué hacer con él, que es exactamente el aviso que faltaba en `RetryService`. Y la comprobación con `Objects.requireNonNull` evita que un `null` llegue al `switch`, que lanzaría una `NullPointerException` sin mensaje útil.

Con la fábrica, los clientes quedan así:

CheckoutService.java

```
public class CheckoutService {

    private final PaymentProcessorFactory factory;

    public CheckoutService(PaymentProcessorFactory factory) {
        this.factory = factory;
    }

    public PaymentResult pay(PaymentType type, Money amount) {
        return factory.create(type).charge(amount);
    }
}
```

La ficha del libro queda así. *Olor:* el mismo `new` o `switch` por tipo repetido en varias clases. *Patrón:* Factory Method, aquí en su variante parametrizada. *Justificación:* el conocimiento de creación vive en un solo sitio (SRP), añadir un método de pago toca la fábrica y no a sus clientes (OCP) y los clientes dependen de la interfaz (DIP). El precio es una clase más y un nivel de indirección.

3.3 Y en Spring: `@Bean` como método de fábrica

Ya declaraste beans con `@Configuration` y `@Bean` en el capítulo anterior. Mira ahora ese método con los ojos de este patrón: un método `@Bean` *es* un método de fábrica. La clase `@Configuration` hace de creador, el tipo que devuelve el método es el producto, y es Spring quien llama al método y registra el resultado con el nombre del método [27]. Un ejemplo con una clase del JDK que no puedes anotar con `@Component`:

ClockConfig.java

```
package shoplab.ejemplos;

import java.time.Clock;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

```

```
@Configuration
public class ClockConfig {

    @Bean
    public Clock clock() {
        return Clock.systemUTC();
    }
}
```

Cualquier bean que pida un `Clock` por constructor recibirá ese. Es un ejemplo solo para leer: no lo añadas a ShopLab. Para tus propias clases tienes dos formas equivalentes de registrar un procesador de pago con el nombre `card`:

Dos formas de registrar el mismo bean

```
// Forma 1: anotas la clase.
@Component("card")
public class CardProcessor implements PaymentProcessor {
    // el cuerpo de la clase no cambia
}

// Forma 2: la clase queda sin anotar y un método @Bean la crea.
@Configuration
public class ProcessorConfig {

    @Bean("card")
    public PaymentProcessor cardProcessor() {
        return new CardProcessor();
    }
}
```

(No uses las dos a la vez: Boot no admite dos beans con el mismo nombre y la aplicación no arrancaría.) La forma 1 es la más corta y la que usará el hito de este capítulo. La forma 2 es la que necesitas cuando la creación tiene lógica propia (leer propiedades, elegir entre varias clases) o la clase no es tuya; la usarás en el capítulo 5.

3.3.1 Varios beans del mismo tipo

Con `CardProcessor` y `BankTransferProcessor` registrados como `@Component("card")` y `@Component("transfer")`, hay dos beans de tipo `PaymentProcessor`. Si una clase pide un solo `PaymentProcessor` por constructor, Spring no sabe cuál darle y la aplicación no arranca: lanza una `NoUniqueBeanDefinitionException` ("expected single matching bean but found 2: card,transfer"). Tienes tres formas de resolverlo [27].

1. `@Qualifier`: elegir uno por nombre. La anotación `org.springframework.beans.factory.annotation.Qualifier` estrecha el conjunto de candidatos; el nombre del bean sirve como valor:

CardOnlyCheckout.java

```
package shoplab.ejemplos;

import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.stereotype.Component;
```

```java
import shoplab.PaymentProcessor;

@Component
public class CardOnlyCheckout {

    private final PaymentProcessor processor;

    public CardOnlyCheckout(
            @Qualifier("card") PaymentProcessor processor) {
        this.processor = processor;
    }
}
```

Sirve cuando la clase *siempre* usa el mismo procesador. No sirve para una fábrica cuyo procesador depende de un dato en tiempo de ejecución.

2. `Map<String, T>`: todos los beans, con su nombre como clave. Si el constructor pide `Map<String, PaymentProcessor>`, Spring entrega un mapa con *todos* los beans de ese tipo y sus nombres como claves. La documentación lo indica así: el tipo de la clave debe ser `String`, los valores son los beans y las claves son los nombres de los beans [27]. Esto es una fábrica que casi no necesita código:

PaymentProcessorFactory.java (versión Spring)

```java
package shoplab;

import java.util.Map;
import java.util.Objects;
import org.springframework.stereotype.Component;

@Component
public class PaymentProcessorFactory {

    private final Map<String, PaymentProcessor> processors;

    public PaymentProcessorFactory(
            Map<String, PaymentProcessor> processors) {
        this.processors = processors;
    }

    public PaymentProcessor create(PaymentType type) {
        Objects.requireNonNull(type, "type");
        return switch (type) {
            case CARD -> processor("card");
            case TRANSFER -> processor("transfer");
        };
    }

    private PaymentProcessor processor(String name) {
        PaymentProcessor processor = processors.get(name);
        if (processor == null) {
            throw new IllegalStateException(
                    "no PaymentProcessor bean named '" + name + "'");
        }
        return processor;
    }
```

```
33 }
```

La fábrica ya no hace `new`: pide al contenedor los procesadores y elige el que corresponde. Eso tiene una ventaja que aprovecharás en el capítulo 5: puedes sustituir el bean `card` (por un adaptador a una pasarela real, por ejemplo) sin tocar esta clase. Y tiene un coste: las cadenas `"card"` y `"transfer"` son ahora parte del contrato entre la fábrica y quien registre los beans. Si alguien renombra un bean, el compilador no avisa; por eso la fábrica falla con un mensaje claro en vez de devolver `null`.

3. `List<T>`: todos los beans, sin nombre. Con `List<PaymentProcessor>` recibes los mismos beans sin clave. Es útil cuando quieres recorrerlos todos. El orden de la lista es el de registro de los beans, salvo que los ordenes con `@Order` o `Ordered` [27], un adelanto: lo verás con detalle en el capítulo 11; no te apoyes en él mientras tanto.

El nombre de un bean por defecto

Una clase con `@Component` sin valor recibe como nombre el de la clase con la primera letra en minúscula (`CardProcessor` sería `cardProcessor`), y un método `@Bean` recibe el nombre del método. Si tu fábrica busca `"card"` y no escribiste el nombre a mano, no lo encontrará. Escribe el nombre en la anotación (`@Component("card")`, `@Bean("card")`) cuando lo uses como clave.

3.4 El olor: objetos que deben ser coherentes entre sí

Ahora la tienda tiene que ofrecer un presupuesto: dado un producto y un país, cuánto cuesta con impuestos y envío. Tres cosas dependen del país: el IVA que se aplica, el coste de envío y la moneda en la que se cobra. Un primer diseño toma cada decisión por separado:

QuoteService.java

```
1  public class QuoteService {
2
3      public Money total(Product product, String region) {
4          BigDecimal rate = region.equals("ES")
5                  ? new BigDecimal("0.21") : new BigDecimal("0.20");
6          Money shipping = region.equals("FR")
7                  ? new Money(new BigDecimal("5.90"), "EUR")
8                  : new Money(new BigDecimal("4.90"), "EUR");
9          Money net = product.price();
10         return net.plus(net.times(rate)).plus(shipping);
11     }
12 }
```

El olor es otro que en la sección 1. Aquí no hay un `new` repetido, sino *tres decisiones independientes* (IVA, envío, moneda) que tienen que dar la misma respuesta para el mismo país. Nada lo garantiza: un descuido deja el IVA de España con el envío de Francia, y el resultado es un precio que ningún país aplica. Si luego el envío o el IVA se calculan en otra clase, el riesgo se multiplica. Lo que se necesita no es crear un objeto, sino crear *una familia* de objetos que pertenecen juntos.

3.5 Abstract Factory en Java puro

Abstract Factory proporciona una interfaz para crear familias de objetos relacionados sin especificar sus clases concretas [7]. La idea es sencilla: en lugar de pedir cada pieza por separado, el cliente recibe *una fábrica de una región* y todas las piezas que le pide salen de la misma familia.

Primero, las tres piezas (los *productos abstractos*) y la fábrica abstracta:

TaxPolicy.java

```
package shoplab;

import java.math.BigDecimal;

public interface TaxPolicy {

    BigDecimal rate(Product product);
}
```

ShippingPolicy.java

```
package shoplab;

public interface ShippingPolicy {

    Money cost(Money subtotal);
}
```

CurrencyPolicy.java

```
package shoplab;

public interface CurrencyPolicy {

    String code();
}
```

RegionFactory.java

```
package shoplab;

public interface RegionFactory {

    TaxPolicy taxPolicy();

    ShippingPolicy shippingPolicy();

    CurrencyPolicy currencyPolicy();
}
```

`TaxPolicy.rate` recibe el producto aunque las dos primeras regiones usen una tasa fija: más adelante el IVA podrá variar por categoría y esta firma ya no tendrá que cambiar. Cada interfaz tiene un solo método, así que una lambda basta para implementarla (una lambda es una forma

corta de escribir esa única implementación, `parámetros -> resultado`; el capítulo 6 la repasa). Estas son las dos *fábricas concretas*, una por país:

SpainRegionFactory.java

```java
package shoplab;

import java.math.BigDecimal;

public class SpainRegionFactory implements RegionFactory {

    private static final BigDecimal VAT = new BigDecimal("0.21");
    private static final BigDecimal FREE_FROM = new BigDecimal("50.00");
    private static final Money FEE =
            new Money(new BigDecimal("4.90"), "EUR");

    @Override
    public TaxPolicy taxPolicy() {
        return product -> VAT;
    }

    @Override
    public ShippingPolicy shippingPolicy() {
        return subtotal -> {
            boolean free = subtotal.amount().compareTo(FREE_FROM) >= 0;
            return free ? Money.zero("EUR") : FEE;
        };
    }

    @Override
    public CurrencyPolicy currencyPolicy() {
        return () -> "EUR";
    }
}
```

FranceRegionFactory.java

```java
package shoplab;

import java.math.BigDecimal;

public class FranceRegionFactory implements RegionFactory {

    private static final BigDecimal VAT = new BigDecimal("0.20");
    private static final BigDecimal FREE_FROM = new BigDecimal("60.00");
    private static final Money FEE =
            new Money(new BigDecimal("5.90"), "EUR");

    @Override
    public TaxPolicy taxPolicy() {
        return product -> VAT;
    }

    @Override
    public ShippingPolicy shippingPolicy() {
        return subtotal -> {
```

```
            boolean free = subtotal.amount().compareTo(FREE_FROM) >= 0;
            return free ? Money.zero("EUR") : FEE;
        };
    }

    @Override
    public CurrencyPolicy currencyPolicy() {
        return () -> "EUR";
    }
}
```

España: IVA del 21 %, envío de 4,90 EUR gratis desde 50 EUR. Francia: IVA del 20 %, envío de 5,90 EUR gratis desde 60 EUR. Los importes son inventados para el curso. Ahora el cliente (en el hito, un controlador) solo conoce `RegionFactory`:

Uso desde el cliente

```
static Money total(Product product, RegionFactory region) {
    Money net = product.price();
    Money tax = net.times(region.taxPolicy().rate(product));
    Money shipping = region.shippingPolicy().cost(net);
    return net.plus(tax).plus(shipping);
}
```

El cliente no puede mezclar piezas por descuido, porque no recibe tres políticas sueltas: recibe una región y pide a esa región sus tres piezas. Asocia los papeles:

Papel en el patrón	Clase del ejemplo
Fábrica abstracta	`RegionFactory`
Fábricas concretas	`SpainRegionFactory`, `FranceRegionFactory`
Productos abstractos	`TaxPolicy`, `ShippingPolicy`, `CurrencyPolicy`
Cliente	el método `total` (en el hito, `QuoteController`)

Queda una pregunta: alguien tiene que decidir qué fábrica concreta se usa. En Java puro, un único `switch` sobre el código de la región en un solo lugar (un método estático o una clase selectora), con la misma regla que ya viste: un solo sitio decide. En Spring no hace falta escribirlo.

Añadir un producto a la familia cuesta caro

Abstract Factory facilita añadir una *región* nueva (una clase más) pero dificulta añadir una *pieza* nueva a la familia: si `RegionFactory` gana un método `invoicePolicy()`, hay que implementarlo en todas las fábricas concretas. Es el compromiso del patrón [7]. Si tus familias cambian de piezas más que de variantes, el patrón no es buena idea.

3.6 Y en Spring: las familias como beans

Con Spring, cada fábrica concreta es un bean con el código de la región como nombre, y el `Map<String, T>` de antes sirve para elegir sin ningún `switch`. La única diferencia en las fábricas es la anotación:

Fábricas como beans (solo cambia la cabecera)

```
@Component("ES")
public class SpainRegionFactory implements RegionFactory {
    // igual que en Java puro
}

@Component("FR")
public class FranceRegionFactory implements RegionFactory {
    // igual que en Java puro
}
```

Y un selector pequeño recibe todas las familias por constructor:

RegionSelector.java

```
package shoplab;

import java.util.Map;
import java.util.Optional;
import java.util.Set;
import java.util.TreeSet;
import org.springframework.stereotype.Component;

@Component
public class RegionSelector {

    private final Map<String, RegionFactory> factories;

    public RegionSelector(Map<String, RegionFactory> factories) {
        this.factories = factories;
    }

    public Optional<RegionFactory> find(String region) {
        return Optional.ofNullable(factories.get(region));
    }

    public Set<String> regions() {
        return new TreeSet<>(factories.keySet());
    }
}
```

Esto tiene la consecuencia más útil del capítulo: para añadir Portugal, escribes una clase `PortugalRegionFactory` con `@Component("PT")` y no modificas `RegionSelector` ni el controlador. El mapa se llena solo con los beans que existan. Es el principio abierto/cerrado funcionando; lo comprobarás en el ejercicio 3.5. Una tienda que solo sirviera a un país podría registrar una única familia activa con `@Profile`, que ya conoces, pero ShopLab atiende a varias regiones a la vez, así que las registra todas.

3.7 Factory Method, Abstract Factory y fábricas estáticas

Se parecen y se confunden. La diferencia está en *qué* crean y *para qué*.

	Factory Method	**Abstract Factory**	**Fábrica estática**
Crea	Un producto, de una clase que decide el creador.	Una *familia* de productos relacionados.	Un objeto, con un nombre claro.
Se elige por	Subclase o parámetro.	La fábrica concreta que se pasa al cliente.	Nada: es una llamada estática.
Polimorfismo	Sí.	Sí.	No.
En ShopLab	La fábrica de procesadores de pago.	`RegionFactory`.	`Money.zero`.

Un *método de fábrica estático* (`Money.zero("EUR")`, `List.of`) no es el patrón de Gamma: no hay subclases ni cambia el producto según el creador. Sigue siendo una buena práctica, porque da nombre a la creación, puede devolver una instancia ya existente y no está obligado a crear siempre una nueva [3]. Úsalo para objetos de valor y deja Factory Method para cuando necesites cambiar *qué clase* se crea.

Algunas reglas para decidir:

- **Una sola implementación y sin previsión de otras:** un `new` directo. No crees una fábrica por si acaso; es sobrediseño.
- **Varias implementaciones elegidas por un dato (un enum, un texto) y el mismo código en varios clientes:** Factory Method parametrizado. En Spring, con un `Map<String, T>`.
- **Varios objetos que deben cambiar a la vez y mantenerse coherentes:** Abstract Factory. Si en realidad solo tienes un objeto, es sobrediseño.
- **Crear obliga a leer configuración o a llamar a una biblioteca:** un método `@Bean`, que ya es un método de fábrica.

Abstract Factory suele implementarse con métodos de fábrica: cada método de `RegionFactory` es uno. Para ver más diagramas y ejemplos del mismo patrón, puedes consultar el catálogo de Refactoring.Guru [20], el capítulo de fábricas de *Head First Design Patterns* [5] o los ejemplos en Java de Sarcar [21].

3.8 Primeras pruebas con JUnit

Hasta ahora has comprobado el código arrancando la aplicación y llamando con `curl`, o con una clase `main`. Funciona, pero no se repite solo: cada vez que cambias algo tienes que volver a hacerlo a mano. Una *prueba automática* es un método que ejecuta un trozo de código y comprueba el resultado; con una orden las lanzas todas. Este libro trata de diseño, no de pruebas, así que usaremos solo lo mínimo de JUnit para que puedas verificar el código de este capítulo.

☞ JUnit Jupiter en ShopLab

No hay nada que descargar: el proyecto que generó Initializr ya lo trae. Comprueba tres cosas.

1. En `build.gradle`, las líneas que en el capítulo 1 te dijimos que no tocaras:

build.gradle (extracto)

```
dependencies {
    // (otras dependencias)
    testImplementation 'org.springframework.boot:spring-boot-starter-
    ↪ webmvc-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}

tasks.named('test') {
    useJUnitPlatform()
}
```

2. Qué hacen. `spring-boot-starter-webmvc-test` es el starter de pruebas para aplicaciones Spring MVC y arrastra `spring-boot-starter-test`, que incluye JUnit, AssertJ, Mockito y otras bibliotecas [24]. En este libro usamos solo JUnit Jupiter (y `@SpringBootTest` en la prueba que ya existe); el resto está disponible pero no lo necesitamos. `junit-platform-launcher` lo necesita Gradle para lanzar las pruebas, y `useJUnitPlatform()` le dice a la tarea `test` que las ejecute con JUnit. Boot 4.1.1 gestiona la versión: la de JUnit Jupiter es la 6.0.3, así que no escribes ninguna versión.
3. Las pruebas viven en `src/test/java`, con los mismos paquetes que el código al que prueban. Ahí tienes ya la que generó Initializr, `src/test/java/shoplab/ShopLabApplicationTests.java`. Ejecuta todas desde la carpeta del proyecto:

```
$ ./gradlew test

BUILD SUCCESSFUL in 6s
```

   (El tiempo y el número de tareas cambian.)

La prueba generada es esta:

ShopLabApplicationTests.java

```
package shoplab;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class ShopLabApplicationTests {

    @Test
    void contextLoads() {
    }
}
```

`@SpringBootTest` arranca el contexto completo de Spring, como `bootRun`, y la prueba pasa si arranca sin errores; por eso tarda unos segundos y por eso *rompe* si falta una propiedad `shop.*` o hay dos beans con el mismo nombre. Nosotros escribiremos pruebas más pequeñas, que no necesitan Spring.

3.8.1 Anatomía de una prueba

JUnit Jupiter se compone de pocas piezas:

- Una **clase de prueba**, normalmente con el nombre de la clase probada y el sufijo `Test`. No hace falta que sea pública, pero no puede ser privada.
- Métodos anotados con `@Test` (`org.junit.jupiter.api.Test`), sin parámetros ni valor de retorno. Cada uno es una prueba independiente y su nombre debe decir qué comprueba.
- **Aserciones**, métodos estáticos de `org.junit.jupiter.api.Assertions`. Si una falla, la prueba falla. Las tres que usaremos:
  - `assertEquals(esperado, obtenido)` compara con `equals`. El valor *esperado* va primero; si los pones al revés, el mensaje de error te confundirá.
  - `assertTrue(condicion)` comprueba que algo es cierto.
  - `assertThrows(Excepcion.class, () -> codigo)` comprueba que el código lanza esa excepción.

Una prueba suele tener tres partes: preparar los datos, ejecutar lo que se prueba y comprobar el resultado. Aquí las pruebas de `Money`, una clase del capítulo 1 que no depende de Spring:

src/test/java/shoplab/MoneyTest.java

```
package shoplab;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

import java.math.BigDecimal;
import org.junit.jupiter.api.Test;

class MoneyTest {

    @Test
    void plusAddsAmountsInTheSameCurrency() {
        Money a = new Money(new BigDecimal("10.50"), "EUR");
        Money b = new Money(new BigDecimal("4.50"), "EUR");

        Money sum = a.plus(b);

        assertEquals(new Money(new BigDecimal("15.00"), "EUR"), sum);
    }

    @Test
    void plusRejectsDifferentCurrencies() {
        Money euros = new Money(new BigDecimal("10.00"), "EUR");
        Money pounds = new Money(new BigDecimal("10.00"), "GBP");

        assertThrows(IllegalArgumentException.class,
                () -> euros.plus(pounds));
    }

    @Test
    void timesRoundsToTwoDecimals() {
        Money price = new Money(new BigDecimal("49.90"), "EUR");

```

```
34          Money tax = price.times(new BigDecimal("0.21"));
35
36          assertEquals(new Money(new BigDecimal("10.48"), "EUR"), tax);
37      }
38  }
```

Crea el archivo y ejecuta `./gradlew test`. Es un ejemplo de este apartado y no forma parte del hito: puedes dejarlo en el proyecto, no estorba. Para comprobar que las pruebas *pueden* fallar, cambia el `15.00` esperado por `16.00` y vuelve a ejecutar:

```
$ ./gradlew test

> Task :test FAILED

MoneyTest > plusAddsAmountsInTheSameCurrency() FAILED
    org.opentest4j.AssertionFailedError at MoneyTest.java:18

3 tests completed, 1 failed
```

El formato exacto cambia entre versiones de Gradle. Los detalles (qué se esperaba y qué se obtuvo) están en el informe HTML, `build/reports/tests/test/index.html`. Con `./gradlew test --tests shoplab.MoneyTest` ejecutas una sola clase. Si Gradle decide que nada ha cambiado y no ejecuta las pruebas, fuerza la ejecución con `./gradlew test --rerun`.

`BigDecimal` y `assertEquals`

`BigDecimal.equals` distingue la escala: `new BigDecimal("0.2")` y `new BigDecimal("0.20")` *no* son iguales para `equals`, aunque valgan lo mismo. En las pruebas de `Money` no pasa porque su constructor normaliza siempre a dos decimales. Si pruebas un `BigDecimal` suelto, crea el esperado con la misma escala que el obtenido (o compara con `compareTo`).

3.9 En el proyecto: H2b

Hito 2b: pagos y presupuestos por región

Partes de ShopLab tal como lo dejó el hito 2a. Al terminar el capítulo tiene:

- el puerto de pago `PaymentProcessor`, dos procesadores simulados (`CardProcessor` y `BankTransferProcessor`, beans `card` y `transfer`) y la fábrica `PaymentProcessorFactory` (Factory Method);
- la familia regional `RegionFactory` (Abstract Factory) con `TaxPolicy`, `ShippingPolicy` y `CurrencyPolicy`, implementada por `SpainRegionFactory` y `FranceRegionFactory`, y el selector `RegionSelector`;
- `GET /products/{id}/quote?region=ES`: el precio del producto con IVA, envío y moneda de la región. Solo lee, no cambia ningún estado.

La construyes en el ejercicio 3.4. No hay dependencias nuevas.

Hay cuatro decisiones que conviene entender antes de escribir código.

1. **El puerto de pago es uno solo.** `PaymentProcessor` con `PaymentResult charge(Money`

`amount)` será el único contrato de pago de todo el libro. Los capítulos siguientes lo reutilizan; no habrá un segundo «gateway».

2. **La fábrica de procesadores no hace `new`.** Recibe el `Map<String, PaymentProcessor>` de los beans. El ejercicio guiado empezó con la versión con `new` y este hito la lleva a Spring.
3. **Todavía no hay cobro.** Nadie llama aún a la fábrica de pagos desde un endpoint: el flujo de pedido llega en los capítulos siguientes. Para ver que funciona usarás un `ApplicationRunner` provisional.
4. **El presupuesto es de solo lectura.** `GET /products/{id}/quote` lee el producto del repositorio y usa la fábrica de la región. No modifica nada.

Recordatorio: `@RequestParam`

Un parámetro de la URL (`?region=ES`) se recibe con `@RequestParam` (`org.springframework.web.bind.annotation.RequestParam`).

- Por defecto el parámetro es obligatorio: si falta, Spring responde `400`.
- Con `@RequestParam(defaultValue = "World")` es opcional y toma ese valor si no viene; así lo hace la guía oficial de servicios REST [29].
- El nombre del parámetro es el del argumento de Java. Funciona sin escribirlo porque el plugin de Gradle de Boot compila con la opción `-parameters`.
- Spring convierte el texto al tipo del argumento (`int`, `boolean`, etc.).

En la URL, el `?` separa la ruta de los parámetros. Con `curl`, escribe la URL completa entre comillas dobles para que tu terminal no interprete el `?`.

`ResponseStatusException`

Para responder con un código HTTP concreto desde un controlador, sin escribir todavía un manejador de errores general, se lanza una excepción `ResponseStatusException` (`org.springframework.web.server`) con un `HttpStatus` y un texto, y Spring la traduce a esa respuesta. Por ejemplo, una con `HttpStatus.NOT_FOUND` responde `404`. En el capítulo 4 la sustituiremos por un manejador común.

Moneda del producto y de la región

Las dos regiones de este hito cobran en euros, y `CurrencyPolicy` existe porque una tercera región podría cobrar en otra moneda. ShopLab no convierte monedas: si el producto está en una moneda distinta de la de la región, el presupuesto responde `400` en lugar de mezclar importes.

3.10 Ejercicios

Los ejercicios 3.5 y 3.6 parten del proyecto con el hito 2b hecho, así que están después de él.

Ejercicio 3.1: Una fábrica de procesadores de pago con `new` ★☆☆☆☆

Crea el puerto de pago y la fábrica en su versión de Java puro, la de la sección 2. Estas clases *sí* se quedan en el proyecto: el hito de este capítulo parte de ellas. Por eso van en el paquete `shoplab`, sin anotaciones de Spring todavía.

1. Crea en `src/main/java/shoplab/` los archivos `PaymentType.java`, `PaymentResult.java`, `PaymentProcessor.java` y `CardProcessor.java`, tal como aparecen en la sección 2.
2. Crea `BankTransferProcessor`, que implementa `PaymentProcessor`. Igual que `CardProcessor`, lleva un `AtomicInteger` y su método `charge` devuelve un resultado con `success` verdadero, el identificador `"transfer-" + n` y el mensaje `"waiting for a bank transfer of " + importe + " " + moneda`.
3. Crea `PaymentProcessorFactory` con el método `PaymentProcessor create(PaymentType type)`. Comprueba que `type` no es nulo con `Objects.requireNonNull` y devuelve el procesador con un `switch` *expresión* sobre el enum, sin `default` y con un `new` por caso.
4. Crea en `shoplab.ejercicios` la clase de comprobación de abajo, compila con `./gradlew compileJava` y ejecútala con `java -cp build/classes/java/main shoplab.ejercicios.PaymentFactoryCheck`.
5. Añade un tercer valor, `CASH`, a `PaymentType`, y vuelve a compilar. Observa el error del compilador y después quita `CASH`.

PaymentFactoryCheck.java

```java
package shoplab.ejercicios;

import java.math.BigDecimal;
import shoplab.Money;
import shoplab.PaymentProcessor;
import shoplab.PaymentProcessorFactory;
import shoplab.PaymentType;

public class PaymentFactoryCheck {

    public static void main(String[] args) {
        PaymentProcessorFactory factory = new PaymentProcessorFactory();
        Money amount = new Money(new BigDecimal("25.00"), "EUR");
        for (PaymentType type : PaymentType.values()) {
            PaymentProcessor processor = factory.create(type);
            System.out.println(type + " -> "
                    + processor.getClass().getSimpleName());
            System.out.println(processor.charge(amount));
        }
    }
}
```

Está bien si la salida del paso 4 es exactamente esta y, en el paso 5, el compilador protesta con un mensaje parecido a `the switch expression does not cover all possible input values`:

```
CARD -> CardProcessor
PaymentResult[success=true, transactionId=card-1, message=charged 25.00 EUR by
    ↪ card]
TRANSFER -> BankTransferProcessor
PaymentResult[success=true, transactionId=transfer-1, message=waiting for a
    ↪ bank transfer of 25.00 EUR]
```

Guiado ▪ solución en la pág. 90

Ejercicio 3.2: Qué familia obtiene el cliente ★★☆☆☆

Estas son dos fábricas regionales distintas de las de ShopLab (importes inventados) y un cliente que no usa una fábrica entera, sino piezas sueltas de cada una. No hace falta compilar nada: lee y razona.

GermanyRegionFactory.java

```java
package shoplab;

import java.math.BigDecimal;

public class GermanyRegionFactory implements RegionFactory {

    private static final BigDecimal FREE_FROM = new BigDecimal("100");
    private static final Money FEE =
            new Money(new BigDecimal("5.00"), "EUR");

    @Override
    public TaxPolicy taxPolicy() {
        return product -> new BigDecimal("0.19");
    }

    @Override
    public ShippingPolicy shippingPolicy() {
        return subtotal -> {
            boolean free = subtotal.amount().compareTo(FREE_FROM) >= 0;
            return free ? Money.zero("EUR") : FEE;
        };
    }

    @Override
    public CurrencyPolicy currencyPolicy() {
        return () -> "EUR";
    }
}
```

UkRegionFactory.java

```java
package shoplab;

import java.math.BigDecimal;

public class UkRegionFactory implements RegionFactory {

    private static final BigDecimal FREE_FROM = new BigDecimal("80");
    private static final Money FEE =
            new Money(new BigDecimal("7.00"), "GBP");

    @Override
    public TaxPolicy taxPolicy() {
        return product -> new BigDecimal("0.20");
    }

    @Override
    public ShippingPolicy shippingPolicy() {
        return subtotal -> {
            boolean free = subtotal.amount().compareTo(FREE_FROM) >= 0;
            return free ? Money.zero("GBP") : FEE;
        };
    }

    @Override
    public CurrencyPolicy currencyPolicy() {
        return () -> "GBP";
    }
}
```

PiecemealClient.java

```java
package shoplab;

import java.math.BigDecimal;

public class PiecemealClient {

    static Money total(
            Product product, TaxPolicy tax, ShippingPolicy shipping) {
        Money net = product.price();
        Money vat = net.times(tax.rate(product));
        return net.plus(vat).plus(shipping.cost(net));
    }

    public static void main(String[] args) {
        Product book = new Product("b1", "Libro",
                new Money(new BigDecimal("40.00"), "EUR"),
                new ProductKind.Digital(), 3, "books");
        RegionFactory germany = new GermanyRegionFactory();
        RegionFactory uk = new UkRegionFactory();

        // llamada A
        System.out.println(total(book,
                germany.taxPolicy(), germany.shippingPolicy()));
        // llamada B
        System.out.println(total(book,
                germany.taxPolicy(), uk.shippingPolicy()));
        // llamada C
        System.out.println(total(book,
                uk.taxPolicy(), germany.shippingPolicy()));
    }
}
```

Responde por escrito:

1. ¿Qué piezas de qué país usa cada llamada (A, B y C) y qué importe calcula o qué excepción lanza?
2. ¿Por qué la llamada C es un error peor que la B, aunque no falle?
3. ¿Qué decisión de diseño de `PiecemealClient` permite B y C? Escribe la firma de un método `total` que los haga imposibles.
4. ¿Qué cambiaría si en vez de Alemania y Reino Unido fueran España y Francia, que cobran las dos en euros?

Está bien si identificas la coherencia de familia (IVA, envío y moneda a la vez) como la razón del patrón, y distingues el fallo ruidoso (B) del silencioso (C).

Lectura de código ▪ solución en la pág. 93

Ejercicio 3.3: Que Spring elija el formateador ★★☆☆☆

Aplica a otro tipo la misma técnica que usará el hito: elegir una implementación por nombre con un `Map<String, T>`. Hazlo **en una copia del proyecto** (`cp -r shoplab shoplab-c03`), porque añade beans que no son del hito. Pon las clases en

`shoplab.ejercicios`.

1. Crea la interfaz `DocumentFormatter` con `String format(List<String> items)`.
2. Crea `JsonFormatter`, con `@Component("json")`. Devuelve los elementos entre comillas dobles, separados por comas y entre corchetes: para `a`, `b`, `c`, devuelve `["a","b","c"]`. Usa un bucle `for`.
3. Crea `CsvFormatter` *sin anotaciones*: une los elementos con comas (`a,b,c`). Regístralo con un método `@Bean("csv")` en una clase `FormatterConfig` anotada con `@Configuration`.
4. Crea `DocumentService` (`@Component`). Su constructor recibe un `Map<String, DocumentFormatter>` y, además, el formateador `json` por separado con `@Qualifier("json")`. Ofrece `formatterFor(String name)` (lanza `IllegalArgumentException` con el mensaje `"unknown format: xml, available: [csv, json]"` si no existe), `render(String format, List<String> items)` y `renderDefault(List<String> items)` (usa el formateador `json`).
5. Añade este `ApplicationRunner` y arranca con `./gradlew bootRun`.

FormatterCheck.java

```java
package shoplab.ejercicios;

import java.util.List;
import org.springframework.boot.ApplicationArguments;
import org.springframework.boot.ApplicationRunner;
import org.springframework.stereotype.Component;

@Component
public class FormatterCheck implements ApplicationRunner {

    private final DocumentService service;

    public FormatterCheck(DocumentService service) {
        this.service = service;
    }

    @Override
    public void run(ApplicationArguments args) {
        List<String> items = List.of("a", "b", "c");
        System.out.println("[c03] json: "
                + service.render("json", items));
        System.out.println("[c03] csv: "
                + service.render("csv", items));
        System.out.println("[c03] default: "
                + service.renderDefault(items));
        try {
            service.render("xml", items);
        } catch (IllegalArgumentException e) {
            System.out.println("[c03] xml: " + e.getMessage());
        }
    }
}
```

Está bien si la consola muestra estas cuatro líneas (entre los mensajes de Boot):

c03

```
 json: ["a","b","c"]
[c03] csv: a,b,c
[c03] default: ["a","b","c"]
[c03] xml: unknown format: xml, available: [csv, json]
```

Responde también: ¿qué hay que tocar en `DocumentService` para añadir un formato `xml`?

Pista: *las claves del mapa son los nombres de los beans; si no escribes el nombre en* `@Component`*, la clave será otra.*

Práctica ▪ solución en la pág. 95

Ejercicio 3.4: Hito 2b: pagos y GET /products/{id}/quote ★★★☆☆

Parte del proyecto tal como lo dejó el hito 2a (sin ninguna clase de los ejercicios anteriores de este capítulo, salvo, si lo hiciste, los seis archivos del ejercicio guiado, que puedes reutilizar). Añade los pagos simulados y los presupuestos por región.

1. **Pagos.** Deja en el paquete `shoplab` `PaymentType`, `PaymentResult`, `PaymentProcessor`, `CardProcessor` y `BankTransferProcessor`. Registra los dos procesadores como beans con los nombres `card` y `transfer` (el nombre va en la propia anotación `@Component`).
2. **Fábrica.** `PaymentProcessorFactory` es un `@Component` que recibe por constructor un `Map<String, PaymentProcessor>`. Su `create(PaymentType)` hace un `switch` exhaustivo y devuelve el bean `card` o `transfer` del mapa, sin ningún `new`; si el bean no existe, lanza `IllegalStateException` con un mensaje claro.
3. **Familia regional.** Crea `TaxPolicy` (`BigDecimal rate(Product)`), `ShippingPolicy` (`Money cost(Money subtotal)`), `CurrencyPolicy` (`String code()`) y `RegionFactory`. Implementa `SpainRegionFactory` (bean `ES`: IVA 21 %, envío de 4,90 EUR gratis desde 50,00 EUR, moneda EUR) y `FranceRegionFactory` (bean `FR`: IVA 20 %, envío de 5,90 EUR gratis desde 60,00 EUR, moneda EUR).
4. **Selector.** `RegionSelector` recibe un `Map<String, RegionFactory>`, con `Optional<RegionFactory> find(String)` y `Set<String> regions()` (ordenado).
5. **Presupuesto.** Crea el record `Quote(String productId, String region, Money net, BigDecimal taxRate, Money tax, Money shipping, Money total)` con un método estático `Quote.of(Product, String region, RegionFactory)` que calcula `tax = net` × `taxRate`, `shipping` sobre `net` y `total = net + tax + shipping`. Crea `QuoteController`, con `GET /products/{id}/quote?region=ES`: `404` si el producto no existe, `400` si la región no existe o la moneda del producto no es la de la región. Acepta la región en minúsculas (`es`).
6. **Comprobación provisional.** Crea en `shoplab.ejercicios` el `ApplicationRunner` `PaymentProcessorCheck`, que pide a la fábrica un procesador por cada `PaymentType`, cobra 25,00 EUR e imprime el resultado, y comprueba que dos peticiones de `CARD` devuelven el mismo bean.
7. **curl.** Arranca con `./gradlew bootRun`, crea dos productos y pide presupuestos de ES y FR (con la URL entre comillas).

Está bien si:

- el arranque imprime los dos resultados de pago (`card-1` y `transfer-1`) y `same bean: true`;
- para un producto de 49,90 EUR, `GET /products/p1/quote?region=ES` responde 200 con IVA `10.48`, envío `4.90` y total `65.28`, y con `region=FR`, IVA `9.98`, envío `5.90` y total `65.78`;
- un producto inexistente da `404`; una región desconocida o sin el parámetro `region`, `400`;
- `GET`, `POST /products` y `GET /settings` siguen funcionando.

Pista: *`@Component("ES")` da al bean el nombre `ES`, que es la clave que recibirá el selector.*

Proyecto ▪ solución en la pág. 98

Ejercicio 3.5: Una región nueva sin tocar nada: Portugal ★★★☆☆

Con el hito 2b hecho, demuestra el principio abierto/cerrado. Hazlo **en una copia del proyecto** (`cp -r shoplab shoplab-pt`), porque la región nueva no es del hito. Añade Portugal sin modificar ninguno de los archivos existentes: solo un archivo nuevo, en `shoplab.ejercicios`. Datos (inventados): IVA del 23 % salvo para productos de la categoría `books`, que pagan el 6 %; envío de 5,50 EUR, gratis desde 75,00 EUR; moneda EUR; código de región `PT`.
Está bien si, con la aplicación arrancada y los productos `p1` (49,90 EUR, `hardware`) y `p2` (19,99 EUR, `books`) creados:

- `GET /products/p1/quote?region=PT` da IVA `0.23`, impuesto `11.48`, envío `5.50` y total `66.88`;
- `GET /products/p2/quote?region=PT` da IVA `0.06`, impuesto `1.20`, envío `5.50` y total `26.69`;
- las regiones `ES` y `FR` siguen dando lo mismo que antes;
- no has editado ningún archivo existente.

Responde también: ¿por qué `RegionSelector` encuentra la región nueva sin cambios?

Pista: *`TaxPolicy.rate` recibe el producto entero, no solo el precio.*

Práctica ▪ solución en la pág. 110

Ejercicio 3.6: Pruebas de coherencia de las familias ★★★☆☆

Con el hito 2b hecho, escribe pruebas con JUnit que verifiquen que cada `RegionFactory` devuelve una familia coherente. Crea `src/test/java/shoplab/RegionFactoryTest.java` (sin Spring: construye los objetos con `new`) con estas cinco pruebas:

1. España: IVA `0.21` para productos de dos categorías distintas; envío de 4,90 EUR para un subtotal de 40,00 EUR y gratis para 50,00 EUR; moneda `EUR`.
2. Francia: igual, con IVA `0.20`, envío de 5,90 EUR bajo 60,00 EUR y gratis desde 60,00 EUR; moneda `EUR`.
3. Coherencia: para cada una de las dos fábricas (recórrelas con un bucle), el envío, tanto cuando se paga como cuando es gratis, está en la moneda de su `CurrencyPolicy`.
4. `RegionSelector`, construido con un `Map.of` de las claves `"ES"` y `"FR"`, devuelve España para `"ES"` y un `Optional` vacío para `"XX"`.

5. `Quote.of` para un producto de 40,00 EUR en España da IVA `8.40`, envío `4.90` y total `53.30`.

Está bien si `./gradlew test` pasa y, al cambiar `0.21` por `0.19` en `SpainRegionFactory`, la primera prueba falla (compruébalo y deshaz el cambio).

Pista: *construye los `BigDecimal` esperados con la misma escala que los obtenidos: `new BigDecimal("0.21")`, no `0.21` ni `new BigDecimal("0.2100")`.*

Test ▪ solución en la pág. 111

3.11 Resumen

Resumen

- El olor de Factory Method es el mismo `new` o `switch` por tipo repetido en varias clases. El patrón concentra la creación en un solo lugar: los clientes piden un `PaymentProcessor` y no saben cuál es.
- Estructura clásica: un creador con un método de fábrica que las subclases sobrescriben. La variante parametrizada (`create(PaymentType)`) es la más habitual y, con un `switch` exhaustivo sin `default`, el compilador avisa cuando falta un caso.
- Un método `@Bean` es un método de fábrica. Con varios beans del mismo tipo, `@Qualifier` elige uno, `Map<String, T>` entrega todos con su nombre como clave y `List<T>` entrega todos sin nombre.
- Abstract Factory crea familias de objetos coherentes entre sí (`TaxPolicy`, `ShippingPolicy`, `CurrencyPolicy`) y facilita añadir variantes (una región) pero no piezas nuevas. En Spring, cada fábrica es un bean y un selector con `Map<String, T>` las descubre sin tocarlo.
- Factory Method crea un producto; Abstract Factory, una familia; una fábrica estática (`Money.zero`) es útil pero no es el patrón. Con una sola implementación, un `new` basta.
- JUnit Jupiter ya viene en el proyecto. Una prueba es un método `@Test` con aserciones (`assertEquals`, `assertTrue`, `assertThrows`) y se lanza con `./gradlew test`.
- ShopLab (H2b) tiene `PaymentProcessor`, dos procesadores, la fábrica de pagos con `Map`, las familias regionales y `GET /products/{id}/quote?region=`.

Soluciones de los ejercicios

Solución del ejercicio 3.1 enunciado en la pág. 82

PaymentType.java

```
1 package shoplab;
2
3 public enum PaymentType {
4     CARD,
5     TRANSFER
6 }
```

PaymentResult.java

```java
package shoplab;

public record PaymentResult(
        boolean success,
        String transactionId,
        String message) {
}
```

PaymentProcessor.java

```java
package shoplab;

public interface PaymentProcessor {

    PaymentResult charge(Money amount);
}
```

CardProcessor.java

```java
package shoplab;

import java.util.concurrent.atomic.AtomicInteger;

public class CardProcessor implements PaymentProcessor {

    private final AtomicInteger sequence = new AtomicInteger();

    @Override
    public PaymentResult charge(Money amount) {
        String transactionId = "card-" + sequence.incrementAndGet();
        String message = "charged " + amount.amount() + " "
                + amount.currency() + " by card";
        return new PaymentResult(true, transactionId, message);
    }
}
```

BankTransferProcessor.java

```java
package shoplab;

import java.util.concurrent.atomic.AtomicInteger;

public class BankTransferProcessor implements PaymentProcessor {

    private final AtomicInteger sequence = new AtomicInteger();

    @Override
    public PaymentResult charge(Money amount) {
        String transactionId = "transfer-" + sequence.incrementAndGet();
        String message = "waiting for a bank transfer of "
                + amount.amount() + " " + amount.currency();
        return new PaymentResult(true, transactionId, message);
    }
}
```

PaymentProcessorFactory.java

```java
package shoplab;

import java.util.Objects;

public class PaymentProcessorFactory {

    public PaymentProcessor create(PaymentType type) {
        Objects.requireNonNull(type, "type");
        return switch (type) {
            case CARD -> new CardProcessor();
            case TRANSFER -> new BankTransferProcessor();
        };
    }
}
```

La clase de comprobación no cambia; la repetimos entera:

PaymentFactoryCheck.java

```java
package shoplab.ejercicios;

import java.math.BigDecimal;
import shoplab.Money;
import shoplab.PaymentProcessor;
import shoplab.PaymentProcessorFactory;
import shoplab.PaymentType;

public class PaymentFactoryCheck {

    public static void main(String[] args) {
        PaymentProcessorFactory factory = new PaymentProcessorFactory();
        Money amount = new Money(new BigDecimal("25.00"), "EUR");
        for (PaymentType type : PaymentType.values()) {
            PaymentProcessor processor = factory.create(type);
            System.out.println(type + " -> "
                    + processor.getClass().getSimpleName());
            System.out.println(processor.charge(amount));
        }
    }
}
```

Con ella se obtiene la salida esperada:

```
CARD -> CardProcessor
PaymentResult[success=true, transactionId=card-1, message=charged 25.00 EUR by
    ↪ card]
TRANSFER -> BankTransferProcessor
PaymentResult[success=true, transactionId=transfer-1, message=waiting for a
    ↪ bank transfer of 25.00 EUR]
```

Cómo funciona: el único lugar con `new CardProcessor()` y `new BankTransferProcessor()` es la fábrica. El `switch` es una expresión sobre un `enum` y no tiene `default`, así que el compilador comprueba que cubre todos los valores. Al añadir `CASH`, `javac` rechaza la fábrica con un error del tipo `the switch expression does not cover all possible input values`: la fábrica no compila hasta que decidas qué procesador corresponde a `CASH`.

Errores típicos: añadir un `default` que devuelva la transferencia (el compilador deja de avisar y un `CASH` nuevo se cobraría mal en silencio); poner `@Component` en las clases ahora (no es un fallo, pero los beans aparecen en el paso siguiente, el hito, y entonces la fábrica dejará de usar `new`); olvidar `Objects.requireNonNull`, con lo que un `null` lanza una `NullPointerException` sin mensaje.

Solución del ejercicio 3.2

enunciado en la pág. 84

1. Qué hace cada llamada.

- **A** usa IVA y envío de Alemania (una sola familia). Calcula $40{,}00 + 7{,}60 + 5{,}00 = 52{,}60$ EUR ($40 \times 0{,}19 = 7{,}60$ y el envío es de 5,00 porque $40 < 100$).
- **B** usa el IVA de Alemania y el envío de Reino Unido. El IVA (7,60 EUR) se suma bien al neto, pero el envío es de 7,00 GBP y `Money.plus` lanza `IllegalArgumentException`:

`different currencies: EUR and GBP`.

- **C** usa el IVA de Reino Unido y el envío de Alemania. Calcula 40,00+8,00+5,00 = 53,00 EUR y no lanza nada.

2. Por qué C es peor. B falla con un error claro nada más ejecutarse: se corrige en minutos. C devuelve un número plausible que ningún país aplica (el IVA británico en una venta alemana) y no avisa a nadie: es un error de negocio silencioso que llega a los clientes. Los fallos que no se ven son los caros.

3. La decisión de diseño. `total` recibe las piezas por separado, y eso deja al llamador elegirlas de familias distintas. Si recibe la fábrica entera, todas las piezas salen de la misma región y mezclarlas ya no es posible:

`PiecemealClient.java (corregido)`

```
package shoplab;

import java.math.BigDecimal;

public class PiecemealClient {

    static Money total(Product product, RegionFactory region) {
        Money net = product.price();
        Money tax = net.times(region.taxPolicy().rate(product));
        Money shipping = region.shippingPolicy().cost(net);
        return net.plus(tax).plus(shipping);
    }

    public static void main(String[] args) {
        Product book = new Product("b1", "Libro",
                new Money(new BigDecimal("40.00"), "EUR"),
                new ProductKind.Digital(), 3, "books");
        System.out.println(total(book, new GermanyRegionFactory()));
    }
}
```

Imprime `Money[amount=52.60, currency=EUR]`, el resultado de la llamada A. (Un producto en euros con `UkRegionFactory` fallaría por la moneda, como B: el patrón asegura la coherencia de la familia, no que el producto pertenezca a esa región.)

4. España y Francia. Las dos cobran en euros, así que el caso B no lanzaría ninguna excepción: mezclar IVA de una con envío de la otra daría, como C, un importe plausible pero incorrecto, sin aviso. El fallo ruidoso de B solo aparece cuando las monedas difieren; la coherencia hay que garantizarla por diseño y no esperar a que Java la detecte.

Solución del ejercicio 3.3

enunciado en la pág. 86

DocumentFormatter.java

```java
package shoplab.ejercicios;

import java.util.List;

public interface DocumentFormatter {

    String format(List<String> items);
}
```

JsonFormatter.java

```java
package shoplab.ejercicios;

import java.util.List;
import org.springframework.stereotype.Component;

@Component("json")
public class JsonFormatter implements DocumentFormatter {

    @Override
    public String format(List<String> items) {
        StringBuilder out = new StringBuilder("[");
        for (int i = 0; i < items.size(); i++) {
            if (i > 0) {
                out.append(',');
            }
            out.append('"').append(items.get(i)).append('"');
        }
        return out.append(']').toString();
    }
}
```

CsvFormatter.java

```java
package shoplab.ejercicios;

import java.util.List;

public class CsvFormatter implements DocumentFormatter {

    @Override
    public String format(List<String> items) {
        return String.join(",", items);
    }
}
```

FormatterConfig.java

```java
package shoplab.ejercicios;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class FormatterConfig {

    @Bean("csv")
    public DocumentFormatter csvFormatter() {
        return new CsvFormatter();
    }
}
```

DocumentService.java

```java
package shoplab.ejercicios;

import java.util.List;
import java.util.Map;
import java.util.TreeSet;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.stereotype.Component;

@Component
public class DocumentService {

    private final Map<String, DocumentFormatter> formatters;
    private final DocumentFormatter defaultFormatter;

    public DocumentService(
            Map<String, DocumentFormatter> formatters,
            @Qualifier("json") DocumentFormatter defaultFormatter) {
        this.formatters = formatters;
        this.defaultFormatter = defaultFormatter;
    }

    public DocumentFormatter formatterFor(String name) {
        DocumentFormatter formatter = formatters.get(name);
        if (formatter == null) {
            throw new IllegalArgumentException("unknown format: " + name
                    + ", available: "
                    + new TreeSet<>(formatters.keySet()));
        }
        return formatter;
    }

    public String render(String format, List<String> items) {
        return formatterFor(format).format(items);
    }

    public String renderDefault(List<String> items) {
        return defaultFormatter.format(items);
    }
}
```

La clase de comprobación, entera:

FormatterCheck.java

```java
package shoplab.ejercicios;

import java.util.List;
import org.springframework.boot.ApplicationArguments;
import org.springframework.boot.ApplicationRunner;
import org.springframework.stereotype.Component;

@Component
public class FormatterCheck implements ApplicationRunner {

    private final DocumentService service;

    public FormatterCheck(DocumentService service) {
        this.service = service;
    }

    @Override
    public void run(ApplicationArguments args) {
        List<String> items = List.of("a", "b", "c");
        System.out.println("[c03] json: "
                + service.render("json", items));
        System.out.println("[c03] csv: "
                + service.render("csv", items));
        System.out.println("[c03] default: "
                + service.renderDefault(items));
        try {
            service.render("xml", items);
        } catch (IllegalArgumentException e) {
            System.out.println("[c03] xml: " + e.getMessage());
        }
    }
}
```

Cómo funciona: Spring registra dos beans de tipo `DocumentFormatter`: `json`, por el nombre de `@Component`, y `csv`, por el nombre de `@Bean`. El `Map<String, DocumentFormatter>` contiene ambos con esos nombres como clave. Como hay dos candidatos, el formateador por defecto necesita `@Qualifier("json")`: sin él, el constructor no tendría un único candidato y la aplicación no arrancaría. Para añadir `xml` solo se crea un `XmlFormatter` con `@Component("xml")`; `DocumentService` no cambia, porque el mapa se llena solo.
Errores típicos: escribir `@Component` sin nombre en `JsonFormatter` (la clave sería `jsonFormatter` y `render("json", items)` fallaría); poner `@Component("csv")` *y* el `@Bean("csv")` (dos beans con el mismo nombre: la aplicación no arranca); olvidar `@Configuration` en `FormatterConfig` (el `@Bean` se ignora).

Solución del ejercicio 3.4

enunciado en la pág. 88

Primero los pagos. Si hiciste el ejercicio guiado, solo cambian las cabeceras de `CardProcessor` y `BankTransferProcessor` (la anotación) y la fábrica completa; se reproducen todos los archivos.

PaymentType.java

```java
package shoplab;

public enum PaymentType {
    CARD,
    TRANSFER
}
```

PaymentResult.java

```java
package shoplab;

public record PaymentResult(
        boolean success,
        String transactionId,
        String message) {
}
```

PaymentProcessor.java

```java
package shoplab;

public interface PaymentProcessor {

    PaymentResult charge(Money amount);
}
```

CardProcessor.java

```java
package shoplab;

import java.util.concurrent.atomic.AtomicInteger;
import org.springframework.stereotype.Component;

@Component("card")
public class CardProcessor implements PaymentProcessor {

    private final AtomicInteger sequence = new AtomicInteger();

    @Override
    public PaymentResult charge(Money amount) {
        String transactionId = "card-" + sequence.incrementAndGet();
        String message = "charged " + amount.amount() + " "
                + amount.currency() + " by card";
        return new PaymentResult(true, transactionId, message);
    }
}
```

BankTransferProcessor.java

```java
package shoplab;

import java.util.concurrent.atomic.AtomicInteger;
import org.springframework.stereotype.Component;

@Component("transfer")
public class BankTransferProcessor implements PaymentProcessor {

    private final AtomicInteger sequence = new AtomicInteger();

    @Override
    public PaymentResult charge(Money amount) {
        String transactionId = "transfer-" + sequence.incrementAndGet();
        String message = "waiting for a bank transfer of "
                + amount.amount() + " " + amount.currency();
        return new PaymentResult(true, transactionId, message);
    }
}
```

PaymentProcessorFactory.java

```java
package shoplab;

import java.util.Map;
import java.util.Objects;
import org.springframework.stereotype.Component;

@Component
public class PaymentProcessorFactory {

    private final Map<String, PaymentProcessor> processors;

    public PaymentProcessorFactory(
            Map<String, PaymentProcessor> processors) {
        this.processors = processors;
    }

    public PaymentProcessor create(PaymentType type) {
        Objects.requireNonNull(type, "type");
        return switch (type) {
            case CARD -> processor("card");
            case TRANSFER -> processor("transfer");
        };
    }

    private PaymentProcessor processor(String name) {
        PaymentProcessor processor = processors.get(name);
        if (processor == null) {
            throw new IllegalStateException(
                    "no PaymentProcessor bean named '" + name + "'");
        }
        return processor;
    }
}
```

Ahora la familia regional: las tres políticas, la fábrica abstracta y las dos concretas.

TaxPolicy.java

```java
package shoplab;

import java.math.BigDecimal;

public interface TaxPolicy {

    BigDecimal rate(Product product);
}
```

ShippingPolicy.java

```java
package shoplab;

public interface ShippingPolicy {

    Money cost(Money subtotal);
}
```

CurrencyPolicy.java

```java
package shoplab;

public interface CurrencyPolicy {

    String code();
}
```

RegionFactory.java

```java
package shoplab;

public interface RegionFactory {

    TaxPolicy taxPolicy();

    ShippingPolicy shippingPolicy();

    CurrencyPolicy currencyPolicy();
}
```

SpainRegionFactory.java

```java
package shoplab;

import java.math.BigDecimal;
import org.springframework.stereotype.Component;

@Component("ES")
public class SpainRegionFactory implements RegionFactory {

    private static final BigDecimal VAT = new BigDecimal("0.21");
    private static final BigDecimal FREE_FROM = new BigDecimal("50.00");
    private static final Money FEE =
            new Money(new BigDecimal("4.90"), "EUR");

    @Override
    public TaxPolicy taxPolicy() {
        return product -> VAT;
    }

    @Override
    public ShippingPolicy shippingPolicy() {
        return subtotal -> {
            boolean free = subtotal.amount().compareTo(FREE_FROM) >= 0;
            return free ? Money.zero("EUR") : FEE;
        };
    }

    @Override
    public CurrencyPolicy currencyPolicy() {
        return () -> "EUR";
    }
}
```

FranceRegionFactory.java

```java
package shoplab;

import java.math.BigDecimal;
import org.springframework.stereotype.Component;

@Component("FR")
public class FranceRegionFactory implements RegionFactory {

    private static final BigDecimal VAT = new BigDecimal("0.20");
    private static final BigDecimal FREE_FROM = new BigDecimal("60.00");
    private static final Money FEE =
            new Money(new BigDecimal("5.90"), "EUR");

    @Override
    public TaxPolicy taxPolicy() {
        return product -> VAT;
    }

    @Override
    public ShippingPolicy shippingPolicy() {
        return subtotal -> {
            boolean free = subtotal.amount().compareTo(FREE_FROM) >= 0;
            return free ? Money.zero("EUR") : FEE;
        };
    }

    @Override
    public CurrencyPolicy currencyPolicy() {
        return () -> "EUR";
    }
}
```

RegionSelector.java

```java
package shoplab;

import java.util.Map;
import java.util.Optional;
import java.util.Set;
import java.util.TreeSet;
import org.springframework.stereotype.Component;

@Component
public class RegionSelector {

    private final Map<String, RegionFactory> factories;

    public RegionSelector(Map<String, RegionFactory> factories) {
        this.factories = factories;
    }

    public Optional<RegionFactory> find(String region) {
        return Optional.ofNullable(factories.get(region));
    }

    public Set<String> regions() {
        return new TreeSet<>(factories.keySet());
    }
}
```

Y el presupuesto y su endpoint:

Quote.java

```
package shoplab;

import java.math.BigDecimal;

public record Quote(
        String productId,
        String region,
        Money net,
        BigDecimal taxRate,
        Money tax,
        Money shipping,
        Money total) {

    public static Quote of(
            Product product, String region, RegionFactory family) {
        Money net = product.price();
        BigDecimal rate = family.taxPolicy().rate(product);
        Money tax = net.times(rate);
        Money shipping = family.shippingPolicy().cost(net);
        Money total = net.plus(tax).plus(shipping);
        return new Quote(
                product.id(), region, net, rate, tax, shipping, total);
    }
}
```

QuoteController.java

```java
package shoplab;

import java.util.Locale;
import java.util.Optional;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.server.ResponseStatusException;

@RestController
@RequestMapping("/products")
public class QuoteController {

    private final ProductRepository repository;
    private final RegionSelector regions;

    public QuoteController(ProductRepository repository,
            RegionSelector regions) {
        this.repository = repository;
        this.regions = regions;
    }

    @GetMapping("/{id}/quote")
    public Quote quote(@PathVariable String id,
            @RequestParam String region) {
        Optional<Product> foundProduct = repository.findById(id);
        if (foundProduct.isEmpty()) {
            throw new ResponseStatusException(
                    HttpStatus.NOT_FOUND, "product not found: " + id);
        }
        Product product = foundProduct.get();

        String code = region.toUpperCase(Locale.ROOT);
        Optional<RegionFactory> foundRegion = regions.find(code);
        if (foundRegion.isEmpty()) {
            throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
                    "unknown region: " + code
                            + ", available: " + regions.regions());
        }
        RegionFactory family = foundRegion.get();

        String currency = family.currencyPolicy().code();
        String priced = product.price().currency();
        if (!priced.equals(currency)) {
            throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
                    "product is priced in " + priced + " but region "
                            + code + " uses " + currency);
        }
        return Quote.of(product, code, family);
    }
}
```

Por último, la comprobación provisional de los pagos. Bórrala cuando hayas visto la salida: imprime en cada arranque.

PaymentProcessorCheck.java

```java
package shoplab.ejercicios;

import java.math.BigDecimal;
import org.springframework.boot.ApplicationArguments;
import org.springframework.boot.ApplicationRunner;
import org.springframework.stereotype.Component;
import shoplab.Money;
import shoplab.PaymentProcessorFactory;
import shoplab.PaymentType;

@Component
public class PaymentProcessorCheck implements ApplicationRunner {

    private final PaymentProcessorFactory factory;

    public PaymentProcessorCheck(PaymentProcessorFactory factory) {
        this.factory = factory;
    }

    @Override
    public void run(ApplicationArguments args) {
        Money amount = new Money(new BigDecimal("25.00"), "EUR");
        for (PaymentType type : PaymentType.values()) {
            System.out.println("[c03] " + type + " -> "
                    + factory.create(type).charge(amount));
        }
        boolean same = factory.create(PaymentType.CARD)
                == factory.create(PaymentType.CARD);
        System.out.println("[c03] same bean: " + same);
    }
}
```

Al arrancar con `./gradlew bootRun`, entre los mensajes de Boot:

c03

```
 CARD -> PaymentResult[success=true, transactionId=card-1, message=charged
    ↪ 25.00 EUR by card]
[c03] TRANSFER -> PaymentResult[success=true, transactionId=transfer-1,
    ↪ message=waiting for a bank transfer of 25.00 EUR]
[c03] same bean: true
```

Con el servidor arrancado, crea dos productos y pide los presupuestos (las URL van entre comillas por el `?`):

```
$ curl -i -H "Content-Type: application/json" \
    -d '{"id":"p1","name":"Teclado",
         "price":{"amount":49.9,"currency":"EUR"},
         "kind":{"type":"physical","weightGrams":800},
         "stock":12,"category":"hardware"}' \
    http://localhost:8080/products
HTTP/1.1 201

$ curl -i -H "Content-Type: application/json" \
    -d '{"id":"p2","name":"Curso",
         "price":{"amount":"19.99","currency":"EUR"},
         "kind":{"type":"digital"},"stock":0,"category":"books"}' \
    http://localhost:8080/products
HTTP/1.1 201

$ curl -i -H "Content-Type: application/json" \
    "http://localhost:8080/products/p1/quote?region=ES"
HTTP/1.1 200
Content-Type: application/json
{"productId":"p1","region":"ES",
 "net":{"amount":49.90,"currency":"EUR"},"taxRate":0.21,
 "tax":{"amount":10.48,"currency":"EUR"},
 "shipping":{"amount":4.90,"currency":"EUR"},
 "total":{"amount":65.28,"currency":"EUR"}}

$ curl -i -H "Content-Type: application/json" \
    "http://localhost:8080/products/p1/quote?region=fr"
HTTP/1.1 200
Content-Type: application/json
{"productId":"p1","region":"FR",
 "net":{"amount":49.90,"currency":"EUR"},"taxRate":0.20,
 "tax":{"amount":9.98,"currency":"EUR"},
 "shipping":{"amount":5.90,"currency":"EUR"},
 "total":{"amount":65.78,"currency":"EUR"}}

$ curl -i -H "Content-Type: application/json" \
    "http://localhost:8080/products/zz/quote?region=ES"
HTTP/1.1 404

$ curl -i -H "Content-Type: application/json" \
    "http://localhost:8080/products/p1/quote?region=XX"
HTTP/1.1 400

$ curl -i -H "Content-Type: application/json" \
    "http://localhost:8080/products/p1/quote"
HTTP/1.1 400
```

Cómo funciona: al arrancar, Spring crea los cuatro beans de pago y regiones (`card`, `transfer`, `ES`, `FR`) y construye `PaymentProcessorFactory` y `RegionSelector` pasándoles los `Map` con esos nombres como claves. `QuoteController` no conoce ninguna región concreta: pide al selector la familia, y de ella obtiene las tres políticas. El presupuesto de España para `p1` se calcula así: $49{,}90 \times 0{,}21 = 10{,}479$, que `Money` redondea a 10,48; el envío es 4,90 porque $49{,}90 < 50{,}00$, y el total es $49{,}90 + 10{,}48 + 4{,}90 = 65{,}28$.

Este es el estado del proyecto del que parte el capítulo siguiente: el del hito 2a más estas quince clases de `shoplab` (las cinco de pago, la fábrica de pagos, las cuatro de la familia regional, las dos regiones, `RegionSelector`, `Quote` y `QuoteController`). `PaymentProcessorCheck` es provisional y no forma parte del estado.

Errores típicos: olvidar el nombre en `@Component("card")` (la clave del mapa sería `cardProcessor` y la fábrica lanzaría `IllegalStateException`); usar `@Component("es")` en minúscula cuando el controlador busca `ES`; escribir la URL sin comillas en una terminal que interprete el `?`; devolver `null` en vez de lanzar una excepción cuando falta un bean; olvidar `-i`, con lo que no ves el código de estado.

Solución del ejercicio 3.5

enunciado en la pág. 89

Solo hay que añadir un archivo; ninguno de los existentes cambia.

PortugalRegionFactory.java

```java
package shoplab.ejercicios;

import java.math.BigDecimal;
import org.springframework.stereotype.Component;
import shoplab.CurrencyPolicy;
import shoplab.Money;
import shoplab.RegionFactory;
import shoplab.ShippingPolicy;
import shoplab.TaxPolicy;

@Component("PT")
public class PortugalRegionFactory implements RegionFactory {

    private static final BigDecimal VAT = new BigDecimal("0.23");
    private static final BigDecimal BOOKS_VAT = new BigDecimal("0.06");
    private static final BigDecimal FREE_FROM = new BigDecimal("75.00");
    private static final Money FEE =
            new Money(new BigDecimal("5.50"), "EUR");

    @Override
    public TaxPolicy taxPolicy() {
        return product -> product.category().equals("books")
                ? BOOKS_VAT : VAT;
    }

    @Override
    public ShippingPolicy shippingPolicy() {
        return subtotal -> {
            boolean free = subtotal.amount().compareTo(FREE_FROM) >= 0;
            return free ? Money.zero("EUR") : FEE;
        };
    }

    @Override
    public CurrencyPolicy currencyPolicy() {
        return () -> "EUR";
    }
}
```

Comprobación (con `p1` y `p2` creados como en el hito):

```
$ curl -i -H "Content-Type: application/json" \
    "http://localhost:8080/products/p1/quote?region=PT"
HTTP/1.1 200
{"productId":"p1","region":"PT",
 "net":{"amount":49.90,"currency":"EUR"},"taxRate":0.23,
 "tax":{"amount":11.48,"currency":"EUR"},
 "shipping":{"amount":5.50,"currency":"EUR"},
 "total":{"amount":66.88,"currency":"EUR"}}

$ curl -i -H "Content-Type: application/json" \
    "http://localhost:8080/products/p2/quote?region=PT"
HTTP/1.1 200
{"productId":"p2","region":"PT",
 "net":{"amount":19.99,"currency":"EUR"},"taxRate":0.06,
 "tax":{"amount":1.20,"currency":"EUR"},
 "shipping":{"amount":5.50,"currency":"EUR"},
 "total":{"amount":26.69,"currency":"EUR"}}
```

Por qué funciona: `@Component("PT")` registra un bean de tipo `RegionFactory` con el nombre `PT`. `RegionSelector` recibe un `Map<String, RegionFactory>` con todos los beans de ese tipo; el nuevo entra en el mapa sin que nadie lo mencione. `QuoteController` tampoco cambia, porque solo conoce la interfaz. Es el principio abierto/cerrado: extiendes añadiendo código, no editando el existente. La diferencia con España y Francia está en el `TaxPolicy`: usa `product.category()`, que es lo que justifica que `rate` reciba el producto entero.
Errores típicos: poner la clase fuera del paquete `shoplab` o de sus subpaquetes (Spring no la escanea y `region=PT` da `400`); escribir `@Component("pt")` (el controlador pasa la región a mayúsculas); editar `RegionSelector` para «registrar» Portugal, que es justo lo que el diseño evita; comparar la categoría con `==` en vez de `equals`.

Solución del ejercicio 3.6

enunciado en la pág. 89

Primero, el código que se prueba (el del hito, completo). Las tres políticas y `RegionFactory`:

TaxPolicy.java

```java
package shoplab;

import java.math.BigDecimal;

public interface TaxPolicy {

    BigDecimal rate(Product product);
}
```

ShippingPolicy.java

```java
package shoplab;

public interface ShippingPolicy {

    Money cost(Money subtotal);
}
```

CurrencyPolicy.java

```java
package shoplab;

public interface CurrencyPolicy {

    String code();
}
```

RegionFactory.java

```java
package shoplab;

public interface RegionFactory {

    TaxPolicy taxPolicy();

    ShippingPolicy shippingPolicy();

    CurrencyPolicy currencyPolicy();
}
```

SpainRegionFactory.java

```java
package shoplab;

import java.math.BigDecimal;
import org.springframework.stereotype.Component;

@Component("ES")
public class SpainRegionFactory implements RegionFactory {

    private static final BigDecimal VAT = new BigDecimal("0.21");
    private static final BigDecimal FREE_FROM = new BigDecimal("50.00");
    private static final Money FEE =
            new Money(new BigDecimal("4.90"), "EUR");

    @Override
    public TaxPolicy taxPolicy() {
        return product -> VAT;
    }

    @Override
    public ShippingPolicy shippingPolicy() {
        return subtotal -> {
            boolean free = subtotal.amount().compareTo(FREE_FROM) >= 0;
            return free ? Money.zero("EUR") : FEE;
        };
    }

    @Override
    public CurrencyPolicy currencyPolicy() {
        return () -> "EUR";
    }
}
```

FranceRegionFactory.java

```java
package shoplab;

import java.math.BigDecimal;
import org.springframework.stereotype.Component;

@Component("FR")
public class FranceRegionFactory implements RegionFactory {

    private static final BigDecimal VAT = new BigDecimal("0.20");
    private static final BigDecimal FREE_FROM = new BigDecimal("60.00");
    private static final Money FEE =
            new Money(new BigDecimal("5.90"), "EUR");

    @Override
    public TaxPolicy taxPolicy() {
        return product -> VAT;
    }

    @Override
    public ShippingPolicy shippingPolicy() {
        return subtotal -> {
            boolean free = subtotal.amount().compareTo(FREE_FROM) >= 0;
            return free ? Money.zero("EUR") : FEE;
        };
    }

    @Override
    public CurrencyPolicy currencyPolicy() {
        return () -> "EUR";
    }
}
```

RegionSelector.java

```java
package shoplab;

import java.util.Map;
import java.util.Optional;
import java.util.Set;
import java.util.TreeSet;
import org.springframework.stereotype.Component;

@Component
public class RegionSelector {

    private final Map<String, RegionFactory> factories;

    public RegionSelector(Map<String, RegionFactory> factories) {
        this.factories = factories;
    }

    public Optional<RegionFactory> find(String region) {
        return Optional.ofNullable(factories.get(region));
    }

    public Set<String> regions() {
        return new TreeSet<>(factories.keySet());
    }
}
```

Quote.java

```java
package shoplab;

import java.math.BigDecimal;

public record Quote(
        String productId,
        String region,
        Money net,
        BigDecimal taxRate,
        Money tax,
        Money shipping,
        Money total) {

    public static Quote of(
            Product product, String region, RegionFactory family) {
        Money net = product.price();
        BigDecimal rate = family.taxPolicy().rate(product);
        Money tax = net.times(rate);
        Money shipping = family.shippingPolicy().cost(net);
        Money total = net.plus(tax).plus(shipping);
        return new Quote(
                product.id(), region, net, rate, tax, shipping, total);
    }
}
```

Y las pruebas:

src/test/java/shoplab/RegionFactoryTest.java

```java
package shoplab;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertTrue;

import java.math.BigDecimal;
import java.util.List;
import java.util.Map;
import org.junit.jupiter.api.Test;

class RegionFactoryTest {

    private static Money eur(String amount) {
        return new Money(new BigDecimal(amount), "EUR");
    }

    private static Product product(String category) {
        return new Product("p1", "Item", eur("40.00"),
                new ProductKind.Digital(), 5, category);
    }

    @Test
    void spainFamilyUsesSpanishVatShippingAndEuro() {
        RegionFactory spain = new SpainRegionFactory();

        assertEquals(new BigDecimal("0.21"),
                spain.taxPolicy().rate(product("hardware")));
        assertEquals(new BigDecimal("0.21"),
                spain.taxPolicy().rate(product("books")));
        assertEquals(eur("4.90"),
                spain.shippingPolicy().cost(eur("40.00")));
        assertEquals(Money.zero("EUR"),
                spain.shippingPolicy().cost(eur("50.00")));
        assertEquals("EUR", spain.currencyPolicy().code());
    }

    @Test
    void franceFamilyUsesFrenchVatShippingAndEuro() {
        RegionFactory france = new FranceRegionFactory();

        assertEquals(new BigDecimal("0.20"),
                france.taxPolicy().rate(product("hardware")));
        assertEquals(new BigDecimal("0.20"),
                france.taxPolicy().rate(product("books")));
        assertEquals(eur("5.90"),
                france.shippingPolicy().cost(eur("59.99")));
        assertEquals(Money.zero("EUR"),
                france.shippingPolicy().cost(eur("60.00")));
        assertEquals("EUR", france.currencyPolicy().code());
    }

    @Test
    void everyFamilyShipsInItsOwnCurrency() {
        List<RegionFactory> families = List.of(
                new SpainRegionFactory(), new FranceRegionFactory());

        for (RegionFactory family : families) {
            String currency = family.currencyPolicy().code();
            Money paid = family.shippingPolicy().cost(eur("10.00"));
            Money free = family.shippingPolicy().cost(eur("500.00"));

```

Con `./gradlew test` las cinco pruebas pasan. Por qué están así:

- No usan Spring: crean las fábricas con `new`, así que se ejecutan en milisegundos. Es posible porque las fábricas no dependen del contenedor.
- Los `BigDecimal` esperados tienen la misma escala que los obtenidos (dos decimales los de `Money`; `0.21` y `0.20` los de las tasas). Los `Money` se comparan con `assertEquals` porque un record implementa `equals` por componentes.
- La tercera prueba no fija valores: comprueba una *propiedad* de la familia (el envío está en la moneda de su política). Si mañana alguien añade una región con el envío en otra moneda, la prueba la detecta.
- Al cambiar `0.21` por `0.19`, `spainFamilyUsesSpanishVatShippingAndEuro` falla con `expected: <0.21> but was: <0.19>` en el informe HTML.

Errores típicos: poner el valor obtenido como primer argumento de `assertEquals` (el mensaje de error invierte «esperado» y «obtenido»); comparar `new BigDecimal("0.2")` con `0.20`, que `equals` considera distintos; usar `@SpringBootTest` para algo que se prueba con `new` (la prueba tarda segundos en vez de milisegundos); poner la clase de prueba en `src/main/java`, donde JUnit no la encuentra ni la dependencia de prueba está disponible.

CAPÍTULO 4

Builder y Prototype, con Lombok

Objetivos del capítulo

- Reconocer el olor del constructor telescópico y de los objetos a medio construir, y resolverlo con un Builder.
- Instalar Lombok (Gradle y NeoVim) y usar `@Getter`, `@Value`, `@With`, `@Builder`, `@Builder.Default` y `@Singular`.
- Copiar un objeto con Prototype (constructor de copia) distinguiendo copia superficial y profunda, y explicar por qué se evita `Cloneable`.
- Recordar cómo traducir excepciones a respuestas HTTP con `@RestControllerAdvice`.
- Añadir a ShopLab los pedidos: `POST /orders`, `GET /orders/{id}` y `POST /orders/{id}/repeat`.

Un pedido tiene muchos datos: líneas, dirección de envío, región, forma de pago. Unos son obligatorios y otros no, y no puede existir a medias. Con constructores largos el código se vuelve ilegible; con *setters* aparecen pedidos a medio construir. El Builder construye el objeto paso a paso y lo entrega ya válido. Prototype resuelve la otra cara: crear un objeto nuevo a partir de uno existente («repetir el pedido») sin compartir por error su estado mutable. Para no escribir el Builder a mano cada vez, usaremos Lombok.

Java sin compilar

Desde este capítulo el código Java del libro está escrito con cuidado contra la documentación, pero no lo ha compilado ni ejecutado su autor. Si algo no compila, `./gradlew compileJava` te dirá dónde; contrasta con los archivos de las soluciones.

4.1 El olor: constructor telescópico y objetos a medio construir

Una dirección de envío tiene seis datos: tres obligatorios y tres opcionales. El primer intento suele ser un constructor con todos los parámetros:

ShippingAddress.java (antes)

```
public class ShippingAddress {
    public ShippingAddress(String recipient, String street, String city,
                           String postalCode, String country,
                           List<String> deliveryNotes) { /* ... */ }
}

// Al usarlo, ¿qué es cada cosa?
new ShippingAddress("Ana", "Calle Sol 1", "Madrid", null, null, null);
```

Tres problemas: no se lee qué significa cada argumento, los `null` esconden lo opcional y cambiar

el orden de dos `String` compila sin avisar. Añadir constructores más cortos (el «constructor telescópico») multiplica las sobrecargas. La otra salida, un constructor vacío y *setters*, deja el objeto inválido entre una llamada y otra, y obliga a que sea mutable. Es el olor que Kerievsky llama refactorizar *hacia* un patrón[11], y Bloch recomienda el Builder para clases con muchos parámetros[3].

4.2 Builder en Java puro

El **Builder** es un objeto auxiliar con un método por dato (que devuelve `this`, para encadenar) y un `build()` final que valida y crea el objeto. Así se lee cada dato por su nombre y el objeto nace completo[7, 5]. Esta es la ilustración con `Order` (más adelante lo hará Lombok por nosotros):

OrderBuilder.java (ilustración)

```
package shoplab;

import java.util.ArrayList;
import java.util.List;

public class OrderBuilder {
    private String id;
    private final List<OrderLine> lines = new ArrayList<>();
    private ShippingAddress shippingAddress;
    private String region;
    private PaymentType paymentType;

    public OrderBuilder id(String id) { this.id = id; return this; }

    public OrderBuilder line(OrderLine line) {
        this.lines.add(line);
        return this;
    }

    public OrderBuilder shippingAddress(ShippingAddress address) {
        this.shippingAddress = address;
        return this;
    }

    public OrderBuilder region(String region) {
        this.region = region;
        return this;
    }

    public OrderBuilder paymentType(PaymentType paymentType) {
        this.paymentType = paymentType;
        return this;
    }

    public Order build() {
        // la validación vive aquí, en un solo sitio
        return new Order(id, List.copyOf(lines), shippingAddress,
                         region, paymentType);
    }
}
```

Fíjate en dos detalles. `build()` copia la lista con `List.copyOf`: si el builder se reutiliza o se sigue modificando, el pedido ya creado no cambia. Y toda la validación está en un punto: si faltan datos, `build()` lanza `IllegalArgumentException` y nunca existe un pedido inválido. El coste es el código repetitivo: tantas líneas de *boilerplate* como campos. Lombok lo genera.

4.3 Instalación de Lombok

☞ Lombok (dependencia y editor)

Lombok es un *procesador de anotaciones*: durante la compilación genera código (*getters*, constructores, builders) a partir de anotaciones. No añade nada en tiempo de ejecución, por eso va solo en tiempo de compilación[19].

1. Gradle. En el bloque `dependencies` de tu `build.gradle` añade dos líneas. No lleva número de versión: la gestiona Spring Boot (Lombok 1.18.46 con Boot 4.1.1[23], compatible con Java 21):

`build.gradle (fragmento)`

```
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    compileOnly 'org.projectlombok:lombok'
    annotationProcessor 'org.projectlombok:lombok'
    // ... el resto sin cambios
}
```

`compileOnly` pone Lombok en el classpath de compilación; `annotationProcessor` lo activa como procesador[8]. Hacen falta las dos. Si tus pruebas usan Lombok, añade también `testCompileOnly` y `testAnnotationProcessor`. Recarga el proyecto y comprueba con `./gradlew compileJava`.

2. NeoVim con jdtls. Gradle ya compila bien, pero el servidor de lenguaje (jdtls) no conoce los métodos que Lombok genera y marcará errores falsos (`getName()` «no existe»). Se arregla pasándole el `lombok.jar` como *agente de Java*. Esta configuración consta en la documentación de `nvim-lspconfig`[15], pero **no la he probado** con un jdtls real:

Descargar lombok.jar y activarlo en jdtls

```
mkdir -p ~/.local/share/java
curl -L -o ~/.local/share/java/lombok.jar https://projectlombok.org/downloads/
    ↪ lombok.jar
export JDTLS_JVM_ARGS="-javaagent:$HOME/.local/share/java/lombok.jar"
```

Añade el `export` a tu `/.bashrc` (o equivalente) y reinicia NeoVim. Si siguen los errores falsos, revisa la ruta del `.jar` y reinicia el servidor (`:LspRestart`). Lo que manda es Gradle: si `./gradlew compileJava` pasa, tu código es correcto aunque el editor se queje.

`@Value` de Lombok no es `@Value` de Spring

Hay dos anotaciones con el mismo nombre. La de Spring (`org.springframework.beans.factory.annotation.Value`, capítulo 2) *inyecta una propiedad* en un campo o parámetro. La de Lombok (`lombok.Value`) convierte una *clase* en inmutable. Mira siempre el `import` antes de fiarte de lo que hace.

4.4 Las anotaciones de Lombok que usaremos

Estas son las que usa el capítulo, con lo esencial de la documentación oficial[18]:

- `@Getter` y `@ToString`: generan `getX()` para cada campo y un `toString()` legible. `@EqualsAndHashCode` genera `equals` y `hashCode` por los campos.
- `@Value`: la variante inmutable de `@Data`. Hace la clase `final` y los campos `private final`, sin *setters*, con `getX()`, `equals`, `hashCode`, `toString` y un constructor con todos los campos. Si escribes un constructor tú, Lombok no genera el suyo.
- `@With`: en un campo genera `withX(valor)`, que devuelve una copia del objeto con ese campo cambiado. Necesita un constructor con todos los campos en el orden de declaración.
- `@Builder`: genera una clase `XBuilder`, el método estático `X.builder()` y `build()`. Con `@Builder(toBuilder = true)` añade `toBuilder()`, que arranca un builder con los valores de la instancia. En una clase con varios campos funciona como si el constructor con todos los campos fuera el anotado; si escribes tú ese constructor (para validar), Lombok lo usa.
- `@Builder.Default`: da un valor por defecto a un campo que el builder no recibe. Sin ella, el inicializador del campo se ignora al construir con el builder.
- `@Singular`: sobre un `List`, `Set` o `Map` del builder genera un método que añade *un* elemento (`deliveryNote("portero")`), otro que añade una colección y `clear...`. El resultado es una lista inmutable.

Ejemplo mínimo que reúne varias:

Point.java (ejemplo)

```java
import lombok.Builder;
import lombok.Value;
import lombok.With;

@Value
@Builder(toBuilder = true)
public class Point {
    @With int x;
    int y;
    @Builder.Default String label = "origin";
}

// Point p = Point.builder().x(1).y(2).build();   label = "origin"
// Point q = p.withX(5);                          x = 5, y = 2
// Point r = p.toBuilder().label("A").build();    x = 1, y = 2
```

4.5 Builder con Lombok, validación y límites frente a un `record`

Para `ShippingAddress` queremos: tres campos obligatorios, `country` con valor por defecto `"ES"`, notas de entrega opcionales (`@Singular`) y validación. Recibirá además JSON en `POST /orders`. Jackson no sabe usar un builder por sí solo; Lombok lo resuelve con `@Jacksonized`, que configura el builder generado para que lo use la deserialización. Está documentada para `@Builder`[18] y desde Lombok 1.18.44 admite Jackson 2 y Jackson 3 (el de Boot 4), según el registro de cambios; no lo he podido probar.

ShippingAddress.java

```java
package shoplab;

import java.util.List;
import lombok.Builder;
import lombok.Singular;
import lombok.Value;
import lombok.extern.jackson.Jacksonized;

@Value
@Builder
@Jacksonized
public class ShippingAddress {
    String recipient;
    String street;
    String city;
    String postalCode;
    @Builder.Default String country = "ES";
    @Singular("deliveryNote") List<String> deliveryNotes;

    // Constructor explícito, con todos los campos y en este orden:
    // el builder lo usa, y aquí vive la validación.
    public ShippingAddress(String recipient, String street, String city,
                           String postalCode, String country,
                           List<String> deliveryNotes) {
        requireText(recipient, "recipient");
        requireText(street, "street");
        requireText(city, "city");
        requireText(country, "country");
        this.recipient = recipient;
        this.street = street;
        this.city = city;
        this.postalCode = postalCode;
        this.country = country;
        this.deliveryNotes = List.copyOf(deliveryNotes);
    }

    private static void requireText(String value, String field) {
        if (value == null || value.isBlank()) {
            throw new IllegalArgumentException(field + " is required");
        }
    }
}
```

Con ello se escribe `ShippingAddress.builder().recipient("Ana").street("Calle Sol 1").city("Madrid").deliveryNote("portero").build()` y `country` vale `"ES"` sin pasarlo. Si falta `city`, `build()` lanza `IllegalArgumentException`.

Límites frente a un `record`. Un `record` (capítulo 1) ya es inmutable, tiene `equals`, `hashCode` y `toString`, y no necesita Lombok ni editor configurado. Si tienes pocos campos y todos son obligatorios, un `record` gana. Lombok compensa cuando hay muchos campos, valores por defecto, colecciones opcionales con `@Singular` o necesitas `toBuilder()`. El precio es una dependencia más, código generado que no ves y un editor que debe conocerla. En ShopLab, `Money`, `Product` y `OrderLine` siguen siendo `record`; `ShippingAddress` y `Order` usan Lombok.

4.6 El olor: copiar un objeto a mano

«Repetir pedido» es crear un pedido nuevo igual a otro. Copiar a mano campo a campo funciona hasta que añades un campo y olvidas copiarlo, o hasta que dos objetos comparten sin querer una lista:

Copia a mano (el problema)

```
Wishlist copy = new Wishlist(original.getOwner());
copy.setItems(original.getItems());   // la MISMA lista en los dos
copy.getItems().add(new WishItem("monitor", 1));
// ¡también aparece en original!
```

4.7 Prototype: constructor de copia, copia profunda y `Cloneable`

El **Prototype** crea objetos nuevos *copiando* una instancia existente en lugar de construirla desde cero[7]. En Java lo más claro es un **constructor de copia**: un constructor que recibe otro objeto del mismo tipo.

La diferencia clave es entre copia superficial y profunda:

- **Superficial**: copia las referencias. Original y copia comparten los mismos objetos internos (la misma lista, los mismos `WishItem`).
- **Profunda**: copia también los objetos internos que son mutables. Cambiar la copia no afecta al original.

Wishlist.java (ejemplo)

```
import java.util.ArrayList;
import java.util.List;

public class Wishlist {
    private final String owner;
    private final List<WishItem> items = new ArrayList<>();

    public Wishlist(String owner) { this.owner = owner; }

    // Constructor de copia: copia profunda
    public Wishlist(Wishlist other) {
        this.owner = other.owner;              // String: inmutable
        for (WishItem item : other.items) {
            this.items.add(new WishItem(item)); // cada WishItem, copiado
        }
    }
    // ... getters y add(WishItem)
}
```

Si solo hubieras hecho `this.items.addAll(other.items)`, la lista sería nueva pero los `WishItem` los compartirían los dos objetos: cambiar la cantidad en la copia la cambiaría en el original. Eso es copia superficial de los elementos.

¿Y `Cloneable`? Java tiene `Object.clone()` y la interfaz `Cloneable`, pero Bloch recomienda evitarlos[3]: `clone()` crea el objeto sin llamar a un constructor, es superficial por defecto, obliga

a *casts* y a tratar excepciones comprobadas, y se rompe con campos `final`. Un constructor de copia o un método `copy()` lo hacen todo mejor.

Qué pasa con `Order`. `Order` será `@Value`: sus campos son `final` y sus tipos (`Money`, `OrderLine`, `ShippingAddress`) también son inmutables. Si nada se puede mutar, compartir referencias es seguro: la «copia profunda» se reduce a un `List.copyOf` y copiar el pedido es casi trivial (`withId` o `toBuilder`). El bug típico de Prototype no puede darse en `Order`; ocurre con clases mutables como `Wishlist` o el `DraftOrder` de los ejercicios.

4.8 Y en Spring: ámbito `prototype` y traducción de excepciones

Ámbito `prototype`. En el capítulo 2 viste que Spring crea un bean singleton por defecto y que `@Scope("prototype")` entrega una instancia *nueva* cada vez que se pide[27]. Es la versión del contenedor de la idea de Prototype, pero con un matiz: Spring *construye* la instancia desde la definición del bean, no la copia de otra. Para datos de dominio como un pedido no se usa: se crean con un builder o con un constructor de copia, como en este capítulo.

Recordatorio: de excepciones a HTTP. Si un controlador lanza una excepción, Spring responde 500 salvo que le digas otra cosa. Una clase con `@RestControllerAdvice` agrupa métodos `@ExceptionHandler(X.class)` que convierten cada excepción en una respuesta con su código. Todo el traductor está en un sitio y los controladores no se llenan de `try/catch`:

ApiExceptionHandler.java

```
package shoplab;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice
public class ApiExceptionHandler {

    @ExceptionHandler(ProductNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ApiError notFound(ProductNotFoundException e) {
        return new ApiError(e.getMessage());
    }

    @ExceptionHandler(IllegalArgumentException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ApiError badRequest(IllegalArgumentException e) {
        return new ApiError(e.getMessage());
    }
}
```

Los hitos posteriores añadirán aquí sus excepciones (conflicto 409 en H7, validación 422 en H8b).

4.9 En el proyecto: Hito 3

Hito 3: pedidos con Builder y Prototype

Partes del estado tras el hito 2b. Este hito añade Lombok al `build.gradle` y estas piezas en `shoplab`: `ShippingAddress` y `Order` (Lombok, inmutables, con Builder); `OrderLine` (`record`) con `subtotal()`; `Order.total()`, la suma de subtotales, que es el importe que cobrará el checkout del hito 4; `CreateOrderRequest` y `OrderItemRequest` (el cuerpo de `POST /orders`); `OrderService`; `OrderRepository` con `InMemoryOrderRepository`; `ProductNotFoundException`, `ApiError` y `ApiExceptionHandler`; y `OrderController` con `POST /orders` (201), `GET /orders/{id}` y `POST /orders/{id}/repeat` (Prototype). En el proyecto queda solo la versión Lombok; el Builder a mano es ilustración. Está en el ejercicio 4.6.

4.10 Ejercicios

Ejercicio 4.1: Añade Lombok y anota una clase ★★☆☆☆

Trabaja en una copia del proyecto. Sigue los pasos:

1. Añade las dos líneas de Lombok al `build.gradle` y, si usas NeoVim, descarga el `lombok.jar` y exporta `JDTLS_JVM_ARGS` como en la instalación.
2. Crea `shoplab.ejercicios.Customer` con los campos `String name` y `String email`, un constructor con ambos y las anotaciones `@Getter` y `@ToString`, sin escribir ningún *getter*.
3. Crea `LombokCheck` con un `main` que construya un `Customer`, imprima `getName()` y el propio objeto.
4. Ejecuta `./gradlew compileJava`.

Está bien si compila, `main` imprime `Ana` y `Customer(name=Ana, email=ana@mail.com)`, y NeoVim no marca `getName()` como error.

Pista: *Si Gradle compila pero NeoVim marca errores, el fallo está en el `-javaagent`: revisa la ruta.*

Guiado ▪ solución en la pág. 129

Ejercicio 4.2: Un Builder a mano ★★☆☆☆

En una clase propia `shoplab.ejercicios.ManualShippingAddress` (no es la del proyecto) escribe, sin Lombok, una dirección con seis campos: `recipient`, `street` y `city` obligatorios; `postalCode`, `country` (por defecto `"ES"`) y `List<String> deliveryNotes` opcionales. Debe tener un `Builder` interno estático con métodos encadenados, un método `deliveryNote(String)` que añade una nota, y un `build()` que valide y copie la lista. Está bien si `build()` sin `city` lanza `IllegalArgumentException`, si `country` vale `ES` sin indicarlo y si modificar el builder después de `build()` no cambia el objeto ya creado.

Pista: *El constructor de la dirección puede ser privado y recibir el builder.*

Práctica ▪ solución en la pág. 131

Ejercicio 4.3: Un Builder con Lombok y su comparación ★★☆☆☆

Escribe `shoplab.ejercicios.CustomerProfile` con `@Value` y `@Builder`: `name` y `email` obligatorios, `phone` opcional, `plan` con valor por defecto `"free"` (`@Builder.Default`) y `tags` con `@Singular`. Valida `name` y `email` en un constructor explícito. Después compara en una frase cuántas líneas ahorras frente a `c04-builder-manual` y qué riesgo nuevo introduces. Está bien si `CustomerProfile.builder().name("Ana").email("a@b.c").tag("vip").tag("beta").build()` da `plan = free` y dos etiquetas, y si sin `email` lanza `IllegalArgumentException`.

Práctica ▪ solución en la pág. 133

Ejercicio 4.4: Copiar una lista de deseos ★★★☆☆

Con clases propias, escribe `WishItem` (`name`, `quantity`, mutable) y `Wishlist` (`id`, `owner` y una lista de `WishItem`). Implementa `Wishlist copy()`: devuelve una lista *nueva*, con identificador nuevo (`UUID`), mismo propietario y sus propios `WishItem` copiados. Está bien si, tras copiar, añadir un elemento a la copia o cambiar la cantidad de uno de sus elementos no altera el original, y si los identificadores son distintos.

Pista: *Un constructor de copia privado en cada clase y* `copy()` *que lo invoca.*

Práctica ▪ solución en la pág. 134

Ejercicio 4.5: El pedido repetido que cambia el original ★★★☆☆

Esta clase propia es mutable (no es el `Order` del proyecto). Al repetir un borrador y añadir un producto a la copia, el original también cambia. Localiza la copia superficial y arréglala.

DraftOrder.java

```java
package shoplab.ejercicios;

import java.util.ArrayList;
import java.util.List;

public class DraftOrder {
    private final String id;
    private final List<String> productIds;

    public DraftOrder(String id) {
        this.id = id;
        this.productIds = new ArrayList<>();
    }

    private DraftOrder(DraftOrder other, String newId) {
        this.id = newId;
        this.productIds = other.productIds;
    }

    public DraftOrder repeat(String newId) {
        return new DraftOrder(this, newId);
    }

    public void add(String productId) { productIds.add(productId); }
    public List<String> getProductIds() { return productIds; }
    public String getId() { return id; }
}
```

Está bien si, con `a = new DraftOrder("1")`, `a.add("p1")`, `b = a.repeat("2")` y `b.add("p2")`, `a` sigue con `[p1]` y `b` tiene `[p1, p2]`.

Depuración ▪ solución en la pág. 136

Ejercicio 4.6: Hito 3: pedidos con Builder y Prototype ★★★☆☆

Parte del hito 2b. Implementa el hito 3 completo.

1. Añade Lombok al `build.gradle` (compileOnly y annotationProcessor).
2. `ShippingAddress` como en el capítulo y `OrderLine` (record con `productId`, `quantity > 0`, `Money unitPrice` y `subtotal()`).
3. `Order` (`@Value`, `@Builder(toBuilder = true)`) con `id`, `lines` (`@Singular("line")`), `shippingAddress`, `region`, `paymentType`, `@With` en `id` y `Money total()`.
4. `CreateOrderRequest`, `OrderItemRequest`, `ProductNotFoundException`, `ApiError` y `ApiExceptionHandler`.
5. `OrderRepository`, `InMemoryOrderRepository`, `OrderService` (`create`, `find`, `repeat`) y `OrderController`.

Está bien si, con un producto `p1` de 49,90 EUR creado antes, los `curl` de la solución devuelven 201 al crear (con `total` 99.80 para dos unidades), 201 al repetir con un `id` distinto y el mismo total, 404 para un `productId` inexistente y 400 sin `city`.

Pista: *Un `productId` desconocido lanza `ProductNotFoundException` en `OrderService`; el 400*

sale de `build()`.

Proyecto ▪ solución en la pág. 137

4.11 Resumen

Resumen

- El Builder construye objetos con muchos datos por nombre y valida en `build()`; evita el constructor telescópico y los objetos a medio construir.
- Lombok es un procesador de anotaciones: `compileOnly` y `annotationProcessor` en Gradle, y `-javaagent` con el `lombok.jar` en jdtls para el editor (configuración sin probar).
- `@Value` (de Lombok) hace la clase inmutable; no es el `@Value` de Spring.
- `@Builder`, `@Builder.Default`, `@Singular`, `@With` y `@Jacksonized` cubren valores por defecto, colecciones, copias con un cambio y JSON.
- Un `record` basta con pocos campos obligatorios; Lombok compensa con muchos campos y opcionales.
- Prototype copia con un constructor de copia; copia profunda si el estado es mutable. Se evita `Cloneable`. En un objeto inmutable como `Order` la copia es casi trivial.
- `@RestControllerAdvice` con `@ExceptionHandler` traduce excepciones a HTTP en un solo sitio.
- ShopLab (H3) crea, consulta y repite pedidos con `POST /orders`, `GET /orders/{id}` y `POST /orders/{id}/repeat`.

Soluciones de los ejercicios

Solución del ejercicio 4.1 enunciado en la pág. 126

build.gradle (fragmento)

```
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    compileOnly 'org.projectlombok:lombok'
    annotationProcessor 'org.projectlombok:lombok'
    testImplementation 'org.springframework.boot:spring-boot-starter-webmvc-
    ↪ test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}
```

Customer.java

```java
package shoplab.ejercicios;

import lombok.Getter;
import lombok.ToString;

@Getter
@ToString
public class Customer {
    private final String name;
    private final String email;

    public Customer(String name, String email) {
        this.name = name;
        this.email = email;
    }
}
```

LombokCheck.java

```java
package shoplab.ejercicios;

public class LombokCheck {
    public static void main(String[] args) {
        Customer customer = new Customer("Ana", "ana@mail.com");
        System.out.println(customer.getName());
        System.out.println(customer);
    }
}
```

Ejecución: `./gradlew compileJava` y, para la salida, `java -cp build/classes/java/main shoplab.ejercicios.LombokCheck`. Lombok genera `getName()`, `getEmail()` y `toString()` al compilar. **Errores típicos:** poner solo `compileOnly` (sin `annotationProcessor` no se genera nada y `getName()` no existe); olvidar recargar Gradle; si solo falla el editor, la ruta del `-javaagent` o no haber reiniciado NeoVim.

Solución del ejercicio 4.2 enunciado en la pág. 126

ManualShippingAddress.java

```java
package shoplab.ejercicios;

import java.util.ArrayList;
import java.util.List;

public class ManualShippingAddress {
    private final String recipient;
    private final String street;
    private final String city;
    private final String postalCode;
    private final String country;
    private final List<String> deliveryNotes;

    private ManualShippingAddress(Builder b) {
        this.recipient = b.recipient;
        this.street = b.street;
        this.city = b.city;
        this.postalCode = b.postalCode;
        this.country = b.country;
        this.deliveryNotes = List.copyOf(b.deliveryNotes);
    }

    public static Builder builder() { return new Builder(); }

    public String getCountry() { return country; }
    public List<String> getDeliveryNotes() { return deliveryNotes; }
    public String getCity() { return city; }

    public static class Builder {
        private String recipient;
        private String street;
        private String city;
        private String postalCode;
        private String country = "ES";
        private final List<String> deliveryNotes = new ArrayList<>();

        public Builder recipient(String v) { recipient = v; return this; }
        public Builder street(String v) { street = v; return this; }
        public Builder city(String v) { city = v; return this; }
        public Builder postalCode(String v) { postalCode = v; return this; }
        public Builder country(String v) { country = v; return this; }

        public Builder deliveryNote(String note) {
            deliveryNotes.add(note);
            return this;
        }

        public ManualShippingAddress build() {
            require(recipient, "recipient");
            require(street, "street");
            require(city, "city");
            require(country, "country");
            return new ManualShippingAddress(this);
        }

        private static void require(String value, String field) {
            if (value == null || value.isBlank()) {
                throw new IllegalArgumentException(field + " is required");
            }
        }
    }
```

Por qué funciona: los métodos devuelven `this` (encadenado), `country` arranca en `"ES"` y el constructor privado hace `List.copyOf`, así que añadir notas al builder después no toca el objeto ya creado; `build()` valida los obligatorios. **Errores típicos:** guardar la misma lista del builder en el objeto (se comparte); poner la validación en cada *setter* en lugar de en `build()` (el orden de las llamadas importaría).

Solución del ejercicio 4.3

enunciado en la pág. 127

CustomerProfile.java

```java
package shoplab.ejercicios;

import java.util.List;
import lombok.Builder;
import lombok.Singular;
import lombok.Value;

@Value
@Builder
public class CustomerProfile {
    String name;
    String email;
    String phone;
    @Builder.Default String plan = "free";
    @Singular List<String> tags;

    public CustomerProfile(String name, String email, String phone,
                           String plan, List<String> tags) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("name is required");
        }
        if (email == null || email.isBlank()) {
            throw new IllegalArgumentException("email is required");
        }
        this.name = name;
        this.email = email;
        this.phone = phone;
        this.plan = plan;
        this.tags = List.copyOf(tags);
    }
}
```

ProfileCheck.java

```java
package shoplab.ejercicios;

public class ProfileCheck {
    public static void main(String[] args) {
        CustomerProfile p = CustomerProfile.builder()
                .name("Ana").email("a@b.c").tag("vip").tag("beta").build();
        System.out.println(p.getPlan() + " " + p.getTags());
        try {
            CustomerProfile.builder().name("Ana").build();
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Salida esperada: `free [vip, beta]` y `email is required`. **Comparación:** el Builder manual de seis campos ocupa unas 60 líneas; este, unas 25, sin el `builder()` ni los métodos encadenados. El riesgo nuevo: el código del builder no se ve, depende de que el procesador de anotaciones y el editor estén bien configurados, y el constructor explícito debe mantener el *mismo orden* que los campos (si añades un campo y olvidas el constructor, falla la compilación). **Errores típicos:** olvidar `@Builder.Default` (`plan` sería `null`); `@Singular` en un campo no `List/Set/Map`.

Solución del ejercicio 4.4

enunciado en la pág. 127

WishItem.java

```java
package shoplab.ejercicios;

public class WishItem {
    private final String name;
    private int quantity;

    public WishItem(String name, int quantity) {
        this.name = name;
        this.quantity = quantity;
    }

    private WishItem(WishItem other) {
        this.name = other.name;
        this.quantity = other.quantity;
    }

    public WishItem copy() { return new WishItem(this); }

    public String getName() { return name; }
    public int getQuantity() { return quantity; }
    public void setQuantity(int quantity) { this.quantity = quantity; }
}
```

Wishlist.java

```java
package shoplab.ejercicios;

import java.util.ArrayList;
import java.util.List;
import java.util.UUID;

public class Wishlist {
    private final String id;
    private final String owner;
    private final List<WishItem> items = new ArrayList<>();

    public Wishlist(String id, String owner) {
        this.id = id;
        this.owner = owner;
    }

    private Wishlist(Wishlist other) {
        this.id = UUID.randomUUID().toString();
        this.owner = other.owner;
        for (WishItem item : other.items) {
            this.items.add(item.copy());
        }
    }

    public Wishlist copy() { return new Wishlist(this); }

    public void add(WishItem item) { items.add(item); }
    public List<WishItem> getItems() { return items; }
    public String getId() { return id; }
    public String getOwner() { return owner; }
}
```

WishlistCheck.java

```java
package shoplab.ejercicios;

public class WishlistCheck {
    public static void main(String[] args) {
        Wishlist a = new Wishlist("w1", "Ana");
        a.add(new WishItem("monitor", 1));
        Wishlist b = a.copy();
        b.getItems().get(0).setQuantity(5);
        b.add(new WishItem("raton", 2));
        System.out.println(a.getItems().size() + " "
                + a.getItems().get(0).getQuantity());   // 1 1
        System.out.println(b.getItems().size() + " "
                + b.getItems().get(0).getQuantity());   // 2 5
        System.out.println(a.getId().equals(b.getId())); // false
    }
}
```

Por qué funciona: la copia crea una lista nueva *y* un `WishItem` nuevo por elemento. **Errores típicos:** `addAll(other.items)` (lista nueva, elementos compartidos); copiar también

el `id` y dejar dos listas con el mismo identificador.

Solución del ejercicio 4.5 enunciado en la pág. 127

El fallo está en el constructor de copia: `this.productIds = other.productIds;` asigna la *misma* lista a las dos instancias (copia superficial). Arreglo:

DraftOrder.java (constructor corregido)

```java
package shoplab.ejercicios;

import java.util.ArrayList;
import java.util.List;

public class DraftOrder {
    private final String id;
    private final List<String> productIds;

    public DraftOrder(String id) {
        this.id = id;
        this.productIds = new ArrayList<>();
    }

    private DraftOrder(DraftOrder other, String newId) {
        this.id = newId;
        this.productIds = new ArrayList<>(other.productIds);
    }

    public DraftOrder repeat(String newId) {
        return new DraftOrder(this, newId);
    }

    public void add(String productId) { productIds.add(productId); }
    public List<String> getProductIds() { return productIds; }
    public String getId() { return id; }
}
```

DraftOrderCheck.java

```java
package shoplab.ejercicios;

public class DraftOrderCheck {
    public static void main(String[] args) {
        DraftOrder a = new DraftOrder("1");
        a.add("p1");
        DraftOrder b = a.repeat("2");
        b.add("p2");
        System.out.println(a.getProductIds()); // [p1]
        System.out.println(b.getProductIds()); // [p1, p2]
    }
}
```

Con `new ArrayList<>(...)` la copia tiene su propia lista. Basta una copia de la lista porque los `String` son inmutables; si los elementos fueran mutables habría que copiar también cada uno (como en `c04-prototype-copy`). **Errores típicos:**

`Collections.unmodifiableList(other.productIds)`, que es una *vista* de la misma lista (el original sigue pudiendo cambiarla y la copia lo vería); copiar la lista en `repeat` pero no en otros constructores.

Solución del ejercicio 4.6 enunciado en la pág. 128

1. build.gradle: añade `compileOnly 'org.projectlombok:lombok'` y `annotationProcessor 'org.projectlombok:lombok'` en `dependencies`.
2. Dominio. `ShippingAddress` es el del capítulo:

ShippingAddress.java

```
package shoplab;

import java.util.List;
import lombok.Builder;
import lombok.Singular;
import lombok.Value;
import lombok.extern.jackson.Jacksonized;

@Value
@Builder
@Jacksonized
public class ShippingAddress {
    String recipient;
    String street;
    String city;
    String postalCode;
    @Builder.Default String country = "ES";
    @Singular("deliveryNote") List<String> deliveryNotes;

    public ShippingAddress(String recipient, String street, String city,
                           String postalCode, String country,
                           List<String> deliveryNotes) {
        requireText(recipient, "recipient");
        requireText(street, "street");
        requireText(city, "city");
        requireText(country, "country");
        this.recipient = recipient;
        this.street = street;
        this.city = city;
        this.postalCode = postalCode;
        this.country = country;
        this.deliveryNotes = List.copyOf(deliveryNotes);
    }

    private static void requireText(String value, String field) {
        if (value == null || value.isBlank()) {
            throw new IllegalArgumentException(field + " is required");
        }
    }
}
```

OrderLine.java

```java
package shoplab;

public record OrderLine(String productId, int quantity, Money unitPrice) {

    public OrderLine {
        if (productId == null || productId.isBlank()) {
            throw new IllegalArgumentException("productId is required");
        }
        if (quantity <= 0) {
            throw new IllegalArgumentException("quantity must be > 0");
        }
        if (unitPrice == null) {
            throw new IllegalArgumentException("unitPrice is required");
        }
    }

    public Money subtotal() {
        return unitPrice.times(quantity);
    }
}
```

Order.java

```java
package shoplab;

import com.fasterxml.jackson.annotation.JsonProperty;
import java.util.List;
import lombok.Builder;
import lombok.Singular;
import lombok.Value;
import lombok.With;

@Value
@Builder(toBuilder = true)
public class Order {
    @With String id;
    @Singular("line") List<OrderLine> lines;
    ShippingAddress shippingAddress;
    String region;
    PaymentType paymentType;

    public Order(String id, List<OrderLine> lines,
                 ShippingAddress shippingAddress, String region,
                 PaymentType paymentType) {
        if (id == null || id.isBlank()) {
            throw new IllegalArgumentException("id is required");
        }
        if (lines == null || lines.isEmpty()) {
            throw new IllegalArgumentException("an order needs lines");
        }
        if (shippingAddress == null) {
            throw new IllegalArgumentException("shippingAddress is required
    ↪ ");
        }
        if (region == null || region.isBlank()) {
            throw new IllegalArgumentException("region is required");
        }
        if (paymentType == null) {
            throw new IllegalArgumentException("paymentType is required");
        }
        this.id = id;
        this.lines = List.copyOf(lines);
        this.shippingAddress = shippingAddress;
        this.region = region;
        this.paymentType = paymentType;
    }

    @JsonProperty("total")
    public Money total() {
        Money sum = Money.zero(lines.get(0).unitPrice().currency());
        for (OrderLine line : lines) {
            sum = sum.plus(line.subtotal());
        }
        return sum;
    }
}
```

CreateOrderRequest.java

```java
package shoplab;

import java.util.List;

public record CreateOrderRequest(List<OrderItemRequest> items,
                                 ShippingAddress shippingAddress,
                                 String region,
                                 PaymentType paymentType) { }
```

OrderItemRequest.java

```java
package shoplab;

public record OrderItemRequest(String productId, int quantity) { }
```

ProductNotFoundException.java

```java
package shoplab;

public class ProductNotFoundException extends RuntimeException {

    public ProductNotFoundException(String productId) {
        super("product not found: " + productId);
    }
}
```

ApiError.java

```java
package shoplab;

public record ApiError(String error) { }
```

3. ApiExceptionHandler.java: tal como aparece en el capítulo (sección «Y en Spring»).
4. Persistencia, servicio y controlador.

OrderRepository.java

```java
package shoplab;

import java.util.Optional;

public interface OrderRepository {
    Order save(Order order);
    Optional<Order> findById(String id);
}
```

InMemoryOrderRepository.java

```java
package shoplab;

import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;
import org.springframework.stereotype.Component;

@Component
public class InMemoryOrderRepository implements OrderRepository {

    private final Map<String, Order> orders = new ConcurrentHashMap<>();

    @Override
    public Order save(Order order) {
        orders.put(order.getId(), order);
        return order;
    }

    @Override
    public Optional<Order> findById(String id) {
        return Optional.ofNullable(orders.get(id));
    }
}
```

OrderService.java

```java
package shoplab;

import java.util.Optional;
import java.util.UUID;
import org.springframework.stereotype.Service;

@Service
public class OrderService {

    private final ProductRepository products;
    private final OrderRepository orders;

    public OrderService(ProductRepository products,
                        OrderRepository orders) {
        this.products = products;
        this.orders = orders;
    }

    public Order create(CreateOrderRequest request) {
        if (request.items() == null || request.items().isEmpty()) {
            throw new IllegalArgumentException("items are required");
        }
        Order.OrderBuilder builder = Order.builder()
                .id(UUID.randomUUID().toString())
                .shippingAddress(request.shippingAddress())
                .region(request.region())
                .paymentType(request.paymentType());
        for (OrderItemRequest item : request.items()) {
            Product product = products.findById(item.productId())
                    .orElseThrow(() ->
                            new ProductNotFoundException(item.productId()));
            builder.line(new OrderLine(item.productId(), item.quantity(),
                                       product.price()));
        }
        return orders.save(builder.build());
    }

    public Optional<Order> find(String id) {
        return orders.findById(id);
    }

    public Optional<Order> repeat(String id) {
        return orders.findById(id)
                .map(order -> orders.save(
                        order.withId(UUID.randomUUID().toString())));
    }
}
```

OrderController.java

```java
package shoplab;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/orders")
public class OrderController {

    private final OrderService service;

    public OrderController(OrderService service) {
        this.service = service;
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Order create(@RequestBody CreateOrderRequest request) {
        return service.create(request);
    }

    @GetMapping("/{id}")
    public ResponseEntity<Order> get(@PathVariable String id) {
        return service.find(id)
                .map(ResponseEntity::ok)
                .orElseGet(() -> ResponseEntity.notFound().build());
    }

    @PostMapping("/{id}/repeat")
    public ResponseEntity<Order> repeat(@PathVariable String id) {
        return service.repeat(id)
                .map(o -> ResponseEntity.status(HttpStatus.CREATED).body(o))
    ↪
                .orElseGet(() -> ResponseEntity.notFound().build());
    }
}
```

5. Prueba (la ejecuta el alumno). Con la aplicación en marcha (`./gradlew bootRun`) y `p1` creado en el capítulo 1:

Crear, repetir y comparar

```
curl -i -H "Content-Type: application/json" -X POST localhost:8080/orders \
  -d '{"items":[{"productId":"p1","quantity":2}],"shippingAddress":{"recipient
    ↪ ":"Ana","street":"Calle Sol 1","city":"Madrid","deliveryNotes":["
    ↪ portero"]},"region":"ES","paymentType":"CARD"}'
curl -i -X POST localhost:8080/orders/ID/repeat
curl -i localhost:8080/orders/ID
```

Sustituye `ID` por el `id` de la primera respuesta. La primera da 201 con `"country":"ES"` (valor por defecto) y `total` 99.80; la segunda, 201 con otro `id` y el mismo `total`; la tercera, 200. Con `productId zz`, 404 y `{"error":"product not found: zz"}`; sin `city`, 400.
Por qué funciona: el builder de `Order` lo genera Lombok y llama al constructor explícito, que valida. `withId` (de `@With`) reutiliza ese mismo constructor, y como todo es inmutable la copia no necesita copia profunda: es Prototype en su forma más simple. `@Jacksonized` permite que `ShippingAddress` se lea desde JSON a través de su builder, y `@JsonProperty("total")` añade `total()` a la respuesta. **Errores típicos:** constructor explícito con los campos en distinto orden que la clase; quitar `@Jacksonized` (Jackson no sabe usar el builder y la petición da 400); importar el `@Value` de Spring en lugar del de Lombok; mezclar monedas en un pedido (`Money.plus` lanza `IllegalArgumentException`, 400).

Parte III

Patrones estructurales

CAPÍTULO 5

Adapter, Facade y Bridge

Objetivos del capítulo

- Reconocer el olor del dominio lleno de llamadas a clientes ajenos y resolverlo con un Adapter (de objeto y de clase).
- Registrar adaptadores como beans de Spring y elegir uno por propiedad y por `@Qualifier`.
- Reducir el acoplamiento de un controlador con una Facade.
- Separar dos ejes de variación (mensaje y canal) con un Bridge y recordar cómo escribir en el log con SLF4J.
- Añadir a ShopLab el cobro con dos pasarelas de tarjeta y `POST /orders/{id}/checkout?channel=email|sms`.

Hasta ahora las piezas del proyecto eran nuestras. En el mundo real no: la pasarela de tarjeta la entrega un proveedor con su propia API, y el controlador que cobra un pedido acaba conociendo media aplicación. Este capítulo cubre tres patrones estructurales que colocan cada cosa en su sitio: el Adapter traduce una interfaz ajena a la que espera tu código, la Facade ofrece una sola entrada a un subsistema y el Bridge separa dos cosas que cambian por motivos distintos[7, 5].

Java sin compilar

Como en los capítulos 3 y 4, el código Java está escrito con cuidado contra la documentación, pero no lo ha compilado ni ejecutado su autor. Si algo falla, `./gradlew compileJava` te dirá dónde.

5.1 El olor: el dominio lleno de llamadas a clientes ajenos

Imagina que el proveedor de tarjetas te da este cliente, que no puedes modificar:

`LegacyCardClient (del proveedor)`

```
public class LegacyCardClient {
    // Importe en céntimos; devuelve "OK|LC-1" o "KO|motivo"
    public String doPayment(long amountInCents, String currencyCode) { ... }
}
```

Y tu dominio ya habla de `PaymentProcessor.charge(Money)`. Si cada punto que cobra convierte a céntimos, monta la cadena y trocea la respuesta, el formato del proveedor se filtra por todo el código. Cambiar de proveedor obliga a tocar todos esos sitios. Es el mismo olor del capítulo 3, ahora con una API que no controlas, y viola la inversión de dependencias: tu código de alto nivel depende de un detalle ajeno[14].

5.2 Adapter en Java puro

El **Adapter** es una clase que implementa la interfaz que tu código espera (`PaymentProcessor`) y por dentro traduce las llamadas a la API ajena. El resto del código no se entera de que existe el cliente del proveedor[7, 21]. Hay dos formas:

- **De objeto:** el adaptador *tiene* el cliente (composición). Es la forma habitual y la que usaremos.
- **De clase:** el adaptador *hereda* del cliente e implementa la interfaz a la vez. Solo es posible si el cliente no es `final` y arrastra toda su API pública.

LegacyCardAdapter.java (de objeto)

```
package shoplab;

public class LegacyCardAdapter implements PaymentProcessor {

    private final LegacyCardClient client;

    public LegacyCardAdapter(LegacyCardClient client) {
        this.client = client;
    }

    @Override
    public PaymentResult charge(Money amount) {
        long cents = amount.amount().movePointRight(2).longValueExact();
        String raw = client.doPayment(cents, amount.currency());
        String[] parts = raw.split("\\|", 2);
        boolean ok = parts[0].equals("OK");
        return new PaymentResult(ok,
                ok ? parts[1] : null,
                ok ? "charged " + amount.amount() + " "
                        + amount.currency() + " by legacy card"
                   : parts[1]);
    }
}
```

`Money` normaliza a dos decimales, así que `movePointRight(2).longValueExact()` no pierde información. La misma idea de clase, para comparar:

LegacyCardClassAdapter.java (ilustración)

```
package shoplab;

public class LegacyCardClassAdapter extends LegacyCardClient
        implements PaymentProcessor {

    @Override
    public PaymentResult charge(Money amount) {
        String raw = doPayment(
                amount.amount().movePointRight(2).longValueExact(),
                amount.currency());
        boolean ok = raw.startsWith("OK|");
        return new PaymentResult(ok, ok ? raw.substring(3) : null, raw);
    }
```

```
}
```

El adaptador de clase expone también `doPayment` a quien lo use como `LegacyCardClassAdapter`, y está atado al cliente concreto. Por eso en el proyecto preferimos composición. **Error típico:** meter lógica de negocio (descuentos, validaciones) en el adaptador. Su única tarea es traducir.

5.3 Y en Spring: adaptadores como beans y selección por `@Qualifier`

En el capítulo 3 la fábrica recibía un `Map<String, PaymentProcessor>` con los beans `card` y `transfer`. Ahora el bean `card` será uno de dos adaptadores, según la propiedad `shop.gateway`. Los adaptadores *no* llevan `@Component`: los crea un método `@Bean` que decide cuál instanciar[27].

PaymentConfig.java

```
package shoplab;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class PaymentConfig {

    @Bean("card")
    PaymentProcessor cardProcessor(
            @Value("${shop.gateway}") String gateway,
            LegacyCardClient legacy,
            FastPayClient fastPay) {
        return switch (gateway) {
            case "legacy" -> new LegacyCardAdapter(legacy);
            case "fastpay" -> new FastPayAdapter(fastPay);
            default -> throw new IllegalStateException(
                    "unknown shop.gateway: " + gateway);
        };
    }
}
```

Como `PaymentProcessorFactory` ya recibe el mapa de beans por nombre, no cambia: el bean `card` sigue llamándose igual aunque ahora sea otro tipo de objeto. Esa es la ventaja de haber diseñado la fábrica contra el contenedor.

Cuando hay varios beans del mismo tipo y quieres inyectar uno concreto por constructor, Spring necesita ayuda: `@Qualifier("nombre")` en el parámetro elige el bean por su nombre[27]. Sin él, Spring lanza `NoUniqueBeanDefinitionException` porque hay dos candidatos de tipo `PaymentProcessor`.

Uso de @Qualifier (ilustración)

```
@Component
public class ReceiptPrinter {
    private final PaymentProcessor processor;

    public ReceiptPrinter(@Qualifier("transfer") PaymentProcessor processor) {
```

```
        this.processor = processor;
    }
}
```

5.4 El olor: el controlador conoce siete servicios

Un checkout real necesita varias piezas: fábrica de procesadores, cobro, canal de aviso, mensajes... Si el controlador las coordina, cada endpoint nuevo copia ese guion:

OrderController (antes)

```
@PostMapping("/{id}/checkout")
public CheckoutResult checkout(@PathVariable String id, String channel) {
    Order order = orders.find(id).orElseThrow();
    PaymentProcessor p = factory.create(order.getPaymentType());
    PaymentResult r = p.charge(order.total());
    NotificationChannel c = channels.get(channel);
    new PaymentConfirmation(c, order, r).send(
            order.getShippingAddress().getRecipient());
    return new CheckoutResult(order.getId(), order.total(), r);
}
```

El controlador depende de cuatro tipos de colaboradores y sabe en qué orden se llaman. Probarlo o reutilizar el flujo desde otro sitio obliga a repetirlo.

5.5 Facade en Java puro y en Spring

La **Facade** es una clase con una interfaz sencilla (a menudo una sola operación) que oculta cómo colaboran las piezas de un subsistema[7, 20]; Kerievsky lo presenta también como destino de refactorizaciones[11]. No elimina el subsistema: sus clases siguen siendo accesibles para quien las necesite. Y no debe inventar reglas de negocio nuevas, solo orquestar.

En Spring la fachada es un bean (`@Service`) cuyas dependencias llegan por constructor. Para los canales, Spring inyecta un `Map<String, NotificationChannel>` con el nombre de cada bean como clave, igual que hicimos con los procesadores. Esa es la fachada completa de ShopLab; los tipos `NotificationChannel` y `PaymentConfirmation` se explican en la sección siguiente.

CheckoutFacade.java

```
package shoplab;

import java.util.Map;
import org.springframework.stereotype.Service;

@Service
public class CheckoutFacade {

    private final PaymentProcessorFactory processors;
    private final Map<String, NotificationChannel> channels;

    public CheckoutFacade(PaymentProcessorFactory processors,
                          Map<String, NotificationChannel> channels) {
```

```
        this.processors = processors;
        this.channels = channels;
    }

    public CheckoutResult checkout(Order order, String channel) {
        NotificationChannel selected = channels.get(channel);
        if (selected == null) {
            throw new IllegalArgumentException(
                    "unknown channel: " + channel);
        }
        Money amount = order.total();
        PaymentResult payment = processors
                .create(order.getPaymentType())
                .charge(amount);
        new PaymentConfirmation(selected, order, payment)
                .send(order.getShippingAddress().getRecipient());
        return new CheckoutResult(order.getId(), amount, payment);
    }
}
```

El canal se valida *antes* de cobrar: un canal desconocido no debe dejar un cobro hecho y un error al final. El resultado es un `record`:

CheckoutResult.java

```
package shoplab;

public record CheckoutResult(String orderId, Money amount,
                             PaymentResult payment) {
}
```

El controlador pasa a depender solo de la fachada (ver el hito).

5.6 El olor: explosión de subclases (mensaje por canal)

Hay dos mensajes (confirmación de pago, aviso de envío) y dos canales (email, SMS). Con herencia, cada combinación es una clase: `PaymentEmail`, `PaymentSms`, `ShippingEmail`, `ShippingSms`. Añadir un tercer canal obliga a escribir dos clases más, y un tercer mensaje, tres. El número de clases crece como el producto de los dos ejes, porque la herencia mezcla en una sola jerarquía dos cosas que varían de forma independiente.

5.7 Bridge en Java puro y en Spring

El **Bridge** separa en dos jerarquías lo que varía por motivos distintos: la *abstracción* (qué mensaje enviamos) y la *implementación* (por qué canal). La abstracción contiene una referencia a la implementación y le delega[7, 21]. Cada eje crece sin tocar el otro: m mensajes más c canales, no $m \times c$ clases.

Recordatorio: registro de mensajes con SLF4J

Spring Boot trae SLF4J y Logback, sin dependencias nuevas[24]. Se pide un `Logger` por clase y se escribe con `log.info`; los `{}` se sustituyen por los argumentos, sin concatenar cadenas:

Un Logger

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class Example {
    private static final Logger log =
            LoggerFactory.getLogger(Example.class);

    void run() {
        log.info("charged {} EUR", 12);
    }
}
```

Con Lombok instalado, `@Slf4j` genera ese campo, pero aquí lo escribimos a mano. En consola verás la hora, el nivel (`INFO`), el nombre de la clase y el mensaje.

La implementación del Bridge es una interfaz con una operación:

NotificationChannel.java

```
package shoplab;

public interface NotificationChannel {
    void deliver(String recipient, String text);
}
```

En Spring, cada canal es un bean con nombre; así el mapa de la fachada ya los recibe:

EmailChannel.java

```
package shoplab;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;

@Component("email")
public class EmailChannel implements NotificationChannel {

    private static final Logger log =
            LoggerFactory.getLogger(EmailChannel.class);

    @Override
    public void deliver(String recipient, String text) {
        log.info("[EMAIL to {}] {}", recipient, text);
    }
}
```

`SmsChannel` es idéntica con `@Component("sms")` y el prefijo `[SMS to {}]` (la tienes completa

en la solución del hito). La abstracción guarda el canal y deja a las subclases decidir el contenido:

Notification.java y PaymentConfirmation.java

```java
package shoplab;

public abstract class Notification {

    protected final NotificationChannel channel;

    protected Notification(NotificationChannel channel) {
        this.channel = channel;
    }

    public abstract void send(String recipient);
}

// ----- archivo aparte: PaymentConfirmation.java -----
package shoplab;

public class PaymentConfirmation extends Notification {

    private final Order order;
    private final PaymentResult result;

    public PaymentConfirmation(NotificationChannel channel,
                               Order order, PaymentResult result) {
        super(channel);
        this.order = order;
        this.result = result;
    }

    @Override
    public void send(String recipient) {
        channel.deliver(recipient, "Order " + order.getId() + ": "
                + result.message() + " (" + result.transactionId() + ")");
    }
}
```

Un mensaje nuevo es una subclase de `Notification`; un canal nuevo, una clase que implementa `NotificationChannel`. No hay productos cartesianos. **Error típico:** copiar el texto del mensaje dentro de cada canal; el canal solo transporta.

5.8 Comparación: Adapter frente a Facade

Se parecen porque ambos ponen una clase delante de otras, pero responden a preguntas distintas[7, 20]:

	Adapter	Facade
Problema	Una interfaz no coincide con la esperada	Un subsistema es complejo de usar
Qué envuelve	Normalmente una clase ajena	Varias clases propias
Interfaz que ofrece	Una ya existente (`PaymentProcessor`)	Una nueva y más simple
Cuándo	Integrar código que no controlas	Dar una entrada única a un flujo

El Bridge se decide antes (en el diseño, para que dos ejes crezcan por separado); el Adapter, después (cuando ya tienes dos interfaces incompatibles).

5.9 En el proyecto: Hito 4

Hito 4: pasarelas, fachada y notificaciones

Partes del hito 3. Se añaden `LegacyCardClient` y `FastPayClient` (con `FastPayReceipt`), los adaptadores `LegacyCardAdapter` y `FastPayAdapter` y `PaymentConfig`, que define `@Bean("card")` según `shop.gateway`. Se *elimina* `CardProcessor`; `BankTransferProcessor` y `PaymentProcessorFactory` no cambian. Se añaden el Bridge de notificaciones (`NotificationChannel`, `EmailChannel`, `SmsChannel`, `Notification`, `PaymentConfirmation`), `CheckoutResult` y `CheckoutFacade.checkout(Order, String)`. `OrderController` gana `POST /orders/{id}/checkout?channel=email|sms` (`@RequestParam`, por defecto `email`). El pedido sigue sin estado: el checkout cobra y avisa, nada más. Está en el ejercicio 5.6.

5.10 Ejercicios

Ejercicio 5.1: Adapta una pasarela de criptomonedas ★★☆☆☆

Trabaja en clases propias del paquete `shoplab.ejercicios` (no entran en el proyecto). El proveedor te da este cliente, que no puedes cambiar:

CryptoPayClient.java

```java
package shoplab.ejercicios;

public class CryptoPayClient {
    // Devuelve el hash de la operación; lanza IllegalArgumentException
    // si la cartera está en blanco o el importe no es positivo.
    public String send(String wallet, double euros) {
        if (wallet == null || wallet.isBlank()) {
            throw new IllegalArgumentException("wallet is required");
        }
        if (euros <= 0) {
            throw new IllegalArgumentException("euros must be positive");
        }
        return "0x" + Integer.toHexString((wallet + euros).hashCode());
    }
}
```

Sigue los pasos:

1. Crea `CryptoPayAdapter`, que implementa `PaymentProcessor`, con un constructor `(CryptoPayClient client, String wallet)`.
2. En `charge(Money)` convierte el importe a `double` y llama a `send`.
3. Si `send` devuelve el hash, devuelve un `PaymentResult` con `success = true` y el hash como `transactionId`.
4. Si lanza `IllegalArgumentException`, devuelve `success = false`, sin `transactionId` y con el mensaje de la excepción.
5. Crea `CryptoPayCheck` con un `main` que cobre 10,00 EUR y 0,00 EUR.

Está bien si el primer cobro da `success = true` con un hash que empieza por `0x` y el segundo, `success = false` con `euros must be positive`.

Pista: *`Money` guarda un `BigDecimal`: `amount.amount().doubleValue()`.*

Guiado ▪ solución en la pág. 157

Ejercicio 5.2: Dos controladores, una fachada ★★☆☆☆

Lee estos dos controladores que hacen lo mismo:

Dos versiones

```java
// Versión A
@PostMapping("/{id}/checkout")
public CheckoutResult checkoutA(@PathVariable String id) {
    Order order = orders.find(id).orElseThrow();
    PaymentProcessor p = factory.create(order.getPaymentType());
    PaymentResult r = p.charge(order.total());
    stock.reserve(order);
    emailChannel.deliver("ana", "paid");
    audit.record(order.getId(), r);
    return new CheckoutResult(order.getId(), order.total(), r);
}

// Versión B
@PostMapping("/{id}/checkout")
public CheckoutResult checkoutB(@PathVariable String id) {
    Order order = orders.find(id).orElseThrow();
    return facade.checkout(order, "email");
}
```

Responde en tres o cuatro frases: (a) qué se gana con B; (b) qué *no* cambia; (c) si en B puede seguir llamándose directamente a `stock.reserve` desde otro sitio. Está bien si mencionas el menor acoplamiento del controlador, que el subsistema sigue existiendo y accesible, y que la fachada no añade reglas de negocio nuevas.

Lectura de código ▪ solución en la pág. 158

Ejercicio 5.3: Mensajes por canales sin explosión de subclases ★★★☆☆

En el paquete `shoplab.ejercicios.bridge`, sin Spring, tienes esta jerarquía:

Versión con herencia

```
abstract class Message { abstract void send(String to); }
class WelcomeEmail extends Message { /* escribe "mail: welcome" */ }
class WelcomeSms   extends Message { /* escribe "sms: welcome" */ }
class ReminderEmail extends Message { /* escribe "mail: reminder" */ }
class ReminderSms   extends Message { /* escribe "sms: reminder" */ }
```

Rediséñala con Bridge: una interfaz `Channel` (`deliver(String to, String text)`), los canales `MailChannel` y `TextChannel`, la abstracción `Message` con los mensajes `WelcomeMessage` y `ReminderMessage`, y añade un tercer canal `PushChannel` *sin tocar* los mensajes. Los canales escriben con SLF4J (`log.info`). Un `main` (`BridgeCheck`) envía cada mensaje por los tres canales. Está bien si hay seis combinaciones con solo 2 + 3 clases de contenido y transporte, y si añadir `PushChannel` no modifica ningún archivo existente.

Pista: *La clase base de los mensajes recibe el `Channel` en su constructor.*

Práctica ▪ solución en la pág. 159

Ejercicio 5.4: Pasarela de respaldo elegida por propiedad ★★★☆☆

En una copia del proyecto (`cp -r`), usa `CryptoPayClient` y `CryptoPayAdapter` de `c05-adapter-guiado` (cópialos al paquete `shoplab`). Registra como beans un `CryptoPayAdapter` (nombre `cryptoBackup`) y un `BankTransferProcessor` extra (nombre `bankBackup`), y define un `@Bean("backup")` de tipo `PaymentProcessor` que elija entre ellos con la propiedad `shop.backup-gateway` (`crypto` o `bank`, por defecto `bank`). Comprueba con un `ApplicationRunner` que inyecta el bean con `@Qualifier("backup")` y cobra 5,00 EUR, escribiendo el resultado en el log. Está bien si con `shop.backup-gateway=crypto` el log muestra un hash `0x...` y con `bank` un mensaje `waiting for a bank transfer`.

Pista: *Los parámetros del método `@Bean("backup")` también admiten `@Qualifier`.*

Práctica ▪ solución en la pág. 162

Ejercicio 5.5: ¿Controlador, servicio o fachada? ★★★★☆

ShopLab debe cobrar un pedido, avisar al cliente y (más adelante) reservar stock. Decide dónde va ese flujo: en el controlador, en un servicio de aplicación o en una fachada. Escribe un párrafo de justificación para un revisor y el esqueleto de la clase elegida. Está bien si: el controlador queda sin lógica de orquestación; las dependencias de la clase elegida son explícitas (constructor); no se duplica la lógica si otro endpoint necesita el mismo flujo; y señalas al menos un riesgo de tu elección (por ejemplo, que la fachada crezca hasta ser una «clase dios»).

Diseño ▪ solución en la pág. 163

Ejercicio 5.6: Hito 4: pasarelas, fachada y notificaciones ★★★☆☆

Parte del hito 3. Implementa el hito 4 completo.

1. Añade `shop.gateway=legacy` a `application.properties`.
2. Crea `LegacyCardClient`, `FastPayClient` y `FastPayReceipt` (clientes ajenos simulados) y sus adaptadores `LegacyCardAdapter` y `FastPayAdapter`.
3. Borra `CardProcessor` y crea `PaymentConfig` con `@Bean("card")`. No toques `PaymentProcessorFactory`.

4. Crea `NotificationChannel`, `EmailChannel`, `SmsChannel`, `Notification` y `PaymentConfirmation`.
5. Crea `CheckoutResult` y `CheckoutFacade`.
6. Añade a `OrderController` el endpoint `POST /orders/{id}/checkout`.

Está bien si, con un pedido `CARD` de 99,80 EUR, el checkout responde 200 con `amount` 99.80 y `payment.success = true`; el log muestra una línea `[EMAIL to Ana]` con `?channel=email` (o sin parámetro) y `[SMS to Ana]` con `?channel=sms`; `?channel=fax` da 400; un pedido inexistente da 404; y al cambiar `shop.gateway` a `fastpay` el identificador de la transacción empieza por `FP-`.

Pista: *Valida el canal en la fachada antes de cobrar.*

Proyecto ▪ solución en la pág. 164

5.11 Resumen

Resumen

- El Adapter traduce una interfaz ajena a la que espera tu código; de objeto (composición) es la forma preferida, de clase (herencia) arrastra la API del cliente.
- En Spring, los adaptadores se crean en un `@Bean` que elige según una propiedad; `@Qualifier` escoge un bean concreto cuando hay varios del mismo tipo.
- La Facade ofrece una entrada simple a un subsistema; no añade reglas de negocio ni oculta el subsistema.
- El Bridge separa abstracción e implementación en dos jerarquías y evita el producto cartesiano de subclases.
- SLF4J viene con Spring Boot: un `Logger` por clase y `log.info("{}", dato)`.
- Adapter se usa para integrar lo ajeno; Facade, para simplificar un flujo propio.
- ShopLab (H4) cobra con dos pasarelas intercambiables por propiedad y avisa por email o SMS con `POST /orders/{id}/checkout`.

Soluciones de los ejercicios

Solución del ejercicio 5.1

enunciado en la pág. 153

CryptoPayAdapter.java

```java
package shoplab.ejercicios;

import shoplab.Money;
import shoplab.PaymentProcessor;
import shoplab.PaymentResult;

public class CryptoPayAdapter implements PaymentProcessor {

    private final CryptoPayClient client;
    private final String wallet;

    public CryptoPayAdapter(CryptoPayClient client, String wallet) {
        this.client = client;
        this.wallet = wallet;
    }

    @Override
    public PaymentResult charge(Money amount) {
        try {
            String hash = client.send(wallet,
                    amount.amount().doubleValue());
            return new PaymentResult(true, hash,
                    "sent " + amount.amount() + " " + amount.currency()
                            + " to " + wallet);
        } catch (IllegalArgumentException e) {
            return new PaymentResult(false, null, e.getMessage());
        }
    }
}
```

CryptoPayCheck.java

```java
package shoplab.ejercicios;

import java.math.BigDecimal;
import shoplab.Money;
import shoplab.PaymentProcessor;

public class CryptoPayCheck {

    public static void main(String[] args) {
        PaymentProcessor processor =
                new CryptoPayAdapter(new CryptoPayClient(), "wallet-1");
        System.out.println(processor.charge(
                new Money(new BigDecimal("10.00"), "EUR")));
        System.out.println(processor.charge(Money.zero("EUR")));
    }
}
```

Ejecuta `./gradlew compileJava` y `java -cp build/classes/java/main shoplab.ejercicios.CryptoPayCheck`. **Por qué funciona:** el adaptador imple-

menta el puerto del dominio y traduce `Money` a `double` y la excepción del proveedor a un `PaymentResult` fallido, de modo que quien cobra no conoce `CryptoPayClient`. **Errores típicos:** dejar escapar la excepción del proveedor al dominio; olvidar que `Money.zero` es válido para `Money` pero no para el proveedor.

Solución del ejercicio 5.2 enunciado en la pág. 154

(a) El controlador B depende de una sola clase y no conoce el orden del flujo (cobrar, reservar, avisar, auditar): está menos acoplado, es más corto y cualquier otro endpoint puede reutilizar `facade.checkout`. La versión A repite ese guion en cada sitio y cambia cada vez que cambia el flujo.
(b) No cambia el subsistema: los mismos servicios y procesadores siguen existiendo, con la misma lógica y los mismos resultados. La fachada solo reubica la coordinación.
(c) Sí: `stock.reserve` sigue siendo un bean público y se puede llamar desde donde se quiera. La fachada no oculta el subsistema, ofrece un camino cómodo. Y tampoco debe añadir reglas de negocio propias: si lo hace, deja de ser una fachada. **Error típico:** responder que la fachada «esconde» las clases y que nadie más puede usarlas.

Solución del ejercicio 5.3 enunciado en la pág. 154

Channel.java, MailChannel.java, TextChannel.java y PushChannel.java

```java
package shoplab.ejercicios.bridge;

public interface Channel {
    void deliver(String to, String text);
}

// ----- MailChannel.java -----
package shoplab.ejercicios.bridge;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class MailChannel implements Channel {
    private static final Logger log =
            LoggerFactory.getLogger(MailChannel.class);

    @Override
    public void deliver(String to, String text) {
        log.info("mail to {}: {}", to, text);
    }
}

// ----- TextChannel.java -----
package shoplab.ejercicios.bridge;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class TextChannel implements Channel {
    private static final Logger log =
            LoggerFactory.getLogger(TextChannel.class);

    @Override
    public void deliver(String to, String text) {
        log.info("sms to {}: {}", to, text);
    }
}

// ----- PushChannel.java -----
package shoplab.ejercicios.bridge;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class PushChannel implements Channel {
    private static final Logger log =
            LoggerFactory.getLogger(PushChannel.class);

    @Override
    public void deliver(String to, String text) {
        log.info("push to {}: {}", to, text);
    }
}
```

Message.java, WelcomeMessage.java, ReminderMessage.java y BridgeCheck.java

```java
package shoplab.ejercicios.bridge;

public abstract class Message {
    protected final Channel channel;

    protected Message(Channel channel) {
        this.channel = channel;
    }

    public abstract void send(String to);
}

// ----- WelcomeMessage.java -----
package shoplab.ejercicios.bridge;

public class WelcomeMessage extends Message {
    public WelcomeMessage(Channel channel) {
        super(channel);
    }

    @Override
    public void send(String to) {
        channel.deliver(to, "welcome");
    }
}

// ----- ReminderMessage.java -----
package shoplab.ejercicios.bridge;

public class ReminderMessage extends Message {
    public ReminderMessage(Channel channel) {
        super(channel);
    }

    @Override
    public void send(String to) {
        channel.deliver(to, "reminder");
    }
}

// ----- BridgeCheck.java -----
package shoplab.ejercicios.bridge;

import java.util.List;

public class BridgeCheck {
    public static void main(String[] args) {
        List<Channel> channels = List.of(
                new MailChannel(), new TextChannel(), new PushChannel());
        for (Channel channel : channels) {
            new WelcomeMessage(channel).send("ana");
            new ReminderMessage(channel).send("ana");
        }
    }
}
```

Por qué funciona: los mensajes (qué decir) y los canales (cómo enviarlo) son dos jerarquías unidas por una referencia. Dos mensajes y tres canales son 5 clases en lugar de 6, y con un cuarto canal serían 6 en lugar de 8: la diferencia crece. `PushChannel` se añade sin editar nada existente salvo el `main`. **Errores típicos:** poner el texto en el canal (el mensaje deja de ser independiente); que `Message` cree el canal con `new` en lugar de recibirlo.

Solución del ejercicio 5.4

enunciado en la pág. 155

En la copia, `CryptoPayClient` y `CryptoPayAdapter` se copian al paquete `shoplab` cambiando la línea `package` (y borrando los `import shoplab.*`).

application.properties (línea añadida)

```
shop.backup-gateway=crypto
```

BackupConfig.java

```
package shoplab;

import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class BackupConfig {

    @Bean("cryptoBackup")
    PaymentProcessor cryptoBackup() {
        return new CryptoPayAdapter(new CryptoPayClient(), "backup-wallet");
    }

    @Bean("bankBackup")
    PaymentProcessor bankBackup() {
        return new BankTransferProcessor();
    }

    @Bean("backup")
    PaymentProcessor backup(
            @Value("${shop.backup-gateway:bank}") String gateway,
            @Qualifier("cryptoBackup") PaymentProcessor crypto,
            @Qualifier("bankBackup") PaymentProcessor bank) {
        return gateway.equals("crypto") ? crypto : bank;
    }
}
```

BackupCheck.java

```java
package shoplab;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.boot.ApplicationArguments;
import org.springframework.boot.ApplicationRunner;
import org.springframework.stereotype.Component;
import java.math.BigDecimal;

@Component
public class BackupCheck implements ApplicationRunner {

    private static final Logger log =
            LoggerFactory.getLogger(BackupCheck.class);

    private final PaymentProcessor backup;

    public BackupCheck(@Qualifier("backup") PaymentProcessor backup) {
        this.backup = backup;
    }

    @Override
    public void run(ApplicationArguments args) {
        log.info("backup charge: {}",
                backup.charge(new Money(new BigDecimal("5.00"), "EUR")));
    }
}
```

Por qué funciona: los dos `@Bean` tienen nombre, y el tercero recibe ambos con `@Qualifier` y devuelve el elegido; `BackupCheck` pide `backup` por nombre, así que Spring no tiene ambigüedad. Se asume que `BankTransferProcessor` tiene constructor sin parámetros, como en el estado del proyecto. **Errores típicos:** no poner `@Qualifier` en `BackupCheck` (`NoUniqueBeanDefinitionException`); escribir mal el nombre del bean; olvidar que `PaymentProcessorFactory` también recibirá estos beans extra en su mapa (inofensivo, porque busca `card` y `transfer`).

Solución del ejercicio 5.5

enunciado en la pág. 155

Solución de referencia: fachada. Justificación: el flujo de checkout coordina varias piezas (cobro, aviso y, luego, stock); si va en el controlador, cada endpoint nuevo (por ejemplo el de H10, que crea el pedido desde un carrito) lo copiaría. Una fachada con una sola operación `checkout` lo concentra, deja el controlador en traducir HTTP y hace explícitas las dependencias. El riesgo: si cada hito añade pasos, la fachada se convierte en una clase dios; se mitiga extrayendo colaboradores cuando crezca.

CheckoutFacade.java (esqueleto)

```java
package shoplab;

import java.util.Map;
import org.springframework.stereotype.Service;

@Service
public class CheckoutFacade {

    private final PaymentProcessorFactory processors;
    private final Map<String, NotificationChannel> channels;

    public CheckoutFacade(PaymentProcessorFactory processors,
                          Map<String, NotificationChannel> channels) {
        this.processors = processors;
        this.channels = channels;
    }

    public CheckoutResult checkout(Order order, String channel) {
        NotificationChannel selected = channels.get(channel);
        if (selected == null) {
            throw new IllegalArgumentException(
                    "unknown channel: " + channel);
        }
        PaymentResult payment = processors
                .create(order.getPaymentType()).charge(order.total());
        selected.deliver(order.getShippingAddress().getRecipient(),
                payment.message());
        return new CheckoutResult(order.getId(), order.total(), payment);
    }
}
```

Criterios para valorar otras soluciones: controlador sin orquestación; una única operación pública con nombre de negocio; dependencias por constructor; sin lógica duplicada si se añade otro endpoint; riesgo señalado. **Alternativa válida:** un servicio de aplicación (`CheckoutService`). La diferencia es de intención: el servicio contiene la lógica del caso de uso; la fachada solo la coordina. Es válida si no mezcla HTTP dentro. **No válida:** dejarlo en el controlador por «ser pocas líneas».

Solución del ejercicio 5.6

enunciado en la pág. 155

application.properties (línea añadida)

```
shop.gateway=legacy
```

LegacyCardClient.java

```java
package shoplab;

import java.util.concurrent.atomic.AtomicInteger;
import org.springframework.stereotype.Component;

// Cliente ajeno simulado: no se modifica al integrarlo.
@Component
public class LegacyCardClient {

    private final AtomicInteger counter = new AtomicInteger();

    // Devuelve "OK|LC-n" o "KO|motivo".
    public String doPayment(long amountInCents, String currencyCode) {
        if (amountInCents <= 0) {
            return "KO|invalid amount";
        }
        return "OK|LC-" + counter.incrementAndGet();
    }
}
```

FastPayReceipt.java y FastPayClient.java

```java
package shoplab;

public record FastPayReceipt(boolean accepted, String receiptId,
                             String reason) {
}

// ----- FastPayClient.java -----
package shoplab;

import java.util.concurrent.atomic.AtomicInteger;
import org.springframework.stereotype.Component;

@Component
public class FastPayClient {

    private final AtomicInteger counter = new AtomicInteger();

    public FastPayReceipt submit(double euros) {
        if (euros <= 0) {
            return new FastPayReceipt(false, null, "invalid amount");
        }
        return new FastPayReceipt(true,
                "FP-" + counter.incrementAndGet(), "accepted");
    }
}
```

LegacyCardAdapter.java y FastPayAdapter.java

```java
package shoplab;

public class LegacyCardAdapter implements PaymentProcessor {

    private final LegacyCardClient client;

    public LegacyCardAdapter(LegacyCardClient client) {
        this.client = client;
    }

    @Override
    public PaymentResult charge(Money amount) {
        long cents = amount.amount().movePointRight(2).longValueExact();
        String raw = client.doPayment(cents, amount.currency());
        String[] parts = raw.split("\\|", 2);
        boolean ok = parts[0].equals("OK");
        return new PaymentResult(ok,
                ok ? parts[1] : null,
                ok ? "charged " + amount.amount() + " "
                        + amount.currency() + " by legacy card"
                   : parts[1]);
    }
}

// ----- FastPayAdapter.java -----
package shoplab;

public class FastPayAdapter implements PaymentProcessor {

    private final FastPayClient client;

    public FastPayAdapter(FastPayClient client) {
        this.client = client;
    }

    @Override
    public PaymentResult charge(Money amount) {
        FastPayReceipt receipt =
                client.submit(amount.amount().doubleValue());
        return new PaymentResult(receipt.accepted(),
                receipt.receiptId(),
                receipt.accepted()
                        ? "charged " + amount.amount() + " "
                                + amount.currency() + " by fastpay"
                        : receipt.reason());
    }
}
```

PaymentConfig.java

```java
package shoplab;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class PaymentConfig {

    @Bean("card")
    PaymentProcessor cardProcessor(
            @Value("${shop.gateway}") String gateway,
            LegacyCardClient legacy,
            FastPayClient fastPay) {
        return switch (gateway) {
            case "legacy" -> new LegacyCardAdapter(legacy);
            case "fastpay" -> new FastPayAdapter(fastPay);
            default -> throw new IllegalStateException(
                    "unknown shop.gateway: " + gateway);
        };
    }
}
```

Borra `CardProcessor.java`; `BankTransferProcessor` y `PaymentProcessorFactory` se quedan como estaban.

NotificationChannel.java, EmailChannel.java y SmsChannel.java

```java
package shoplab;

public interface NotificationChannel {
    void deliver(String recipient, String text);
}

// ----- EmailChannel.java -----
package shoplab;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;

@Component("email")
public class EmailChannel implements NotificationChannel {

    private static final Logger log =
            LoggerFactory.getLogger(EmailChannel.class);

    @Override
    public void deliver(String recipient, String text) {
        log.info("[EMAIL to {}] {}", recipient, text);
    }
}

// ----- SmsChannel.java -----
package shoplab;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;

@Component("sms")
public class SmsChannel implements NotificationChannel {

    private static final Logger log =
            LoggerFactory.getLogger(SmsChannel.class);

    @Override
    public void deliver(String recipient, String text) {
        log.info("[SMS to {}] {}", recipient, text);
    }
}
```

Notification.java y PaymentConfirmation.java

```java
package shoplab;

public abstract class Notification {

    protected final NotificationChannel channel;

    protected Notification(NotificationChannel channel) {
        this.channel = channel;
    }

    public abstract void send(String recipient);
}

// ----- PaymentConfirmation.java -----
package shoplab;

public class PaymentConfirmation extends Notification {

    private final Order order;
    private final PaymentResult result;

    public PaymentConfirmation(NotificationChannel channel,
                               Order order, PaymentResult result) {
        super(channel);
        this.order = order;
        this.result = result;
    }

    @Override
    public void send(String recipient) {
        channel.deliver(recipient, "Order " + order.getId() + ": "
                + result.message() + " (" + result.transactionId() + ")");
    }
}
```

CheckoutResult.java y CheckoutFacade.java

```java
package shoplab;

public record CheckoutResult(String orderId, Money amount,
                             PaymentResult payment) {
}

// ----- CheckoutFacade.java -----
package shoplab;

import java.util.Map;
import org.springframework.stereotype.Service;

@Service
public class CheckoutFacade {

    private final PaymentProcessorFactory processors;
    private final Map<String, NotificationChannel> channels;

    public CheckoutFacade(PaymentProcessorFactory processors,
                          Map<String, NotificationChannel> channels) {
        this.processors = processors;
        this.channels = channels;
    }

    public CheckoutResult checkout(Order order, String channel) {
        NotificationChannel selected = channels.get(channel);
        if (selected == null) {
            throw new IllegalArgumentException(
                    "unknown channel: " + channel);
        }
        Money amount = order.total();
        PaymentResult payment = processors
                .create(order.getPaymentType())
                .charge(amount);
        new PaymentConfirmation(selected, order, payment)
                .send(order.getShippingAddress().getRecipient());
        return new CheckoutResult(order.getId(), amount, payment);
    }
}
```

OrderController.java

```java
package shoplab;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/orders")
public class OrderController {

    private final OrderService service;
    private final CheckoutFacade facade;

    public OrderController(OrderService service,
                           CheckoutFacade facade) {
        this.service = service;
        this.facade = facade;
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Order create(@RequestBody CreateOrderRequest request) {
        return service.create(request);
    }

    @GetMapping("/{id}")
    public ResponseEntity<Order> get(@PathVariable String id) {
        return service.find(id)
                .map(ResponseEntity::ok)
                .orElseGet(() -> ResponseEntity.notFound().build());
    }

    @PostMapping("/{id}/repeat")
    public ResponseEntity<Order> repeat(@PathVariable String id) {
        return service.repeat(id)
                .map(o -> ResponseEntity.status(HttpStatus.CREATED).body(o))
                .orElseGet(() -> ResponseEntity.notFound().build());
    }

    @PostMapping("/{id}/checkout")
    public ResponseEntity<CheckoutResult> checkout(
            @PathVariable String id,
            @RequestParam(defaultValue = "email") String channel) {
        return service.find(id)
                .map(order -> ResponseEntity.ok(
                        facade.checkout(order, channel)))
                .orElseGet(() -> ResponseEntity.notFound().build());
    }
}
```

5. Prueba (la ejecuta el alumno). Con `p1` creado y la aplicación en marcha:

Crear un pedido y cobrarlo

```
curl -i -H "Content-Type: application/json" -X POST localhost:8080/orders \
  -d '{"items":[{"productId":"p1","quantity":2}],"shippingAddress":{"recipient
    ↪ ":"Ana","street":"Calle Sol 1","city":"Madrid"},"region":"ES","
    ↪ paymentType":"CARD"}'
curl -i -X POST localhost:8080/orders/ID/checkout
curl -i -X POST "localhost:8080/orders/ID/checkout?channel=sms"
curl -i -X POST "localhost:8080/orders/ID/checkout?channel=fax"
curl -i -X POST localhost:8080/orders/zz/checkout
```

Sustituye `ID` por el `id` de la primera respuesta. Las dos primeras dan 200 con `amount` 99.80 y `payment.transactionId` `LC-1`, `LC-2`; en el log de `bootRun` aparecen `[EMAIL to Ana] Order ...` y `[SMS to Ana] Order ....` `fax` da 400 con `{"error":"unknown channel: fax"}` y sin cobro; el pedido inexistente, 404. Cambia `shop.gateway=fastpay`, reinicia y los identificadores serán `FP-1`, `FP-2`.

Por qué funciona: `PaymentConfig` registra el adaptador con el nombre `card`, justo el que busca `PaymentProcessorFactory` en su mapa, así que la fábrica no cambia. Spring inyecta en la fachada el mapa de todos los `NotificationChannel` (`email`, `sms`). `ApiExceptionHandler` convierte la `IllegalArgumentException` del canal desconocido en 400. **Errores típicos:** dejar `CardProcessor` con `@Component("card")` (dos beans con el mismo nombre: la aplicación no arranca); poner `@Component` en los adaptadores (aparecerían dos `PaymentProcessor` extra y el `@Bean("card")` sobraría); olvidar `shop.gateway` (`PlaceholderResolutionException`); validar el canal después de cobrar.

CAPÍTULO 6

Composite y Decorator

Objetivos del capítulo

- Reconocer el olor del código que trata distinto lo simple y lo compuesto, y resolverlo con un Composite escrito con `sealed` y `record`.
- Recordar lambdas, streams y `Comparator` y usarlos para recorrer y agrupar un árbol.
- Añadir comportamiento por capas, sin herencia, con un Decorator, y entender por qué el orden de las capas cambia el resultado.
- Decorar un bean de Spring con `@Primary` y `@Bean`.
- Añadir a ShopLab `GET /catalog` y `GET /orders/{id}/total?gift=&express=`, y hacer que el checkout cobre con la cadena de precios por defecto.

Una tienda tiene productos, categorías que agrupan productos y packs que venden varios productos a la vez. Y un pedido se cobra con recargos que se suman según el caso: regalo, urgencia, gestión. Son dos problemas de estructura. El Composite deja tratar igual lo simple y lo compuesto; el Decorator añade comportamiento a un objeto envolviéndolo, sin crear una subclase por cada combinación[7, 5, 20].

Java sin compilar

Como en los capítulos 3 a 5, el código Java está escrito con cuidado contra la documentación, pero no lo ha compilado ni ejecutado su autor. Si algo falla, `./gradlew compileJava` te dirá dónde.

6.1 El olor: tratar distinto un producto, una categoría y un pack

Quieres el precio de un nodo del catálogo. Sin patrón, el código pregunta por el tipo en cada sitio:

CatalogPricing.java (olor)

```
1  package shoplab.ejemplos;
2
3  import java.util.List;
4  import shoplab.Money;
5
6  public class CatalogPricing {
7
8      // node es Object: producto, lista de nodos (categoría) o ...?
9      public static Money price(Object node, String currency) {
10         if (node instanceof shoplab.Product p) {
11             return p.price();
12         } else if (node instanceof List<?> children) {
13             Money sum = Money.zero(currency);
```

```
            for (Object child : children) {
                sum = sum.plus(price(child, currency));
            }
            return sum;
        }
        throw new IllegalArgumentException("unknown node: " + node);
    }
}
```

Cada operación nueva (contar, buscar, mostrar) repite el `if` con `instanceof`, y cuando aparece el pack hay que acordarse de tocar todos. El olor es ese: quien recorre la estructura conoce todos sus tipos.

6.2 Composite en Java puro con `sealed`

El **Composite** define una interfaz común para los elementos simples (hojas) y los compuestos (que contienen otros elementos de la misma interfaz). El cliente llama a la misma operación y no sabe si tiene delante una hoja o un árbol entero[7]. Con Java 21 lo escribimos con una interfaz `sealed`, cuyos únicos implementadores son tres `record`[16]: el compilador conoce el conjunto cerrado de variantes[12].

CatalogNode.java

```
package shoplab;

public sealed interface CatalogNode permits Product, Category, Pack {

    String name();

    // Precio de catálogo de este nodo; currency da el cero de un
    // nodo vacío (una categoría sin productos).
    Money totalPrice(String currency);
}
```

Recordatorio: lambdas, streams y `Comparator`

Los necesitas para recorrer el árbol. Una **lambda** es una función sin nombre: `c -> c.totalPrice(currency)`. Un **stream** encadena operaciones sobre una colección: `map` transforma cada elemento, `filter` se queda con los que cumplen una condición y `reduce` los combina en uno solo a partir de un valor inicial. `collect(Collectors.groupingBy(f))` agrupa en un `Map` por el valor de `f`. Una *referencia a método* como `Money::plus` es una lambda abreviada. Un `Comparator` dice cómo ordenar: `Comparator.comparing(Product::id)` ordena por `id`.

Recordatorio.java

```
package shoplab.ejemplos;

import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

```

```
public class Recordatorio {

    public static void main(String[] args) {
        List<String> names = List.of("teclado", "raton", "pantalla");

        List<String> longNames = names.stream()
                .filter(n -> n.length() > 5)
                .map(String::toUpperCase)
                .sorted(Comparator.naturalOrder())
                .toList();
        System.out.println(longNames); // [PANTALLA, TECLADO]

        int totalLetters = names.stream()
                .map(String::length)
                .reduce(0, Integer::sum);
        System.out.println(totalLetters); // 20

        Map<Integer, List<String>> byLength = names.stream()
                .collect(Collectors.groupingBy(String::length));
        System.out.println(byLength);
    }
}
```

Ahora los tres nodos. La hoja es el `Product` que ya tienes: solo cambia su declaración, sus campos no.

Product.java (cambia solo la cabecera)

```
package shoplab;

import java.util.Objects;

public record Product(String id, String name, Money price,
                      ProductKind kind, int stock, String category)
        implements CatalogNode {

    public Product {
        if (id == null || id.isBlank() || name == null || name.isBlank()
                || category == null || category.isBlank()) {
            throw new IllegalArgumentException(
                    "id, name and category are required");
        }
        if (stock < 0) {
            throw new IllegalArgumentException("stock must be >= 0");
        }
        Objects.requireNonNull(price);
        Objects.requireNonNull(kind);
    }

    @Override
    public Money totalPrice(String currency) {
        return price;
    }
}
```

`Product` no lleva anotaciones de Jackson. Las categorías y los packs son compuestos:

Category.java y Pack.java

```
package shoplab;

import java.util.List;

public record Category(String name, List<CatalogNode> children)
        implements CatalogNode {

    public Category {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("name is required");
        }
        children = List.copyOf(children);
    }

    @Override
    public Money totalPrice(String currency) {
        return children.stream()
                .map(child -> child.totalPrice(currency))
                .reduce(Money.zero(currency), Money::plus);
    }
}

// ----- Pack.java -----
package shoplab;

import java.math.BigDecimal;
import java.util.List;

public record Pack(String name, List<Product> products,
                   BigDecimal discount) implements CatalogNode {

    public Pack {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("name is required");
        }
        if (discount.signum() < 0
                || discount.compareTo(BigDecimal.ONE) > 0) {
            throw new IllegalArgumentException(
                    "discount must be between 0 and 1");
        }
        products = List.copyOf(products);
    }

    @Override
    public Money totalPrice(String currency) {
        Money sum = products.stream()
                .map(Product::price)
                .reduce(Money.zero(currency), Money::plus);
        return sum.times(BigDecimal.ONE.subtract(discount));
    }
}
```

El `Pack` aplica un descuento a la suma de sus productos. `Category` y `Pack` llaman a `totalPrice` de sus hijos sin preguntar de qué tipo son: esa es la idea del Composite. Los tres `record` son inmutables y `List.copyOf` copia la lista de entrada; es la defensa habitual para que nadie

modifique el árbol desde fuera[3].

La operación `totalPrice` vive en los propios nodos. Otras operaciones (contar, buscar) se pueden escribir fuera con un `switch` sobre la interfaz sellada, y el compilador exige cubrir los tres tipos (lo harás en el ejercicio 6.4):

Switch exhaustivo sobre CatalogNode

```
static String kindOf(CatalogNode node) {
    return switch (node) {
        case Product p -> "product";
        case Category c -> "category";
        case Pack k -> "pack";
    };
}
```

Error típico: en un `switch` sobre una interfaz sellada, añadir un `default`. Con él, si mañana añades una variante, el compilador ya no te avisa de los sitios que olvidaste. Los estudios empíricos sobre patrones coinciden en que Composite suele aportar valor cuando de verdad hay una estructura recursiva[30].

6.3 Y en Spring: el árbol como servicio y recorrido desde un controlador

El catálogo se construye a partir del repositorio, así que lo ofrece un servicio. La interfaz `CatalogService` oculta de dónde sale el árbol:

CatalogService.java

```
package shoplab;

public interface CatalogService {
    CatalogNode tree();
}
```

La implementación agrupa los productos por su campo `category`, y añade los packs que se declaran en una configuración. Un pack se describe con un `PackSpec` (nombre, categoría donde vive, ids de productos y descuento). Los productos que van dentro de un pack dejan de aparecer sueltos en la categoría: se venden en el pack, y así no se cuentan dos veces.

PackSpec.java, CatalogPacks.java y CatalogConfig.java

```
package shoplab;

import java.math.BigDecimal;
import java.util.List;

public record PackSpec(String name, String category,
                       List<String> productIds, BigDecimal discount) {
}

// ----- CatalogPacks.java -----
package shoplab;
```

```

import java.util.List;

public record CatalogPacks(List<PackSpec> packs) {
}

// ----- CatalogConfig.java -----
package shoplab;

import java.math.BigDecimal;
import java.util.List;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class CatalogConfig {

    @Bean
    CatalogPacks catalogPacks() {
        return new CatalogPacks(List.of(
                new PackSpec("starter-pack", "hardware",
                        List.of("p1", "p2"), new BigDecimal("0.10"))));
    }
}
```

DefaultCatalogService.java

```
package shoplab;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.Set;
import java.util.TreeMap;
import java.util.stream.Collectors;
import org.springframework.stereotype.Service;

@Service
public class DefaultCatalogService implements CatalogService {

    private final ProductRepository repository;
    private final CatalogPacks packs;

    public DefaultCatalogService(ProductRepository repository,
                                 CatalogPacks packs) {
        this.repository = repository;
        this.packs = packs;
    }

    @Override
    public CatalogNode tree() {
        Map<String, List<Product>> byCategory = repository.findAll()
                .stream()
```

```
                .sorted(Comparator.comparing(Product::id))
                .collect(Collectors.groupingBy(Product::category,
                        TreeMap::new, Collectors.toList()));
        List<CatalogNode> categories = new ArrayList<>();
        for (Map.Entry<String, List<Product>> entry
                : byCategory.entrySet()) {
            categories.add(category(entry.getKey(), entry.getValue()));
        }
        return new Category("catalog", categories);
    }

    private Category category(String name, List<Product> inCategory) {
        List<CatalogNode> children = new ArrayList<>();
        Set<String> packed = new HashSet<>();
        for (PackSpec spec : packs.packs()) {
            if (!spec.category().equals(name)) {
                continue;
            }
            List<Product> items = spec.productIds().stream()
                    .map(repository::findById)
                    .flatMap(Optional::stream)
                    .toList();
            if (items.size() == spec.productIds().size()) {
                children.add(new Pack(spec.name(), items,
                        spec.discount()));
                items.forEach(p -> packed.add(p.id()));
            }
        }
        inCategory.stream()
                .filter(p -> !packed.contains(p.id()))
                .forEach(children::add);
        return new Category(name, children);
    }
}
```

Un pack solo aparece si existen *todos* sus productos. Para recorrerlo desde la API basta un controlador delgado:

CatalogController.java

```
package shoplab;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/catalog")
public class CatalogController {

    private final CatalogService catalog;

    public CatalogController(CatalogService catalog) {
        this.catalog = catalog;
    }

```

```
    @GetMapping
    public CatalogNode get() {
        return catalog.tree();
    }
}
```

Jackson serializa cada `record` con sus componentes; el JSON no lleva un campo de tipo, pero se distinguen por su forma (`children` es una categoría; `discount` con `products`, un pack; `id`, un producto). No anotamos `CatalogNode` con `@JsonTypeInfo` a propósito: heredaría `Product` y `POST /products` dejaría de aceptar el JSON de siempre.

6.4 El olor: recargos que se acumulan con subclases

El checkout debe poder cobrar el importe del pedido más un recargo de regalo, uno de urgencia, los dos, o ninguno. Con herencia, cada combinación es una clase:

Jerarquía que explota (olor)

```
public class OrderPrice { Money price(Order o) { return o.total(); } }
public class GiftOrderPrice extends OrderPrice { /* + regalo */ }
public class ExpressOrderPrice extends OrderPrice { /* + urgencia */ }
public class GiftExpressOrderPrice extends OrderPrice { /* + ambos */ }
// y con "seguro" otra vez el doble de clases...
```

Con n recargos hay 2^n combinaciones. Además la combinación queda fijada al compilar: no puedes decidir con un parámetro de la petición que este pedido lleve regalo. Es el mismo problema que Bridge resolvía con dos ejes (capítulo 5), solo que aquí no son dos ejes, sino capas que se pueden apilar en cualquier orden.

6.5 Decorator en Java puro: composición y orden

El **Decorator** envuelve un objeto con otro que implementa la *misma interfaz*. El decorador guarda el objeto envuelto, delega en él y añade su parte antes o después[7, 5]. Como cada decorador es también un `PriceCalculator`, se pueden apilar tantos como se quiera.

PriceCalculator.java y BaseOrderPrice.java

```
package shoplab;

public interface PriceCalculator {
    Money price(Order order);
}

// ----- BaseOrderPrice.java -----
package shoplab;

public class BaseOrderPrice implements PriceCalculator {

    @Override
    public Money price(Order order) {
        return order.total();
```

```
    }
}
```

SurchargePrice.java

```
package shoplab;

import java.math.BigDecimal;
import java.util.Objects;

public class SurchargePrice implements PriceCalculator {

    private final PriceCalculator inner;
    private final String label;
    private final BigDecimal rate;   // recargo porcentual, o null
    private final Money fixed;       // recargo fijo, o null

    private SurchargePrice(PriceCalculator inner, String label,
                           BigDecimal rate, Money fixed) {
        this.inner = Objects.requireNonNull(inner);
        this.label = Objects.requireNonNull(label);
        this.rate = rate;
        this.fixed = fixed;
    }

    public static SurchargePrice percent(PriceCalculator inner,
                                         String label, BigDecimal rate) {
        return new SurchargePrice(inner, label,
                Objects.requireNonNull(rate), null);
    }

    public static SurchargePrice fixed(PriceCalculator inner,
                                       String label, Money amount) {
        return new SurchargePrice(inner, label, null,
                Objects.requireNonNull(amount));
    }

    public String label() {
        return label;
    }

    @Override
    public Money price(Order order) {
        Money current = inner.price(order);
        Money extra = fixed != null ? fixed : current.times(rate);
        return current.plus(extra);
    }
}
```

El recargo porcentual se calcula sobre lo que devuelve la capa de dentro, que ya puede incluir otros recargos. Por eso **el orden importa**. Con un pedido de 100,00 EUR, un recargo fijo de 5,00 EUR (F) y uno del 10 % (P):

El orden cambia el resultado

```
PriceCalculator base = new BaseOrderPrice();
Money five = new Money(new BigDecimal("5.00"), "EUR");
BigDecimal ten = new BigDecimal("0.10");

// primero F, luego P: (100 + 5) * 1.10 = 115.50
PriceCalculator fp = SurchargePrice.percent(
        SurchargePrice.fixed(base, "F", five), "P", ten);

// primero P, luego F: 100 * 1.10 + 5 = 115.00
PriceCalculator pf = SurchargePrice.fixed(
        SurchargePrice.percent(base, "P", ten), "F", five);
```

Se lee de dentro hacia fuera: la capa más interna se aplica primero. **Errores típicos:** olvidar delegar en `inner` (el decorador ignora lo que envuelve); meter lógica del recargo en el `BaseOrderPrice`; y creer que el orden «da igual».

6.6 Y en Spring: decorar un bean con `@Primary` y `@Bean`

Queremos que el resto del código pida `PriceCalculator` y reciba, sin saberlo, la cadena ya montada. Hay dos beans que son un `PriceCalculator`: el base y la cadena por defecto. Cuando se inyecta por tipo y hay dos candidatos, Spring falla con `NoUniqueBeanDefinitionException`, salvo que uno sea `@Primary`[27]. El método `@Bean` que fabrica la cadena recibe por parámetro el bean base y lo envuelve:

PricingConfig.java

```
package shoplab;

import java.math.BigDecimal;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Primary;

@Configuration
public class PricingConfig {

    @Bean
    BaseOrderPrice baseOrderPrice() {
        return new BaseOrderPrice();
    }

    // Cadena por defecto: el pedido con una capa de gestión al 0 %.
    // Es el punto donde enchufar recargos permanentes.
    @Bean
    @Primary
    PriceCalculator defaultPriceCalculator(BaseOrderPrice base) {
        return SurchargePrice.percent(base, "handling", BigDecimal.ZERO);
    }
}
```

Dos detalles. El parámetro se declara como `BaseOrderPrice` (el tipo concreto) y no como `PriceCalculator`: pedir la interfaz sería pedir un bean primario y la cadena se envolvería a sí

misma. Y la capa de gestión va al 0 % a propósito: la cadena por defecto no cambia ningún importe todavía, pero cambiar esa tasa cambiaría lo que cobra el checkout. Cualquier clase que pida `PriceCalculator` por constructor recibirá la cadena por defecto; si necesitas el bean base, pídelo por su tipo concreto o con `@Qualifier("baseOrderPrice")`.

6.7 Decorator frente a herencia y composición simple

- **Herencia:** cada combinación es una clase, se fija al compilar y la subclase queda atada a la implementación de su padre. Es lo que ocurre en la sección del olor[3].
- **Composición simple:** una sola clase que guarda una lista de recargos (por ejemplo, un `List<Surcharge>`) y los aplica en un bucle. Basta cuando todos los recargos son del mismo tipo y solo cambian los datos.
- **Decorator:** cada capa es un objeto con su propia lógica y se apila por configuración, también en tiempo de ejecución. Cuesta una clase por capa y un poco de indirección al depurar.

Si lo único que cambia son unos números, una lista de datos es más simple que cuatro decoradores. Si cada capa tiene lógica distinta (un tope, una condición, una consulta), el Decorator gana. El ejercicio 6.5 te pide decidir.

Qué falta por comparar

El Decorator se parece a otros patrones que aún no has visto. Su comparación con Strategy y con Chain of Responsibility se hace en los capítulos 8 y 11 y se resume en el capítulo 13.

6.8 En el proyecto: Hito 5a

Hito 5a: catálogo en árbol y precio por capas

Partes del hito 4. `Product` pasa a implementar `CatalogNode` (sus campos no cambian). Se añaden `CatalogNode`, `Category`, `Pack`, `PackSpec`, `CatalogPacks`, `CatalogConfig`, `CatalogService`, `DefaultCatalogService` y `CatalogController` con `GET /catalog`. Se añaden `PriceCalculator`, `BaseOrderPrice`, `SurchargePrice`, `PricingConfig` (cadena por defecto `@Primary`: recargo de gestión al 0 %), `OrderTotal` y `PricingController` con `GET /orders/{id}/total?gift=&express=`. `CheckoutFacade` cobra `calculator.price(order)` con la cadena por defecto. `/total` sin parámetros da el mismo importe que el checkout; con `gift` o `express`, `/total` es una *simulación* («cuánto costaría») y difiere del checkout, que no acepta esas banderas. Está en el ejercicio 6.6.

6.9 Ejercicios

Ejercicio 6.1: Un árbol de categorías con precio total ★★☆☆☆

Trabaja en clases propias del paquete `shoplab.tree` (no entran en el proyecto). Quieres modelar una tienda con artículos, grupos y un lote de precio cerrado.

1. Crea `sealed interface Node permits Item, Group, Bundle` con `String name()` y `Money totalPrice(String currency)`.

2. Crea `record Item(String name, Money price)`: su total es su precio.
3. Crea `record Group(String name, List<Node> children)`: su total es la suma de los totales de sus hijos (con `reduce` sobre un stream).
4. Crea `record Bundle(String name, List<Item> items, Money price)`: su total es `price`, el precio cerrado del lote.
5. Crea `TreeCheck` con un `main` que monte el grupo `store` con el grupo `hardware` (teclado 49,90 EUR y ratón 19,90 EUR) y el lote `desk-kit` de 60,00 EUR (con esos mismos dos artículos), e imprima el total.

Está bien si imprime `129.80` (49,90 + 19,90 + 60,00).

Pista: *Los hijos son de tipo `Node`: llama a `child.totalPrice(currency)` sin preguntar su tipo.*

Guiado ▪ solución en la pág. 187

Ejercicio 6.2: Tres cadenas, tres precios ★★☆☆☆

Tienes un pedido de 100,00 EUR y estas tres cadenas. G es un recargo fijo de 3,00 EUR, F uno fijo de 5,00 EUR y P uno del 10 %.

Las tres cadenas

```
PriceCalculator base = new BaseOrderPrice();
Money five = new Money(new BigDecimal("5.00"), "EUR");
Money three = new Money(new BigDecimal("3.00"), "EUR");
BigDecimal ten = new BigDecimal("0.10");

PriceCalculator a = SurchargePrice.fixed(SurchargePrice.percent(
        SurchargePrice.fixed(base, "F", five), "P", ten), "G", three);

PriceCalculator b = SurchargePrice.fixed(SurchargePrice.fixed(
        SurchargePrice.percent(base, "P", ten), "F", five), "G", three);

PriceCalculator c = SurchargePrice.fixed(SurchargePrice.percent(
        SurchargePrice.fixed(base, "G", three), "P", ten), "F", five);
```

Calcula `a.price(order)`, `b.price(order)` y `c.price(order)`, e indica en una frase por qué no coinciden. Está bien si das los tres importes con dos decimales y nombras la capa que, aplicada antes o después, cambia el resultado.

Lectura de código ▪ solución en la pág. 189

Ejercicio 6.3: Descuento de temporada y precio mínimo ★★★☆☆

Trabaja *sobre una copia* del proyecto en el hito 5a (añades clases y un `ApplicationRunner`). Crea dos decoradores que *no* repitan a `SurchargePrice`:

- `SeasonalDiscountPrice(PriceCalculator inner, String season, BigDecimal rate)`: resta al precio de `inner` el porcentaje `rate` (entre 0 y 1; si no, `IllegalArgumentException`). Usa `Money.times(BigDecimal)` y `Money.minus`.
- `MinimumPrice(PriceCalculator inner, Money minimum)`: si el precio de `inner` es menor que `minimum`, devuelve `minimum`.

Escribe un `DiscountCheck` que construya un pedido de 2 unidades de 49,90 EUR (total 99,80) y pruebe las dos cadenas posibles con un 10 % de descuento y un mínimo de 95,00

EUR. Está bien si imprime `95.00` cuando el mínimo envuelve al descuento y `89.82` cuando el descuento envuelve al mínimo.

Pista: *`BigDecimal` se compara con `compareTo`, no con `equals`.*

Práctica ▪ solución en la pág. 190

Ejercicio 6.4: Operaciones recursivas con `switch` ★★★☆☆

Trabaja *sobre una copia* del proyecto en el hito 5a. Crea la clase `CatalogOps` con tres métodos estáticos que usen un `switch` exhaustivo (sin `default`) sobre `CatalogNode`:

- `int countProducts(CatalogNode node)`: cuántos productos hay (los de un pack cuentan).
- `int depth(CatalogNode node)`: profundidad del árbol; un producto vale 1, un pack vale 2 y una categoría vale 1 más que su hijo más profundo (1 si está vacía).
- `Optional<CatalogNode> find(CatalogNode node, String name)`: el primer nodo con ese nombre, buscando en profundidad.

Añade un `OpsCheck` (`ApplicationRunner`) que, con `p1` (49,90 EUR) y `p2` (19,90 EUR) de categoría `hardware` y `p3` de `office`, imprima los tres resultados sobre `catalog.tree()`. Está bien si, con esos tres productos, `countProducts` da 3, `depth` da 4 y `find(tree, "starter-pack")` devuelve el pack. Prueba además a borrar el caso `Pack` del `switch` y comprueba que el compilador se queja.

Pista: *Para `find`, en las categorías y packs usa `map(child -> find(child, name))`, `flatMap(Optional::stream)` y `findFirst()`.*

Práctica ▪ solución en la pág. 193

Ejercicio 6.5: Seis combinaciones de recargos ★★★★☆

ShopLab debe soportar seis combinaciones de recargos sobre el precio de un pedido: regalo, urgencia, seguro, regalo+urgencia, regalo+seguro y urgencia+seguro. Compara tres diseños (herencia, Decorator y una lista de recargos como datos), elige uno y justifica tu elección en un párrafo. Entrega el código del diseño elegido. Está bien si: el diseño elegido no necesita una clase por combinación; el orden de aplicación es explícito; cada pieza (capa o recargo) se puede probar por separado; y nombras una situación en la que elegirías uno de los otros dos diseños.

Diseño ▪ solución en la pág. 194

Ejercicio 6.6: Hito 5a: catálogo en árbol y precio por capas ★★★☆☆

Parte del hito 4. Implementa el hito 5a completo.

1. Cambia la declaración de `Product` para que implemente `CatalogNode`.
2. Crea `CatalogNode`, `Category`, `Pack`, `PackSpec` y `CatalogPacks`.
3. Crea `CatalogConfig`, `CatalogService`, `DefaultCatalogService` y `CatalogController`.
4. Crea `PriceCalculator`, `BaseOrderPrice`, `SurchargePrice` y `PricingConfig` con el bean `@Primary`.
5. Crea `OrderTotal` y `PricingController` con `GET /orders/{id}/total?gift=&express=`.
6. Cambia `CheckoutFacade` para que cobre `calculator.price(order)`.

Está bien si: con `p1` (49,90 EUR) y `p2` (19,90 EUR) en `hardware`, `GET /catalog` muestra la categoría `hardware` con el pack `starter-pack` dentro; para un pedido de 99,80 EUR, `/total` sin parámetros y el `amount` del checkout dan los dos 99.80; con `?express=true`, `/total` da 109.78, más que lo que cobra el checkout.

Pista: *La cadena por defecto se pide por constructor como `PriceCalculator`; el controlador añade capas encima de ella en cada petición.*

Proyecto ▪ solución en la pág. 196

6.10 Resumen

Resumen

- El Composite da una interfaz común a hojas y compuestos; con `sealed` y `record` el compilador conoce todas las variantes y un `switch` sin `default` las cubre.
- Lambdas, `map`/`filter`/`reduce`, `groupingBy` y `Comparator` sirven para recorrer, agrupar y ordenar el árbol.
- El Decorator envuelve un objeto con otro de la misma interfaz y añade comportamiento por capas; evita la explosión de subclases.
- El orden de las capas cambia el resultado: se lee de dentro hacia fuera.
- En Spring, un `@Bean` `@Primary` monta la cadena sobre el bean base, que se pide por su tipo concreto.
- Herencia, Decorator y lista de datos resuelven lo mismo con costes distintos; si solo cambian números, basta una lista.
- ShopLab (H5a) ofrece `GET /catalog` en árbol y `GET /orders/{id}/total`; el checkout cobra con la cadena por defecto y `/total` con `gift` o `express` es una simulación.

Soluciones de los ejercicios

Solución del ejercicio 6.1 enunciado en la pág. 183

Node.java, Item.java, Group.java y Bundle.java

```java
package shoplab.tree;

import shoplab.Money;

public sealed interface Node permits Item, Group, Bundle {

    String name();

    Money totalPrice(String currency);
}

// ----- Item.java -----
package shoplab.tree;

import shoplab.Money;

public record Item(String name, Money price) implements Node {

    @Override
    public Money totalPrice(String currency) {
        return price;
    }
}

// ----- Group.java -----
package shoplab.tree;

import java.util.List;
import shoplab.Money;

public record Group(String name, List<Node> children) implements Node {

    public Group {
        children = List.copyOf(children);
    }

    @Override
    public Money totalPrice(String currency) {
        return children.stream()
                .map(child -> child.totalPrice(currency))
                .reduce(Money.zero(currency), Money::plus);
    }
}

// ----- Bundle.java -----
package shoplab.tree;

import java.util.List;
import shoplab.Money;

public record Bundle(String name, List<Item> items, Money price)
        implements Node {

    public Bundle {
        items = List.copyOf(items);
    }

    @Override
    public Money totalPrice(String currency) {
        return price;
    }
}
```

TreeCheck.java

```java
package shoplab.tree;

import java.math.BigDecimal;
import java.util.List;
import shoplab.Money;

public class TreeCheck {

    public static void main(String[] args) {
        Item keyboard = new Item("keyboard",
                new Money(new BigDecimal("49.90"), "EUR"));
        Item mouse = new Item("mouse",
                new Money(new BigDecimal("19.90"), "EUR"));
        Group hardware = new Group("hardware", List.of(keyboard, mouse));
        Bundle kit = new Bundle("desk-kit", List.of(keyboard, mouse),
                new Money(new BigDecimal("60.00"), "EUR"));
        Group store = new Group("store", List.of(hardware, kit));
        System.out.println(store.totalPrice("EUR").amount()); // 129.80
    }
}
```

Ejecuta `./gradlew compileJava` y `java -cp build/classes/java/main shoplab.tree.TreeCheck`. **Por qué funciona:** `Group` llama a `totalPrice` de cada hijo sin saber si es un artículo, otro grupo o un lote; el lote corta la recursión porque su precio es cerrado. **Errores típicos:** olvidar el valor inicial de `reduce` (sin él el resultado es un `Optional`); usar `List<Node>` en `Bundle` y perder que solo admite artículos.

Solución del ejercicio 6.2 enunciado en la pág. 184

Los tres cálculos, de dentro hacia fuera:

- `a`: F, P, G $\rightarrow$ $100 + 5 = 105{,}00$; $105 \cdot 1{,}10 = 115{,}50$; $115{,}50 + 3 = \mathbf{118.50}$.
- `b`: P, F, G $\rightarrow$ $100 \cdot 1{,}10 = 110{,}00$; $+5 = 115{,}00$; $+3 = \mathbf{118.00}$.
- `c`: G, P, F $\rightarrow$ $100 + 3 = 103{,}00$; $103 \cdot 1{,}10 = 113{,}30$; $+5 = \mathbf{118.30}$.

No coinciden porque el recargo porcentual se aplica sobre lo que haya acumulado hasta ese momento: cuantos más recargos fijos entran antes de `P`, más grande es la base sobre la que se calcula el 10 %. **Error típico:** leer la cadena de fuera hacia dentro (aplicar primero la capa más externa).

Solución del ejercicio 6.3 enunciado en la pág. 184

SeasonalDiscountPrice.java y MinimumPrice.java

```java
package shoplab;

import java.math.BigDecimal;
import java.util.Objects;

public class SeasonalDiscountPrice implements PriceCalculator {

    private final PriceCalculator inner;
    private final String season;
    private final BigDecimal rate;

    public SeasonalDiscountPrice(PriceCalculator inner, String season,
                                 BigDecimal rate) {
        if (rate.signum() < 0 || rate.compareTo(BigDecimal.ONE) > 0) {
            throw new IllegalArgumentException(
                    "rate must be between 0 and 1");
        }
        this.inner = Objects.requireNonNull(inner);
        this.season = Objects.requireNonNull(season);
        this.rate = rate;
    }

    public String season() {
        return season;
    }

    @Override
    public Money price(Order order) {
        Money current = inner.price(order);
        return current.minus(current.times(rate));
    }
}

// ----- MinimumPrice.java -----
package shoplab;

import java.util.Objects;

public class MinimumPrice implements PriceCalculator {

    private final PriceCalculator inner;
    private final Money minimum;

    public MinimumPrice(PriceCalculator inner, Money minimum) {
        this.inner = Objects.requireNonNull(inner);
        this.minimum = Objects.requireNonNull(minimum);
    }

    @Override
    public Money price(Order order) {
        Money current = inner.price(order);
        if (!current.currency().equals(minimum.currency())) {
            throw new IllegalArgumentException("currency mismatch");
        }
        return current.amount().compareTo(minimum.amount()) < 0
                ? minimum
                : current;
    }
}
```

DiscountCheck.java

```java
package shoplab;

import java.math.BigDecimal;
import org.springframework.boot.ApplicationArguments;
import org.springframework.boot.ApplicationRunner;
import org.springframework.stereotype.Component;

@Component
public class DiscountCheck implements ApplicationRunner {

    @Override
    public void run(ApplicationArguments args) {
        Order order = Order.builder()
                .id("o1")
                .line(new OrderLine("p1", 2,
                        new Money(new BigDecimal("49.90"), "EUR")))
                .shippingAddress(ShippingAddress.builder()
                        .recipient("Ana").street("Calle Sol 1")
                        .city("Madrid").build())
                .region("ES")
                .paymentType(PaymentType.CARD)
                .build();
        PriceCalculator base = new BaseOrderPrice();
        BigDecimal tenPercent = new BigDecimal("0.10");
        Money minimum = new Money(new BigDecimal("95.00"), "EUR");

        PriceCalculator minimumOutside = new MinimumPrice(
                new SeasonalDiscountPrice(base, "summer", tenPercent),
                minimum);
        PriceCalculator discountOutside = new SeasonalDiscountPrice(
                new MinimumPrice(base, minimum), "summer", tenPercent);

        System.out.println("minimum outside: "
                + minimumOutside.price(order).amount());   // 95.00
        System.out.println("discount outside: "
                + discountOutside.price(order).amount());  // 89.82
    }
}
```

Arranca la copia con `./gradlew bootRun` y mira la salida. **Por qué funciona:** cada decorador pide el precio a `inner` y lo transforma. En la primera cadena, el descuento deja 89,82 y el mínimo lo sube a 95,00; en la segunda, el mínimo no actúa (99,80 ya lo supera) y el descuento baja a 89,82. Es otra prueba de que el orden importa. **Errores típicos:** comparar `BigDecimal` con `equals` (distingue `95.0` de `95.00`); que `DiscountCheck` se quede en el proyecto real: por eso se hace en la copia, porque un `ApplicationRunner` se ejecuta en cada arranque.

Solución del ejercicio 6.4

enunciado en la pág. 185

CatalogOps.java

```
package shoplab;

import java.util.Optional;

public final class CatalogOps {

    private CatalogOps() {
    }

    public static int countProducts(CatalogNode node) {
        return switch (node) {
            case Product p -> 1;
            case Category c -> c.children().stream()
                    .mapToInt(CatalogOps::countProducts)
                    .sum();
            case Pack k -> k.products().size();
        };
    }

    public static int depth(CatalogNode node) {
        return switch (node) {
            case Product p -> 1;
            case Category c -> 1 + c.children().stream()
                    .mapToInt(CatalogOps::depth)
                    .max()
                    .orElse(0);
            case Pack k -> 2;
        };
    }

    public static Optional<CatalogNode> find(CatalogNode node,
                                             String name) {
        if (node.name().equals(name)) {
            return Optional.of(node);
        }
        return switch (node) {
            case Product p -> Optional.<CatalogNode>empty();
            case Category c -> c.children().stream()
                    .map(child -> find(child, name))
                    .flatMap(Optional::stream)
                    .findFirst();
            case Pack k -> k.products().stream()
                    .map(product -> find(product, name))
                    .flatMap(Optional::stream)
                    .findFirst();
        };
    }
}
```

OpsCheck.java

```java
package shoplab;

import org.springframework.boot.ApplicationArguments;
import org.springframework.boot.ApplicationRunner;
import org.springframework.stereotype.Component;

@Component
public class OpsCheck implements ApplicationRunner {

    private final CatalogService catalog;

    public OpsCheck(CatalogService catalog) {
        this.catalog = catalog;
    }

    @Override
    public void run(ApplicationArguments args) {
        CatalogNode tree = catalog.tree();
        System.out.println("products: " + CatalogOps.countProducts(tree));
        System.out.println("depth: " + CatalogOps.depth(tree));
        System.out.println("starter-pack: "
                + CatalogOps.find(tree, "starter-pack"));
        System.out.println("nothing: " + CatalogOps.find(tree, "zz"));
    }
}
```

Con los tres productos, el árbol es `catalog` → `hardware` → `starter-pack` → `p1`/`p2`, más `office` → `p3`: tres productos y profundidad 4 (catálogo, categoría, pack, producto; el pack vale 2 porque cuenta a sus productos). Como los productos del pack solo están dentro del pack, no se cuentan dos veces. **Por qué funciona:** los `switch` sin `default` son exhaustivos porque `CatalogNode` es sellada; si borras el caso `Pack`, el compilador dice que el `switch` no cubre todos los casos. **Errores típicos:** añadir un `default -> 0`, que silencia el aviso cuando aparece una variante nueva; olvidar `orElse(0)` en `depth` (`max()` devuelve un `OptionalInt`).

Solución del ejercicio 6.5

enunciado en la pág. 185

Comparación.

- *Herencia:* seis clases (y 8 con los tres a la vez), la combinación se fija al compilar y añadir un cuarto recargo duplica el trabajo. Descartada.
- *Decorator:* tres decoradores (`GiftPrice`, `ExpressPrice`, `InsurancePrice`) que se apilan según lo que pida la petición; el orden es el del código que monta la cadena.
- *Lista de recargos:* una clase `ListPrice` que aplica en orden una lista de `Surcharge` (datos: etiqueta, porcentaje o importe fijo).

Referencia: Decorator. Los recargos de ShopLab ya son decoradores (`SurchargePrice`), cada capa se puede probar sola con un `BaseOrderPrice` y nuevos recargos con lógica propia (por ejemplo, un seguro que depende del valor del pedido) encajan sin tocar el resto. La cadena se monta así:

CombinationsCheck.java

```java
package shoplab.ejercicios;

import java.math.BigDecimal;
import java.util.Set;
import shoplab.BaseOrderPrice;
import shoplab.Money;
import shoplab.PriceCalculator;
import shoplab.SurchargePrice;

public class CombinationsCheck {

    static PriceCalculator chain(Set<String> flags, String currency) {
        PriceCalculator calculator = new BaseOrderPrice();
        if (flags.contains("gift")) {
            calculator = SurchargePrice.fixed(calculator, "gift",
                    new Money(new BigDecimal("3.00"), currency));
        }
        if (flags.contains("express")) {
            calculator = SurchargePrice.percent(calculator, "express",
                    new BigDecimal("0.10"));
        }
        if (flags.contains("insurance")) {
            calculator = SurchargePrice.percent(calculator, "insurance",
                    new BigDecimal("0.02"));
        }
        return calculator;
    }

    public static void main(String[] args) {
        System.out.println(chain(Set.of("gift", "express"), "EUR")
                .getClass().getSimpleName());
        System.out.println(chain(Set.of(), "EUR")
                .getClass().getSimpleName());
    }
}
```

Una sola función monta las seis combinaciones (y las demás) y el orden es el de los `if`. **Alternativa válida: lista de recargos como datos.**

Surcharge.java y ListPrice.java

```java
package shoplab.ejercicios;

import java.math.BigDecimal;
import java.util.List;
import shoplab.Money;
import shoplab.Order;
import shoplab.PriceCalculator;

public record Surcharge(String label, BigDecimal rate, Money fixed) {

    public Money applyTo(Money current) {
        Money extra = fixed != null ? fixed : current.times(rate);
        return current.plus(extra);
    }
}

// ----- ListPrice.java -----
package shoplab.ejercicios;

import java.util.List;
import shoplab.Money;
import shoplab.Order;
import shoplab.PriceCalculator;

public class ListPrice implements PriceCalculator {

    private final List<Surcharge> surcharges;

    public ListPrice(List<Surcharge> surcharges) {
        this.surcharges = List.copyOf(surcharges);
    }

    @Override
    public Money price(Order order) {
        Money current = order.total();
        for (Surcharge surcharge : surcharges) {
            current = surcharge.applyTo(current);
        }
        return current;
    }
}
```

Es más corta y sirve si todos los recargos son «un porcentaje o un importe». **Criterios para valorar otras soluciones:** no una clase por combinación; orden explícito (aquí, el de la lista o el de las capas); cada pieza se puede probar sola; y la justificación menciona un coste de la opción elegida (el Decorator pide una clase por capa; la lista no admite capas con lógica propia sin ensuciar `Surcharge`).

Solución del ejercicio 6.6

enunciado en la pág. 185

1. Producto. Sustituye `Product.java` por el de la sección del Composite (cambia solo la cabecera `implements CatalogNode` y el método `totalPrice`):

Product.java

```
package shoplab;

import java.util.Objects;

public record Product(String id, String name, Money price,
                      ProductKind kind, int stock, String category)
        implements CatalogNode {

    public Product {
        if (id == null || id.isBlank() || name == null || name.isBlank()
                || category == null || category.isBlank()) {
            throw new IllegalArgumentException(
                    "id, name and category are required");
        }
        if (stock < 0) {
            throw new IllegalArgumentException("stock must be >= 0");
        }
        Objects.requireNonNull(price);
        Objects.requireNonNull(kind);
    }

    @Override
    public Money totalPrice(String currency) {
        return price;
    }
}
```

2. Nodos.

CatalogNode.java, Category.java y Pack.java

```java
package shoplab;

public sealed interface CatalogNode permits Product, Category, Pack {

    String name();

    Money totalPrice(String currency);
}

// ----- Category.java -----
package shoplab;

import java.util.List;

public record Category(String name, List<CatalogNode> children)
        implements CatalogNode {

    public Category {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("name is required");
        }
        children = List.copyOf(children);
    }

    @Override
    public Money totalPrice(String currency) {
        return children.stream()
                .map(child -> child.totalPrice(currency))
                .reduce(Money.zero(currency), Money::plus);
    }
}

// ----- Pack.java -----
package shoplab;

import java.math.BigDecimal;
import java.util.List;

public record Pack(String name, List<Product> products,
                   BigDecimal discount) implements CatalogNode {

    public Pack {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("name is required");
        }
        if (discount.signum() < 0
                || discount.compareTo(BigDecimal.ONE) > 0) {
            throw new IllegalArgumentException(
                    "discount must be between 0 and 1");
        }
        products = List.copyOf(products);
    }

    @Override
    public Money totalPrice(String currency) {
        Money sum = products.stream()
                .map(Product::price)
                .reduce(Money.zero(currency), Money::plus);
        return sum.times(BigDecimal.ONE.subtract(discount));
    }
}
```

3. Catálogo.

PackSpec.java, CatalogPacks.java, CatalogConfig.java y CatalogService.java

```java
package shoplab;

import java.math.BigDecimal;
import java.util.List;

public record PackSpec(String name, String category,
                       List<String> productIds, BigDecimal discount) {
}

// ----- CatalogPacks.java -----
package shoplab;

import java.util.List;

public record CatalogPacks(List<PackSpec> packs) {
}

// ----- CatalogConfig.java -----
package shoplab;

import java.math.BigDecimal;
import java.util.List;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class CatalogConfig {

    @Bean
    CatalogPacks catalogPacks() {
        return new CatalogPacks(List.of(
                new PackSpec("starter-pack", "hardware",
                        List.of("p1", "p2"), new BigDecimal("0.10"))));
    }
}

// ----- CatalogService.java -----
package shoplab;

public interface CatalogService {
    CatalogNode tree();
}
```

DefaultCatalogService.java y CatalogController.java

```
package shoplab;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.Set;
import java.util.TreeMap;
import java.util.stream.Collectors;
import org.springframework.stereotype.Service;

@Service
public class DefaultCatalogService implements CatalogService {

    private final ProductRepository repository;
    private final CatalogPacks packs;

    public DefaultCatalogService(ProductRepository repository,
                                 CatalogPacks packs) {
        this.repository = repository;
        this.packs = packs;
    }

    @Override
    public CatalogNode tree() {
        Map<String, List<Product>> byCategory = repository.findAll()
                .stream()
                .sorted(Comparator.comparing(Product::id))
                .collect(Collectors.groupingBy(Product::category,
                        TreeMap::new, Collectors.toList()));
        List<CatalogNode> categories = new ArrayList<>();
        for (Map.Entry<String, List<Product>> entry
                : byCategory.entrySet()) {
            categories.add(category(entry.getKey(), entry.getValue()));
        }
        return new Category("catalog", categories);
    }

    private Category category(String name, List<Product> inCategory) {
        List<CatalogNode> children = new ArrayList<>();
        Set<String> packed = new HashSet<>();
        for (PackSpec spec : packs.packs()) {
            if (!spec.category().equals(name)) {
                continue;
            }
            List<Product> items = spec.productIds().stream()
                    .map(repository::findById)
                    .flatMap(Optional::stream)
                    .toList();
            if (items.size() == spec.productIds().size()) {
                children.add(new Pack(spec.name(), items,
                        spec.discount()));
                items.forEach(p -> packed.add(p.id()));
            }
        }
        inCategory.stream()
                .filter(p -> !packed.contains(p.id()))
                .forEach(children::add);
        return new Category(name, children);
    }
```

4. Precio por capas.

PriceCalculator.java, BaseOrderPrice.java y SurchargePrice.java

```
package shoplab;

public interface PriceCalculator {
    Money price(Order order);
}

// ----- BaseOrderPrice.java -----
package shoplab;

public class BaseOrderPrice implements PriceCalculator {

    @Override
    public Money price(Order order) {
        return order.total();
    }
}

// ----- SurchargePrice.java -----
package shoplab;

import java.math.BigDecimal;
import java.util.Objects;

public class SurchargePrice implements PriceCalculator {

    private final PriceCalculator inner;
    private final String label;
    private final BigDecimal rate;
    private final Money fixed;

    private SurchargePrice(PriceCalculator inner, String label,
                           BigDecimal rate, Money fixed) {
        this.inner = Objects.requireNonNull(inner);
        this.label = Objects.requireNonNull(label);
        this.rate = rate;
        this.fixed = fixed;
    }

    public static SurchargePrice percent(PriceCalculator inner,
                                         String label, BigDecimal rate) {
        return new SurchargePrice(inner, label,
                Objects.requireNonNull(rate), null);
    }

    public static SurchargePrice fixed(PriceCalculator inner,
                                       String label, Money amount) {
        return new SurchargePrice(inner, label, null,
                Objects.requireNonNull(amount));
    }

    public String label() {
        return label;
    }

    @Override
    public Money price(Order order) {
        Money current = inner.price(order);
        Money extra = fixed != null ? fixed : current.times(rate);
        return current.plus(extra);
    }
}
```

PricingConfig.java, OrderTotal.java y PricingController.java

```java
package shoplab;

import java.math.BigDecimal;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Primary;

@Configuration
public class PricingConfig {

    @Bean
    BaseOrderPrice baseOrderPrice() {
        return new BaseOrderPrice();
    }

    @Bean
    @Primary
    PriceCalculator defaultPriceCalculator(BaseOrderPrice base) {
        return SurchargePrice.percent(base, "handling", BigDecimal.ZERO);
    }
}

// ----- OrderTotal.java -----
package shoplab;

public record OrderTotal(String orderId, Money total,
                         boolean simulation) {
}

// ----- PricingController.java -----
package shoplab;

import java.math.BigDecimal;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/orders")
public class PricingController {

    private final OrderService orders;
    private final PriceCalculator defaultPrice;

    public PricingController(OrderService orders,
                             PriceCalculator defaultPrice) {
        this.orders = orders;
        this.defaultPrice = defaultPrice;
    }

    @GetMapping("/{id}/total")
    public ResponseEntity<OrderTotal> total(
            @PathVariable String id,
            @RequestParam(defaultValue = "false") boolean gift,
            @RequestParam(defaultValue = "false") boolean express) {
        return orders.find(id)
                .map(order -> ResponseEntity.ok(
                        simulate(order, gift, express)))
                .orElseGet(() -> ResponseEntity.notFound().build());
```

5. Checkout. `CheckoutFacade` recibe la cadena por defecto y cobra con ella:

CheckoutFacade.java

```java
package shoplab;

import java.util.Map;
import org.springframework.stereotype.Service;

@Service
public class CheckoutFacade {

    private final PaymentProcessorFactory processors;
    private final Map<String, NotificationChannel> channels;
    private final PriceCalculator calculator;

    public CheckoutFacade(PaymentProcessorFactory processors,
                          Map<String, NotificationChannel> channels,
                          PriceCalculator calculator) {
        this.processors = processors;
        this.channels = channels;
        this.calculator = calculator;
    }

    public CheckoutResult checkout(Order order, String channel) {
        NotificationChannel selected = channels.get(channel);
        if (selected == null) {
            throw new IllegalArgumentException(
                    "unknown channel: " + channel);
        }
        Money amount = calculator.price(order);
        PaymentResult payment = processors
                .create(order.getPaymentType())
                .charge(amount);
        new PaymentConfirmation(selected, order, payment)
                .send(order.getShippingAddress().getRecipient());
        return new CheckoutResult(order.getId(), amount, payment);
    }
}
```

6. Prueba (la ejecuta el alumno). Con la aplicación en marcha:

Catálogo, total y checkout

```
curl -i -H "Content-Type: application/json" -X POST localhost:8080/products \
  -d '{"id":"p1","name":"Teclado","price":{"amount":49.9,"currency":"EUR"},"
    ↪ kind":{"type":"physical","weightGrams":800},"stock":12,"category":"
    ↪ hardware"}'
curl -i -H "Content-Type: application/json" -X POST localhost:8080/products \
  -d '{"id":"p2","name":"Raton","price":{"amount":19.9,"currency":"EUR"},"kind
    ↪ ":{"type":"physical","weightGrams":90},"stock":30,"category":"hardware
    ↪ "}'
curl -i localhost:8080/catalog
curl -i -H "Content-Type: application/json" -X POST localhost:8080/orders \
  -d '{"items":[{"productId":"p1","quantity":2}],"shippingAddress":{"recipient
    ↪ ":"Ana","street":"Calle Sol 1","city":"Madrid"},"region":"ES","
    ↪ paymentType":"CARD"}'
curl -i localhost:8080/orders/ID/total
curl -i -X POST localhost:8080/orders/ID/checkout
curl -i "localhost:8080/orders/ID/total?express=true"
curl -i "localhost:8080/orders/ID/total?gift=true&express=true"
```

Sustituye `ID` por el `id` del pedido. `/catalog` devuelve `catalog` con la categoría `hardware`, que contiene el pack `starter-pack` con `p1` y `p2` y descuento `0.10` (62.82 EUR). `/total` sin parámetros da `99.80` con `simulation: false` y el checkout cobra `amount` `99.80`: coinciden. Con `?express=true`, `/total` da `109.78` (simulación): el checkout no acepta esa bandera y seguirá cobrando 99.80. Con las dos banderas, $(99{,}80 + 3{,}00) \cdot 1{,}10 = 113{,}08$.

Por qué funciona: el bean `@Primary` es la cadena por defecto, así que `CheckoutFacade` y `PricingController` reciben el mismo `PriceCalculator`. El controlador no modifica ese bean: crea capas nuevas encima en cada petición. **Errores típicos:** declarar el parámetro de `defaultPriceCalculator` como `PriceCalculator` (el bean se pide a sí mismo); olvidar `@Primary` (`NoUniqueBeanDefinitionException`); poner `@Component` en `BaseOrderPrice` además del `@Bean` (dos beans del mismo tipo con nombres distintos); que el pack use productos que no existen (el pack simplemente no aparece); y comparar el `/total` con `?express=true` con lo que cobra el checkout como si fuera un error.

CAPÍTULO 7

Proxy y Flyweight

Objetivos del capítulo

- Reconocer el olor de las consultas repetidas y del código transversal mezclado con la lógica, y resolverlo con un Proxy.
- Escribir a mano un proxy de caché, uno de protección y uno virtual, y crear un proxy dinámico con `java.lang.reflect.Proxy`.
- Añadir el starter de AOP al proyecto y escribir un `@Aspect` con `@Around` y un pointcut `execution(...)`.
- Explicar por qué una llamada de un método a otro de la misma clase no pasa por el proxy.
- Compartir objetos inmutables con un Flyweight y servirlos desde un bean singleton.
- Añadir a ShopLab una caché del catálogo, un aspecto de tiempos y tipos de IVA compartidos (IVA reducido para `books`).

A veces quieres controlar el acceso a un objeto sin que su cliente lo note: guardar resultados caros, medir tiempos, comprobar permisos o crear el objeto solo cuando hace falta. Eso es un Proxy: un sustituto con la misma interfaz que el objeto real que decide cuándo y cómo llega la llamada hasta él[7, 5]. Y a veces tienes miles de objetos idénticos y quieres uno solo: es el Flyweight. Spring usa proxies por todas partes, y su programación orientada a aspectos (AOP) es el Proxy industrializado.

Java sin compilar

Como en los capítulos 3 a 6, el código Java está escrito con cuidado contra la documentación, pero no lo ha compilado ni ejecutado su autor. Si algo falla, `./gradlew compileJava` te dirá dónde.

7.1 El olor: consultas repetidas y código transversal

Tu `GET /catalog` reconstruye el árbol entero en cada petición, aunque el catálogo casi nunca cambie. Y quieres medir cuánto tarda cada servicio. Sin patrón, lo resuelves metiendo código en el propio servicio:

PriceService.java (olor)

```
public Money priceOf(String id) {
    long start = System.nanoTime();              // medir
    Money cached = cache.get(id);                // cachear
    if (cached != null) {
        return cached;                           // (y no se mide este caso)
    }
    Money price = repository.findPrice(id);      // lo único que importa
```

```
    cache.put(id, price);
    log.info("priceOf took {} ns", System.nanoTime() - start);
    return price;
}
```

El método hace tres cosas, y dos de ellas se repetirán en cada servicio. Cambiar la política de caché obliga a tocar todos. La salida es sacar esas preocupaciones a un sustituto que se coloque delante.

7.2 Proxy en Java puro

El Proxy tiene la misma interfaz que el objeto real (el *sujeto*) y lo guarda dentro. Para el cliente es indistinguible; por dentro decide qué hacer con cada llamada. Se usa en tres sabores clásicos[7, 21].

Proxy de caché. Delega la primera vez y devuelve lo guardado después:

CachingTariffService.java

```
package shoplab.ejemplos;

import java.math.BigDecimal;
import java.util.HashMap;
import java.util.Map;

interface TariffService {
    BigDecimal tariff(String zone);
}

class CachingTariffService implements TariffService {

    private final TariffService delegate;
    private final Map<String, BigDecimal> cache = new HashMap<>();

    CachingTariffService(TariffService delegate) {
        this.delegate = delegate;
    }

    @Override
    public BigDecimal tariff(String zone) {
        BigDecimal known = cache.get(zone);
        if (known != null) {
            return known;
        }
        BigDecimal fresh = delegate.tariff(zone);
        cache.put(zone, fresh);
        return fresh;
    }
}
```

El cliente recibe un `TariffService` y no sabe si es el real o el proxy.

Proxy de protección. Decide si la llamada se permite:

GuardedAdmin.java

```java
package shoplab.ejemplos;

interface AdminOperations {
    void deleteProduct(String id);
}

class GuardedAdmin implements AdminOperations {

    private final AdminOperations delegate;
    private final boolean admin;

    GuardedAdmin(AdminOperations delegate, boolean admin) {
        this.delegate = delegate;
        this.admin = admin;
    }

    @Override
    public void deleteProduct(String id) {
        if (!admin) {
            throw new SecurityException("admin role required");
        }
        delegate.deleteProduct(id);
    }
}
```

Proxy virtual. Crea el objeto caro solo cuando se usa por primera vez:

LazyReport.java

```java
package shoplab.ejemplos;

import java.util.function.Supplier;

interface Report {
    String body();
}

class LazyReport implements Report {

    private final Supplier<Report> factory;
    private Report real;                    // aún no existe

    LazyReport(Supplier<Report> factory) {
        this.factory = factory;
    }

    @Override
    public String body() {
        if (real == null) {
            real = factory.get();           // se crea en la primera llamada
        }
        return real.body();
    }
}
```

Parecido a Decorator y a Adapter

El Proxy, el Decorator del capítulo 6 y el Adapter del capítulo 5 envuelven un objeto. Se distinguen por la intención: el Adapter cambia la interfaz, el Decorator *añade* responsabilidades y el Proxy *controla el acceso* con la misma interfaz y suele decidir él cuándo crear o llamar al sujeto[7, 9].

7.3 Proxy dinámico del JDK

Escribir un proxy a mano por interfaz es repetitivo: si quieres medir tiempos en veinte interfaces, son veinte clases casi iguales. El JDK trae `java.lang.reflect.Proxy`, que fabrica en tiempo de ejecución una clase que implementa las interfaces que le digas y reenvía cada llamada a un único `InvocationHandler`[17].

TimingProxyDemo.java

```
package shoplab.ejemplos;

import java.lang.reflect.InvocationHandler;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
import java.lang.reflect.Proxy;

interface Greeter {
    String greet(String name);
}

public class TimingProxyDemo {

    public static void main(String[] args) {
        Greeter real = name -> "Hello, " + name;

        InvocationHandler handler = (Object proxy, Method method,
                                     Object[] arguments) -> {
            long start = System.nanoTime();
            try {
                return method.invoke(real, arguments);
            } catch (InvocationTargetException e) {
                throw e.getTargetException();   // la excepción real
            } finally {
                System.out.println(method.getName() + " took "
                        + (System.nanoTime() - start) + " ns");
            }
        };

        Greeter proxy = (Greeter) Proxy.newProxyInstance(
                Greeter.class.getClassLoader(),
                new Class<?>[] {Greeter.class},
                handler);

        System.out.println(proxy.greet("Ana"));
    }
}
```

`Proxy.newProxyInstance` recibe el cargador de clases, la lista de interfaces y el manejador, y

devuelve un objeto que puedes convertir a cualquiera de esas interfaces. Cada llamada llega a `invoke` con el método y sus argumentos; tú decides si lo ejecutas con `method.invoke` y qué haces antes y después. Error típico: olvidar desenvolver `InvocationTargetException`; sin ello el cliente recibiría esa excepción envoltorio en lugar de la que lanzó el método. Limitación: el proxy del JDK solo funciona con *interfaces*, no con clases.

7.4 Instalar el starter de AOP

El proxy dinámico del JDK te da la idea, pero Spring ya tiene un mecanismo completo para aplicarlo a muchos beans a la vez: la programación orientada a aspectos (AOP)[26]. Necesita una dependencia que el proyecto aún no tiene.

☞ Starter de AOP de Spring Boot 4.1

En Boot 3 la dependencia se llamaba `spring-boot-starter-aop`. Comprobado el 2026-10-08 en Maven Central, ese artefacto se quedó en `4.0.0-M2`: en Boot 4 se sustituye por `spring-boot-starter-aspectj`, publicado en la versión 4.1.1 con la descripción «Starter for using aspect-oriented programming with AspectJ». La referencia de Boot explica que, si AspectJ está en el classpath, la autoconfiguración activa el soporte de aspectos y no hace falta `@EnableAspectJAutoProxy`; y que por defecto se usan proxies CGLIB[24].
Añade esta línea al bloque `dependencies` de `build.gradle` (la versión la pone el BOM de Boot):

`build.gradle (añadir)`

```
implementation 'org.springframework.boot:spring-boot-starter-aspectj'
```

Comprueba que resuelve con `./gradlew dependencies -configuration compileClasspath`: debe aparecer `org.aspectj:aspectjweaver`. Si tu IDE no ve las anotaciones, recarga el proyecto Gradle.

7.5 Y en Spring: `@Aspect`, `@Around` y pointcuts

Un *aspecto* es una clase con `@Aspect` (y `@Component`, para que Spring la registre) que agrupa una preocupación transversal. Su *consejo* (*advice*) es un método anotado con `@Around`, que recibe un `ProceedingJoinPoint`: tú decides cuándo llamar a `proceed()`, que ejecuta el método real. El *pointcut* dice a qué métodos se aplica[26].

La forma más usada de pointcut es `execution(...)`, con este esquema:

```
execution(modificador? tipo-retorno paquete.Clase.método(parámetros))
```

y comodines: `*` sustituye un nombre o un tipo, y `..` en los parámetros significa «cualquier cantidad». Ejemplos:

- `execution(* shoplab.*Service.*(..))`: cualquier método de las clases del paquete `shoplab` cuyo nombre acaba en `Service`.
- `execution(public * shoplab.OrderService.create(..))`: solo `create`.
- `execution(* shoplab.*Controller.*(..))`: todos los controladores.

El aspecto de tiempos que usará el hito:

TimingAspect.java

```java
package shoplab;

import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;

@Aspect
@Component
public class TimingAspect {

    private static final Logger log =
            LoggerFactory.getLogger(TimingAspect.class);

    @Around("execution(* shoplab.*Service.*(..))")
    public Object time(ProceedingJoinPoint joinPoint) throws Throwable {
        long start = System.nanoTime();
        try {
            return joinPoint.proceed();
        } finally {
            double ms = (System.nanoTime() - start) / 1_000_000.0;
            log.info("[TIMING] {} took {} ms",
                    joinPoint.getSignature().toShortString(),
                    String.format("%.2f", ms));
        }
    }
}
```

Spring detecta el aspecto y envuelve en un proxy cada bean cuyos métodos encajan con el pointcut. Para ti el bean sigue llamándose igual: el controlador que pedía un `OrderService` recibe, sin saberlo, el proxy. Ese proxy es un proxy JDK si el bean implementa interfaces y se configura así, o una subclase generada con CGLIB; Boot usa CGLIB por defecto, así que los beans que se advierten no pueden ser `final`.

Autoinvocación: la llamada interna no pasa por el proxy

El proxy solo ve las llamadas que le llegan desde fuera. Cuando un método del bean llama a otro método *del mismo bean* (`this.otro()`), la llamada ocurre ya dentro del objeto real, sin pasar por el proxy, y el aspecto no se ejecuta. La documentación de Spring lo describe con un ejemplo igual de sencillo y propone, entre otras vías, refactorizar para que la llamada cruce el límite de un bean; el tejido de AspectJ en compilación o carga no tiene este problema[26]. Lo verás en el ejercicio 7.5.

Lo que no debe hacer un aspecto: contener lógica de negocio. Si quitas el aspecto, la aplicación debe seguir funcionando igual, solo sin la medición, auditoría o caché.

7.6 El olor: miles de objetos idénticos

El catálogo de IVA es pequeño: un tipo general y uno reducido por país. Pero si cada producto de cada pedido crea su propio `new TaxRate("ES", "STANDARD", 0.21)`, en memoria hay miles de copias del mismo valor:

Olor

```
for (OrderLine line : order.getLines()) {
    // un objeto nuevo por línea, todos iguales
    TaxRate rate = new TaxRate("ES", "STANDARD", new BigDecimal("0.21"));
    ...
}
```

7.7 Flyweight: compartir lo que no cambia

El Flyweight (peso mosca) comparte objetos inmutables en lugar de crear uno por uso. Distingue dos clases de estado[7, 10]:

- *Intrínseco*: lo que es igual para todos los usos y vive dentro del objeto compartido (región, tipo y porcentaje de un IVA).
- *Extrínseco*: lo que cambia en cada uso y se pasa desde fuera (el producto concreto al que se aplica).

Una fábrica guarda los objetos ya creados y devuelve siempre el mismo para la misma clave. En Java, `ConcurrentHashMap.computeIfAbsent` lo hace de forma atómica, también con varios hilos[17]:

TaxRate.java y TaxRateFactory.java

```
package shoplab;

import java.math.BigDecimal;
import java.util.Map;
import java.util.Objects;
import java.util.concurrent.ConcurrentHashMap;
import org.springframework.stereotype.Component;

public record TaxRate(String region, String type, BigDecimal rate) { }

// ----- TaxRateFactory.java -----
@Component
public class TaxRateFactory {

    // valores didácticos
    private static final Map<String, BigDecimal> RATES = Map.of(
            "ES|STANDARD", new BigDecimal("0.21"),
            "ES|REDUCED", new BigDecimal("0.10"),
            "FR|STANDARD", new BigDecimal("0.20"),
            "FR|REDUCED", new BigDecimal("0.055"));

    private final Map<String, TaxRate> cache = new ConcurrentHashMap<>();

```

```
24      public TaxRate get(String region, String type) {
25          Objects.requireNonNull(region);
26          Objects.requireNonNull(type);
27          return cache.computeIfAbsent(region + "|" + type, key -> {
28              BigDecimal rate = RATES.get(key);
29              if (rate == null) {
30                  throw new IllegalArgumentException(
31                          "unknown tax rate: " + key);
32              }
33              return new TaxRate(region, type, rate);
34          });
35      }
36  }
```

(En el fichero real, `TaxRate` y `TaxRateFactory` van en archivos distintos.) Dos llamadas con la misma clave devuelven la *misma instancia*: `get("ES", "STANDARD") == get("ES", "STANDARD")`. Si la clave no existe, la excepción sale de `computeIfAbsent` sin guardar nada.

7.8 Y en Spring: la fábrica como bean singleton

Un Flyweight necesita *una sola* fábrica, o cada una tendría su propia caché. En Spring no hace falta escribir un Singleton a mano: ya viste en el capítulo 2 que un bean tiene ámbito singleton por defecto[27]. Basta con `@Component` en `TaxRateFactory` y pedirla por constructor donde haga falta, como harán `SpainRegionFactory` y `FranceRegionFactory`: dejan de llevar la tasa fija y preguntan a la fábrica con el tipo adecuado al producto.

7.9 En el proyecto: Hito 5b

Hito 5b: caché del catálogo, tiempos y tipos de IVA compartidos

Partes del hito 5a. Se añade el starter de AOP. `CachingCatalogService` es un proxy de caché manual sobre `CatalogService`: lleva `@Primary` y guarda el árbol durante 30 segundos; la implementación real (`DefaultCatalogService`) pasa a llamarse bean `"default"` y se pide con `@Qualifier("default")`. `TimingAspect` escribe en el log lo que tarda cada `*Service`. `TaxRate` y `TaxRateFactory` (`TaxRate get(String region, String type)`) se comparten, y `SpainRegionFactory` y `FranceRegionFactory` reciben la fábrica por constructor: `rate(product)` pide el tipo `"REDUCED"` si la categoría es `books` y `"STANDARD"` en otro caso. El presupuesto de un producto que no es de `books` no cambia; el de un libro baja su IVA. Está en el ejercicio 7.6.

7.10 Ejercicios

Ejercicio 7.1: Un proxy de caché para tipos de cambio ★★☆☆☆

Trabaja en clases propias del paquete `shoplab.ejercicios` (no entran en el proyecto).

1. Crea `interface ExchangeRateService` con `BigDecimal rate(String currency)`.
2. Crea `SlowExchangeRateService`: duerme 300 ms con `Thread.sleep` y devuelve 1,08 para `USD` y 0,85 para `GBP`; otra moneda lanza `IllegalArgumentException`.

3. Crea `CachingExchangeRateService`, que implementa la misma interfaz, recibe el servicio real y guarda en un `Map` los resultados ya consultados. Añade los contadores `hits()` y `misses()`.
4. Crea `ExchangeRateCheck` con un `main` que pida `USD` tres veces y `GBP` una, y escriba el tiempo de cada llamada.

Está bien si la primera petición de cada moneda tarda unos 300 ms y las demás casi 0, y `hits()` es 2 y `misses()` es 2.

Pista: *Es el mismo esquema de `CachingTariffService` de la teoría: comprueba la caché, delega si no está y guarda.*

Guiado ▪ solución en la pág. 218

Ejercicio 7.2: Un proxy dinámico que mide llamadas ★★★☆☆

Trabaja en clases propias de `shoplab.ejercicios`. Define una interfaz propia `ReportService` con `String generate(String name)` y `int pages(String name)`, y una implementación sencilla. Con `java.lang.reflect.Proxy` y un `InvocationHandler`, crea un proxy que escriba con SLF4J el nombre del método y su duración en microsegundos en cada llamada, y que deje pasar las excepciones del método real tal cual. Pruébalo con un `ApplicationRunner` en un `@Component`. Está bien si al arrancar con `./gradlew bootRun` se ven dos líneas `[CALL]` (una por método) y, si `generate("")` lanza `IllegalArgumentException`, el llamante recibe esa excepción y no una `InvocationTargetException`.

Pista: *El manejador es una clase con un campo `Object target`; una fábrica estática genérica puede devolver el proxy ya convertido a la interfaz.*

Práctica ▪ solución en la pág. 221

Ejercicio 7.3: Auditar los controladores con un aspecto ★★★☆☆

Trabaja *sobre una copia* del proyecto en el hito 5b (añades un bean, el aspecto). Escribe un `@Aspect` con `@Around` que audite las llamadas a los controladores: nombre del método, argumentos y resultado en el log, y cuántas veces se ha llamado a cada endpoint. Es una preocupación transversal distinta de los tiempos de `TimingAspect`. Está bien si, tras llamar con `curl` varias veces a `GET /products`, el log muestra líneas `[AUDIT]` con el contador subiendo (`calls=1`, `calls=2`, …) y los argumentos y el resultado de cada llamada.

Pista: *Pointcut sobre el paquete de controladores, `getArgs()` del `ProceedingJoinPoint` y un `ConcurrentHashMap` de contadores.*

Práctica ▪ solución en la pág. 223

Ejercicio 7.4: Una instancia por moneda ★★★☆☆

Trabaja en clases propias de `shoplab.ejercicios`. Crea un record `CurrencyFormat(String code, String symbol, int decimals)` con un método estático `of(String code)` que devuelva *siempre la misma instancia* para el mismo código (`EUR`, `USD` y `JPY`; otro código lanza `IllegalArgumentException`), también si lo piden varios hilos a la vez. Añade `String format(BigDecimal amount)`, que escriba el importe con sus decimales y el símbolo. Comprueba con 8 hilos y 100 llamadas que todas las referencias son iguales con `==`, y explica qué parte es el estado intrínseco y cuál el extrínseco. Está bien si la comprobación imprime `same instance: true` y `CurrencyFormat.of("EUR").format(new BigDecimal("3.5"))` da `3.50 €`.

Pista: *Es el mismo mecanismo que* `TaxRateFactory.get(region, type)`*:* `computeIfAbsent` *sobre un* `ConcurrentHashMap`*.*

Práctica ▪ solución en la pág. 225

Ejercicio 7.5: El contador que no sube ★★★☆☆

Trabaja *sobre una copia* del proyecto (añades beans y un aspecto). Tienes este servicio y este aspecto, que cuenta las llamadas a los métodos de las clases cuyo nombre empieza por `Invoice` en `shoplab.ejercicios`:

InvoiceService.java y CallCountAspect.java

```java
package shoplab.ejercicios;

import java.math.BigDecimal;
import org.springframework.stereotype.Service;

@Service
public class InvoiceService {

    public String issue(String customer) {
        BigDecimal total = computeTotal(customer);   // llamada interna
        return customer + ":" + total;
    }

    public BigDecimal computeTotal(String customer) {
        return new BigDecimal("100.00");
    }
}

// ----- CallCountAspect.java -----
package shoplab.ejercicios;

import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.springframework.stereotype.Component;

@Aspect
@Component
public class CallCountAspect {

    private final Map<String, Integer> counts = new ConcurrentHashMap<>();

    @Around("execution(* shoplab.ejercicios.Invoice*.*(..))")
    public Object count(ProceedingJoinPoint joinPoint) throws Throwable {
        counts.merge(joinPoint.getSignature().getName(), 1, Integer::sum);
        return joinPoint.proceed();
    }

    public Map<String, Integer> counts() {
        return Map.copyOf(counts);
    }
}
```

Un `ApplicationRunner` llama dos veces a `issue("ana")` y luego imprime `counts()`. Sale `{issue=2}`, pero esperabas también `computeTotal=2`. Explica por qué falta y arréglalo para que salga `{issue=2, computeTotal=2}` (en cualquier orden). Está bien si el contador cuenta las dos y la explicación menciona el proxy.

Pista: *Piensa en quién recibe cada llamada: ¿el proxy o el objeto real?*

Depuración ▪ solución en la pág. 227

Ejercicio 7.6: Hito 5b: caché, tiempos e IVA compartido ★★★★☆

Parte del hito 5a. Implementa el hito 5b completo.

1. Añade el starter `spring-boot-starter-aspectj` a `build.gradle`.
2. Anota `DefaultCatalogService` con `@Service("default")`.
3. Crea `CachingCatalogService` (`@Service @Primary`, 30 segundos).
4. Crea `TimingAspect`.
5. Crea `TaxRate` y `TaxRateFactory`.
6. Cambia `SpainRegionFactory` y `FranceRegionFactory` para que reciban `TaxRateFactory` y calculen el tipo según la categoría.

Está bien si: dos llamadas seguidas a `GET /catalog` escriben `[CACHE] miss` la primera y `[CACHE] hit` la segunda, y cada llamada va acompañada de líneas `[TIMING]`; el `quote` de `p1` (49,90 EUR) sigue dando `tax 10.48` y `total 65.28` en `ES`; y un libro de 20,00 EUR de la categoría `books` da en `ES` `taxRate 0.10`, `tax 2.00` y `total 26.90` (neto más IVA pasa de 24,20 a 22,00; con el envío de 4,90 el total pasa de 29,10 a 26,90).

Pista: *El constructor de las fábricas regionales cambia: si tenías un test que hacía `new SpainRegionFactory()`, ahora necesita `new SpainRegionFactory(new TaxRateFactory())`.*

Proyecto ▪ solución en la pág. 229

7.11 Resumen

Resumen

- Un Proxy tiene la misma interfaz que su sujeto y controla el acceso: de caché, de protección o virtual.
- `java.lang.reflect.Proxy` fabrica proxies en ejecución para interfaces, con un único `InvocationHandler`; desenvuelve `InvocationTargetException`.
- En Boot 4.1 el starter de AOP es `spring-boot-starter-aspectj`, no `spring-boot-starter-aop`.
- Un `@Aspect` con `@Around` y un pointcut `execution(...)` aplica una preocupación transversal a muchos beans sin tocarlos.
- Una llamada de un método a otro de la misma clase no pasa por el proxy: el aspecto no se ejecuta; mueve el método a otro bean.
- Un Flyweight comparte objetos inmutables (estado intrínseco) y recibe el estado extrínseco desde fuera; `computeIfAbsent` lo hace seguro con hilos.
- En Spring, la fábrica del Flyweight es un bean singleton que se inyecta donde hace falta.

Soluciones de los ejercicios

Solución del ejercicio 7.1

enunciado en la pág. 213

ExchangeRateService.java, SlowExchangeRateService.java

```
package shoplab.ejercicios;

import java.math.BigDecimal;

public interface ExchangeRateService {

    BigDecimal rate(String currency);
}

// ----- SlowExchangeRateService.java -----
package shoplab.ejercicios;

import java.math.BigDecimal;
import java.util.Map;

public class SlowExchangeRateService implements ExchangeRateService {

    private static final Map<String, BigDecimal> RATES = Map.of(
            "USD", new BigDecimal("1.08"),
            "GBP", new BigDecimal("0.85"));

    @Override
    public BigDecimal rate(String currency) {
        try {
            Thread.sleep(300);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            throw new IllegalStateException("interrupted", e);
        }
        BigDecimal rate = RATES.get(currency);
        if (rate == null) {
            throw new IllegalArgumentException(
                    "unknown currency: " + currency);
        }
        return rate;
    }
}
```

CachingExchangeRateService.java y ExchangeRateCheck.java

```java
package shoplab.ejercicios;

import java.math.BigDecimal;
import java.util.HashMap;
import java.util.Map;

public class CachingExchangeRateService implements ExchangeRateService {

    private final ExchangeRateService delegate;
    private final Map<String, BigDecimal> cache = new HashMap<>();
    private int hits;
    private int misses;

    public CachingExchangeRateService(ExchangeRateService delegate) {
        this.delegate = delegate;
    }

    @Override
    public BigDecimal rate(String currency) {
        BigDecimal known = cache.get(currency);
        if (known != null) {
            hits++;
            return known;
        }
        misses++;
        BigDecimal fresh = delegate.rate(currency);
        cache.put(currency, fresh);
        return fresh;
    }

    public int hits() {
        return hits;
    }

    public int misses() {
        return misses;
    }
}

// ----- ExchangeRateCheck.java -----
package shoplab.ejercicios;

public class ExchangeRateCheck {

    public static void main(String[] args) {
        CachingExchangeRateService service = new CachingExchangeRateService
    ↪ (
                new SlowExchangeRateService());
        String[] requests = {"USD", "USD", "USD", "GBP"};
        for (String currency : requests) {
            long start = System.nanoTime();
            var rate = service.rate(currency);
            long ms = (System.nanoTime() - start) / 1_000_000;
            System.out.println(currency + " = " + rate + " in " + ms + " ms
    ↪ ");
        }
        System.out.println("hits=" + service.hits()
                + " misses=" + service.misses());
    }
}
```

Se ejecuta con `./gradlew compileJava` y `java -cp build/classes/java/main shoplab.ejercicios.ExchangeRateCheck`. Funciona porque el proxy implementa la misma interfaz: `ExchangeRateCheck` usaría igual el servicio real. Salida esperada: `USD` tarda 300 ms la primera vez y 0 después; `hits=2 misses=2`. Errores típicos: guardar en la caché antes de delegar (si el servicio real lanza, queda un valor falso) y usar este `HashMap` desde varios hilos: no es seguro; para eso usarías `ConcurrentHashMap`.

Solución del ejercicio 7.2 enunciado en la pág. 214

ReportService.java, SimpleReportService.java, TimingHandler.java

```
package shoplab.ejercicios;

public interface ReportService {

    String generate(String name);

    int pages(String name);
}

// ----- SimpleReportService.java -----
package shoplab.ejercicios;

public class SimpleReportService implements ReportService {

    @Override
    public String generate(String name) {
        if (name.isBlank()) {
            throw new IllegalArgumentException("name is required");
        }
        return "Report " + name;
    }

    @Override
    public int pages(String name) {
        return name.length();
    }
}

// ----- TimingHandler.java -----
package shoplab.ejercicios;

import java.lang.reflect.InvocationHandler;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
import java.lang.reflect.Proxy;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class TimingHandler implements InvocationHandler {

    private static final Logger log =
            LoggerFactory.getLogger(TimingHandler.class);

    private final Object target;

    private TimingHandler(Object target) {
        this.target = target;
    }

    public static <T> T wrap(Class<T> type, T target) {
        Object proxy = Proxy.newProxyInstance(
                type.getClassLoader(),
                new Class<?>[] {type},
                new TimingHandler(target));
        return type.cast(proxy);
    }

    @Override
    public Object invoke(Object proxy, Method method, Object[] args)
            throws Throwable {
        long start = System.nanoTime();
        try {
```

ReportCheck.java

```java
package shoplab.ejercicios;

import org.springframework.boot.ApplicationArguments;
import org.springframework.boot.ApplicationRunner;
import org.springframework.stereotype.Component;

@Component
public class ReportCheck implements ApplicationRunner {

    @Override
    public void run(ApplicationArguments args) {
        ReportService service = TimingHandler.wrap(
                ReportService.class, new SimpleReportService());
        System.out.println(service.generate("sales"));
        System.out.println(service.pages("sales"));
        try {
            service.generate("");
        } catch (IllegalArgumentException e) {
            System.out.println("caught " + e.getMessage());
        }
    }
}
```

Con `./gradlew bootRun` salen las líneas `[CALL] generate` y `[CALL] pages` (más otra `generate` de la llamada que falla) y `caught name is required`. Funciona porque `newProxyInstance` implementa `ReportService` y manda todo a `invoke`; al desenvolver `InvocationTargetException` el llamante recibe la excepción original. Error típico: devolver `null` en lugar del resultado de `method.invoke`, o dejar pasar la `InvocationTargetException` (el `catch (IllegalArgumentException)` no la capturaría). Al terminar, quita `@Component` de `ReportCheck`, para que no se ejecute en cada arranque.

Solución del ejercicio 7.3 enunciado en la pág. 214

En tu copia, añade el aspecto:

AuditAspect.java

```java
package shoplab;

import java.util.Arrays;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;

@Aspect
@Component
public class AuditAspect {

    private static final Logger log =
            LoggerFactory.getLogger(AuditAspect.class);

    private final Map<String, Integer> calls = new ConcurrentHashMap<>();

    @Around("execution(* shoplab.*Controller.*(..))")
    public Object audit(ProceedingJoinPoint joinPoint) throws Throwable {
        String endpoint = joinPoint.getSignature().getDeclaringType()
                .getSimpleName() + "." + joinPoint.getSignature().getName();
        int count = calls.merge(endpoint, 1, Integer::sum);
        String args = Arrays.toString(joinPoint.getArgs());
        try {
            Object result = joinPoint.proceed();
            log.info("[AUDIT] {} args={} result={} calls={}",
                    endpoint, args, result, count);
            return result;
        } catch (Throwable error) {
            log.warn("[AUDIT] {} args={} failed={} calls={}",
                    endpoint, args, error.toString(), count);
            throw error;
        }
    }
}
```

Prueba: `curl -i localhost:8080/products` tres veces; el log muestra `[AUDIT] ProductController.findAll args=[] ... calls=1`, luego `calls=2` y `calls=3`. Funciona porque el pointcut encaja con los métodos de todos los controladores del paquete `shoplab` y el proxy los intercepta; el contador es un `ConcurrentHashMap` porque las peticiones llegan en hilos distintos. (Si tu método se llama de otra forma, mira el nombre real en `ProductController`.) Errores típicos: patrón con `shoplab.Controller.*` (no existe esa clase; hace falta el comodín `*Controller`) y olvidar relanzar la excepción, con lo que el cliente recibiría una respuesta 200 con `null`.

Solución del ejercicio 7.4 enunciado en la pág. 214

CurrencyFormat.java y CurrencyFormatCheck.java

```java
package shoplab.ejercicios;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.Map;
import java.util.Objects;
import java.util.concurrent.ConcurrentHashMap;

public record CurrencyFormat(String code, String symbol, int decimals) {

    private static final Map<String, CurrencyFormat> CACHE =
            new ConcurrentHashMap<>();

    private static final Map<String, String> SYMBOLS = Map.of(
            "EUR", "€", "USD", "$", "JPY", "yen");

    public static CurrencyFormat of(String code) {
        Objects.requireNonNull(code);
        return CACHE.computeIfAbsent(code, c -> {
            String symbol = SYMBOLS.get(c);
            if (symbol == null) {
                throw new IllegalArgumentException(
                        "unknown currency: " + c);
            }
            return new CurrencyFormat(c, symbol, "JPY".equals(c) ? 0 : 2);
        });
    }

    public String format(BigDecimal amount) {
        return amount.setScale(decimals, RoundingMode.HALF_UP)
                .toPlainString() + " " + symbol;
    }
}

// ----- CurrencyFormatCheck.java -----
package shoplab.ejercicios;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

public class CurrencyFormatCheck {

    public static void main(String[] args) throws Exception {
        ExecutorService pool = Executors.newFixedThreadPool(8);
        List<Future<CurrencyFormat>> results = new ArrayList<>();
        for (int i = 0; i < 100; i++) {
            results.add(pool.submit(() -> CurrencyFormat.of("EUR")));
        }
        CurrencyFormat first = results.get(0).get();
        boolean same = true;
        for (Future<CurrencyFormat> result : results) {
            same &= result.get() == first;
        }
        pool.shutdown();
        System.out.println("same instance: " + same);
        System.out.println(first.format(new BigDecimal("3.5")));
    }
}
```

Salida: `same instance: true` y `3.50 €`. `computeIfAbsent` sobre un `ConcurrentHashMap` es atómico por clave: solo un hilo crea la instancia. Estado *intrínseco*: `code`, `symbol` y `decimals`, iguales para todos los usos y guardados en el objeto compartido. Estado *extrínseco*: el importe, que llega por parámetro a `format` y no se guarda. Aviso honesto: un record tiene constructor público, así que nada impide un `new CurrencyFormat(...)`; el Flyweight funciona por disciplina (todos usan `of`). Errores típicos: comprobar con `containsKey` y luego `put` (dos hilos pueden crear dos instancias) y usar un `HashMap` normal.

Solución del ejercicio 7.5 enunciado en la pág. 215

Por qué. Spring no usa tu `InvoiceService`: inyecta un proxy que lo envuelve, y el aspecto vive en el proxy. El `ApplicationRunner` llama a `issue` a través del proxy, así que se cuenta. Pero dentro de `issue` la llamada `computeTotal(customer)` es `this.computeTotal(...)`: ocurre ya en el objeto real, no cruza el proxy y el aspecto no la ve[26].
Arreglo. Mueve `computeTotal` a otro bean; así la llamada pasa por su propio proxy:

InvoiceCalculator.java, InvoiceService.java y CallCountCheck.java

```java
package shoplab.ejercicios;

import java.math.BigDecimal;
import org.springframework.stereotype.Service;

@Service
public class InvoiceCalculator {

    public BigDecimal computeTotal(String customer) {
        return new BigDecimal("100.00");
    }
}

// ----- InvoiceService.java -----
package shoplab.ejercicios;

import java.math.BigDecimal;
import org.springframework.stereotype.Service;

@Service
public class InvoiceService {

    private final InvoiceCalculator calculator;

    public InvoiceService(InvoiceCalculator calculator) {
        this.calculator = calculator;
    }

    public String issue(String customer) {
        BigDecimal total = calculator.computeTotal(customer);
        return customer + ":" + total;
    }
}

// ----- CallCountCheck.java -----
package shoplab.ejercicios;

import org.springframework.boot.ApplicationArguments;
import org.springframework.boot.ApplicationRunner;
import org.springframework.stereotype.Component;

@Component
public class CallCountCheck implements ApplicationRunner {

    private final InvoiceService service;
    private final CallCountAspect aspect;

    public CallCountCheck(InvoiceService service, CallCountAspect aspect) {
        this.service = service;
        this.aspect = aspect;
    }

    @Override
    public void run(ApplicationArguments args) {
        service.issue("ana");
        service.issue("ana");
        System.out.println(aspect.counts());
    }
}
```

El aspecto no cambia: su pointcut `Invoice*` ya encaja con `InvoiceCalculator`. Ahora sale `{issue=2, computeTotal=2}`. Alternativas peores: inyectar el propio bean en sí mismo (funciona, pero es un olor) o pasar a tejido AspectJ en compilación, que no usa proxy. Error típico: ampliar el pointcut o añadir anotaciones de estilo `@Transactional` (este curso no las usa) esperando que arreglen la autoinvocación: no lo hacen. Quita `@Component` de `CallCountCheck` al terminar.

Solución del ejercicio 7.6 enunciado en la pág. 217

1. Dependencia. En `build.gradle`, dentro de `dependencies`:

`build.gradle (añadir)`

```
implementation 'org.springframework.boot:spring-boot-starter-aspectj'
```

2. Implementación real con nombre. Solo cambia la anotación de `DefaultCatalogService` (antes `@Service`); se da la clase completa:

DefaultCatalogService.java

```
package shoplab;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.Set;
import java.util.TreeMap;
import java.util.stream.Collectors;
import org.springframework.stereotype.Service;

@Service("default")
public class DefaultCatalogService implements CatalogService {

    private final ProductRepository repository;
    private final CatalogPacks packs;

    public DefaultCatalogService(ProductRepository repository,
                                 CatalogPacks packs) {
        this.repository = repository;
        this.packs = packs;
    }

    @Override
    public CatalogNode tree() {
        Map<String, List<Product>> byCategory = repository.findAll()
                .stream()
                .sorted(Comparator.comparing(Product::id))
                .collect(Collectors.groupingBy(Product::category,
                        TreeMap::new, Collectors.toList()));
        List<CatalogNode> categories = new ArrayList<>();
        for (Map.Entry<String, List<Product>> entry
                : byCategory.entrySet()) {
            categories.add(category(entry.getKey(), entry.getValue()));
        }
        return new Category("catalog", categories);
    }

    private Category category(String name, List<Product> inCategory) {
        List<CatalogNode> children = new ArrayList<>();
        Set<String> packed = new HashSet<>();
        for (PackSpec spec : packs.packs()) {
            if (!spec.category().equals(name)) {
                continue;
            }
            List<Product> items = spec.productIds().stream()
                    .map(repository::findById)
                    .flatMap(Optional::stream)
                    .toList();
            if (items.size() == spec.productIds().size()) {
                children.add(new Pack(spec.name(), items,
                        spec.discount()));
                items.forEach(p -> packed.add(p.id()));
            }
        }
        inCategory.stream()
                .filter(p -> !packed.contains(p.id()))
                .forEach(children::add);
        return new Category(name, children);
    }
```

3. Proxy de caché.

CachingCatalogService.java

```java
package shoplab;

import java.time.Duration;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.context.annotation.Primary;
import org.springframework.stereotype.Service;

@Service
@Primary
public class CachingCatalogService implements CatalogService {

    private static final Logger log =
            LoggerFactory.getLogger(CachingCatalogService.class);
    private static final long TTL_NANOS = Duration.ofSeconds(30).toNanos();

    private final CatalogService delegate;
    private CatalogNode cached;
    private long loadedAt;

    public CachingCatalogService(
            @Qualifier("default") CatalogService delegate) {
        this.delegate = delegate;
    }

    @Override
    public synchronized CatalogNode tree() {
        long now = System.nanoTime();
        if (cached != null && now - loadedAt < TTL_NANOS) {
            log.info("[CACHE] hit: catalog tree");
            return cached;
        }
        log.info("[CACHE] miss: loading catalog tree");
        cached = delegate.tree();
        loadedAt = now;
        return cached;
    }
}
```

4. Aspecto de tiempos. Crea `TimingAspect.java`:

TimingAspect.java

```java
package shoplab;

import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;

@Aspect
@Component
public class TimingAspect {

    private static final Logger log =
            LoggerFactory.getLogger(TimingAspect.class);

    @Around("execution(* shoplab.*Service.*(..))")
    public Object time(ProceedingJoinPoint joinPoint) throws Throwable {
        long start = System.nanoTime();
        try {
            return joinPoint.proceed();
        } finally {
            double ms = (System.nanoTime() - start) / 1_000_000.0;
            log.info("[TIMING] {} took {} ms",
                    joinPoint.getSignature().toShortString(),
                    String.format("%.2f", ms));
        }
    }
}
```

5. Flyweight.

TaxRate.java y TaxRateFactory.java

```java
package shoplab;

import java.math.BigDecimal;

public record TaxRate(String region, String type, BigDecimal rate) { }

// ----- TaxRateFactory.java -----
package shoplab;

import java.math.BigDecimal;
import java.util.Map;
import java.util.Objects;
import java.util.concurrent.ConcurrentHashMap;
import org.springframework.stereotype.Component;

@Component
public class TaxRateFactory {

    private static final Map<String, BigDecimal> RATES = Map.of(
            "ES|STANDARD", new BigDecimal("0.21"),
            "ES|REDUCED", new BigDecimal("0.10"),
            "FR|STANDARD", new BigDecimal("0.20"),
            "FR|REDUCED", new BigDecimal("0.055"));

    private final Map<String, TaxRate> cache = new ConcurrentHashMap<>();

    public TaxRate get(String region, String type) {
        Objects.requireNonNull(region);
        Objects.requireNonNull(type);
        return cache.computeIfAbsent(region + "|" + type, key -> {
            BigDecimal rate = RATES.get(key);
            if (rate == null) {
                throw new IllegalArgumentException(
                        "unknown tax rate: " + key);
            }
            return new TaxRate(region, type, rate);
        });
    }
}
```

6. Fábricas regionales. Sustituye ambas clases (el envío y la moneda no cambian; solo la política de IVA y el constructor):

SpainRegionFactory.java y FranceRegionFactory.java

```java
package shoplab;

import java.math.BigDecimal;
import org.springframework.stereotype.Component;

@Component("ES")
public class SpainRegionFactory implements RegionFactory {

    private static final BigDecimal FREE_FROM = new BigDecimal("50.00");
    private static final Money SHIPPING =
            new Money(new BigDecimal("4.90"), "EUR");

    private final TaxRateFactory taxRates;

    public SpainRegionFactory(TaxRateFactory taxRates) {
        this.taxRates = taxRates;
    }

    private static String typeOf(Product product) {
        return "books".equals(product.category()) ? "REDUCED" : "STANDARD";
    }

    @Override
    public TaxPolicy taxPolicy() {
        return product -> taxRates.get("ES", typeOf(product)).rate();
    }

    @Override
    public ShippingPolicy shippingPolicy() {
        return subtotal -> subtotal.amount().compareTo(FREE_FROM) >= 0
                ? Money.zero("EUR") : SHIPPING;
    }

    @Override
    public CurrencyPolicy currencyPolicy() {
        return () -> "EUR";
    }
}

// ----- FranceRegionFactory.java -----
package shoplab;

import java.math.BigDecimal;
import org.springframework.stereotype.Component;

@Component("FR")
public class FranceRegionFactory implements RegionFactory {

    private static final BigDecimal FREE_FROM = new BigDecimal("60.00");
    private static final Money SHIPPING =
            new Money(new BigDecimal("5.90"), "EUR");

    private final TaxRateFactory taxRates;

    public FranceRegionFactory(TaxRateFactory taxRates) {
        this.taxRates = taxRates;
    }

    private static String typeOf(Product product) {
        return "books".equals(product.category()) ? "REDUCED" : "STANDARD";
    }
```

Prueba. Arranca con `./gradlew bootRun`. Crea un libro y pide dos veces el catálogo y el presupuesto (URL entre comillas):

```
curl -i -H "Content-Type: application/json" -d '{"id":"b1","name":"Java Book
    ↪ ","price":{"amount":20.00,"currency":"EUR"},"kind":{"type":"digital"},"
    ↪ stock":5,"category":"books"}' localhost:8080/products
curl localhost:8080/catalog
curl localhost:8080/catalog
curl "localhost:8080/products/b1/quote?region=ES"
```

En el log, la primera llamada a `/catalog` da `[CACHE] miss`, un `[TIMING] DefaultCatalogService.tree() took ...` y un `[TIMING] CachingCatalogService.tree() took ...`; la segunda, `[CACHE] hit` y solo el `[TIMING]` de `CachingCatalogService`. El `quote` de `b1` da `taxRate 0.10`, `tax 2.00`, `shipping 4.90`, `total 26.90` (en `FR`: `0.055`, `1.10`, `5.90`, `27.00`); el de `p1` sigue en `65.28`.
Explicación: `@Primary` hace que el controlador reciba la caché y `@Qualifier("default")` que la caché reciba la implementación real; el controlador no cambia. Como la caché dura 30 segundos, un producto nuevo puede tardar en aparecer en `/catalog`. Errores típicos: olvidar el nombre `"default"` (dos candidatos y sin `@Qualifier`, o la caché se inyectaría a sí misma y el contexto no arranca), tener un test con `new SpainRegionFactory()` sin argumento, y poner `TimingAspect` en el paquete equivocado (el pointcut mira `shoplab.*Service`).

Parte IV

Patrones de comportamiento y cierre

CAPÍTULO 8

Strategy y Template Method

Objetivos del capítulo

- Reconocer el olor de los `if/else` por regla y sustituirlos por un Strategy, con clases y con lambdas.
- Inyectar todas las estrategias como una `List` de beans y elegir la mejor sin tocar el servicio al añadir una.
- Reconocer el olor de los procesos casi iguales y fijar su esqueleto con un Template Method.
- Leer `JdbcTemplate` como una plantilla con callbacks.
- Decidir entre Strategy y Template Method (composición o herencia) y defenderlo.
- Añadir a ShopLab descuentos intercambiables y un flujo de pago con esqueleto fijo.

Hasta ahora has envuelto, adaptado y compartido objetos. Ahora el problema es otro: un mismo trabajo se puede hacer de varias maneras (calcular un descuento) o se hace siempre con los mismos pasos, pero con detalles distintos (validar, autorizar, cobrar, avisar). Strategy intercambia el *algoritmo* entero; Template Method fija el *esqueleto* y deja huecos[7, 5]. Los dos resuelven el mismo olor, el código que varía mezclado con el que no, desde direcciones opuestas: uno por composición y el otro por herencia.

Java sin compilar

Como en los capítulos 3 a 7, el código Java está escrito con cuidado contra la documentación, pero no lo ha compilado ni ejecutado su autor. Si algo falla, `./gradlew compileJava` te dirá dónde. Usamos sin reexplicarlos lambdas, streams y `Comparator`, que repasaste en el capítulo 6.

8.1 El olor: un `if/else` por cada regla

Quieres descuentos en ShopLab: un 10 % de rebaja, «compra dos y llévate el tercero gratis», quizá más. Sin patrón, escribes lo mismo que ya viste en el ejercicio 1.5:

DiscountService.java (olor)

```
public Money discountFor(String rule, Order order, Money price) {
    if (rule.equals("TEN_PERCENT")) {
        return price.times(new BigDecimal("0.10"));
    } else if (rule.equals("BUY_TWO_GET_ONE")) {
        Money free = Money.zero(price.currency());
        for (OrderLine line : order.getLines()) {
            free = free.plus(line.unitPrice().times(line.quantity() / 3));
        }
        return free;
    }
```

```
    throw new IllegalArgumentException("unknown rule: " + rule);
}
```

Cada regla nueva obliga a editar este método (rompe el principio abierto/cerrado de Martin[14]), los algoritmos no se pueden probar por separado y la elección de regla está pegada a su cálculo. Kerievsky lo recoge entre sus refactorizaciones: sustituir la lógica condicional por una estrategia[11].

8.2 Strategy en Java puro

Strategy define una familia de algoritmos, encapsula cada uno detrás de la misma interfaz y los hace intercambiables. El *contexto* guarda una estrategia y le delega el trabajo[7, 5]. Un ejemplo con otro dominio, los puntos de fidelidad:

PointsRule.java, PerEuro.java, Loyalty.java

```
package shoplab.ejemplos;

@FunctionalInterface
public interface PointsRule {

    int points(Money spent);
}

// ----- PerEuro.java -----
package shoplab.ejemplos;

public class PerEuro implements PointsRule {

    @Override
    public int points(Money spent) {
        return spent.amount().intValue();   // un punto por euro entero
    }
}

// ----- Loyalty.java (el contexto) -----
package shoplab.ejemplos;

public class Loyalty {

    private PointsRule rule;

    public Loyalty(PointsRule rule) {
        this.rule = rule;
    }

    public void changeRule(PointsRule rule) {
        this.rule = rule;
    }

    public int earn(Money spent) {
        return rule.points(spent);
    }
}
```

El contexto no sabe qué regla usa y la puedes cambiar en ejecución:

LoyaltyDemo.java

```
package shoplab.ejemplos;

import java.math.BigDecimal;

public class LoyaltyDemo {

    public static void main(String[] args) {
        Money spent = new Money(new BigDecimal("49.90"), "EUR");
        Loyalty loyalty = new Loyalty(new PerEuro());
        System.out.println(loyalty.earn(spent));    // 49
        loyalty.changeRule(m -> 10);                 // regla simple: lambda
        System.out.println(loyalty.earn(spent));    // 10
    }
}
```

Como `PointsRule` tiene un solo método abstracto (es una interfaz funcional), un algoritmo corto cabe en una lambda; uno con estado o con varias piezas merece su clase. Es la misma idea que el `Comparator` de `List.sort`: un algoritmo de orden que pasas como argumento. Elige clase o lambda según la complejidad, no por costumbre[3].

Error típico

Dejar un `switch` o un `if` dentro de la «estrategia». Si tu clase `Discount` sigue preguntando «¿qué tipo soy?», no has hecho Strategy: has movido el olor de sitio. Cada variante es una clase (o lambda) y quien elige es el contexto o el contenedor.

8.3 Y en Spring: todas las estrategias como beans en una `List`

Si cada estrategia es un bean, Spring puede inyectarlas todas a la vez: un parámetro de constructor de tipo `List<T>` recibe todos los beans de tipo `T` (y `Map<String, T>`, como ya hiciste con los procesadores, los recibe indexados por nombre)[27]. El servicio no nombra ninguna estrategia concreta:

Strategy con List (esquema)

```
@Service
public class RuleService {

    private final List<PointsRule> rules;

    public RuleService(List<PointsRule> rules) {   // Spring las inyecta todas
        this.rules = rules;
    }

    public int best(Money spent) {
        return rules.stream()
                .mapToInt(rule -> rule.points(spent))
                .max()
                .orElse(0);
    }
```

```
16 }
```

Añadir una regla es crear una clase con `@Component`: el servicio no cambia. Eso es el principio abierto/cerrado funcionando. El orden de la lista no está garantizado salvo que anotes los beans con `@Order`; aquí da igual, porque `max` no depende del orden.

8.4 El olor: procesos casi iguales

El otro olor aparece cuando varias clases hacen *lo mismo en el mismo orden* y solo difiere un paso. Tu cobro tiene esa forma: comprobar el importe, comprobar las reglas del método de pago, cobrar y avisar. Con tarjeta y con transferencia son casi idénticos:

Dos flujos casi iguales (olor)

```
1  public PaymentResult payByCard(Order order, Money amount) {
2      if (amount.amount().signum() <= 0) { throw new IllegalArgumentException("
       ↪ amount"); }
3      if (amount.amount().compareTo(new BigDecimal("1000")) > 0) { throw new
       ↪ IllegalArgumentException("limit"); }
4      PaymentResult result = cardProcessor.charge(amount);
5      notifyCustomer(order, result);
6      return result;
7  }
8
9  public PaymentResult payByTransfer(Order order, Money amount) {
10     if (amount.amount().signum() <= 0) { throw new IllegalArgumentException("
       ↪ amount"); }
11     PaymentResult result = transferProcessor.charge(amount);
12     notifyCustomer(order, result);
13     return result;
14 }
```

El orden y la validación están duplicados. El día que alguien olvide notificar en un tercer método, nadie se dará cuenta.

8.5 Template Method en Java puro

Template Method pone el esqueleto de un algoritmo en un método de una clase base y delega algunos pasos en las subclases[7, 5]. Tiene tres piezas:

- el *método plantilla*: público y `final`, llama a los pasos en orden;
- los *pasos abstractos*: las subclases están obligadas a rellenarlos;
- los *hooks*: pasos con una implementación por defecto (a menudo vacía) que una subclase *puede* sustituir.

OrderReport.java, TextReport.java

```
1  package shoplab.ejemplos;
2
3  public abstract class OrderReport {
```

```
4
5       // El método plantilla: final, nadie cambia el orden ni se salta un paso.
6       public final String render(Order order) {
7           StringBuilder out = new StringBuilder();
8           out.append(header(order)).append('\n');
9           for (OrderLine line : order.getLines()) {
10              out.append(line(line)).append('\n');
11          }
12          out.append(footer(order));
13          return out.toString();
14      }
15
16      protected abstract String header(Order order);
17
18      protected abstract String line(OrderLine line);
19
20      protected String footer(Order order) {      // hook: vacío por defecto
21          return "";
22      }
23  }
24
25  // ----- TextReport.java -----
26  package shoplab.ejemplos;
27
28  public class TextReport extends OrderReport {
29
30      @Override
31      protected String header(Order order) {
32          return "Order " + order.getId();
33      }
34
35      @Override
36      protected String line(OrderLine line) {
37          return "- " + line.quantity() + " x " + line.productId();
38      }
39
40      @Override
41      protected String footer(Order order) {
42          return "Total: " + order.total().amount();
43      }
44  }
```

Dos reglas de diseño. Primero, el método plantilla es `final`: si una subclase pudiera sobreescribirlo, el esqueleto dejaría de ser una garantía. Segundo, los pasos que solo deben llamarse desde la plantilla son `protected`. Bloch recuerda que las clases pensadas para heredar deben documentar qué métodos se pueden sobreescribir y en qué orden se llaman; si no lo están, deben prohibir la herencia[3]. La contrapartida es que la herencia acopla la subclase a la base.

Error típico

Llamar desde el método plantilla a un método que la subclase puede romper (por ejemplo, sin documentar qué debe hacer un hook) o dejar el método plantilla sin `final`. En ambos casos, el esqueleto ya no es fijo.

8.6 Y en Spring: `JdbcTemplate` y callbacks

Spring usa Template Method en muchos sitios, pero con un giro: en lugar de heredar, *pasas el paso variable como argumento*. `JdbcTemplate` es el ejemplo clásico (solo lo leemos; ShopLab no usa base de datos y no añadimos ninguna dependencia) [28]. Mira esta consulta:

Lectura: JdbcTemplate (no se ejecuta en ShopLab)

```
List<ProductRow> rows = jdbcTemplate.query(
        "select id, name from product where category = ?",
        (rs, rowNum) -> new ProductRow(rs.getString("id"),
                                       rs.getString("name")),
        "hardware");
```

La parte fija (la plantilla) es todo lo que no ves: obtener una conexión, preparar la sentencia, ejecutarla, recorrer el resultado, traducir las excepciones de SQL y cerrar los recursos pase lo que pase. La parte variable es la función `(rs, rowNum) -> ...`: un `RowMapper` que convierte *una fila* en un objeto. Fíjate en la mezcla de patrones: `JdbcTemplate` se comporta como Template Method (esqueleto fijo con un paso que rellenas tú), pero el paso se pasa como un objeto intercambiable, es decir, como una Strategy. A esa forma de Template Method se le llama a menudo *callback*. Lo aprovechas igual en tu propio código cuando el paso variable es pequeño: un método con el esqueleto y un parámetro de tipo interfaz funcional, en vez de una jerarquía de clases.

8.7 Strategy frente a Template Method

	Strategy	Template Method
Mecanismo	composición: el contexto guarda la estrategia	herencia: la subclase rellena pasos
Qué varía	el algoritmo entero	algunos pasos de un algoritmo
Cuándo se elige	en ejecución (se puede cambiar)	al crear la clase (en compilación)
Qué garantiza	nada sobre el orden	el orden de los pasos
Coste	más objetos pequeños	jerarquía rígida, acoplamiento a la base

Regla práctica: si lo que cambia es *todo el cálculo* y quieres elegirlo o cambiarlo en ejecución, usa Strategy. Si el orden de los pasos es una regla de negocio que no se debe romper y varían solo algunos, usa Template Method. Y ante la duda, prefiere composición[3]: puedes construir un Template Method con Strategy (el esqueleto recibe sus pasos por constructor) pero no al revés.

8.8 En el proyecto: Hito 6

Hito 6: descuentos con Strategy y pago con Template Method

Partes del hito 5b. `DiscountStrategy` es una interfaz funcional (`Money discount(Order order, Money price)`: devuelve lo que se resta). Hay tres estrategias: `PercentageDiscount` (un porcentaje), `BuyTwoGetOne` (una unidad gratis por cada tres de la misma línea) y

`NoDiscount`. `DiscountService` recibe todas por `List` y aplica la mejor, sin superar nunca el precio. `AbstractPaymentFlow` es un método plantilla `pay(order, amount, channel)` con cuatro pasos: *validar* (importe mayor que cero), *autorizar* (hook de cada subclase: tarjeta rechaza importes por encima de un límite didáctico de 1000,00; la transferencia no tiene límite; no cobra), *capturar* (el único paso que llama a `PaymentProcessor.charge`, una sola vez) y *notificar* (con la `PaymentConfirmation` del hito 4). `CheckoutFacade` ya no cobra ni notifica: delega en el flujo de su `paymentType`. El importe es `amountDue(order)` = precio de la cadena por defecto menos el mejor descuento; `/total` sin parámetros y el checkout usan lo mismo. Con `gift` o `express`, `/total` usa la sobrecarga `amountDue(order, calculator)` (simulación). Está en el ejercicio 8.6.

8.9 Ejercicios

Ejercicio 8.1: Reglas de promoción con clases y lambdas ★★☆☆☆

Trabaja en clases propias del paquete `shoplab.ejercicios.promo` (no entran en el proyecto y no usan la `DiscountStrategy` del hito).

1. Crea `@FunctionalInterface interface PromoRule` con `Money apply(Money price)`: devuelve el precio ya rebajado.
2. Crea `ThresholdRule implements PromoRule`: recibe un `Money threshold` y un `BigDecimal rate`; si el precio es mayor o igual que el umbral, resta ese porcentaje; si no, lo deja igual.
3. Crea `PromoCheck` con un método `static Money best(Money price, PromoRule... rules)` que devuelva el precio más bajo de entre las reglas.
4. En su `main`, usa un precio de 100,00 EUR y pásale *dos lambdas* (un 10 % menos y 5,00 EUR menos) y un `ThresholdRule` (umbral 100,00 y 20 %). Escribe el resultado.

Está bien si imprime un importe de 80.00 EUR; si quitas el `ThresholdRule`, 90.00.

Pista: *Para comparar dos `Money` usa `amount().compareTo(...)`.*

Guiado ▪ solución en la pág. 247

Ejercicio 8.2: Elegir el envío más barato ★★★☆☆

En una copia del proyecto, crea `interface ShippingRateStrategy` con `boolean supports(Order order)`, `Money cost(Order order)` y `String name()`. Implementa tres estrategias como `@Component`: estándar (siempre aplicable, 4,90), exprés (siempre aplicable, 9,90) y recogida en tienda (solo si la ciudad del envío es `Madrid`, 0,00). Crea `ShippingService`, que reciba `List<ShippingRateStrategy>` y devuelva la opción aplicable más barata de un pedido. Añade `GET /orders/{id}/shipping` que la escriba como `ShippingOption(String name, Money cost)`.
Está bien si un pedido enviado a Madrid responde `pickup` a 0.00, uno enviado a Sevilla responde `standard` a 4.90 y añadir una cuarta estrategia no obliga a tocar `ShippingService`.

Pista: *Filtra con `supports`, y busca el mínimo con `Comparator.comparing` sobre el importe.*

Práctica ▪ solución en la pág. 250

Ejercicio 8.3: Exportar pedidos con un método plantilla ★★★☆☆

En `shoplab.ejercicios.export` (clases propias), escribe `AbstractOrderExport` con el constructor `(OrderRepository repository)` y un método plantilla `final Path export(List<String> ids, Path target)` que ejecute, en este orden, *cargar* (busca los pedidos que existan), *formatear* (abstracto, devuelve las líneas del archivo), *escribir* (con `Files.write`) y un hook `afterWrite(Path target)` vacío por defecto. Crea dos subclases: `CsvOrderExport` (cabecera `id,recipient,total` y una línea por pedido) y `TextOrderExport` (una línea `Order ID for NOMBRE: TOTAL MONEDA` por pedido; además usa el hook para escribir en consola la ruta escrita). Explica por qué el método plantilla es `final`.

Está bien si, con un pedido `o1` de Ana por 99,80 EUR, el CSV tiene la cabecera y la línea `o1,Ana,99.80`, el texto tiene `Order o1 for Ana: 99.80 EUR` y un id inexistente se ignora. (El hito aplica la misma estructura al pago.)

Pista: *`Files.write(Path, Iterable<? extends CharSequence>)` escribe una lista de líneas en UTF-8.*

Práctica ▪ solución en la pág. 253

Ejercicio 8.4: Qué patrones hay en JdbcTemplate ★★★☆☆

Lee este fragmento (`Customer` es un `record` de dos campos y `jdbcTemplate` un `JdbcTemplate` ya configurado; no se ejecuta en ShopLab):

```
List<Customer> customers = jdbcTemplate.query(
        "select id, name from customer where city = ?",
        (rs, rowNum) -> new Customer(rs.getLong("id"),
                                     rs.getString("name")),
        "Madrid");
```

Responde: (a) ¿qué es la plantilla y qué pasos fijos hace? (b) ¿cuál es el paso variable? (c) ¿qué patrón es el `RowMapper` y qué relación tiene con Template Method? (d) ¿qué ganas frente a hacerlo con JDBC a mano?

Está bien si identificas la plantilla, el paso variable, el patrón del `RowMapper` y la ventaja principal.

Lectura de código ▪ solución en la pág. 255

Ejercicio 8.5: Comisiones de pago: herencia o composición ★★★★☆

ShopLab calcula comisiones de pago de tres formas: tarjeta (1,5 % más 0,25 fijos), transferencia (0,50 fijos) y monedero (1 % con un mínimo de 0,30). Todas comparten dos partes: sumar importes con la misma moneda y aplicar un mínimo. Decide cómo modelarlo y justifícalo: Strategy o Template Method, dónde viven las partes comunes y cómo lo probarías.

Está bien si: no hay herencia rígida entre las tres formas de cálculo, las partes comunes viven en un único sitio, añadir una cuarta no toca las anteriores, y cada cálculo se puede probar sin Spring. Entrega el diseño (clases y relaciones) y el código de las piezas.

Diseño ▪ solución en la pág. 255

Ejercicio 8.6: Hito 6: descuentos y flujo de pago ★★★★☆

Parte del hito 5b. Implementa el hito 6 completo.

1. Crea `DiscountStrategy`, `PercentageDiscount`, `BuyTwoGetOne` y `NoDiscount`, y

`DiscountConfig` que registre un `PercentageDiscount` del 10 % como bean.

2. Crea `DiscountService` (recibe la `List`).
3. Crea `AbstractPaymentFlow`, `CardPaymentFlow` y `TransferPaymentFlow`, sin crear ningún procesador nuevo.
4. Cambia `CheckoutFacade`: añade `amountDue(order)` y `amountDue(order, calculator)` y delega en el flujo.
5. Cambia `PricingController` para que use `amountDue`.
6. Prueba con curl (ver abajo) y mira el log.

Está bien si: con un pedido de 3 × `p1` (49,90) a pagar con tarjeta, `/total` da 99.80 (149,70 menos 49,90 de la oferta, mejor que el 10 %) y el checkout cobra `amount` 99.80; con `?express=true`, `/total` da 114.77 y el checkout sigue cobrando 99.80; y en el log del checkout salen, en este orden, `[FLOW] validate`, `[FLOW] authorize`, `[FLOW] capture` (una sola vez) y `[FLOW] notify`.

Pista: *Un pedido de 40 unidades de* `p1` *con tarjeta supera el límite de 1000,00: responde 400 y el log no llega a* `capture`*.*

Proyecto ▪ solución en la pág. 257

8.10 Resumen

Resumen

- Strategy encapsula cada algoritmo detrás de una interfaz y lo hace intercambiable en ejecución; sustituye cadenas de `if/else`.
- Una estrategia pequeña cabe en una lambda (interfaz funcional); una con estado o varias piezas, en una clase.
- En Spring, `List<T>` inyecta todas las estrategias: añadir una es crear un bean, sin tocar el servicio.
- Template Method fija el esqueleto en un método `final` y deja pasos abstractos y hooks a las subclases.
- `JdbcTemplate` es una plantilla con callbacks: lo fijo son los recursos y errores; el paso variable (`RowMapper`) es una estrategia.
- Strategy usa composición y se elige en ejecución; Template Method usa herencia y garantiza el orden. Ante la duda, composición.
- En ShopLab, `DiscountService` elige el mejor descuento y `AbstractPaymentFlow` garantiza que el cobro llama a `charge` una sola vez.

Soluciones de los ejercicios

Solución del ejercicio 8.1 enunciado en la pág. 243

PromoRule.java, ThresholdRule.java

```
package shoplab.ejercicios.promo;

import shoplab.Money;

@FunctionalInterface
public interface PromoRule {

    Money apply(Money price);
}

// ----- ThresholdRule.java -----
package shoplab.ejercicios.promo;

import java.math.BigDecimal;
import shoplab.Money;

public class ThresholdRule implements PromoRule {

    private final Money threshold;
    private final BigDecimal rate;

    public ThresholdRule(Money threshold, BigDecimal rate) {
        this.threshold = threshold;
        this.rate = rate;
    }

    @Override
    public Money apply(Money price) {
        if (price.amount().compareTo(threshold.amount()) >= 0) {
            return price.minus(price.times(rate));
        }
        return price;
    }
}

// ----- PromoCheck.java -----
package shoplab.ejercicios.promo;

import java.math.BigDecimal;
import shoplab.Money;

public class PromoCheck {

    public static Money best(Money price, PromoRule... rules) {
        Money best = price;
        for (PromoRule rule : rules) {
            Money candidate = rule.apply(price);
            if (candidate.amount().compareTo(best.amount()) < 0) {
                best = candidate;
            }
        }
        return best;
    }

    public static void main(String[] args) {
        Money price = new Money(new BigDecimal("100.00"), "EUR");
        Money five = new Money(new BigDecimal("5.00"), "EUR");
        Money result = best(price,
                p -> p.minus(p.times(new BigDecimal("0.10"))),
                p -> p.minus(five),
                new ThresholdRule(price, new BigDecimal("0.20")));
        System.out.println(result);   // Money[amount=80.00, currency=EUR]
```

Las dos reglas simples son lambdas porque son una expresión; la que necesita datos (umbral y porcentaje) es una clase. Con las tres da 90,00, 95,00 y 80,00 y gana la última; sin el `ThresholdRule` gana 90,00. **Errores típicos:** olvidar `@FunctionalInterface` (compila igual, pero pierdes la protección si añades un segundo método); restar en vez de multiplicar por `0.90` y obtener un precio negativo con porcentajes grandes (`Money` lanza excepción si el resultado es negativo).

Solución del ejercicio 8.2 enunciado en la pág. 243

ShippingRateStrategy.java y estrategias

```
package shoplab;

public interface ShippingRateStrategy {

    boolean supports(Order order);

    Money cost(Order order);

    String name();
}

// ----- StandardShipping.java -----
package shoplab;

import java.math.BigDecimal;
import org.springframework.stereotype.Component;

@Component
public class StandardShipping implements ShippingRateStrategy {

    @Override
    public boolean supports(Order order) {
        return true;
    }

    @Override
    public Money cost(Order order) {
        return new Money(new BigDecimal("4.90"), order.total().currency());
    }

    @Override
    public String name() {
        return "standard";
    }
}

// ----- ExpressShipping.java -----
package shoplab;

import java.math.BigDecimal;
import org.springframework.stereotype.Component;

@Component
public class ExpressShipping implements ShippingRateStrategy {

    @Override
    public boolean supports(Order order) {
        return true;
    }

    @Override
    public Money cost(Order order) {
        return new Money(new BigDecimal("9.90"), order.total().currency());
    }

    @Override
    public String name() {
        return "express";
    }
}

// ----- StorePickup.java -----
```

Es la misma técnica que `DiscountService`: el servicio solo conoce la interfaz y Spring le inyecta lo que haya. Una cuarta estrategia (por ejemplo, envío gratuito desde cierto importe) es una clase más con `@Component`. **Errores típicos:** olvidar `@Component` en una estrategia (no aparece en la lista y no hay error); meter la elección de «la más barata» dentro de una estrategia en lugar del servicio.

Solución del ejercicio 8.3 enunciado en la pág. 244

AbstractOrderExport.java y subclases

```java
package shoplab.ejercicios.export;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;
import java.util.Optional;
import shoplab.Order;
import shoplab.OrderRepository;

public abstract class AbstractOrderExport {

    private final OrderRepository repository;

    protected AbstractOrderExport(OrderRepository repository) {
        this.repository = repository;
    }

    // Final: el orden cargar -> formatear -> escribir -> hook no se cambia.
    public final Path export(List<String> ids, Path target)
            throws IOException {
        List<Order> orders = load(ids);
        List<String> lines = format(orders);
        write(lines, target);
        afterWrite(target);
        return target;
    }

    protected List<Order> load(List<String> ids) {
        return ids.stream()
                .map(repository::findById)
                .flatMap(Optional::stream)
                .toList();
    }

    protected abstract List<String> format(List<Order> orders);

    protected void write(List<String> lines, Path target)
            throws IOException {
        Files.write(target, lines);
    }

    protected void afterWrite(Path target) {
        // hook: por defecto no hace nada
    }
}

// ----- CsvOrderExport.java -----
package shoplab.ejercicios.export;

import java.util.ArrayList;
import java.util.List;
import shoplab.Order;
import shoplab.OrderRepository;

public class CsvOrderExport extends AbstractOrderExport {

    public CsvOrderExport(OrderRepository repository) {
        super(repository);
    }
```

El método plantilla es `final` porque el orden de los pasos es lo que el patrón garantiza: si una subclase pudiera sobreescribir `export`, podría escribir antes de formatear o saltarse la carga. Las subclases solo rellenan `format` (obligatorio) y, si quieren, `afterWrite` (opcional). **Errores típicos:** olvidar `final` en la plantilla; hacer `public` los pasos (cualquiera podría llamarlos fuera de orden); olvidar que `Files.write` puede lanzar `IOException`.

Solución del ejercicio 8.4

enunciado en la pág. 244

(a) La plantilla es `JdbcTemplate.query`: abre o toma una conexión del *pool*, prepara la sentencia y le asigna el parámetro `"Madrid"`, la ejecuta, recorre el `ResultSet`, traduce las `SQLException` a la jerarquía de excepciones de Spring y libera conexión, sentencia y resultado aunque haya un error. Todo eso es el esqueleto fijo del algoritmo.
(b) El paso variable es qué hacer con cada fila: la lambda `(rs, rowNum) -> new Customer(...)`, que construye un `Customer` a partir de la fila actual. Solo tú sabes qué columnas tiene y a qué objeto se convierten.
(c) El `RowMapper` es una **Strategy**: un algoritmo (convertir una fila) detrás de una interfaz funcional, intercambiable (podrías pasar otro `RowMapper` para otro tipo). La relación con Template Method es que `JdbcTemplate` es una plantilla cuyo paso variable no se rellena heredando de una clase, sino pasando ese paso como un *callback*: Template Method implementado por composición.
(d) Dejas de escribir y de olvidar el código repetitivo y propenso a fugas (`try/finally`, cierre de recursos, traducción de errores); tu código solo dice qué consulta hace y cómo se convierte una fila. **Error típico:** decir que `RowMapper` es un Template Method: el que lo es es `JdbcTemplate`; el `RowMapper` es el paso que se le pasa.

Solución del ejercicio 8.5

enunciado en la pág. 244

Diseño de referencia: Strategy con composición. Una interfaz `CommissionStrategy` con tres implementaciones independientes, una clase auxiliar `Fees` con las partes comunes (crear un importe en la moneda dada y aplicar un mínimo) y un `CommissionService` que recibe un `Map<String, CommissionStrategy>` (los beans, por nombre).

CommissionStrategy.java, Fees.java y estrategias

```
package shoplab.ejercicios.fees;

import shoplab.Money;

public interface CommissionStrategy {

    Money commission(Money amount);
}

// ----- Fees.java (partes comunes) -----
package shoplab.ejercicios.fees;

import java.math.BigDecimal;
import shoplab.Money;

public final class Fees {

    private Fees() {
    }

    public static Money of(String value, String currency) {
        return new Money(new BigDecimal(value), currency);
    }

    public static Money atLeast(Money value, Money minimum) {
        return value.amount().compareTo(minimum.amount()) < 0
                ? minimum : value;
    }
}

// ----- CardCommission.java -----
package shoplab.ejercicios.fees;

import java.math.BigDecimal;
import shoplab.Money;

public class CardCommission implements CommissionStrategy {

    @Override
    public Money commission(Money amount) {
        return amount.times(new BigDecimal("0.015"))
                .plus(Fees.of("0.25", amount.currency()));
    }
}

// ----- TransferCommission.java -----
package shoplab.ejercicios.fees;

import shoplab.Money;

public class TransferCommission implements CommissionStrategy {

    @Override
    public Money commission(Money amount) {
        return Fees.of("0.50", amount.currency());
    }
}

// ----- WalletCommission.java -----
package shoplab.ejercicios.fees;

import java.math.BigDecimal;
```

Por qué así. Las tres formas son *algoritmos completos distintos*, no variaciones de un mismo orden de pasos; Strategy las deja independientes, y las partes comunes viven en `Fees` sin herencia. Cada estrategia se prueba con `new CardCommission().commission(...)`, sin Spring; una cuarta es una clase más. Una prueba: `WalletCommission` con 10,00 EUR da 0,30 (el mínimo) y con 100,00 EUR, 1,00.
Criterios para valorar otras soluciones: no hay jerarquía rígida; las partes comunes están en un único sitio; añadir un método no toca los existentes; se prueba sin Spring; la elección por nombre o por tipo está en un único punto.
Alternativa válida. Un Template Method `AbstractCommission` con `final commission(amount)` que llame a `base(amount)`, `fixed(amount)` y `minimum(amount)`, si el *orden* de los pasos fuera una regla de negocio (por ejemplo, el mínimo siempre se aplica al final). Aquí no lo es, así que se acoplaría a la base sin ganar nada. **Error típico:** una clase base con `if (method.equals("CARD"))`, que reintroduce el olor.

Solución del ejercicio 8.6 enunciado en la pág. 244

1. Descuentos.

DiscountStrategy.java y estrategias

```java
package shoplab;

@FunctionalInterface
public interface DiscountStrategy {

    Money discount(Order order, Money price);
}

// ----- PercentageDiscount.java (sin @Component: lo crea DiscountConfig) -----
package shoplab;

import java.math.BigDecimal;

public class PercentageDiscount implements DiscountStrategy {

    private final BigDecimal rate;

    public PercentageDiscount(BigDecimal rate) {
        this.rate = rate;
    }

    @Override
    public Money discount(Order order, Money price) {
        return price.times(rate);
    }
}

// ----- BuyTwoGetOne.java -----
package shoplab;

import org.springframework.stereotype.Component;

@Component
public class BuyTwoGetOne implements DiscountStrategy {

    @Override
    public Money discount(Order order, Money price) {
        Money free = Money.zero(price.currency());
        for (OrderLine line : order.getLines()) {
            free = free.plus(line.unitPrice().times(line.quantity() / 3));
        }
        return free;
    }
}

// ----- NoDiscount.java -----
package shoplab;

import org.springframework.stereotype.Component;

@Component
public class NoDiscount implements DiscountStrategy {

    @Override
    public Money discount(Order order, Money price) {
        return Money.zero(price.currency());
    }
}

// ----- DiscountConfig.java -----
```

2. Flujo de pago. Reutiliza el `PaymentProcessor` existente; la fábrica de procesadores entrega el bean `card` o `transfer`.

AbstractPaymentFlow.java y subclases

```java
package shoplab;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public abstract class AbstractPaymentFlow {

    protected final Logger log = LoggerFactory.getLogger(getClass());

    private final PaymentProcessor processor;

    protected AbstractPaymentFlow(PaymentProcessor processor) {
        this.processor = processor;
    }

    public abstract PaymentType type();

    // Método plantilla: el orden es fijo y charge se llama solo en capture.
    public final PaymentResult pay(Order order, Money amount,
                                   NotificationChannel channel) {
        validate(amount);
        authorize(order, amount);
        PaymentResult result = capture(amount);
        notifyCustomer(order, result, channel);
        return result;
    }

    protected void validate(Money amount) {
        log.info("[FLOW] validate {} {}", amount.amount(), amount.currency());
        if (amount.amount().signum() <= 0) {
            throw new IllegalArgumentException("amount must be positive");
        }
    }

    /** Hook de cada método de pago: comprueba reglas, NO cobra. */
    protected abstract void authorize(Order order, Money amount);

    protected PaymentResult capture(Money amount) {
        log.info("[FLOW] capture (charge once)");
        return processor.charge(amount);
    }

    protected void notifyCustomer(Order order, PaymentResult result,
                                  NotificationChannel channel) {
        log.info("[FLOW] notify");
        new PaymentConfirmation(channel, order, result)
                .send(order.getShippingAddress().getRecipient());
    }
}

// ----- CardPaymentFlow.java -----
package shoplab;

import java.math.BigDecimal;
import org.springframework.stereotype.Component;

@Component
public class CardPaymentFlow extends AbstractPaymentFlow {

    // Límite didáctico de la autenticación de tarjeta
```

3. Fachada y controlador.

CheckoutFacade.java y PricingController.java

```
package shoplab;

import java.util.List;
import java.util.Map;
import org.springframework.stereotype.Service;

@Service
public class CheckoutFacade {

    private final List<AbstractPaymentFlow> flows;
    private final Map<String, NotificationChannel> channels;
    private final PriceCalculator calculator;
    private final DiscountService discounts;

    public CheckoutFacade(List<AbstractPaymentFlow> flows,
                          Map<String, NotificationChannel> channels,
                          PriceCalculator calculator,
                          DiscountService discounts) {
        this.flows = flows;
        this.channels = channels;
        this.calculator = calculator;
        this.discounts = discounts;
    }

    public Money amountDue(Order order) {
        return amountDue(order, calculator);
    }

    public Money amountDue(Order order, PriceCalculator custom) {
        Money price = custom.price(order);
        return price.minus(discounts.best(order, price));
    }

    public CheckoutResult checkout(Order order, String channel) {
        NotificationChannel selected = channels.get(channel);
        if (selected == null) {
            throw new IllegalArgumentException(
                    "unknown channel: " + channel);
        }
        AbstractPaymentFlow flow = flows.stream()
                .filter(f -> f.type() == order.getPaymentType())
                .findFirst()
                .orElseThrow(() -> new IllegalStateException(
                        "no flow for " + order.getPaymentType()));
        Money amount = amountDue(order);
        PaymentResult payment = flow.pay(order, amount, selected);
        return new CheckoutResult(order.getId(), amount, payment);
    }
}

// ----- PricingController.java -----
package shoplab;

import java.math.BigDecimal;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RestController
```

4. Prueba (la ejecuta el alumno). Con los productos `p1` y `p2` de capítulos anteriores creados:

Descuentos y flujo de pago

```
curl -i -H "Content-Type: application/json" -X POST localhost:8080/orders \
  -d '{"items":[{"productId":"p1","quantity":3}],"shippingAddress":{"recipient
    ↪ ":"Ana","street":"Calle Sol 1","city":"Madrid"},"region":"ES","
    ↪ paymentType":"CARD"}'
curl -i localhost:8080/orders/ID/total
curl -i "localhost:8080/orders/ID/total?express=true"
curl -i -X POST localhost:8080/orders/ID/checkout
```

Resultados: el pedido de 149,70 EUR tiene dos descuentos posibles, 14,97 (10 %) y 49,90 (tres unidades, una gratis); `DiscountService` elige el segundo. `/total` da `99.80` con `simulation: false` y el checkout cobra `amount 99.80`. Con `?express=true`, el precio es 164,67 y `/total` da `114.77` (simulación); el checkout sigue cobrando 99.80. En el log del checkout verás, en este orden: `[FLOW] validate 99.80 EUR`, `[FLOW] authorize card (limit 1000.00)`, `[FLOW] capture (charge once)` y `[FLOW] notify`, seguidas de la línea `[EMAIL to Ana] ...`; `capture` sale una vez, y es el único sitio que llama a `charge`. Con 40 unidades de `p1` (1996,00 menos 648,70 de la oferta: 1347,30) y tarjeta, el log se detiene en `authorize` y la respuesta es 400 `card payments are limited to 1000.00`: no se cobra.

Errores típicos: llamar a `charge` desde `authorize` (se cobraría dos veces); calcular el descuento sobre `order.total()` en vez del precio de la cadena; no limitar el descuento al precio (`Money.minus` lanzaría excepción); olvidar que `PricingController` ahora necesita `CheckoutFacade`.

CAPÍTULO 9

State, Command y Memento

Objetivos del capítulo

- Reconocer el olor de las banderas booleanas y modelar un ciclo de vida con State, usando `sealed` y `switch`.
- Encapsular acciones como objetos Command con `execute` y `undo`, y llevar su historial.
- Guardar y restaurar instantáneas con Memento y `record`.
- Decidir entre comandos inversos e instantáneas para deshacer.
- Registrar la máquina de estados, el historial y el almacén de instantáneas como beans de Spring.
- Dar a ShopLab un ciclo de vida del pedido y un carrito con deshacer e instantáneas.

Un pedido se crea, se paga, se envía o se cancela, y no todos los saltos tienen sentido: no se envía un pedido sin pagar. Un carrito se llena y se vacía, y quieres poder decir «deshaz lo último» o «vuelve a como estaba hace un rato». Este capítulo resuelve esos tres problemas con tres patrones: State representa el ciclo de vida, Command convierte cada acción en un objeto que sabe deshacerse, y Memento guarda una foto del estado para recuperarla después[7, 5].

Java sin compilar

Como en los capítulos 3 a 8, el código Java está escrito con cuidado contra la documentación, pero no lo ha compilado ni ejecutado su autor. Si algo falla, `./gradlew compileJava` te dirá dónde. Usamos sin reexplicarlos `record`, `sealed` y `switch` con patrones (capítulo 1), Lombok (capítulo 4) y las colecciones de `java.util`.

9.1 El olor: banderas y transiciones ilegales

La primera idea para saber en qué punto está un documento es una bandera por cada hecho:

Report.java (olor)

```
public class Report {

    private boolean submitted;
    private boolean approved;
    private boolean rejected;

    public void approve() {
        approved = true;            // ¿y si ya estaba rechazado?
    }

    public void reject() {
        rejected = true;            // ¿y si ya estaba aprobado?
```

```
    }
}
```

Tres booleanos permiten $2^3 = 8$ combinaciones, y solo unas pocas tienen sentido. Nada impide llegar a «aprobado y rechazado a la vez»; cada método debería comprobar las demás banderas, y cada bandera nueva duplica las combinaciones. El estado real del objeto es uno solo (y lo expresan mal tres variables independientes). Refactoring Guru lo describe como el síntoma que State viene a curar: condicionales enormes que dependen del estado[20].

9.2 State en Java puro con `sealed` y `switch`

State representa cada estado como un tipo propio y deja que el contexto delegue en él o decida según él[7, 5]. En Java moderno, una `sealed interface` con un `record` por estado hace que el compilador conozca todos los estados posibles[12], y un `switch` con patrones sobre ella es exhaustivo: si añades un estado y olvidas tratarlo, no compila [2].

Report.java, ReportState.java, ReportEvent.java

```
package shoplab.ejemplos;

public sealed interface ReportState
        permits ReportState.Draft, ReportState.InReview,
                ReportState.Published {

    record Draft() implements ReportState { }
    record InReview() implements ReportState { }
    record Published() implements ReportState { }
}

// ----- ReportEvent.java -----
package shoplab.ejemplos;

public sealed interface ReportEvent
        permits ReportEvent.Submit, ReportEvent.Publish {

    record Submit() implements ReportEvent { }
    record Publish() implements ReportEvent { }
}

// ----- Report.java -----
package shoplab.ejemplos;

public record Report(String title, ReportState state) {

    public Report next(ReportEvent event) {
        ReportState target = switch (state) {
            case ReportState.Draft draft -> switch (event) {
                case ReportEvent.Submit submit ->
                        new ReportState.InReview();
                case ReportEvent.Publish publish -> throw bad(event);
            };
            case ReportState.InReview review -> switch (event) {
                case ReportEvent.Publish publish ->
                        new ReportState.Published();
```

```
                case ReportEvent.Submit submit -> throw bad(event);
            };
            case ReportState.Published published -> throw bad(event);
        };
        return new Report(title, target);
    }

    private IllegalStateException bad(ReportEvent event) {
        return new IllegalStateException("cannot apply "
                + event.getClass().getSimpleName() + " in state "
                + state.getClass().getSimpleName());
    }
}
```

Qué ha pasado: cada estado y cada evento es un tipo; `next` devuelve un `Report` *nuevo* (el registro es inmutable) o lanza una excepción si la transición no está permitida. Las transiciones legales están en un único sitio, y no hay combinaciones imposibles. El error típico es poner un `default` en el `switch`: te quita la exhaustividad y los estados nuevos caen en silencio en el caso por defecto. Sin `default`, el compilador te avisa.

State clásico y State con `switch`

La versión de Gamma y colaboradores pone el comportamiento *dentro* de cada estado (un método `submit()` por clase, con polimorfismo)[7]. Funciona bien cuando cada estado se comporta de forma muy distinta. Cuando lo que varía es sobre todo *qué transiciones se permiten*, como aquí, una tabla de transiciones con `switch` es más corta y se lee de un vistazo. Es una variante de datos más que de objetos; el patrón sigue siendo el mismo: el estado actual decide qué puede pasar.

9.3 Y en Spring: la máquina de estados como bean

Spring no trae un State propio dentro del contenedor: existe un proyecto aparte, Spring Statemachine, pero para un ciclo de vida de cuatro estados sería disparar con cañón. No añadimos ninguna librería. Nuestra máquina es una clase sin estado propio (el estado vive en el pedido), así que un `@Component` singleton, que ya sabes usar desde el capítulo 2, es perfecto: los servicios la piden por constructor.

Un bean sin estado propio

```
@Component
public class ReportMachine {

    public Report apply(Report report, ReportEvent event) {
        return report.next(event);
    }
}
```

Como el bean es singleton y no guarda nada mutable, es seguro con varios hilos. Si la máquina guardase el estado de un pedido en un campo, dos peticiones se pisarían; es la trampa clásica del singleton con estado.

9.4 El olor: acciones que no se pueden deshacer

Un carrito con métodos que modifican directamente su contenido no recuerda qué hizo:

Basket (olor)

```
basket.add("p1", 2);
basket.remove("p2");
// ¿Cómo deshaces el remove? Ya no sabes cuántos había.
```

Para deshacer necesitas *recordar* cada acción y lo que borró. Eso es exactamente lo que hace Command.

9.5 Command con deshacer en Java puro

Command convierte una petición en un objeto: encapsula la acción y los datos necesarios para ejecutarla, y así se puede guardar, pasar y, si guarda lo necesario, revertir[7, 5]. Los participantes son el *comando* (con `execute` y `undo`), el *receptor* (quien hace el trabajo real) y el *invocador* (quien guarda el historial).

Counter.java, CounterCommand.java, Increment.java, CounterHistory.java

```
package shoplab.ejemplos;

public class Counter {

    private int value;

    public int value() { return value; }
    public void set(int value) { this.value = value; }
}

// ----- CounterCommand.java -----
package shoplab.ejemplos;

public interface CounterCommand {

    void execute();
    void undo();
}

// ----- Increment.java -----
package shoplab.ejemplos;

public class Increment implements CounterCommand {

    private final Counter counter;
    private final int amount;

    public Increment(Counter counter, int amount) {
        this.counter = counter;
        this.amount = amount;
    }

```

```
    @Override
    public void execute() {
        counter.set(counter.value() + amount);
    }

    @Override
    public void undo() {
        counter.set(counter.value() - amount);
    }
}

// ----- CounterHistory.java (el invocador) -----
package shoplab.ejemplos;

import java.util.ArrayDeque;
import java.util.Deque;

public class CounterHistory {

    private final Deque<CounterCommand> done = new ArrayDeque<>();

    public void run(CounterCommand command) {
        command.execute();
        done.push(command);
    }

    public boolean undo() {
        if (done.isEmpty()) {
            return false;
        }
        done.pop().undo();
        return true;
    }
}
```

El historial es una pila: el último comando ejecutado es el primero que se deshace. Para que `undo` funcione, cada comando guarda lo mínimo para revertir: aquí la cantidad sumada. Hay operaciones que *no* se pueden invertir con un dato tan pequeño (vaciar todo, por ejemplo): el comando tiene que guardar lo que borra. Error típico: guardar el valor anterior *después* de modificarlo, o dejar que un comando deshaga algo que no ejecutó (tras un `undo`, el comando sale de la pila).

9.6 Y en Spring: el historial como bean

El invocador, el historial, es el candidato natural a bean: vive tanto como la aplicación y lo usan varios servicios. A diferencia de la máquina de estados, el historial *sí* guarda estado (las pilas), así que hay dos precauciones. Una pila por clave (el carrito), no una global, para que dos carritos no mezclen su historial. Y colecciones seguras con varios hilos, porque el singleton lo comparten todas las peticiones. Lo verás completo en el hito.

9.7 El olor: restaurar un estado anterior

Deshacer comando a comando es incómodo cuando quieres volver a un punto concreto, tras muchos cambios, o cuando la operación no es invertible: «vuelve a como estaba cuando lo guardé». Repetir en orden inverso decenas de `undo` es lento y frágil. Si en lugar de las operaciones guardas el *resultado* en un momento dado, recuperar es copiar de vuelta.

9.8 Memento con records y su relación con Command

Memento captura el estado interno de un objeto (el *originador*) en una instantánea, para devolvérselo más tarde, sin que nadie más conozca sus detalles[7, 21]. Las piezas son el originador, el memento (la foto) y el cuidador, quien la guarda sin abrirla.

Un `record` inmutable es un memento casi gratis: una vez creado, nadie lo cambia. Ojo con las colecciones: copia la lista con `List.copyOf` para que cambios posteriores del originador no alteren la foto.

Note.java, NoteSnapshot.java

```
package shoplab.ejemplos;

import java.util.List;

public record NoteSnapshot(String title, List<String> lines) {

    public NoteSnapshot {
        lines = List.copyOf(lines);
    }
}

// ----- Note.java (el originador) -----
package shoplab.ejemplos;

import java.util.ArrayList;
import java.util.List;

public class Note {

    private String title = "";
    private final List<String> lines = new ArrayList<>();

    public void setTitle(String title) { this.title = title; }
    public void addLine(String line) { lines.add(line); }

    public NoteSnapshot snapshot() {
        return new NoteSnapshot(title, lines);
    }

    public void restore(NoteSnapshot snapshot) {
        this.title = snapshot.title();
        this.lines.clear();
        this.lines.addAll(snapshot.lines());
    }
}
```

Command y Memento se complementan, y tienen costes distintos:

	Comandos inversos	Instantáneas
Memoria	Poca: guardan solo la diferencia	Más: una copia por foto
Complejidad	Cada comando necesita su `undo`	Una foto genérica sirve para todo
Operaciones no invertibles	Guardar lo borrado	Sin problema
Volver a un punto lejano	Muchos `undo`	Un solo `restore`

Una regla práctica: comandos para operaciones pequeñas y frecuentes, instantáneas para operaciones masivas o para puntos de guardado explícitos. En el hito usarás las dos.

9.9 Y en Spring: el almacén de instantáneas como bean

El cuidador del Memento es quien guarda las fotos. En Spring es un bean con una interfaz, igual que los repositorios del curso: `SnapshotStore`, con una implementación en memoria `InMemorySnapshotStore`. Los servicios dependen de la interfaz; si mañana las fotos van a una base de datos, cambia la implementación y no los servicios. Como `CommandHistory`, es un singleton con estado, así que usa colecciones concurrentes.

9.10 En el proyecto: Hito 7

Hito 7: ciclo de vida del pedido y carrito con deshacer

Partes del hito 6. **Pedido:** `Order` gana el campo `state`, un `OrderState` sellado (`Created`, `Paid`, `Shipped`, `Cancelled`), con `@Builder.Default` (vale `Created`) y `@With`. Los eventos son `OrderEvent` (`Pay`, `Ship`, `Cancel`) y `OrderStateMachine.apply(Order, OrderEvent)` devuelve el pedido nuevo o lanza `IllegalTransitionException`, que `ApiExceptionHandler` traduce a 409. Transiciones: `Created` → `Paid` (Pay), `Paid` → `Shipped` (Ship), `Created` o `Paid` → `Cancelled` (Cancel). No hay endpoint `/pay`: `POST /orders/{id}/checkout` comprueba antes de cobrar que `Pay` es legal y, si el cobro sale bien, guarda el pedido en `Paid`. Nuevos: `POST /orders/{id}/ship` y `POST /orders/{id}/cancel`.
Carrito: `Cart(String id, List<CartLine> lines)` y `record CartLine(String productId, int quantity)`, con `CartRepository` (`InMemoryCartRepository`), `CartService`, comandos `CartCommand` (`AddItem`, `RemoveItem`) y `CommandHistory` (una pila por carrito), y la instantánea `CartSnapshot` con `SnapshotStore` (`InMemorySnapshotStore`). Endpoints: `POST /carts`, `GET /carts/{id}`, `POST /carts/{id}/items` (cuerpo: un `CartLine`), `DELETE /carts/{id}/items/{productId}`, `POST /carts/{id}/undo`, `/snapshots` y `/restore`. Está en el ejercicio 9.6.

Algunas decisiones del hito, para que no te sorprendan:

- `Cart` es inmutable: cada comando recibe un carrito y devuelve otro, que el servicio guarda en el repositorio.
- `RemoveItem` se crea con la línea que va a quitar (el servicio la busca antes), y así `undo` sabe qué devolver. `AddItem.undo` resta la cantidad añadida.

- `restore` deja el carrito como la última instantánea y *vacía* su historial, porque los comandos viejos ya no corresponden al contenido.
- Deshacer sin historial o restaurar sin instantánea responde 400 (`IllegalArgumentException`); un carrito inexistente, 404.
- Repetir un pedido (`repeat`) crea un pedido nuevo, así que vuelve a `Created`.

9.11 Ejercicios

Ejercicio 9.1: Del envío con banderas a una interfaz sellada ★★☆☆☆

Trabaja en clases propias del paquete `shoplab.ejercicios.shipping`. Un envío (`Shipment`) tiene hoy tres banderas: `inTransit`, `delivered` y `returned`. Conviértelo a State.

1. Crea `sealed interface ShipmentState` con los `records` `Packed`, `InTransit`, `Delivered` y `Returned`.
2. Crea `sealed interface ShipmentEvent` con los `records` `Dispatch`, `Deliver` y `Return`.
3. Crea `record Shipment(String id, ShipmentState state)` con `Shipment next(ShipmentEvent event)`, con un `switch` sin `default`. Transiciones: `Packed` con `Dispatch` → `InTransit`; `InTransit` con `Deliver` → `Delivered`; `Delivered` con `Return` → `Returned`. Cualquier otra lanza `IllegalStateException`.
4. Crea `ShipmentCheck` con un `main` que lleve un envío por los tres saltos legales, escriba el estado de cada paso, e intente después `Dispatch` sobre el envío devuelto y escriba el mensaje de la excepción.

Está bien si el `main` escribe `InTransit`, `Delivered` y `Returned` y después un mensaje de error que nombra el evento y el estado. (El hito aplica la misma técnica a `Order`.)

Pista: *Usa nombres distintos para las variables de patrón de los dos `switch` anidados: el compilador no permite repetirlos en el mismo ámbito.*

Guiado ▪ solución en la pág. 275

Ejercicio 9.2: Combinaciones imposibles ★★☆☆☆

Lee esta clase (`Parcel` no es de ShopLab):

```
1  public class Parcel {
2
3      private boolean paid;
4      private boolean shipped;
5      private boolean cancelled;
6
7      public void pay()    { paid = true; }
8      public void ship()   { shipped = true; }
9      public void cancel() { cancelled = true; }
10 }
```

Las reglas del negocio son: no se envía sin pagar; un paquete enviado no se cancela; se puede cancelar sin pagar y pagado. Responde: (a) ¿cuántas combinaciones permiten las tres banderas? (b) Haz una tabla y marca cuáles son legales y cuáles ilegales. (c) Escribe

una secuencia de llamadas que llegue a una combinación ilegal. (d) ¿Qué estados tendría la versión State y por qué no hay combinaciones ilegales?
Está bien si encuentras las 8 combinaciones, separas 5 legales de 3 ilegales y explicas que State tiene un valor a la vez.

Lectura de código ▪ solución en la pág. 277

Ejercicio 9.3: Deshacer `SetQuantity` y `ClearAll` ★★★☆☆

En clases propias de `shoplab.ejercicios.basket`, trabaja con esta cesta (solo contiene cantidades por id de producto):

```
package shoplab.ejercicios.basket;

import java.util.LinkedHashMap;
import java.util.Map;

public class Basket {

    private final Map<String, Integer> items = new LinkedHashMap<>();

    public int quantityOf(String productId) {
        return items.getOrDefault(productId, 0);
    }

    public void set(String productId, int quantity) {
        if (quantity <= 0) {
            items.remove(productId);
        } else {
            items.put(productId, quantity);
        }
    }

    public Map<String, Integer> contents() {
        return new LinkedHashMap<>(items);
    }

    public void replaceAll(Map<String, Integer> newItems) {
        items.clear();
        items.putAll(newItems);
    }
}
```

Crea `interface BasketCommand` con `execute()` y `undo()`; los comandos `SetQuantity(Basket, String productId, int quantity)` y `ClearAll(Basket)`; y `BasketEditor` con una pila de deshacer: `run(BasketCommand)` y `boolean undo()`. Escribe `BasketCheck` con un `main` que lo pruebe.
Está bien si: tras `SetQuantity("p1", 3)`, `SetQuantity("p1", 5)` y `ClearAll`, tres `undo` sucesivos dejan la cesta con `p1` en 5, luego en 3 y luego vacía; un cuarto `undo` devuelve `false`. (El hito aplica la técnica a `AddItem` y `RemoveItem` del `Cart`.)

Pista: *`ClearAll` debe guardar una copia de `contents()` antes de vaciar.*

Práctica ▪ solución en la pág. 278

Ejercicio 9.4: Memento para un borrador de precio ★★★☆☆

En una copia del proyecto (`cp -r`), crea en `shoplab.ejercicios.drafts`: `PriceDraft` (originador, mutable: `productId`, `Money price`, con `setPrice`), un `record PriceDraftSnapshot(String productId, Money price)`, los métodos `snapshot()` y `restore(PriceDraftSnapshot)` de `PriceDraft`, y `DraftStore`, un `@Component` singleton con un `Map` por producto: `save(PriceDraftSnapshot)` y `Optional<PriceDraftSnapshot> latest(String productId)`. Añade un `@Component DraftCheck implements ApplicationRunner` que cree un borrador de `p1` a 49,90 EUR, guarde una instantánea, lo cambie a 10,00, restaure la última y escriba el precio.
Está bien si el programa escribe `49.90` tras restaurar, y si guardar otra instantánea no modifica la anterior.

Pista: *Ya usaste `ApplicationRunner` en el capítulo 2. `restore` debe comprobar que la instantánea es del mismo `productId`.*

Práctica ▪ solución en la pág. 281

Ejercicio 9.5: Comandos inversos o instantáneas ★★★★☆

Para el carrito de ShopLab, decide si «deshacer» se implementa con comandos inversos o con instantáneas en cada una de estas tres operaciones, y justifícalo: (a) añadir un producto; (b) vaciar el carrito; (c) aplicar un cupón que recalcula el precio de todas las líneas con una regla que depende de la fecha. Di además cómo combinarías las dos técnicas en un mismo historial.
Está bien si: argumentas con memoria, simplicidad y capacidad de invertir; ves que (c) no se invierte de forma fiable; y tu propuesta de historial común es coherente (por ejemplo, un comando que guarda una instantánea cuando no sabe invertirse).

Diseño ▪ solución en la pág. 283

Ejercicio 9.6: Hito 7: ciclo de vida del pedido y carrito ★★★★☆

Parte del hito 6. Implementa el hito 7 completo.

1. Crea `OrderState`, `OrderEvent`, `IllegalTransitionException` y `OrderStateMachine`.
2. Añade el campo `state` a `Order` y haz que `OrderService.repeat` devuelva el pedido a `Created`.
3. Añade el manejador 409 a `ApiExceptionHandler`.
4. Cambia `OrderService` y `OrderController`: `checkout` comprueba antes de cobrar y guarda `Paid`; nuevos `/ship` y `/cancel`.
5. Crea el carrito: `CartLine`, `Cart`, `CartSnapshot`, `CartCommand`, `CommandHistory`, `CartRepository`, `InMemoryCartRepository`, `SnapshotStore`, `InMemorySnapshotStore`, `CartService` y `CartController`.
6. Prueba con curl (ver la solución).

Está bien si: un pedido nuevo tiene `state created`; tras el checkout, `paid`; `ship` lo deja en `shipped`; después `cancel` responde 409; y en el carrito, añadir `p1` dos veces (2 y 1 unidades), deshacer, guardar una instantánea, quitar `p1` y restaurar deja `p1` con 2 unidades. No existe ningún endpoint `/pay`.

Pista: *Pide `ensureAllowed` antes de cobrar: así un pedido ya pagado responde 409 sin llegar a la pasarela.*

Proyecto ▪ solución en la pág. 283

9.12 Resumen

Resumen

- Varias banderas booleanas permiten combinaciones imposibles; State las sustituye por un único valor que es el estado actual.
- Una `sealed interface` con `record`s y un `switch` sin `default` hace que el compilador compruebe que tratas todos los estados.
- La máquina de estados sin estado propio es un bean singleton seguro; un bean con estado (historial, almacén) necesita colecciones concurrentes y una clave por entidad.
- Command encapsula una acción con `execute` y `undo`; cada comando guarda lo mínimo para revertir, y el invocador lleva la pila.
- Memento guarda una instantánea inmutable (un `record` con copias defensivas) y un cuidador, el `SnapshotStore`, la conserva.
- Comandos para operaciones pequeñas, instantáneas para operaciones masivas o puntos de guardado; se pueden combinar.
- En ShopLab, el pedido tiene ciclo de vida con 409 en transiciones ilegales, y el carrito deshace comandos y restaura instantáneas.

Soluciones de los ejercicios

Solución del ejercicio 9.1 enunciado en la pág. 271

ShipmentState.java, ShipmentEvent.java, Shipment.java, ShipmentCheck.java

```
package shoplab.ejercicios.shipping;

public sealed interface ShipmentState
        permits ShipmentState.Packed, ShipmentState.InTransit,
                ShipmentState.Delivered, ShipmentState.Returned {

    record Packed() implements ShipmentState { }
    record InTransit() implements ShipmentState { }
    record Delivered() implements ShipmentState { }
    record Returned() implements ShipmentState { }
}

// ----- ShipmentEvent.java -----
package shoplab.ejercicios.shipping;

public sealed interface ShipmentEvent
        permits ShipmentEvent.Dispatch, ShipmentEvent.Deliver,
                ShipmentEvent.Return {

    record Dispatch() implements ShipmentEvent { }
    record Deliver() implements ShipmentEvent { }
    record Return() implements ShipmentEvent { }
}

// ----- Shipment.java -----
package shoplab.ejercicios.shipping;

public record Shipment(String id, ShipmentState state) {

    public Shipment next(ShipmentEvent event) {
        ShipmentState target = switch (state) {
            case ShipmentState.Packed packed -> switch (event) {
                case ShipmentEvent.Dispatch dispatch ->
                        new ShipmentState.InTransit();
                case ShipmentEvent.Deliver deliver -> throw bad(event);
                case ShipmentEvent.Return back -> throw bad(event);
            };
            case ShipmentState.InTransit transit -> switch (event) {
                case ShipmentEvent.Deliver deliver ->
                        new ShipmentState.Delivered();
                case ShipmentEvent.Dispatch dispatch -> throw bad(event);
                case ShipmentEvent.Return back -> throw bad(event);
            };
            case ShipmentState.Delivered delivered -> switch (event) {
                case ShipmentEvent.Return back ->
                        new ShipmentState.Returned();
                case ShipmentEvent.Dispatch dispatch -> throw bad(event);
                case ShipmentEvent.Deliver deliver -> throw bad(event);
            };
            case ShipmentState.Returned returned -> throw bad(event);
        };
        return new Shipment(id, target);
    }

    private IllegalStateException bad(ShipmentEvent event) {
        return new IllegalStateException("cannot apply "
                + event.getClass().getSimpleName() + " in state "
                + state.getClass().getSimpleName());
    }
}

// ----- ShipmentCheck.java -----
```

Salida esperada: `InTransit`, `Delivered`, `Returned` y `cannot apply Dispatch in state Returned`. Funciona porque cada estado y evento es un tipo y el `switch` sin `default` es exhaustivo: no hay combinación sin tratar. Errores típicos: repetir el nombre de una variable de patrón en un `switch` anidado (no compila), añadir un `default` que oculta estados nuevos, y usar `Return` como nombre del evento sin recordar que `return` en minúscula es palabra reservada (el nombre de tipo con mayúscula sí es válido, pero la variable de patrón no puede llamarse `return`; por eso se llama `back`).

Solución del ejercicio 9.2 enunciado en la pág. 271

(a) Tres booleanos dan $2^3 = 8$ combinaciones.

paid	shipped	cancelled	Veredicto
no	no	no	legal: creado
sí	no	no	legal: pagado
sí	sí	no	legal: enviado
no	no	sí	legal: cancelado sin pagar
sí	no	sí	legal: cancelado tras pagar
no	sí	no	ilegal: enviado sin pagar
no	sí	sí	ilegal: enviado y cancelado
sí	sí	sí	ilegal: enviado y cancelado

(c) `new Parcel().ship()` deja `shipped = true` y `paid = false`; o `pay(); ship(); cancel();` llega a la combinación enviado y cancelado. Nada lo impide, porque cada método solo cambia su bandera.
(d) La versión State tendría `Created`, `Paid`, `Shipped` y `Cancelled`: cuatro valores (puedes añadir un quinto para distinguir cancelado antes o después de pagar, si el negocio lo pide). El objeto está en *un* estado a la vez, así que «enviado y cancelado» no se puede ni escribir, y las transiciones legales viven en un solo `switch`. Error típico: responder que basta con validar en cada método; funciona, pero repites las reglas y cada bandera nueva duplica las combinaciones a revisar.

Solución del ejercicio 9.3 enunciado en la pág. 272

Basket.java, BasketCommand.java, SetQuantity.java, ClearAll.java, BasketEditor.java, BasketCheck.java

```
package shoplab.ejercicios.basket;

import java.util.LinkedHashMap;
import java.util.Map;

public class Basket {

    private final Map<String, Integer> items = new LinkedHashMap<>();

    public int quantityOf(String productId) {
        return items.getOrDefault(productId, 0);
    }

    public void set(String productId, int quantity) {
        if (quantity <= 0) {
            items.remove(productId);
        } else {
            items.put(productId, quantity);
        }
    }

    public Map<String, Integer> contents() {
        return new LinkedHashMap<>(items);
    }

    public void replaceAll(Map<String, Integer> newItems) {
        items.clear();
        items.putAll(newItems);
    }
}

// ----- BasketCommand.java -----
package shoplab.ejercicios.basket;

public interface BasketCommand {

    void execute();
    void undo();
}

// ----- SetQuantity.java -----
package shoplab.ejercicios.basket;

public class SetQuantity implements BasketCommand {

    private final Basket basket;
    private final String productId;
    private final int quantity;
    private int previous;

    public SetQuantity(Basket basket, String productId, int quantity) {
        this.basket = basket;
        this.productId = productId;
        this.quantity = quantity;
    }

    @Override
    public void execute() {
        previous = basket.quantityOf(productId);
        basket.set(productId, quantity);
    }
```

Salida: `{}`, `{p1=5}`, `{p1=3}`, `{}` y `false`. Cada comando guarda en `execute` lo que necesita para `undo`: la cantidad anterior (`SetQuantity`) o una copia del contenido (`ClearAll`). Errores típicos: guardar `previous` en el constructor (la cesta puede cambiar entre la creación y la ejecución), o guardar el propio mapa de la cesta en lugar de una copia, que se vaciaría también.

Solución del ejercicio 9.4 enunciado en la pág. 273

PriceDraft.java, PriceDraftSnapshot.java, DraftStore.java, DraftCheck.java

```
package shoplab.ejercicios.drafts;

import shoplab.Money;

public record PriceDraftSnapshot(String productId, Money price) { }

// ----- PriceDraft.java -----
package shoplab.ejercicios.drafts;

import shoplab.Money;

public class PriceDraft {

    private final String productId;
    private Money price;

    public PriceDraft(String productId, Money price) {
        this.productId = productId;
        this.price = price;
    }

    public Money price() { return price; }
    public void setPrice(Money price) { this.price = price; }

    public PriceDraftSnapshot snapshot() {
        return new PriceDraftSnapshot(productId, price);
    }

    public void restore(PriceDraftSnapshot snapshot) {
        if (!productId.equals(snapshot.productId())) {
            throw new IllegalArgumentException(
                    "snapshot of another product: " + snapshot.productId());
        }
        this.price = snapshot.price();
    }
}

// ----- DraftStore.java -----
package shoplab.ejercicios.drafts;

import java.util.Deque;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentLinkedDeque;
import org.springframework.stereotype.Component;

@Component
public class DraftStore {

    private final Map<String, Deque<PriceDraftSnapshot>> byProduct =
            new ConcurrentHashMap<>();

    public void save(PriceDraftSnapshot snapshot) {
        byProduct.computeIfAbsent(snapshot.productId(),
                        key -> new ConcurrentLinkedDeque<>())
                 .addLast(snapshot);
    }

    public Optional<PriceDraftSnapshot> latest(String productId) {
```

Escribe `Draft price: 49.90`. La instantánea es un `record` inmutable y `Money` también lo es, así que cambiar el borrador después no altera lo guardado. El almacén es un bean singleton con una cola por producto, de modo que guardar otra instantánea añade al final sin tocar las anteriores. Errores típicos: guardar la referencia al borrador mutable en lugar de una copia, o olvidar que un singleton con estado necesita una colección segura con varios hilos.

Solución del ejercicio 9.5 enunciado en la pág. 273

Referencia.

- *(a) Añadir un producto: comando inverso.* La operación es pequeña y frecuente; el inverso (restar la cantidad añadida) es trivial y casi no ocupa memoria. Una instantánea por cada clic sería un desperdicio.
- *(b) Vaciar el carrito: instantánea (o comando que guarda las líneas borradas).* Es masiva y no se invierte con un dato pequeño; lo que hay que recordar es el contenido entero. Un comando `ClearAll` con una copia de las líneas es en la práctica un mini-memento.
- *(c) Cupón que recalcula con una regla que depende de la fecha: instantánea.* Aplicar la regla al revés no es fiable (la fecha cambió, hay redondeos, la regla pudo cambiar). Lo seguro es guardar el estado anterior completo.

Historial común. Un único historial de `CartCommand`, donde los comandos simples saben invertirse y los masivos (`ClearAll`, `ApplyCoupon`) guardan una `CartSnapshot` del carrito anterior y su `undo` la restaura. Así el invocador no distingue: llama a `undo` y cada comando hace lo que sabe.
Criterios para valorar otras soluciones. Argumenta con memoria, complejidad y posibilidad de invertir; no afirma que una técnica sirva para todo; trata (c) como no invertible; propone un historial coherente. **Alternativa válida:** guardar siempre una instantánea tras cada acción (muy simple, con coste de memoria que se justifica si los carritos son pequeños y el historial se limita a, por ejemplo, 20 pasos).

Solución del ejercicio 9.6 enunciado en la pág. 273

1. Estado del pedido.

OrderState.java, OrderEvent.java, IllegalTransitionException.java

```
package shoplab;

import com.fasterxml.jackson.annotation.JsonSubTypes;
import com.fasterxml.jackson.annotation.JsonTypeInfo;

@JsonTypeInfo(use = JsonTypeInfo.Id.NAME, property = "type")
@JsonSubTypes({
        @JsonSubTypes.Type(value = OrderState.Created.class,
                name = "created"),
        @JsonSubTypes.Type(value = OrderState.Paid.class, name = "paid"),
        @JsonSubTypes.Type(value = OrderState.Shipped.class,
                name = "shipped"),
        @JsonSubTypes.Type(value = OrderState.Cancelled.class,
                name = "cancelled")
})
public sealed interface OrderState
        permits OrderState.Created, OrderState.Paid,
                OrderState.Shipped, OrderState.Cancelled {

    record Created() implements OrderState { }
    record Paid() implements OrderState { }
    record Shipped() implements OrderState { }
    record Cancelled() implements OrderState { }
}

// ----- OrderEvent.java -----
package shoplab;

public sealed interface OrderEvent
        permits OrderEvent.Pay, OrderEvent.Ship, OrderEvent.Cancel {

    record Pay() implements OrderEvent { }
    record Ship() implements OrderEvent { }
    record Cancel() implements OrderEvent { }
}

// ----- IllegalTransitionException.java -----
package shoplab;

public class IllegalTransitionException extends RuntimeException {

    public IllegalTransitionException(OrderState from, OrderEvent event) {
        super("cannot apply " + event.getClass().getSimpleName()
                + " in state " + from.getClass().getSimpleName());
    }
}
```

2. OrderStateMachine.java:

OrderStateMachine.java

```java
package shoplab;

import org.springframework.stereotype.Component;

@Component
public class OrderStateMachine {

    public Order apply(Order order, OrderEvent event) {
        OrderState current = order.getState();
        OrderState next = switch (current) {
            case OrderState.Created created -> switch (event) {
                case OrderEvent.Pay pay -> new OrderState.Paid();
                case OrderEvent.Cancel cancel ->
                        new OrderState.Cancelled();
                case OrderEvent.Ship ship -> throw illegal(current, event);
            };
            case OrderState.Paid paid -> switch (event) {
                case OrderEvent.Ship ship -> new OrderState.Shipped();
                case OrderEvent.Cancel cancel ->
                        new OrderState.Cancelled();
                case OrderEvent.Pay pay -> throw illegal(current, event);
            };
            case OrderState.Shipped shipped ->
                    throw illegal(current, event);
            case OrderState.Cancelled cancelled ->
                    throw illegal(current, event);
        };
        return order.withState(next);
    }

    private static IllegalTransitionException illegal(OrderState from,
                                                      OrderEvent event) {
        return new IllegalTransitionException(from, event);
    }
}
```

3. Order.java (nuevo campo y parámetro al final del constructor; el resto igual que en el capítulo 4):

Order.java

```java
package shoplab;

import com.fasterxml.jackson.annotation.JsonProperty;
import java.util.List;
import lombok.Builder;
import lombok.Singular;
import lombok.Value;
import lombok.With;

@Value
@Builder(toBuilder = true)
public class Order {
    @With String id;
    @Singular("line") List<OrderLine> lines;
    ShippingAddress shippingAddress;
    String region;
    PaymentType paymentType;
    @With @Builder.Default OrderState state = new OrderState.Created();

    public Order(String id, List<OrderLine> lines,
                 ShippingAddress shippingAddress, String region,
                 PaymentType paymentType, OrderState state) {
        if (id == null || id.isBlank()) {
            throw new IllegalArgumentException("id is required");
        }
        if (lines == null || lines.isEmpty()) {
            throw new IllegalArgumentException("an order needs lines");
        }
        if (shippingAddress == null) {
            throw new IllegalArgumentException("shippingAddress is required
    ↪ ");
        }
        if (region == null || region.isBlank()) {
            throw new IllegalArgumentException("region is required");
        }
        if (paymentType == null) {
            throw new IllegalArgumentException("paymentType is required");
        }
        if (state == null) {
            throw new IllegalArgumentException("state is required");
        }
        this.id = id;
        this.lines = List.copyOf(lines);
        this.shippingAddress = shippingAddress;
        this.region = region;
        this.paymentType = paymentType;
        this.state = state;
    }

    @JsonProperty("total")
    public Money total() {
        Money sum = Money.zero(lines.get(0).unitPrice().currency());
        for (OrderLine line : lines) {
            sum = sum.plus(line.subtotal());
        }
        return sum;
    }
}
```

4. ApiExceptionHandler.java: añade este método dentro de la clase (con los `import` que ya tiene):

ApiExceptionHandler.java (método nuevo)

```
    @ExceptionHandler(IllegalTransitionException.class)
    @ResponseStatus(HttpStatus.CONFLICT)
    public ApiError conflict(IllegalTransitionException e) {
        return new ApiError(e.getMessage());
    }
```

5. OrderService.java y OrderController.java:

OrderService.java

```
package shoplab;

import java.util.Optional;
import java.util.UUID;
import org.springframework.stereotype.Service;

@Service
public class OrderService {

    private final ProductRepository products;
    private final OrderRepository orders;
    private final OrderStateMachine machine;

    public OrderService(ProductRepository products,
                        OrderRepository orders,
                        OrderStateMachine machine) {
        this.products = products;
        this.orders = orders;
        this.machine = machine;
    }

    public Order create(CreateOrderRequest request) {
        if (request.items() == null || request.items().isEmpty()) {
            throw new IllegalArgumentException("items are required");
        }
        Order.OrderBuilder builder = Order.builder()
                .id(UUID.randomUUID().toString())
                .shippingAddress(request.shippingAddress())
                .region(request.region())
                .paymentType(request.paymentType());
        for (OrderItemRequest item : request.items()) {
            Product product = products.findById(item.productId())
                    .orElseThrow(() ->
                            new ProductNotFoundException(item.productId()));
            builder.line(new OrderLine(item.productId(), item.quantity(),
                                       product.price()));
        }
        return orders.save(builder.build());
    }

    public Optional<Order> find(String id) {
        return orders.findById(id);
    }

    public Optional<Order> repeat(String id) {
        return orders.findById(id)
                .map(order -> orders.save(
                        order.withId(UUID.randomUUID().toString())
                             .withState(new OrderState.Created())));
    }

    public void ensureAllowed(Order order, OrderEvent event) {
        machine.apply(order, event);
    }

    public Order apply(Order order, OrderEvent event) {
        return orders.save(machine.apply(order, event));
    }

    public Optional<Order> ship(String id) {
```

OrderController.java

```java
package shoplab;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/orders")
public class OrderController {

    private final OrderService service;
    private final CheckoutFacade facade;

    public OrderController(OrderService service,
                           CheckoutFacade facade) {
        this.service = service;
        this.facade = facade;
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Order create(@RequestBody CreateOrderRequest request) {
        return service.create(request);
    }

    @GetMapping("/{id}")
    public ResponseEntity<Order> get(@PathVariable String id) {
        return service.find(id)
                .map(ResponseEntity::ok)
                .orElseGet(() -> ResponseEntity.notFound().build());
    }

    @PostMapping("/{id}/repeat")
    public ResponseEntity<Order> repeat(@PathVariable String id) {
        return service.repeat(id)
                .map(o -> ResponseEntity.status(HttpStatus.CREATED).body(o))
                .orElseGet(() -> ResponseEntity.notFound().build());
    }

    @PostMapping("/{id}/checkout")
    public ResponseEntity<CheckoutResult> checkout(
            @PathVariable String id,
            @RequestParam(defaultValue = "email") String channel) {
        return service.find(id)
                .map(order -> {
                    service.ensureAllowed(order, new OrderEvent.Pay());
                    CheckoutResult result = facade.checkout(order, channel);
                    if (result.payment().success()) {
                        service.apply(order, new OrderEvent.Pay());
                    }
                    return ResponseEntity.ok(result);
                })
```

Solución del ejercicio 9.6 (continuación)

6. Carrito: datos e instantánea.

CartLine.java, Cart.java, CartSnapshot.java

```
package shoplab;

public record CartLine(String productId, int quantity) {

    public CartLine {
        if (productId == null || productId.isBlank()) {
            throw new IllegalArgumentException("productId is required");
        }
        if (quantity <= 0) {
            throw new IllegalArgumentException("quantity must be > 0");
        }
    }
}

// ----- CartSnapshot.java -----
package shoplab;

import java.util.List;

public record CartSnapshot(String cartId, List<CartLine> lines) {

    public CartSnapshot {
        lines = List.copyOf(lines);
    }
}

// ----- Cart.java -----
package shoplab;

import java.util.ArrayList;
import java.util.List;

public record Cart(String id, List<CartLine> lines) {

    public Cart {
        if (id == null || id.isBlank()) {
            throw new IllegalArgumentException("id is required");
        }
        if (lines == null) {
            throw new IllegalArgumentException("lines are required");
        }
        lines = List.copyOf(lines);
    }

    public Cart plus(CartLine added) {
        List<CartLine> result = new ArrayList<>();
        boolean merged = false;
        for (CartLine line : lines) {
            if (line.productId().equals(added.productId())) {
                result.add(new CartLine(line.productId(),
                        line.quantity() + added.quantity()));
                merged = true;
            } else {
                result.add(line);
            }
        }
        if (!merged) {
            result.add(added);
        }
        return new Cart(id, result);
    }
```

7. Comandos e historial.

CartCommand.java, CommandHistory.java

```java
package shoplab;

public sealed interface CartCommand
        permits CartCommand.AddItem, CartCommand.RemoveItem {

    Cart execute(Cart cart);

    Cart undo(Cart cart);

    record AddItem(CartLine line) implements CartCommand {

        @Override
        public Cart execute(Cart cart) {
            return cart.plus(line);
        }

        @Override
        public Cart undo(Cart cart) {
            return cart.minus(line);
        }
    }

    record RemoveItem(CartLine removed) implements CartCommand {

        @Override
        public Cart execute(Cart cart) {
            return cart.without(removed.productId());
        }

        @Override
        public Cart undo(Cart cart) {
            return cart.plus(removed);
        }
    }
}

// ----- CommandHistory.java -----
package shoplab;

import java.util.Deque;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentLinkedDeque;
import org.springframework.stereotype.Component;

@Component
public class CommandHistory {

    private final Map<String, Deque<CartCommand>> stacks =
            new ConcurrentHashMap<>();

    public void push(String cartId, CartCommand command) {
        stacks.computeIfAbsent(cartId,
                        key -> new ConcurrentLinkedDeque<>())
              .push(command);
    }

    public Optional<CartCommand> pop(String cartId) {
        Deque<CartCommand> stack = stacks.get(cartId);
        if (stack == null) {
```

8. Repositorios y almacén.

CartRepository.java, InMemoryCartRepository.java, SnapshotStore.java, InMemorySnapshotStore.java

```
package shoplab;

import java.util.Optional;

public interface CartRepository {

    Cart save(Cart cart);

    Optional<Cart> findById(String id);
}

// ----- InMemoryCartRepository.java -----
package shoplab;

import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;
import org.springframework.stereotype.Component;

@Component
public class InMemoryCartRepository implements CartRepository {

    private final Map<String, Cart> carts = new ConcurrentHashMap<>();

    @Override
    public Cart save(Cart cart) {
        carts.put(cart.id(), cart);
        return cart;
    }

    @Override
    public Optional<Cart> findById(String id) {
        return Optional.ofNullable(carts.get(id));
    }
}

// ----- SnapshotStore.java -----
package shoplab;

import java.util.Optional;

public interface SnapshotStore {

    CartSnapshot save(CartSnapshot snapshot);

    Optional<CartSnapshot> latest(String cartId);
}

// ----- InMemorySnapshotStore.java -----
package shoplab;

import java.util.Deque;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentLinkedDeque;
import org.springframework.stereotype.Component;

@Component
public class InMemorySnapshotStore implements SnapshotStore {
```

9. Servicio y controlador del carrito.

CartService.java, CartController.java

```
package shoplab;

import java.util.List;
import java.util.Optional;
import java.util.UUID;
import org.springframework.stereotype.Service;

@Service
public class CartService {

    private final CartRepository carts;
    private final ProductRepository products;
    private final CommandHistory history;
    private final SnapshotStore snapshots;

    public CartService(CartRepository carts, ProductRepository products,
                       CommandHistory history, SnapshotStore snapshots) {
        this.carts = carts;
        this.products = products;
        this.history = history;
        this.snapshots = snapshots;
    }

    public Cart create() {
        return carts.save(new Cart(UUID.randomUUID().toString(),
                                   List.of()));
    }

    public Optional<Cart> find(String id) {
        return carts.findById(id);
    }

    public Optional<Cart> addItem(String cartId, CartLine line) {
        return carts.findById(cartId).map(cart -> {
            products.findById(line.productId()).orElseThrow(() ->
                    new ProductNotFoundException(line.productId()));
            return run(cart, new CartCommand.AddItem(line));
        });
    }

    public Optional<Cart> removeItem(String cartId, String productId) {
        return carts.findById(cartId).map(cart -> {
            CartLine line = cart.lines().stream()
                    .filter(l -> l.productId().equals(productId))
                    .findFirst()
                    .orElseThrow(() ->
                            new ProductNotFoundException(productId));
            return run(cart, new CartCommand.RemoveItem(line));
        });
    }

    public Optional<Cart> undo(String cartId) {
        return carts.findById(cartId).map(cart -> {
            CartCommand command = history.pop(cartId).orElseThrow(() ->
                    new IllegalArgumentException("nothing to undo"));
            return carts.save(command.undo(cart));
        });
    }

    public Optional<CartSnapshot> snapshot(String cartId) {
        return carts.findById(cartId)
```

10. Prueba (la ejecuta el alumno). Con `p1` creado y la aplicación en marcha, crea un carrito y copia su `id` (aquí `CID`):

Carrito con deshacer e instantáneas

```
curl -i -X POST localhost:8080/carts
curl -i -H "Content-Type: application/json" -X POST localhost:8080/carts/CID/
    ↪ items -d '{"productId":"p1","quantity":2}'
curl -i -H "Content-Type: application/json" -X POST localhost:8080/carts/CID/
    ↪ items -d '{"productId":"p1","quantity":1}'
curl -i -X POST localhost:8080/carts/CID/undo
curl -i -X POST localhost:8080/carts/CID/snapshots
curl -i -X DELETE localhost:8080/carts/CID/items/p1
curl -i -X POST localhost:8080/carts/CID/restore
curl -i -X POST localhost:8080/carts/CID/undo
```

Resultados: tras la segunda llamada, `p1` con 3; tras `undo`, con 2; la instantánea responde 201; tras `DELETE`, `lines` vacío; tras `restore`, `p1` con 2; el último `undo` responde 400 porque `restore` vació el historial. Ahora el pedido (con un pedido `OID` creado como en el capítulo 4):

Ciclo de vida del pedido

```
curl -i -H "Content-Type: application/json" -X POST localhost:8080/orders -d '
    ↪ {"items":[{"productId":"p1","quantity":2}],"shippingAddress":{"
    ↪ recipient":"Ana","street":"Calle Sol 1","city":"Madrid"},"region":"ES
    ↪ ","paymentType":"CARD"}'
curl -i -X POST localhost:8080/orders/OID/checkout
curl -i -X POST localhost:8080/orders/OID/checkout
curl -i -X POST localhost:8080/orders/OID/ship
curl -i -X POST localhost:8080/orders/OID/cancel
```

El pedido nuevo trae `"state":{"type":"created"}`; el primer `checkout` cobra y lo deja en `paid`; el segundo responde 409 sin llegar a la pasarela; `ship` responde 200 con `shipped`; `cancel` responde 409 `{"error":"cannot apply Cancel in state Shipped"}`.

Errores típicos. Olvidar el parámetro `state` en el constructor de `Order` (Lombok no encuentra el constructor de todos los campos y no compila); guardar el pedido como pagado *antes* de cobrar (si el cobro falla, queda pagado); no vaciar el historial en `restore` (un `undo` posterior aplicaría un comando a un contenido que ya no es el suyo); y no devolver el pedido a `Created` en `repeat`.

CAPÍTULO 10

Observer y Mediator

Objetivos del capítulo

- Reconocer el olor de un servicio que avisa a mano a todos los interesados.
- Implementar Observer en Java puro y con los eventos de Spring (`ApplicationEventPublisher` y `@EventListener`).
- Controlar el orden de los listeners con `@Order` y saber que, por defecto, se ejecutan de forma síncrona.
- Implementar Mediator para coordinar colaboradores que no deben conocerse entre sí.
- Distinguir Mediator de Facade y de Observer con una tabla.
- Dar a ShopLab eventos de pedido y un mediador de colocación dentro del checkout.

Cuando un pedido se cobra pasan muchas cosas: hay que descontar stock, programar el envío, avisar al cliente y sumar puntos de fidelidad. Si el código que cobra conoce y llama a todo eso, cada novedad obliga a tocarlo. Este capítulo trae dos patrones cercanos pero distintos. Observer deja que unos objetos reaccionen a un hecho sin que quien lo produce los conozca. Mediator coloca en un solo sitio la coordinación de varios colaboradores que, de otro modo, se llamarían entre sí[7, 5].

Java sin compilar

Como en los capítulos 3 a 9, el código Java está escrito con cuidado contra la documentación, pero no lo ha compilado ni ejecutado su autor. Si algo falla, `./gradlew compileJava` te dirá dónde. Usamos sin reexplicarlos `record`, `sealed` y las colecciones de `java.util`, el contenedor de Spring y la inyección por constructor (capítulos 1 y 2) y SLF4J, que ya usan los canales del capítulo 5.

10.1 El olor: el servicio de pedidos avisa a mano

Imagina que, al colocar un pedido, el servicio llama uno por uno a quien le interesa (`shoplab.ejemplos.observer`; clases de ejemplo, no son de ShopLab):

PlacementService.java (olor)

```
public class PlacementService {

    private final EmailService email;
    private final LoyaltyService loyalty;
    private final WarehouseService warehouse;

    public PlacementService(EmailService email, LoyaltyService loyalty,
                            WarehouseService warehouse) {
        this.email = email;
```

```
        this.loyalty = loyalty;
        this.warehouse = warehouse;
    }

    public void place(String orderId) {
        // ... cobrar ...
        email.sendConfirmation(orderId);
        loyalty.addPoints(orderId);
        warehouse.prepare(orderId);
    }
}
```

Los síntomas son los de siempre: la clase conoce a todos sus interesados, cada nuevo interesado (un aviso por SMS, una analítica) obliga a modificarla y a repetir las pruebas de lo que ya funcionaba, y no puedes probar `place` sin montar tres colaboradores. Quien produce el hecho (*el pedido se colocó*) no debería saber quién reacciona a él.

10.2 Observer en Java puro

Observer define una dependencia de uno a muchos: un *sujeto* mantiene una lista de *observadores* y los avisa cuando ocurre algo[7]. El sujeto solo conoce una interfaz pequeña; cada observador decide qué hace con el aviso.

OrderObserver.java y OrderSubject.java

```
package shoplab.ejemplos.observer;

public interface OrderObserver {
    void onPlaced(String orderId);
}

// ----- archivo aparte: OrderSubject.java -----
package shoplab.ejemplos.observer;

import java.util.ArrayList;
import java.util.List;

public class OrderSubject {

    private final List<OrderObserver> observers = new ArrayList<>();

    public void subscribe(OrderObserver observer) {
        observers.add(observer);
    }

    public void unsubscribe(OrderObserver observer) {
        observers.remove(observer);
    }

    public void place(String orderId) {
        // ... cobrar ...
        for (OrderObserver observer : List.copyOf(observers)) {
            observer.onPlaced(orderId);
        }
```

```
    }
}
```

Dos detalles importan. Como `OrderObserver` tiene un solo método, puedes suscribir una lambda: `subject.subscribe(id -> System.out.println("nuevo: " + id))`. Y el bucle recorre una *copia* (`List.copyOf`): si un observador se da de baja mientras se le avisa, recorrer la lista original lanzaría `ConcurrentModificationException`. **Error típico:** olvidar la baja y dejar observadores vivos para siempre (una fuga de memoria); `subscribe` sin `unsubscribe` es una deuda. Observer es además un patrón cuya implementación puede cambiar mucho de lenguaje a lenguaje (con aspectos, por ejemplo, el sujeto casi desaparece)[9].

10.3 Y en Spring: eventos, `@EventListener` y orden

Spring trae su propio Observer[25]. El sujeto es un `ApplicationEventPublisher`, un bean que Spring inyecta; el evento es un objeto cualquiera (un `record` sirve); y cada observador es un método de un bean marcado con `@EventListener`, cuyo parámetro dice qué tipo de evento escucha. No hay lista que mantener: el contenedor descubre los listeners al arrancar.

Un evento, un publicador y un listener

```java
public record StockLow(String productId, int stock) { }

@Component
public class StockWatcher {
    private final ApplicationEventPublisher publisher;

    public StockWatcher(ApplicationEventPublisher publisher) {
        this.publisher = publisher;
    }

    public void report(String productId, int stock) {
        publisher.publishEvent(new StockLow(productId, stock));
    }
}

@Component
public class StockMailer {
    @EventListener
    public void on(StockLow event) {
        System.out.println("stock bajo: " + event.productId());
    }
}
```

Con los imports `org.springframework.context.ApplicationEventPublisher` y `org.springframework.context.event.EventListener`. Tres propiedades que conviene tener claras:

- **Es síncrono por defecto.** `publishEvent` no vuelve hasta que todos los listeners han terminado; el listener corre en el mismo hilo que el publicador y, si hay transacción, dentro de ella[25]. Hacerlo asíncrono es posible (con `@Async`), pero es otra decisión y este libro no la toma.
- **Una excepción sube al publicador.** Como todo ocurre en la misma llamada, si un

listener lanza, `publishEvent` lanza y los listeners que faltaban no se ejecutan (con la configuración por defecto). Un listener no debe fallar por cosas secundarias.

- **El orden se pide con `@Order`.** Sin él, no puedes confiar en el orden. Con `@Order(n)` en el método, los listeners se ejecutan de menor a mayor valor[25]. La misma anotación ordena los elementos de una `List<T>` inyectada[27], que usarás en el capítulo 11.

Dos listeners ordenados

```
@EventListener
@Order(1)
public void first(StockLow event) { ... }

@EventListener
@Order(2)
public void second(StockLow event) { ... }
```

(Los `...` de este fragmento son solo el cuerpo de los métodos; no es código para pegar.) `@Order` está en `org.springframework.core.annotation.Order`.

`@Order` y la clase `Order` de ShopLab

En el paquete `shoplab` existe la clase `Order` (el pedido). Al importar `org.springframework.core.annotation.Order` en un archivo de ese paquete, el nombre `Order` pasa a significar la anotación en *ese* archivo. Los listeners de este capítulo no usan el tipo `Order` de forma explícita, así que compilan; si algún día necesitas ambos, escribe uno con su nombre completo.

Un listener no es un callback del publicador

El publicador no sabe cuántos listeners hay, ni en qué orden, ni si hay alguno. Si tu lógica necesita *«primero A y después B, y si A falla, no hagas B»*, eso no es un evento: es una secuencia, y su sitio es un mediador (más adelante).

10.4 El olor: colaboradores que se llaman entre sí

Observer resuelve «quien lo produce no conoce a quien reacciona». Otro problema distinto: varios objetos que *sí* deben coordinarse. Si el inventario llama al pago, el pago llama al envío y el envío vuelve a llamar al inventario, cada clase conoce a las otras dos: tres clases, seis flechas, y ninguna se prueba ni se reutiliza sola. Con n colaboradores, el número de enlaces crece como n^2.

10.5 Mediator en Java puro

Mediator saca esas flechas de los colaboradores y las pone en un objeto: el mediador[7]. Cada colaborador solo conoce al mediador y le dice «ha pasado esto»; el mediador decide a quién avisar y en qué orden. Ejemplo mínimo, un formulario cuyo botón solo se activa si el nombre no está vacío (`shoplab.ejemplos.mediator`):

FormMediator.java, NameField.java, SubmitButton.java y OrderForm.java

```
package shoplab.ejemplos.mediator;

```

```
public interface FormMediator {
    void changed(Object source);
}

// ----- archivo aparte: NameField.java -----
package shoplab.ejemplos.mediator;

public class NameField {
    private final FormMediator mediator;
    private String text = "";

    public NameField(FormMediator mediator) {
        this.mediator = mediator;
    }

    public void setText(String text) {
        this.text = text;
        mediator.changed(this);
    }

    public String text() {
        return text;
    }
}

// ----- archivo aparte: SubmitButton.java -----
package shoplab.ejemplos.mediator;

public class SubmitButton {
    private boolean enabled;

    public void setEnabled(boolean enabled) {
        this.enabled = enabled;
    }

    public boolean enabled() {
        return enabled;
    }
}

// ----- archivo aparte: OrderForm.java -----
package shoplab.ejemplos.mediator;

public class OrderForm implements FormMediator {

    public final NameField name = new NameField(this);
    public final SubmitButton submit = new SubmitButton();

    @Override
    public void changed(Object source) {
        if (source == name) {
            submit.setEnabled(!name.text().isBlank());
        }
    }
}
```

`NameField` no sabe que existe un botón; `SubmitButton` no sabe que existe un campo. Toda la

regla vive en `OrderForm.changed`. **Error típico:** que el mediador crezca hasta saber demasiado de cada colega y se convierta en una clase «dios»; si pasa, parte el mediador por responsabilidades.

10.6 Y en Spring: el mediador como bean

En Spring el mediador es un bean más: lo marcas con `@Component`, el contenedor le inyecta por constructor a los colaboradores que coordina y quien lo necesita (un controlador, una fachada) recibe el mediador. Los colaboradores no se inyectan entre sí. Así, cambiar la secuencia es tocar una sola clase. En ShopLab el mediador será `CheckoutMediator`, y conviene decir con franqueza qué versión del patrón es: el GoF clásico usa una interfaz `notify(sender, event)` para que los colegas avisen[7]; aquí el mediador es más directo, un *orquestador* con un método que llama a los colegas en orden. Ambos comparten la idea: las flechas entre colegas desaparecen.

10.7 Mediator frente a Facade y Observer

Los tres reducen acoplamiento y se confunden con facilidad. Esta tabla fija las diferencias[7, 20]:

	Mediator	Facade	Observer
Intención	Coordinar colegas que se hablan entre sí	Dar una entrada simple a un subsistema	Reaccionar a un hecho sin acoplar
Quién conoce a quién	El mediador conoce a los colegas; ellos, solo a él	La fachada conoce al subsistema, no al revés	El sujeto no conoce a los observadores
Dirección	Bidireccional: colegas ↔ mediador	Unidireccional: cliente → subsistema	Unidireccional: sujeto → observadores
Secuencia y orden	Sí: es su trabajo, con control de errores	No añade reglas; reenvía	No: cada observador es independiente
En ShopLab	`CheckoutMediator`	`CheckoutFacade`	`OrderPlaced` y sus listeners

Que un patrón reduzca el acoplamiento no garantiza que mejore el diseño en todos los casos: la evidencia empírica es matizada[30], así que úsalos cuando el olor esté presente. La regla práctica: si el orden importa y un fallo debe deshacer pasos anteriores, mediador; si solo quieres ofrecer una puerta cómoda, fachada; si es «avisa a quien le interese y que cada uno haga lo suyo», observador. Un mismo flujo puede usar los tres, como verás en el hito.

10.8 En el proyecto: H8a

Hito 8a: eventos de pedido y mediador de colocación

Partes del H7. Hasta ahora `CheckoutFacade.checkout` elegía el flujo de pago, cobraba y el controlador marcaba el pedido como pagado. En el H8a:

- Nace el evento `OrderPlaced(orderId, channel)` y el publicador `OrderEventPublisher`. Dos listeners con `@EventListener`: `ConfirmationEmailListener` (usa `Notification`, del H4) y `LoyaltyPointsListener`.
- Nacen los servicios simulados `InventoryService` (descuenta el `stock` de los `Product`)

y `ShippingService` (programa el envío a la `ShippingAddress` del pedido).

- Nace `CheckoutMediator`, que coordina inventario, flujo de pago y envío y publica `OrderPlaced`. `CheckoutFacade.checkout` sigue siendo el único punto de entrada y delega en él.
- **Refactor a propósito:** se quita el paso «notificar» de `AbstractPaymentFlow` (queda validar, autorizar y capturar). Si no, el cliente recibiría dos avisos. El `channel` viaja ahora en el evento.

No hay endpoint `/place`: sigue siendo `POST /orders/{id}/checkout`.

Tres decisiones del diseño, que conviene entender antes de escribir el código:

1. **Stock primero, cobro después.** El mediador reserva el stock *antes* de cobrar: es peor cobrar y descubrir que no queda género que al revés. Si el cobro falla (lanza o devuelve `success=false`), el mediador *devuelve* el stock con `release`. Ese deshacer es justo lo que un evento no sabría hacer.
2. **Quién guarda `Paid`.** En el capítulo 9 lo hacía el controlador tras el cobro. En el H8a lo hace el mediador, con `OrderService.apply(order, new OrderEvent.Pay())`, justo antes de programar el envío y publicar el evento. El controlador conserva el `ensureAllowed(Pay)` del principio, para rechazar con 409 antes de tocar el stock, pero ya *no* llama a `apply`: llamarlo dos veces lanzaría `IllegalTransitionException`.
3. **Tras el cobro, avisos por evento.** Lo estrictamente secuencial (stock, cobro, pago registrado, envío) está en el mediador; lo que es «reacción» (aviso al cliente, puntos) es observador. Cuando se publica `OrderPlaced`, el pedido ya está en `Paid`.

El corazón es el mediador. Aquí está completo (los demás archivos, en la solución del ejercicio 10.6):

CheckoutMediator.java

```
1  package shoplab;
2
3  import java.util.List;
4  import org.springframework.stereotype.Component;
5
6  @Component
7  public class CheckoutMediator {
8
9      private final InventoryService inventory;
10     private final List<AbstractPaymentFlow> flows;
11     private final ShippingService shipping;
12     private final OrderService orders;
13     private final OrderEventPublisher events;
14
15     public CheckoutMediator(InventoryService inventory,
16                             List<AbstractPaymentFlow> flows,
17                             ShippingService shipping,
18                             OrderService orders,
19                             OrderEventPublisher events) {
20         this.inventory = inventory;
21         this.flows = flows;
22         this.shipping = shipping;
23         this.orders = orders;
```

```
        this.events = events;
    }

    public CheckoutResult place(Order order, Money amount,
                                String channel) {
        AbstractPaymentFlow flow = flows.stream()
                .filter(f -> f.type() == order.getPaymentType())
                .findFirst()
                .orElseThrow(() -> new IllegalStateException(
                        "no flow for " + order.getPaymentType()));
        inventory.reserve(order);
        PaymentResult payment;
        try {
            payment = flow.pay(order, amount);
        } catch (RuntimeException e) {
            inventory.release(order);
            throw e;
        }
        if (!payment.success()) {
            inventory.release(order);
            return new CheckoutResult(order.getId(), amount, payment);
        }
        orders.apply(order, new OrderEvent.Pay());
        shipping.schedule(order);
        events.orderPlaced(order.getId(), channel);
        return new CheckoutResult(order.getId(), amount, payment);
    }
}
```

10.9 Ejercicios

Ejercicio 10.1: Un EventBus mínimo ★★★☆☆

Trabaja en clases propias del paquete `shoplab.ejercicios.bus`. Vas a escribir Observer a mano con pasos que se retiran poco a poco.

1. Crea `record OrderPlaced(String orderId, String channel)` (no es el de ShopLab: es otro, de este paquete).
2. Crea la clase `EventBus` con una lista de `Consumer<OrderPlaced>` (`java.util.function.Consumer`), un método `subscribe(Consumer<OrderPlaced> listener)` y un método `publish(OrderPlaced event)` que llama a cada listener.
3. Haz que `subscribe` devuelva un `Runnable` que, al ejecutarse, da de baja al listener.
4. Haz que un listener pueda darse de baja *mientras* se publica, sin excepciones, y que un listener que lance no impida avisar al resto (recoge la excepción con un `try/catch` y sigue).
5. Sin pasos: escribe `BusCheck` con `main` que suscriba tres listeners (uno que se da de baja a sí mismo la primera vez, otro que lanza y otro que imprime) y publique dos eventos.

Está bien si: en la primera publicación imprimen los tres, y en la segunda ya no está el que se dio de baja; el que lanza no impide que imprima el siguiente.

Pista: *Una `CopyOnWriteArrayList` de `java.util.concurrent` admite quitar elementos mientras se recorre; también sirve recorrer `List.copyOf`.*

Guiado ▪ solución en la pág. 310

Ejercicio 10.2: Stock bajo con dos listeners ordenados ★★★☆☆

En una copia del proyecto, paquete `shoplab.ejercicios.stock`, crea el evento `LowStockDetected(String productId, int stock)` y una clase `StockMonitor` con `void check(Product product)`: si `product.stock()` es menor que 5, publica el evento con un `ApplicationEventPublisher` inyectado por constructor. Escúchalo con dos `@EventListener` en beans distintos: `LowStockLogListener` (`@Order(1)`) escribe un mensaje con SLF4J y `LowStockAlertListener` (`@Order(2)`) escribe un aviso. Ambos, además, anotan su nombre (`"log"` y `"alert"`) en este bean registrador, que te damos hecho:

CallRecorder.java

```
package shoplab.ejercicios.stock;

import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;
import org.springframework.stereotype.Component;

@Component
public class CallRecorder {
    private final List<String> calls = new CopyOnWriteArrayList<>();

    public void record(String name) {
        calls.add(name);
    }

    public List<String> calls() {
        return List.copyOf(calls);
    }

    public void clear() {
        calls.clear();
    }
}
```

Está bien si: al comprobar un producto con `stock` 2 aparecen los dos mensajes en el orden log, aviso; con `stock` 20 no aparece ninguno.

Pista: *Los listeners reciben `CallRecorder` por constructor, como cualquier bean.*

Práctica ▪ solución en la pág. 312

Ejercicio 10.3: Un mediador entre inventario, pago y envío ★★★☆☆

En clases propias del paquete `shoplab.ejercicios.mediator`, modela tres colegas, `Inventory`, `Payment` y `Shipping`, que no se conozcan entre sí. Crea la interfaz `Mediator` con `void notify(Object sender, String event)` y el mediador `CheckoutFlow`. `Inventory.reserve()` avisa con `"reserved"`; `Payment.charge()` con `"paid"`; `Shipping.schedule()` con `"shipped"`. El mediador reacciona: al recibir `"reserved"` manda cobrar, y al recibir `"paid"` manda programar el envío. Cada colega imprime una línea al actuar. Está bien si: llamar solo a `inventory.reserve()` imprime

las tres líneas en orden y ninguno de los tres colegas tiene un campo del tipo de otro colega.

Pista: *El mediador construye los colegas pasándose a sí mismo (`this`) por constructor.*

Práctica ▪ solución en la pág. 315

Ejercicio 10.4: Probar que se ejecutan los dos listeners ★★★☆☆

En la copia del ejercicio 10.2, escribe una prueba `@SpringBootTest` (`org.springframework.boot.test.context.SpringBootTest`, con JUnit Jupiter, capítulo 3) que compruebe que al comprobar un producto con `stock` 2 se ejecutan los dos listeners en orden, y que con `stock` 20 no se ejecuta ninguno. Usa el `CallRecorder` como registrador de llamadas. No uses `MockMvc` ni `@MockitoBean`: es el contexto real. Está bien si: ambas pruebas pasan con `./gradlew test` y fallan si cambias el umbral del monitor a 1.

Pista: *Inyecta `StockMonitor` y `CallRecorder` con `@Autowired` y limpia el registrador antes de cada prueba con `@BeforeEach`.*

Test ▪ solución en la pág. 317

Ejercicio 10.5: Un cuarto canal de aviso: whatsapp ★★★★★

Hoy hay tres canales de aviso: `email` y `sms` (H4) y `push` (ejercicio 5.3, que era independiente de ShopLab). Añade un cuarto, `whatsapp`, con reglas *distintas por cliente*: algunos clientes aceptaron whatsapp y quieren recibir ahí la confirmación en lugar del canal pedido; los demás siguen con el canal que pidieron. Presenta tu diseño y decide qué parte es Observer, cuál Mediator y cuál Facade. Criterios de valoración:

- Un solo aviso por pedido; sin ciclos entre clases ni entre eventos.
- Responsabilidades claras: cada clase explica en una frase qué hace.
- Fácil de ampliar: un quinto canal no obliga a tocar el mediador.
- El mediador solo contiene lo que es secuencia estricta.
- Justifica la decisión con la tabla de este capítulo.

Diseño ▪ solución en la pág. 319

Ejercicio 10.6: Hito H8a: eventos de pedido y mediador ★★★★☆

Implementa el hito H8a sobre ShopLab tal como lo dejó el capítulo anterior: crea `OrderPlaced`, `OrderEventPublisher`, `ConfirmationEmailListener`, `LoyaltyPointsListener`, `InventoryService`, `ShippingService` y `CheckoutMediator`; cambia `CheckoutFacade`, `OrderController` y quita «notificar» de `AbstractPaymentFlow`. Está bien si: tras crear un producto `p1` con `stock` 12 y un pedido de 2 unidades, `POST /orders/ID/checkout` responde 200, el `stock` de `p1` es 10, el pedido está en `paid`, el log muestra en este orden la reserva, los pasos `[FLOW]`, el envío, un solo `[EMAIL to ...]` y los puntos; un segundo checkout responde 409 sin tocar el stock; y con un pedido de 50 unidades responde 400 sin cobrar.

Pista: *Empieza por el flujo (quitar «notificar») y el mediador; los listeners al final. Si el controlador llama a `apply(Pay)`, el pedido pasa dos veces por `Paid`.*

Proyecto ▪ solución en la pág. 321

10.10 Resumen

Resumen

- Un servicio que llama a mano a todos sus interesados se acopla a ellos; Observer invierte la dependencia: el sujeto solo conoce una interfaz.
- En Java puro, Observer es una lista de observadores; al publicar se recorre una copia para que las bajas durante el aviso no rompan nada.
- En Spring, `ApplicationEventPublisher` publica un objeto y los métodos `@EventListener` lo reciben; es síncrono por defecto y una excepción en un listener llega al publicador.
- `@Order` fija el orden de los listeners (y de una `List<T>` inyectada); sin él no hay orden garantizado.
- Mediator centraliza la coordinación de colegas que no se conocen; en Spring es un bean con los colaboradores inyectados.
- Mediator coordina con orden y deshace; Facade ofrece una entrada simple; Observer avisa sin acoplar.
- En ShopLab, el checkout reserva stock, cobra, marca `Paid`, programa el envío y publica `OrderPlaced`; el cliente recibe un solo aviso.

Soluciones de los ejercicios

Solución del ejercicio 10.1 enunciado en la pág. 306

OrderPlaced.java, EventBus.java y BusCheck.java

```java
package shoplab.ejercicios.bus;

public record OrderPlaced(String orderId, String channel) {
}

// ----- archivo aparte: EventBus.java -----
package shoplab.ejercicios.bus;

import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.function.Consumer;

public class EventBus {

    private final List<Consumer<OrderPlaced>> listeners =
            new CopyOnWriteArrayList<>();

    public Runnable subscribe(Consumer<OrderPlaced> listener) {
        listeners.add(listener);
        return () -> listeners.remove(listener);
    }

    public void publish(OrderPlaced event) {
        for (Consumer<OrderPlaced> listener : listeners) {
            try {
                listener.accept(event);
            } catch (RuntimeException e) {
                System.out.println("listener failed: " + e.getMessage());
            }
        }
    }
}

// ----- archivo aparte: BusCheck.java -----
package shoplab.ejercicios.bus;

public class BusCheck {

    public static void main(String[] args) {
        EventBus bus = new EventBus();
        Runnable[] unsubscribe = new Runnable[1];
        unsubscribe[0] = bus.subscribe(e -> {
            System.out.println("once: " + e.orderId());
            unsubscribe[0].run();
        });
        bus.subscribe(e -> {
            throw new IllegalStateException("boom " + e.orderId());
        });
        bus.subscribe(e -> System.out.println("print: " + e.orderId()));

        bus.publish(new OrderPlaced("o1", "email"));
        bus.publish(new OrderPlaced("o2", "email"));
    }
}
```

Salida: `once: o1`, `listener failed: boom o1`, `print: o1`; después `listener failed: boom o2` y `print: o2`. Funciona porque `CopyOnWriteArrayList` recorre una instantánea: quitar durante el recorrido no lanza `ConcurrentModificationException`. La baja es un `Runnable` que cierra sobre el listener. El arreglo de un elemento (`unsubscribe`) permite que la lambda se refiera a su propia baja, porque una variable local capturada debe ser efectivamente final. **Errores típicos:** usar un `ArrayList` y recorrerlo directamente (excepción al darse de baja), y no capturar las excepciones (un listener roto silencia a los siguientes).

Solución del ejercicio 10.2

enunciado en la pág. 307

LowStockDetected.java y StockMonitor.java

```java
package shoplab.ejercicios.stock;

public record LowStockDetected(String productId, int stock) {
}

// ----- archivo aparte: StockMonitor.java -----
package shoplab.ejercicios.stock;

import org.springframework.context.ApplicationEventPublisher;
import org.springframework.stereotype.Component;
import shoplab.Product;

@Component
public class StockMonitor {

    public static final int THRESHOLD = 5;

    private final ApplicationEventPublisher publisher;

    public StockMonitor(ApplicationEventPublisher publisher) {
        this.publisher = publisher;
    }

    public void check(Product product) {
        if (product.stock() < THRESHOLD) {
            publisher.publishEvent(
                    new LowStockDetected(product.id(), product.stock()));
        }
    }
}
```

LowStockLogListener.java y LowStockAlertListener.java

```java
package shoplab.ejercicios.stock;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.context.event.EventListener;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;

@Component
public class LowStockLogListener {

    private static final Logger log =
            LoggerFactory.getLogger(LowStockLogListener.class);

    private final CallRecorder recorder;

    public LowStockLogListener(CallRecorder recorder) {
        this.recorder = recorder;
    }

    @EventListener
    @Order(1)
    public void on(LowStockDetected event) {
        log.info("[STOCK] low: {} has {}", event.productId(),
                event.stock());
        recorder.record("log");
    }
}

// ----- archivo aparte: LowStockAlertListener.java -----
package shoplab.ejercicios.stock;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.context.event.EventListener;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;

@Component
public class LowStockAlertListener {

    private static final Logger log =
            LoggerFactory.getLogger(LowStockAlertListener.class);

    private final CallRecorder recorder;

    public LowStockAlertListener(CallRecorder recorder) {
        this.recorder = recorder;
    }

    @EventListener
    @Order(2)
    public void on(LowStockDetected event) {
        log.warn("[ALERT] restock {}", event.productId());
        recorder.record("alert");
    }
}
```

El publicador se inyecta por constructor; cada listener es un bean y el contenedor los encuentra por el tipo del parámetro. Con `@Order(1)` y `@Order(2)` el log sale antes que el aviso. Es la misma técnica que `OrderPlaced` en el hito, con otro evento. **Errores típicos:** olvidar `@Component` (el método nunca se invoca, sin error), poner `@Order` en la clase en lugar de en el método (no ordena el listener), y esperar que el orden sea el alfabético o el de declaración.

Solución del ejercicio 10.3 enunciado en la pág. 307

Mediator.java, Inventory.java, Payment.java y Shipping.java

```java
package shoplab.ejercicios.mediator;

public interface Mediator {
    void notify(Object sender, String event);
}

// ----- archivo aparte: Inventory.java -----
package shoplab.ejercicios.mediator;

public class Inventory {
    private final Mediator mediator;

    public Inventory(Mediator mediator) {
        this.mediator = mediator;
    }

    public void reserve() {
        System.out.println("inventory: reserved");
        mediator.notify(this, "reserved");
    }
}

// ----- archivo aparte: Payment.java -----
package shoplab.ejercicios.mediator;

public class Payment {
    private final Mediator mediator;

    public Payment(Mediator mediator) {
        this.mediator = mediator;
    }

    public void charge() {
        System.out.println("payment: charged");
        mediator.notify(this, "paid");
    }
}

// ----- archivo aparte: Shipping.java -----
package shoplab.ejercicios.mediator;

public class Shipping {
    private final Mediator mediator;

    public Shipping(Mediator mediator) {
        this.mediator = mediator;
    }

    public void schedule() {
        System.out.println("shipping: scheduled");
        mediator.notify(this, "shipped");
    }
}
```

CheckoutFlow.java y MediatorCheck.java

```java
package shoplab.ejercicios.mediator;

public class CheckoutFlow implements Mediator {

    public final Inventory inventory = new Inventory(this);
    public final Payment payment = new Payment(this);
    public final Shipping shipping = new Shipping(this);

    @Override
    public void notify(Object sender, String event) {
        switch (event) {
            case "reserved" -> payment.charge();
            case "paid" -> shipping.schedule();
            default -> { }
        }
    }
}

// ----- archivo aparte: MediatorCheck.java -----
package shoplab.ejercicios.mediator;

public class MediatorCheck {

    public static void main(String[] args) {
        CheckoutFlow flow = new CheckoutFlow();
        flow.inventory.reserve();
    }
}
```

Salida: `inventory: reserved`, `payment: charged`, `shipping: scheduled`. Cada colega solo tiene un `Mediator`; toda la secuencia vive en `CheckoutFlow.notify`. **Errores típicos:** dejar que `Payment` llame directamente a `Shipping` (se pierde el sentido del patrón) y olvidar el `default` del `switch`: como sentencia es opcional, pero deja claro que los demás eventos se ignoran a propósito.

Solución del ejercicio 10.4

enunciado en la pág. 308

Código probado (el de la solución del ejercicio 10.2, sin cambios) y la prueba. La prueba va en `src/test/java/shoplab/ejercicios/stock/StockEventsTest.java`:

StockEventsTest.java

```java
package shoplab.ejercicios.stock;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertTrue;

import java.math.BigDecimal;
import java.util.List;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import shoplab.Money;
import shoplab.Product;
import shoplab.ProductKind;

@SpringBootTest
class StockEventsTest {

    @Autowired
    private StockMonitor monitor;

    @Autowired
    private CallRecorder recorder;

    @BeforeEach
    void clean() {
        recorder.clear();
    }

    private Product product(int stock) {
        return new Product("p9", "Cable",
                new Money(new BigDecimal("5.00"), "EUR"),
                new ProductKind.Digital(), stock, "hardware");
    }

    @Test
    void lowStockRunsBothListenersInOrder() {
        monitor.check(product(2));
        assertEquals(List.of("log", "alert"), recorder.calls());
    }

    @Test
    void enoughStockRunsNoListener() {
        monitor.check(product(20));
        assertTrue(recorder.calls().isEmpty());
    }
}
```

Las demás clases (`LowStockDetected`, `CallRecorder`, `StockMonitor`, `LowStockLogListener` y `LowStockAlertListener`) son exactamente las del ejercicio 10.2. La prueba arranca el contexto real; como los listeners son síncronos, al volver `check` ya han corrido y se puede afirmar sobre la lista sin esperas. Si cambias `THRESHOLD` a 1, la primera prueba falla (el registrador queda vacío). **Errores típicos:** no limpiar el

registrador (las llamadas de una prueba contaminan la otra) y comprobar solo que «hay dos elementos» sin mirar el orden, con lo que `@Order` queda sin probar.

Solución del ejercicio 10.5 enunciado en la pág. 308

Solución de referencia. La secuencia estricta se queda en el mediador: reservar stock, cobrar, marcar `Paid`, programar el envío. Todo lo que es aviso se queda en eventos. Para el canal por cliente no se añade un listener por canal (habría varios avisos), sino una *política* que decide el canal y un único listener que avisa:

WhatsappChannel.java, WhatsappOptIns.java y ChannelPolicy.java

```java
package shoplab;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;

@Component("whatsapp")
public class WhatsappChannel implements NotificationChannel {

    private static final Logger log =
            LoggerFactory.getLogger(WhatsappChannel.class);

    @Override
    public void deliver(String recipient, String text) {
        log.info("[WHATSAPP to {}] {}", recipient, text);
    }
}

// ----- archivo aparte: WhatsappOptIns.java -----
package shoplab;

import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import org.springframework.stereotype.Component;

@Component
public class WhatsappOptIns {

    private final Set<String> recipients = ConcurrentHashMap.newKeySet();

    public void optIn(String recipient) {
        recipients.add(recipient);
    }

    public boolean accepted(String recipient) {
        return recipients.contains(recipient);
    }
}

// ----- archivo aparte: ChannelPolicy.java -----
package shoplab;

public interface ChannelPolicy {
    String channelFor(Order order, String requested);
}

// ----- archivo aparte: OptInChannelPolicy.java -----
package shoplab;

import org.springframework.stereotype.Component;

@Component
public class OptInChannelPolicy implements ChannelPolicy {

    private final WhatsappOptIns optIns;

    public OptInChannelPolicy(WhatsappOptIns optIns) {
        this.optIns = optIns;
    }

    @Override
```

En `ConfirmationEmailListener` solo cambia una línea: en lugar de `channels.get(event.channel())` se usa `channels.get(policy.channelFor(order, event.channel()))`, con el listener recibiendo también un `ChannelPolicy` por constructor. El quinto canal es otra clase `@Component("...")` que implementa `NotificationChannel`; no se toca el mediador.

Reparto. *Observer:* `OrderPlaced` y sus listeners (aviso y puntos), porque son reacciones independientes. *Mediator:* `CheckoutMediator`, porque el orden importa y hay que devolver stock si el cobro falla. *Facade:* `CheckoutFacade` (y el endpoint), que ofrece la entrada única y valida el canal. *Strategy* (capítulo 8) aparece en la política de canal.

Criterios para valorar otras soluciones. Un aviso por pedido (no uno por listener); sin ciclos (el listener no publica eventos que lo vuelvan a activar ni el mediador depende del listener); responsabilidades definibles en una frase; añadir un canal sin modificar mediador ni fachada; y la secuencia estricta fuera de los eventos. **Alternativa válida:** solo eventos, con un listener por canal que decide si le toca según el cliente (`@Order` para que no se solapen). Es más extensible pero repartida: la regla «un solo canal por cliente» queda dispersa y es fácil enviar dos avisos. **Errores típicos:** meter el aviso dentro del mediador (lo acopla a cada canal) o hacer que el listener llame al mediador (ciclo).

Solución del ejercicio 10.6

enunciado en la pág. 308

Orden de trabajo: primero el flujo (quitar «notificar»), luego los servicios, el mediador, la fachada y el controlador, y al final los listeners. El orden entre stock y cobro y la persistencia de `Paid` están explicados en la sección del hito: el stock se reserva antes de cobrar y se devuelve si el cobro falla; `Paid` lo guarda el mediador y el controlador ya no llama a `apply`.

AbstractPaymentFlow.java (sin notificar)

```java
package shoplab;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public abstract class AbstractPaymentFlow {

    protected final Logger log = LoggerFactory.getLogger(getClass());

    private final PaymentProcessor processor;

    protected AbstractPaymentFlow(PaymentProcessor processor) {
        this.processor = processor;
    }

    public abstract PaymentType type();

    // Método plantilla: validar, autorizar y capturar. Sin notificar.
    public final PaymentResult pay(Order order, Money amount) {
        validate(amount);
        authorize(order, amount);
        return capture(amount);
    }

    protected void validate(Money amount) {
        log.info("[FLOW] validate {} {}", amount.amount(),
                amount.currency());
        if (amount.amount().signum() <= 0) {
            throw new IllegalArgumentException("amount must be positive");
        }
    }

    /** Hook de cada método de pago: comprueba reglas, NO cobra. */
    protected abstract void authorize(Order order, Money amount);

    protected PaymentResult capture(Money amount) {
        log.info("[FLOW] capture (charge once)");
        return processor.charge(amount);
    }
}
```

`CardPaymentFlow` y `TransferPaymentFlow` no cambian: solo implementan `type()` y `authorize`. Si otro código llamaba a `pay` con tres argumentos, el compilador lo señala.

InventoryService.java y ShippingService.java

```java
package shoplab;

import org.springframework.stereotype.Service;

@Service
public class InventoryService {

    private final ProductRepository products;

    public InventoryService(ProductRepository products) {
        this.products = products;
    }

    /** Comprueba todas las líneas y después descuenta; o nada. */
    public void reserve(Order order) {
        for (OrderLine line : order.getLines()) {
            Product product = products.findById(line.productId())
                    .orElseThrow(() -> new ProductNotFoundException(
                            line.productId()));
            if (product.stock() < line.quantity()) {
                throw new IllegalArgumentException(
                        "not enough stock for " + product.id());
            }
        }
        for (OrderLine line : order.getLines()) {
            change(line.productId(), -line.quantity());
        }
    }

    public void release(Order order) {
        for (OrderLine line : order.getLines()) {
            change(line.productId(), line.quantity());
        }
    }

    private void change(String productId, int delta) {
        products.findById(productId).ifPresent(p ->
                products.save(new Product(p.id(), p.name(), p.price(),
                        p.kind(), p.stock() + delta, p.category())));
    }
}

// ----- archivo aparte: ShippingService.java -----
package shoplab;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;

@Service
public class ShippingService {

    private static final Logger log =
            LoggerFactory.getLogger(ShippingService.class);

    public void schedule(Order order) {
        ShippingAddress address = order.getShippingAddress();
        log.info("[SHIPPING] order {} scheduled for {}, {}",
                order.getId(), address.getRecipient(), address.getCity());
    }
}
```

`reserve` comprueba primero y descuenta después, para no dejar un pedido a medias reservado. Es una simulación: no es segura frente a peticiones simultáneas.

`OrderPlaced.java, OrderEventPublisher.java y OrderPlacedNotification.java`

```java
package shoplab;

public record OrderPlaced(String orderId, String channel) {
}

// ----- archivo aparte: OrderEventPublisher.java -----
package shoplab;

import org.springframework.context.ApplicationEventPublisher;
import org.springframework.stereotype.Component;

@Component
public class OrderEventPublisher {

    private final ApplicationEventPublisher publisher;

    public OrderEventPublisher(ApplicationEventPublisher publisher) {
        this.publisher = publisher;
    }

    public void orderPlaced(String orderId, String channel) {
        publisher.publishEvent(new OrderPlaced(orderId, channel));
    }
}

// ----- archivo aparte: OrderPlacedNotification.java -----
package shoplab;

public class OrderPlacedNotification extends Notification {

    private final Order order;

    public OrderPlacedNotification(NotificationChannel channel,
                                   Order order) {
        super(channel);
        this.order = order;
    }

    @Override
    public void send(String recipient) {
        channel.deliver(recipient, "Order " + order.getId()
                + " confirmed, total " + order.total().amount()
                + " " + order.total().currency());
    }
}
```

`OrderPlacedNotification` es otra subclase del puente de H4 (`Notification`); `PaymentConfirmation` queda sin uso, porque el resultado del cobro ya no llega al aviso.

ConfirmationEmailListener.java y LoyaltyPointsListener.java

```java
package shoplab;

import java.util.Map;
import org.springframework.context.event.EventListener;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;

@Component
public class ConfirmationEmailListener {

    private final OrderService orders;
    private final Map<String, NotificationChannel> channels;

    public ConfirmationEmailListener(
            OrderService orders,
            Map<String, NotificationChannel> channels) {
        this.orders = orders;
        this.channels = channels;
    }

    @EventListener
    @Order(1)
    public void on(OrderPlaced event) {
        NotificationChannel channel = channels.get(event.channel());
        if (channel == null) {
            return;
        }
        orders.find(event.orderId()).ifPresent(order ->
                new OrderPlacedNotification(channel, order).send(
                        order.getShippingAddress().getRecipient()));
    }
}

// ----- archivo aparte: LoyaltyPointsListener.java -----
package shoplab;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.context.event.EventListener;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;

@Component
public class LoyaltyPointsListener {

    private static final Logger log =
            LoggerFactory.getLogger(LoyaltyPointsListener.class);

    private final OrderService orders;

    public LoyaltyPointsListener(OrderService orders) {
        this.orders = orders;
    }

    @EventListener
    @Order(2)
    public void on(OrderPlaced event) {
        orders.find(event.orderId()).ifPresent(order -> {
            int points = order.total().amount().intValue();
            log.info("[LOYALTY] {} gets {} points",
                    order.getShippingAddress().getRecipient(), points);
        });
```

El nombre `ConfirmationEmailListener` viene del canal por defecto (`email`): el canal real viaja en el evento. Los canales ya se validaron en la fachada antes de llegar aquí, por eso el `null` solo se ignora.
`CheckoutMediator` queda así (igual que en la sección del hito). Falta la fachada y el controlador:

CheckoutMediator.java

```java
package shoplab;

import java.util.List;
import org.springframework.stereotype.Component;

@Component
public class CheckoutMediator {

    private final InventoryService inventory;
    private final List<AbstractPaymentFlow> flows;
    private final ShippingService shipping;
    private final OrderService orders;
    private final OrderEventPublisher events;

    public CheckoutMediator(InventoryService inventory,
                            List<AbstractPaymentFlow> flows,
                            ShippingService shipping,
                            OrderService orders,
                            OrderEventPublisher events) {
        this.inventory = inventory;
        this.flows = flows;
        this.shipping = shipping;
        this.orders = orders;
        this.events = events;
    }

    public CheckoutResult place(Order order, Money amount,
                                String channel) {
        AbstractPaymentFlow flow = flows.stream()
                .filter(f -> f.type() == order.getPaymentType())
                .findFirst()
                .orElseThrow(() -> new IllegalStateException(
                        "no flow for " + order.getPaymentType()));
        inventory.reserve(order);
        PaymentResult payment;
        try {
            payment = flow.pay(order, amount);
        } catch (RuntimeException e) {
            inventory.release(order);
            throw e;
        }
        if (!payment.success()) {
            inventory.release(order);
            return new CheckoutResult(order.getId(), amount, payment);
        }
        orders.apply(order, new OrderEvent.Pay());
        shipping.schedule(order);
        events.orderPlaced(order.getId(), channel);
        return new CheckoutResult(order.getId(), amount, payment);
    }
}
```

CheckoutFacade.java

```
package shoplab;

import java.util.Map;
import org.springframework.stereotype.Service;

@Service
public class CheckoutFacade {

    private final CheckoutMediator mediator;
    private final Map<String, NotificationChannel> channels;
    private final PriceCalculator calculator;
    private final DiscountService discounts;

    public CheckoutFacade(CheckoutMediator mediator,
                          Map<String, NotificationChannel> channels,
                          PriceCalculator calculator,
                          DiscountService discounts) {
        this.mediator = mediator;
        this.channels = channels;
        this.calculator = calculator;
        this.discounts = discounts;
    }

    public Money amountDue(Order order) {
        return amountDue(order, calculator);
    }

    public Money amountDue(Order order, PriceCalculator custom) {
        Money price = custom.price(order);
        return price.minus(discounts.best(order, price));
    }

    public CheckoutResult checkout(Order order, String channel) {
        if (!channels.containsKey(channel)) {
            throw new IllegalArgumentException(
                    "unknown channel: " + channel);
        }
        return mediator.place(order, amountDue(order), channel);
    }
}
```

La fachada sigue calculando el importe y validando el canal antes de nada, y delega la secuencia. No se crea un ciclo: el mediador no depende de la fachada (por eso recibe el importe ya calculado).

OrderController.java (solo el método checkout)

```
    @PostMapping("/{id}/checkout")
    public ResponseEntity<CheckoutResult> checkout(
            @PathVariable String id,
            @RequestParam(defaultValue = "email") String channel) {
        return service.find(id)
                .map(order -> {
                    service.ensureAllowed(order, new OrderEvent.Pay());
                    return ResponseEntity.ok(
                            facade.checkout(order, channel));
                })
                .orElseGet(() -> ResponseEntity.notFound().build());
    }
```

El resto del controlador no cambia. Prueba, con ShopLab en marcha (`ID` es el identificador del pedido; el producto `p1` se creó con `stock` 12):

```
curl -i -X POST localhost:8080/orders/ID/checkout
curl -s localhost:8080/products/p1
curl -i -X POST localhost:8080/orders/ID/checkout
```

La primera llamada responde 200 y, en el log, aparecen en orden `[FLOW] validate`, `[FLOW] capture`, `[SHIPPING]`, un único `[EMAIL to ...]` y `[LOYALTY]`. El `GET` muestra el `stock` descontado y la tercera llamada responde 409 sin tocarlo (`ensureAllowed` lanza antes de reservar). **Errores típicos:** dejar `apply(Pay)` también en el controlador (la segunda llamada lanza `IllegalTransitionException`); conservar el paso «notificar» en el flujo (llegan dos avisos); reservar tras cobrar (se cobra sin stock); y olvidar devolver el stock si `pay` lanza.

CAPÍTULO 11

Chain of Responsibility e Iterator

Objetivos del capítulo

- Reconocer el olor de un método de validación enorme, con `if` anidados.
- Implementar Chain of Responsibility en Java puro, con eslabones enlazados.
- Montar la cadena en Spring con una `List<T>` de beans ordenada con `@Order`.
- Implementar Iterator con `Iterator<T>` e `Iterable<T>` y usarlo con `for-each`.
- Decidir entre una cadena y una simple lista de reglas.
- Dar a ShopLab una validación en cadena dentro del checkout y un recorrido plano del catálogo.

Antes de cobrar un pedido hay que comprobar muchas cosas: que hay stock, que la dirección es válida, que el importe tiene sentido. Si todo eso vive en un solo método, cada regla nueva lo agranda y lo hace más difícil de probar. Chain of Responsibility reparte cada comprobación en un eslabón independiente y deja que el pedido recorra la cadena hasta que alguien lo rechace. Iterator resuelve otro dolor: recorrer una estructura (como el árbol del catálogo del capítulo 6) sin que quien la recorre conozca su interior[7, 5].

Java sin compilar

Como en los capítulos 3 a 10, el código Java está escrito con cuidado contra la documentación, pero no lo ha compilado ni ejecutado su autor. Si algo falla, `./gradlew compileJava` te dirá dónde. Usamos sin reexplicarlos `record`, `sealed`, `switch` con patrones y las colecciones de `java.util` (capítulo 1), la inyección por constructor y las listas de beans (capítulos 2 y 8) y `@Order` (capítulo 10).

11.1 El olor: validaciones en un método de 80 líneas

Un validador «todo en uno» suele acabar así (clase de ejemplo en `shoplab.ejemplos.chain`; no es de ShopLab):

OrderChecker.java (olor)

```
1  package shoplab.ejemplos.chain;
2
3  import shoplab.Order;
4  import shoplab.OrderLine;
5  import shoplab.Product;
6  import shoplab.ProductRepository;
7
8  public class OrderChecker {
9
10     private final ProductRepository products;
```

```
11
12      public OrderChecker(ProductRepository products) {
13          this.products = products;
14      }
15
16      public String check(Order order) {
17          if (order != null) {
18              if (!order.getLines().isEmpty()) {
19                  for (OrderLine line : order.getLines()) {
20                      Product p = products.findById(line.productId())
21                              .orElse(null);
22                      if (p == null) {
23                          return "unknown product " + line.productId();
24                      } else {
25                          if (p.stock() < line.quantity()) {
26                              return "not enough stock for " + p.id();
27                          }
28                      }
29                  }
30                  if (order.getShippingAddress() != null) {
31                      if (order.getShippingAddress().getCity().isBlank()) {
32                          return "city is blank";
33                      }
34                      // ... 60 líneas más de direcciones, importes ...
35                  }
36              }
37          }
38          return null;
39      }
40  }
```

Los síntomas: la anidación crece con cada regla, las reglas no se pueden reordenar ni desactivar sin tocar el método, para probar la dirección tienes que montar también el stock, y devuelve `null` para «todo bien», una convención que invita a errores. El refactor que buscamos es repartir la clase en piezas pequeñas con una sola responsabilidad[11].

11.2 Chain of Responsibility en Java puro

Chain of Responsibility da a varios objetos la oportunidad de tratar una petición: cada uno hace lo suyo y, si no la resuelve, la pasa al siguiente[7]. Quien envía la petición no sabe quién la atenderá ni cuántos eslabones hay.

En la variante más conocida, cada eslabón guarda una referencia al siguiente:

Handler.java y los eslabones

```
1  package shoplab.ejemplos.chain;
2
3  import java.util.Optional;
4  import shoplab.Order;
5
6  public abstract class Handler {
7
8      private Handler next;
9
```

```
    /** Enlaza y devuelve el siguiente, para encadenar llamadas. */
    public Handler setNext(Handler next) {
        this.next = next;
        return next;
    }

    /** Devuelve el primer error de la cadena, si lo hay. */
    public final Optional<String> handle(Order order) {
        Optional<String> error = check(order);
        if (error.isPresent() || next == null) {
            return error;
        }
        return next.handle(order);
    }

    protected abstract Optional<String> check(Order order);
}

// ----- archivo aparte: LinesHandler.java -----
package shoplab.ejemplos.chain;

import java.util.Optional;
import shoplab.Order;

public class LinesHandler extends Handler {

    @Override
    protected Optional<String> check(Order order) {
        if (order.getLines().size() > 10) {
            return Optional.of("too many lines");
        }
        return Optional.empty();
    }
}

// ----- archivo aparte: CityHandler.java -----
package shoplab.ejemplos.chain;

import java.util.Optional;
import shoplab.Order;

public class CityHandler extends Handler {

    @Override
    protected Optional<String> check(Order order) {
        if (order.getShippingAddress().getCity().isBlank()) {
            return Optional.of("city is blank");
        }
        return Optional.empty();
    }
}
```

El método `handle` es una plantilla (capítulo 8): decide si cortar o seguir; cada subclase solo implementa `check`. Se usa así:

Montar y usar la cadena

```
Handler first = new LinesHandler();
first.setNext(new CityHandler());   // más eslabones: .setNext(...)

Optional<String> error = first.handle(order);
error.ifPresent(System.out::println);
```

Hay dos formas de cadena. En la *clásica*, el primer eslabón que puede tratar la petición la trata y los demás no la ven (un manejador de eventos de ratón). En la de *validación*, todos los eslabones miran la petición y el primero que la rechaza la corta. Aquí usamos la segunda. Un error típico es olvidar enlazar un eslabón: la cadena termina antes de tiempo y el resto de reglas nunca se ejecuta, sin ningún aviso.

11.3 Y en Spring: `List<T>` ordenada con `@Order`

En Spring no hace falta enlazar eslabones a mano. Si declaras una interfaz y varias clases `@Component` que la implementan, el contenedor inyecta todas en un `List<T>`[27], y con `@Order` fijas el orden de esa lista (capítulo 10). La «cadena» es entonces la lista, y un pequeño servicio la recorre y se detiene en el primer error. Es la misma técnica que ya usó `DiscountService` en el capítulo 8. Añadir una regla es crear una clase: no se toca ninguna otra.

Importar `@Order` en un paquete que tiene `Order`

En ShopLab existe la clase `shoplab.Order` (el pedido). Si en un archivo del paquete `shoplab` escribes `import org.springframework.core.annotation.Order;`, esa importación tapa a la clase del pedido y deja de compilar cualquier uso de `Order` como tipo. En los validadores del hito usaremos el nombre completo en la anotación (`@org.springframework.core.annotation.Order(1)`) y sin `import`.

El diseño tiene un matiz. Una cadena enlazada «de verdad» (la sección anterior) es una estructura que se puede construir en tiempo de ejecución, enlazar de otra manera según el caso o dejar que cada eslabón decida si continúa. La lista ordenada es más simple y encaja con la inyección de Spring, pero el servicio decide por todos cuándo parar. Para validar es más que suficiente; ver la sección 11.7.

11.4 El olor: recorrer un árbol conociendo su estructura

En el capítulo 6 el catálogo se convirtió en un árbol de `CatalogNode`: `Category`, `Pack` y `Product`. Imagina que un controlador quiere una lista plana de productos y lo hace así:

Recorrido que conoce la estructura (olor)

```
static void collect(CatalogNode node, List<Product> out) {
    if (node instanceof Category c) {
        for (CatalogNode child : c.children()) {
            collect(child, out);
        }
    } else if (node instanceof Pack k) {
        out.addAll(k.products());
    } else if (node instanceof Product p) {
```

```
            out.add(p);
        }
    }
```

El controlador sabe qué es una `Category`, qué es un `Pack` y cómo están anidados; si mañana aparece otro nodo, hay que tocarlo. Además construye la lista entera en memoria aunque el cliente solo quisiera los tres primeros.

11.5 Iterator, `Iterable` y el bucle `for-each`

Iterator ofrece una manera de recorrer los elementos de una colección sin exponer cómo está hecha por dentro[7]. En Java el patrón ya está en el lenguaje, con dos interfaces de `java.util`[17]:

- `Iterator<T>`: el cursor. Tiene `boolean hasNext()` y `T next()` (que lanza `NoSuchElementException` si ya no quedan elementos).
- `Iterable<T>`: «lo que se puede recorrer». Su único método abstracto es `Iterator<T> iterator()`.

El bucle `for-each` funciona con cualquier `Iterable` y es la forma recomendada de recorrer colecciones[3]. Estas dos formas son la misma:

for-each y su equivalente

```
for (Product p : catalog.products()) {
    System.out.println(p.name());
}

Iterator<Product> it = catalog.products().iterator();
while (it.hasNext()) {
    System.out.println(it.next().name());
}
```

Como `Iterable` tiene un solo método abstracto, también se puede escribir como lambda: `() -> new MiIterador(...)`. Cada llamada a `iterator()` devuelve un cursor nuevo, así que la misma estructura se puede recorrer varias veces.

Para recorrer un árbol en profundidad sin recursión necesitamos guardar los nodos pendientes. Usamos una pila, y en Java la pila recomendada es `ArrayDeque`.

Recordatorio: `Deque`

`java.util.Deque` es una cola de doble extremo. `push` y `pop` trabajan por la cabeza (comportamiento de pila), `add` añade por el final y `poll` saca por la cabeza (comportamiento de cola). `ArrayDeque` la implementa y no admite `null`. Cuidado: `pop` sobre una pila vacía lanza excepción; `poll` devuelve `null`.

Este es el iterador del catálogo, que recorre en profundidad y solo devuelve `Product`:

CatalogIterator.java

```
package shoplab;

import java.util.ArrayDeque;
```

```
import java.util.Deque;
import java.util.Iterator;
import java.util.List;
import java.util.NoSuchElementException;

public class CatalogIterator implements Iterator<Product> {

    private final Deque<CatalogNode> pending = new ArrayDeque<>();
    private Product ready;

    public CatalogIterator(CatalogNode root) {
        pending.push(root);
        advance();
    }

    /** Busca el siguiente producto y lo deja en 'ready'. */
    private void advance() {
        ready = null;
        while (ready == null && !pending.isEmpty()) {
            CatalogNode node = pending.pop();
            switch (node) {
                case Product p -> ready = p;
                case Category c -> pushInOrder(c.children());
                case Pack k -> pushInOrder(List.copyOf(k.products()));
            }
        }
    }

    /** Apila al revés para que el primer hijo sea el primero en salir. */
    private void pushInOrder(List<? extends CatalogNode> nodes) {
        for (int i = nodes.size() - 1; i >= 0; i--) {
            pending.push(nodes.get(i));
        }
    }

    @Override
    public boolean hasNext() {
        return ready != null;
    }

    @Override
    public Product next() {
        if (ready == null) {
            throw new NoSuchElementException();
        }
        Product result = ready;
        advance();
        return result;
    }
}
```

Dos detalles. El `switch` sobre `CatalogNode` no tiene `default`: como la interfaz es `sealed`, el compilador comprueba que cubres los tres casos y te avisa si algún día se añade otro. Y el iterador es *perezoso*: no construye la lista de productos, solo avanza hasta el siguiente cuando se lo piden. El error típico es apilar los hijos en su orden natural: como la pila saca el último en entrar, los hijos saldrían al revés (es el fallo del ejercicio 11.3).

11.6 Y en Spring: el catálogo expone `Iterable`

Aquí hay que ser honestos: Spring no aporta un Iterator propio. El patrón es de Java y lo que hace Spring es dejarte construir un servicio que lo exponga. En ShopLab, `CatalogService` gana un método `products()` que devuelve un `Iterable<Product>` sobre el árbol, y el controlador lo recorre sin saber nada del árbol. Como es un método `default` de la interfaz, ni `DefaultCatalogService` ni el proxy `CachingCatalogService` cambian: heredan el recorrido y la caché les sigue sirviendo el árbol.

Si el controlador devuelve ese `Iterable` tal cual, Jackson lo serializa como un array de JSON y lo va leyendo con su iterador, así que no hace falta construir antes la lista. Eso es lo que en el libro llamamos «recorrido perezoso». Si la fuente fuese una base de datos, el beneficio sería mayor; en ShopLab, con datos en memoria, es sobre todo una demostración del diseño.

11.7 Cadena frente a lista de reglas

	Cadena enlazada	**Lista ordenada**
Quién decide continuar	Cada eslabón	El recorrido, para todos
Cómo se monta	Se enlazan a mano (`setNext`)	Spring inyecta `List<T>`
Orden	El de los enlaces	`@Order`
Cambiar el recorrido según el caso	Fácil: otra cadena	Hay que cambiar el servicio
Riesgo típico	Eslabón sin enlazar	Reglas sin `@Order` (orden no garantizado)

Una regla práctica: si todas las reglas se aplican siempre y solo importa el orden, usa la lista. Si cada una decide qué viene después (o hay cadenas distintas para casos distintos), usa eslabones enlazados. Hannemann y Kiczales mostraron que un mismo patrón puede implementarse de varias maneras con distinta modularidad[9]: elegir la variante es parte del diseño, no una cuestión de «el patrón dice que...».

11.8 En el proyecto: H8b

Hito 8b: validación en cadena y recorrido del catálogo

Partes del H8a (`CheckoutFacade` delega en `CheckoutMediator`). En el H8b:

- `OrderValidator` (interfaz) con tres validadores ordenados: `StockValidator` (stock suficiente), `AddressValidator` (la `ShippingAddress` del H3 sin campos en blanco y con un `country` que sea una región conocida) y `AmountValidator` (líneas dentro de `ShopSettings.maxItemsPerOrder` y total mayor que cero).
- `ValidationChain` recorre la lista y lanza `ValidationException(message)` con el primer error. `ApiExceptionHandler` la traduce a 422 con `ApiError`.
- `POST /orders/validate` (mismo cuerpo que `POST /orders`): construye un `Order` sin guardarlo y valida.
- `CheckoutFacade.checkout` ejecuta la cadena antes de nada.
- `CatalogIterator` y `CatalogService.products()`; `GET /catalog/flat`.

Dos decisiones que conviene conocer:

1. **Un 422 antes del 400.** Hasta el H8a, un pedido sin stock suficiente respondía 400 porque lo detectaba `InventoryService.reserve`. Ahora la cadena lo detecta antes y responde 422 con el mensaje `not enough stock for ...`. La reserva sigue comprobando el stock (puede cambiar entre la validación y la reserva), pero ya es una segunda red.
2. **Validar sin guardar.** `OrderService.create` construye y guarda. Para validar necesitamos solo lo primero, así que se extrae `OrderService.draft`, que `create` reutiliza. Si el cuerpo es tan incorrecto que el constructor de `Order` o de `ShippingAddress` lanza `IllegalArgumentException` (destinatario o calle en blanco), eso se traduce a 400 antes de llegar a la cadena; los validadores cubren lo que esos constructores no comprueban (código postal, región).

El código completo del hito está en la solución del ejercicio 11.6.

11.9 Ejercicios

Ejercicio 11.1: Una cadena manual de tres validadores ★★★☆☆

En clases propias (paquete `shoplab.ejercicios.chain`), valida el alta de un usuario con una cadena de eslabones enlazados con `setNext`. Crea `record Signup(String email, String password, int age)` y tres eslabones: el correo debe contener `@`, la contraseña debe tener al menos 8 caracteres y la edad debe ser al menos 18. Cada eslabón devuelve `Optional<String>` con su error. Está bien si: una clase `ChainCheck` con `main` imprime `OK` para un alta válida y el *primer* error en un alta con dos fallos (sin mencionar el segundo).

Pista: *Reutiliza la forma de `Handler`: un método `handle` final que llama a `check` y decide si seguir.*

Práctica ▪ solución en la pág. 341

Ejercicio 11.2: Validadores de borrador de producto con `@Order` ★★★☆☆

En una copia del proyecto (`cp -r`), crea en `shoplab.ejercicios.drafts` un `record ProductDraft(String name, BigDecimal price, String category)` y la interfaz `ProductDraftValidator` con `Optional<String> validate(ProductDraft)`. Implementa tres beans ordenados con `@Order`: nombre no vacío, precio positivo y categoría conocida (`hardware`, `books` o `music`). Un `ProductDraftService` recibe `List<ProductDraftValidator>` y devuelve el primer error. Después añade un *cuarto* validador (nombre de 40 caracteres como máximo) sin modificar ninguna de las otras clases. Expón `POST /drafts/check` para probarlo con `curl`. Está bien si: un borrador válido responde 200; un borrador sin nombre y con precio negativo responde 422 con el error del nombre; y un nombre de 41 caracteres responde 422 gracias al cuarto validador.

Pista: *El servicio no sabe cuántos validadores hay: solo recorre la lista.*

Práctica ▪ solución en la pág. 343

Ejercicio 11.3: ¿Por qué B sale después de C? ★★★☆☆

Lee este iterador, pensado para recorrer un árbol en profundidad, de izquierda a derecha:

PreOrder.java

```
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Iterator;
import java.util.List;

record Node(String name, List<Node> children) {}

class PreOrder implements Iterator<Node> {

    private final Deque<Node> stack = new ArrayDeque<>();

    PreOrder(Node root) {
        stack.push(root);
    }

    public boolean hasNext() {
        return !stack.isEmpty();
    }

    public Node next() {
        Node n = stack.pop();
        for (Node c : n.children()) {
            stack.push(c);
        }
        return n;
    }
}
```

Se aplica al árbol `A` con hijos `B` y `C`, y `B` con hijos `D` y `E` (los demás nodos no tienen hijos). Responde: (1) ¿en qué orden devuelve los nodos? (2) ¿Por qué `B` parece «saltarse» su turno y aparece después de `C`? (3) ¿Cómo lo arreglas? Está bien si: das el orden exacto, explicas el efecto de la pila (último en entrar, primero en salir) y corriges el bucle.

Lectura de código ▪ solución en la pág. 346

Ejercicio 11.4: Un hilo de comentarios recorrido por niveles ★★★★☆

En clases propias (paquete `shoplab.ejercicios.comments`), crea `record Comment(String text, List<Comment> replies)` de modo que sea `Iterable<Comment>`: su `iterator()` recorre el hilo *por niveles* (primero la raíz, luego sus respuestas directas, luego las respuestas de estas, etc.) y de forma perezosa, sin construir una lista con todos los comentarios. Añade también un método `depthFirst()` que devuelve un `Iterable<Comment>` en profundidad para poder comparar. Una clase `ThreadCheck` con `main` imprime ambos órdenes para un hilo con raíz `Q`, respuestas `A` (con `A1`, `A2`) y `B` (con `B1`). Está bien si: el recorrido por niveles imprime `Q A B A1 A2 B1`, el recorrido en profundidad imprime `Q A A1 A2 B B1`, y funciona con `for-each`.

Pista: *Por niveles necesitas una* cola*: `add` al final y `poll` de la cabeza de un `ArrayDeque`. En profundidad era una pila (`push`/`pop`).*

Práctica ▪ solución en la pág. 347

Ejercicio 11.5: Un pipeline de validación configurable ★★★★★

Diseña, en clases propias (`shoplab.ejercicios.pipeline`), un pipeline genérico de validación que pueda trabajar de dos maneras: *cortocircuitar* en el primer error o *acumular todos* los errores. El orden de las reglas debe ser explícito y configurable, no depender del orden en que se registran. Entrega las interfaces y clases, una demostración con reglas sobre una contraseña, y una nota de tres líneas sobre cómo ampliarlo. Criterios de valoración: es extensible (una regla o un modo nuevos no obligan a tocar el resto), el orden es explícito, no admite ciclos ni órdenes repetidos, y no duplica la lógica de recorrido entre los dos modos. La solución explica por qué elegiste esa separación.

Diseño ▪ solución en la pág. 349

Ejercicio 11.6: Hito H8b: validación en cadena y recorrido del catálogo ★★★★☆

Implementa el hito H8b sobre ShopLab tal como lo dejó el capítulo anterior: crea `OrderValidator`, `StockValidator`, `AddressValidator`, `AmountValidator`, `ValidationException`, `ValidationChain` y `CatalogIterator`; añade `OrderService.draft`, `CatalogService.products()`, `POST /orders/validate` y `GET /catalog/flat`; haz que `checkout` valide antes de actuar y que `ApiExceptionHandler` traduzca `ValidationException` a 422. Está bien si: con `p1` de stock 12, `POST /orders/validate` con 50 unidades responde 422 con `not enough stock for p1`; con `country XX` responde 422 con `unknown country: XX`; con un pedido válido responde 200 y no se guarda nada; `POST /orders/ID/checkout` de un pedido con stock insuficiente responde 422 sin cobrar; y `GET /catalog/flat` lista los productos en un solo array.

Pista: *Recuerda el aviso de la sección sobre `@Order`: en el paquete `shoplab` usa el nombre completo de la anotación.*

Proyecto ▪ solución en la pág. 352

11.10 Resumen

Resumen

- Un método de validación con `if` anidados mezcla reglas, se prueba mal y no se puede reordenar; la cadena reparte una regla por eslabón.
- Chain of Responsibility pasa la petición de eslabón en eslabón; en la variante de validación, el primer eslabón que rechaza corta.
- En Spring, una `List<T>` de beans con `@Order` hace de cadena: añadir una regla es añadir una clase.
- En el paquete `shoplab`, importar la anotación `@Order` tapa a `shoplab.Order`: usa el nombre completo.
- Iterator separa el recorrido de la estructura; en Java son `Iterator<T>` e `Iterable<T>`, y `for-each` los usa.
- Un iterador perezoso guarda lo pendiente (una pila para profundidad, una cola para anchura) y no construye la lista.
- Spring no aporta un Iterator propio: el servicio de catálogo expone un `Iterable` y el controlador lo recorre.
- En ShopLab, el checkout valida con la cadena (422 con el primer error) y el catálogo se

lista plano con `GET /catalog/flat`.

Soluciones de los ejercicios

Solución del ejercicio 11.1 enunciado en la pág. 337

Signup.java, SignupHandler.java y los eslabones

```java
package shoplab.ejercicios.chain;

public record Signup(String email, String password, int age) {
}

// ----- archivo aparte: SignupHandler.java -----
package shoplab.ejercicios.chain;

import java.util.Optional;

public abstract class SignupHandler {

    private SignupHandler next;

    public SignupHandler setNext(SignupHandler next) {
        this.next = next;
        return next;
    }

    public final Optional<String> handle(Signup signup) {
        Optional<String> error = check(signup);
        if (error.isPresent() || next == null) {
            return error;
        }
        return next.handle(signup);
    }

    protected abstract Optional<String> check(Signup signup);
}

// ----- archivo aparte: EmailHandler.java -----
package shoplab.ejercicios.chain;

import java.util.Optional;

public class EmailHandler extends SignupHandler {

    @Override
    protected Optional<String> check(Signup signup) {
        if (signup.email() == null
                || !signup.email().contains("@")) {
            return Optional.of("email must contain @");
        }
        return Optional.empty();
    }
}

// ----- archivo aparte: PasswordHandler.java -----
package shoplab.ejercicios.chain;

import java.util.Optional;

public class PasswordHandler extends SignupHandler {

    @Override
    protected Optional<String> check(Signup signup) {
        if (signup.password() == null
                || signup.password().length() < 8) {
            return Optional.of("password needs 8 characters");
        }
        return Optional.empty();
    }
```

Salida: `OK` y `email must contain @`. `setNext` devuelve el eslabón que recibe, y por eso se puede encadenar: `first.setNext(a).setNext(b)` enlaza `first` con `a` y `a` con `b`. El segundo alta falla en dos reglas, pero solo se ve el primer error porque el correo corta la cadena. **Errores típicos:** encadenar `first.setNext(a)` y `first.setNext(b)` (el segundo sustituye al primero); devolver `this` en `setNext`; y olvidar que `next` puede ser `null` en el último eslabón.

Solución del ejercicio 11.2 enunciado en la pág. 337

Se hace en una copia del proyecto; nada de esto forma parte del estado de ShopLab.

ProductDraft.java y los validadores

```java
package shoplab.ejercicios.drafts;

import java.math.BigDecimal;

public record ProductDraft(String name, BigDecimal price,
                           String category) {
}

// ----- archivo aparte: ProductDraftValidator.java -----
package shoplab.ejercicios.drafts;

import java.util.Optional;

public interface ProductDraftValidator {

    Optional<String> validate(ProductDraft draft);
}

// ----- archivo aparte: DraftNameValidator.java -----
package shoplab.ejercicios.drafts;

import java.util.Optional;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;

@Component
@Order(1)
public class DraftNameValidator implements ProductDraftValidator {

    @Override
    public Optional<String> validate(ProductDraft draft) {
        if (draft.name() == null || draft.name().isBlank()) {
            return Optional.of("name must not be empty");
        }
        return Optional.empty();
    }
}

// ----- archivo aparte: DraftPriceValidator.java -----
package shoplab.ejercicios.drafts;

import java.util.Optional;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;

@Component
@Order(2)
public class DraftPriceValidator implements ProductDraftValidator {

    @Override
    public Optional<String> validate(ProductDraft draft) {
        if (draft.price() == null || draft.price().signum() <= 0) {
            return Optional.of("price must be positive");
        }
        return Optional.empty();
    }
}

// ----- archivo aparte: DraftCategoryValidator.java -----
package shoplab.ejercicios.drafts;

import java.util.Optional;
```

El cuarto validador, el servicio y el controlador

```
package shoplab.ejercicios.drafts;

import java.util.Optional;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;

@Component
@Order(4)
public class DraftNameLengthValidator
        implements ProductDraftValidator {

    @Override
    public Optional<String> validate(ProductDraft draft) {
        if (draft.name().length() > 40) {
            return Optional.of("name is longer than 40 characters");
        }
        return Optional.empty();
    }
}

// ----- archivo aparte: ProductDraftService.java -----
package shoplab.ejercicios.drafts;

import java.util.List;
import java.util.Optional;
import org.springframework.stereotype.Service;

@Service
public class ProductDraftService {

    private final List<ProductDraftValidator> validators;

    public ProductDraftService(List<ProductDraftValidator> validators) {
        this.validators = validators;
    }

    public Optional<String> firstError(ProductDraft draft) {
        for (ProductDraftValidator validator : validators) {
            Optional<String> error = validator.validate(draft);
            if (error.isPresent()) {
                return error;
            }
        }
        return Optional.empty();
    }
}

// ----- archivo aparte: DraftController.java -----
package shoplab.ejercicios.drafts;

import java.util.Map;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import shoplab.ApiError;

@RestController
@RequestMapping("/drafts")
public class DraftController {
```

Con el servidor en marcha:

```
curl -i -H "Content-Type: application/json" -d '{"name":"Lamp","price":9.9,"
    ↪ category":"hardware"}' localhost:8080/drafts/check
curl -i -H "Content-Type: application/json" -d '{"name":"","price":-1,"
    ↪ category":"hardware"}' localhost:8080/drafts/check
```

La primera responde 200; la segunda, 422 con `name must not be empty`: el precio también falla, pero el nombre (`@Order(1)`) corta primero. El cuarto validador es una clase nueva y no toca ninguna otra: ese es el beneficio. Aquí sí se importa `org.springframework.core.annotation.Order`, porque este paquete no tiene una clase `Order`. **Errores típicos:** olvidar `@Order` (el orden deja de estar garantizado); poner `@Component` sin que el paquete esté bajo `shoplab`, de modo que no se escanea; y que un validador falle con `NullPointerException` si no comprueba `null` (el cuarto da por hecho que el nombre existe porque el primero ya lo comprobó: otra razón para fijar el orden).

Solución del ejercicio 11.3

enunciado en la pág. 337

(1) El orden es `A C B E D`. Traza: la pila empieza con `[A]`. `next` saca `A` y apila `B` y luego `C`; la cabeza es `C`. Devuelve `C` (no tiene hijos). Después saca `B` y apila `D` y luego `E`; la cabeza es `E`, que sale antes que `D`.
(2) `B` no se salta: espera en la pila debajo de `C`. Una pila devuelve primero lo último que entró, y el bucle apila los hijos de izquierda a derecha, así que el último hijo sale el primero. Todo el recorrido sale con los hermanos al revés.
(3) Hay que apilar los hijos al revés, para que el primero quede arriba:

PreOrder.java (next corregido)

```
1     public Node next() {
2         Node n = stack.pop();
3         List<Node> children = n.children();
4         for (int i = children.size() - 1; i >= 0; i--) {
5             stack.push(children.get(i));
6         }
7         return n;
8     }
```

Ahora la salida es `A B D E C`. Otra solución válida es no cambiar el bucle y usar `stack.addFirst` en lugar de `push` sobre una copia invertida, pero apilar al revés es lo habitual. **Error típico:** corregirlo invirtiendo el resultado final (devolver el recorrido al revés): ya no es un recorrido en profundidad de izquierda a derecha.

Solución del ejercicio 11.4 enunciado en la pág. 338

Comment.java y ThreadCheck.java

```java
package shoplab.ejercicios.comments;

import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Iterator;
import java.util.List;
import java.util.NoSuchElementException;

public record Comment(String text, List<Comment> replies)
        implements Iterable<Comment> {

    public Comment {
        replies = List.copyOf(replies);
    }

    /** Por niveles (anchura): lo que usa for-each. */
    @Override
    public Iterator<Comment> iterator() {
        return new LevelIterator(this);
    }

    /** En profundidad, para comparar. */
    public Iterable<Comment> depthFirst() {
        return () -> new DepthIterator(this);
    }

    private static final class LevelIterator
            implements Iterator<Comment> {

        private final Deque<Comment> queue = new ArrayDeque<>();

        LevelIterator(Comment root) {
            queue.add(root);
        }

        @Override
        public boolean hasNext() {
            return !queue.isEmpty();
        }

        @Override
        public Comment next() {
            Comment current = queue.poll();
            if (current == null) {
                throw new NoSuchElementException();
            }
            queue.addAll(current.replies());
            return current;
        }
    }

    private static final class DepthIterator
            implements Iterator<Comment> {

        private final Deque<Comment> stack = new ArrayDeque<>();

        DepthIterator(Comment root) {
            stack.push(root);
        }

        @Override
```

Salida: `levels: Q A B A1 A2 B1` y `depth: Q A A1 A2 B B1`. El iterador por niveles usa el `ArrayDeque` como *cola*: `poll` saca de la cabeza y `addAll` añade las respuestas al final, así que los hijos de un nivel esperan a que termine el anterior. El de profundidad es la pila de siempre. Ninguno construye una lista con todo el hilo: cada `next` hace trabajo para un solo comentario. Que el *record* sea el propio `Iterable` es cómodo para un hilo pequeño; para estructuras grandes suele ser mejor una clase aparte que ofrezca la vista. **Errores típicos:** usar `push` en el iterador por niveles (sale en profundidad invertida); pasar `null` como respuestas (`List.copyOf` lanza NPE: usa `List.of()` para las hojas); y usar `pop` sobre una cola vacía, que lanza excepción (por eso `next` comprueba antes).

Solución del ejercicio 11.5 enunciado en la pág. 339

Solución de referencia. Separo tres responsabilidades: la regla (qué se comprueba), el modo de ejecución (cortocircuitar o acumular, una estrategia como las del capítulo 8) y el pipeline (orden explícito y validación de su configuración).

Rule.java, Step.java, RunMode.java y los modos

```java
package shoplab.ejercicios.pipeline;

import java.util.Optional;

public interface Rule<T> {

    Optional<String> check(T value);
}

// ----- archivo aparte: Step.java -----
package shoplab.ejercicios.pipeline;

/** Una regla con un orden explícito (menor = antes) y un nombre. */
public record Step<T>(int order, String name, Rule<T> rule) {
}

// ----- archivo aparte: RunMode.java -----
package shoplab.ejercicios.pipeline;

import java.util.List;

public interface RunMode {

    /** 'steps' ya llega ordenada. Devuelve los errores encontrados. */
    <T> List<String> run(List<Step<T>> steps, T value);
}

// ----- archivo aparte: FailFast.java -----
package shoplab.ejercicios.pipeline;

import java.util.List;
import java.util.Optional;

public final class FailFast implements RunMode {

    @Override
    public <T> List<String> run(List<Step<T>> steps, T value) {
        for (Step<T> step : steps) {
            Optional<String> error = step.rule().check(value);
            if (error.isPresent()) {
                return List.of(error.get());
            }
        }
        return List.of();
    }
}

// ----- archivo aparte: CollectAll.java -----
package shoplab.ejercicios.pipeline;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

public final class CollectAll implements RunMode {

    @Override
    public <T> List<String> run(List<Step<T>> steps, T value) {
        List<String> errors = new ArrayList<>();
        for (Step<T> step : steps) {
            Optional<String> error = step.rule().check(value);
```

Pipeline.java y PipelineCheck.java

```java
package shoplab.ejercicios.pipeline;

import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

public final class Pipeline<T> {

    private final List<Step<T>> steps;
    private final RunMode mode;

    public Pipeline(List<Step<T>> steps, RunMode mode) {
        Set<Integer> seen = new HashSet<>();
        for (Step<T> step : steps) {
            if (!seen.add(step.order())) {
                throw new IllegalArgumentException(
                        "duplicate order: " + step.order());
            }
        }
        List<Step<T>> sorted = new ArrayList<>(steps);
        sorted.sort((a, b) -> Integer.compare(a.order(), b.order()));
        this.steps = List.copyOf(sorted);
        this.mode = mode;
    }

    public List<String> validate(T value) {
        return mode.run(steps, value);
    }
}

// ----- archivo aparte: PipelineCheck.java -----
package shoplab.ejercicios.pipeline;

import java.util.List;
import java.util.Optional;

public class PipelineCheck {

    public static void main(String[] args) {
        Rule<String> length = pw -> pw.length() >= 8
                ? Optional.empty() : Optional.of("min 8 characters");
        Rule<String> digit = pw -> pw.chars().anyMatch(Character::isDigit)
                ? Optional.empty() : Optional.of("needs a digit");
        Rule<String> noSpace = pw -> pw.contains(" ")
                ? Optional.of("no spaces") : Optional.empty();

        // Registradas en desorden: el orden lo da 'order'.
        List<Step<String>> steps = List.of(
                new Step<>(30, "spaces", noSpace),
                new Step<>(10, "length", length),
                new Step<>(20, "digit", digit));

        String bad = "ab c";
        System.out.println(new Pipeline<>(steps, new FailFast())
                .validate(bad));
        System.out.println(new Pipeline<>(steps, new CollectAll())
                .validate(bad));
    }
}
```

Salida: `[min 8 characters]` con `FailFast` y `[min 8 characters, needs a digit, no spaces]` con `CollectAll`. Para ampliar: una regla nueva es otro `Step` con un `order` libre; un modo nuevo (por ejemplo, «parar tras N errores») es otra implementación de `RunMode`. Ninguno obliga a tocar el resto.

Criterios para valorar otras soluciones. Extensible sin modificar clases existentes; orden explícito (no el de inserción); sin ciclos (una lista ordenada no puede tenerlos, a diferencia de una cadena enlazada donde un eslabón puede enlazar a un anterior); sin duplicar el recorrido (cada modo tiene su bucle, pero el pipeline no sabe cuál es). **Alternativa válida:** una cadena enlazada donde cada eslabón recibe y devuelve un resultado acumulado (`Result` con la lista de errores) y cada eslabón decide si continúa mirando una bandera de modo. Funciona, pero cada eslabón conoce el modo y es más fácil crear un ciclo al enlazar. **Errores típicos:** ordenar por posición de registro; meter el modo dentro de cada regla; y no avisar de órdenes repetidos (el desempate queda al azar).

Solución del ejercicio 11.6 enunciado en la pág. 339

Orden de trabajo: excepción y validadores, cadena y manejador, `OrderService.draft`, fachada y controlador, y por último el catálogo.

OrderValidator.java, ValidationException.java y StockValidator.java

```java
package shoplab;

import java.util.Optional;

public interface OrderValidator {

    /** Devuelve el error, o vacío si el pedido cumple la regla. */
    Optional<String> validate(Order order);
}

// ----- archivo aparte: ValidationException.java -----
package shoplab;

public class ValidationException extends RuntimeException {

    public ValidationException(String message) {
        super(message);
    }
}

// ----- archivo aparte: StockValidator.java -----
package shoplab;

import java.util.Optional;
import org.springframework.stereotype.Component;

@Component
@org.springframework.core.annotation.Order(1)
public class StockValidator implements OrderValidator {

    private final ProductRepository products;

    public StockValidator(ProductRepository products) {
        this.products = products;
    }

    @Override
    public Optional<String> validate(Order order) {
        for (OrderLine line : order.getLines()) {
            Optional<Product> product =
                    products.findById(line.productId());
            if (product.isEmpty()) {
                return Optional.of("unknown product " + line.productId());
            }
            if (product.get().stock() < line.quantity()) {
                return Optional.of(
                        "not enough stock for " + line.productId());
            }
        }
        return Optional.empty();
    }
}
```

AddressValidator.java y AmountValidator.java

```java
package shoplab;

import java.util.Optional;
import org.springframework.stereotype.Component;

@Component
@org.springframework.core.annotation.Order(2)
public class AddressValidator implements OrderValidator {

    private final RegionSelector regions;

    public AddressValidator(RegionSelector regions) {
        this.regions = regions;
    }

    @Override
    public Optional<String> validate(Order order) {
        ShippingAddress a = order.getShippingAddress();
        if (blank(a.getRecipient())) {
            return Optional.of("recipient is blank");
        }
        if (blank(a.getStreet())) {
            return Optional.of("street is blank");
        }
        if (blank(a.getCity())) {
            return Optional.of("city is blank");
        }
        if (blank(a.getPostalCode())) {
            return Optional.of("postalCode is blank");
        }
        if (!regions.regions().contains(a.getCountry())) {
            return Optional.of("unknown country: " + a.getCountry());
        }
        return Optional.empty();
    }

    private static boolean blank(String text) {
        return text == null || text.isBlank();
    }
}

// ----- archivo aparte: AmountValidator.java -----
package shoplab;

import java.util.Optional;
import org.springframework.stereotype.Component;

@Component
@org.springframework.core.annotation.Order(3)
public class AmountValidator implements OrderValidator {

    private final ShopSettings settings;

    public AmountValidator(ShopSettings settings) {
        this.settings = settings;
    }

    @Override
    public Optional<String> validate(Order order) {
        if (order.getLines().size() > settings.maxItemsPerOrder()) {
            return Optional.of("too many lines: max "
```

`ShippingAddress` es un `@Value` de Lombok, por eso los getters son `getRecipient()`, `getStreet()`...; `ShopSettings` es un *record* y usa `maxItemsPerOrder()`. Los constructores de `ShippingAddress` y `Order` ya rechazan algunos campos en blanco (da 400 antes de llegar aquí); el validador es la red para el resto (código postal, región).

`ValidationChain.java`

```
package shoplab;

import java.util.List;
import java.util.Optional;
import org.springframework.stereotype.Service;

@Service
public class ValidationChain {

    private final List<OrderValidator> validators;

    public ValidationChain(List<OrderValidator> validators) {
        this.validators = validators;
    }

    /** Recorre en orden y lanza con el primer error. */
    public void validate(Order order) {
        for (OrderValidator validator : validators) {
            Optional<String> error = validator.validate(order);
            if (error.isPresent()) {
                throw new ValidationException(error.get());
            }
        }
    }
}
```

`ApiExceptionHandler.java (método nuevo)`

```
    // Imports nuevos: org.springframework.http.ResponseEntity y
    // org.springframework.web.bind.annotation.ExceptionHandler
    // (ya estarán por los otros manejadores).
    @ExceptionHandler(ValidationException.class)
    public ResponseEntity<ApiError> validation(ValidationException ex) {
        return ResponseEntity.status(422)
                .body(new ApiError(ex.getMessage()));
    }
```

Se añade a la clase existente; los demás manejadores (404, 400, 409) no cambian. Se usa el número 422 para no depender del nombre de la constante del `HttpStatus`, que ha cambiado entre versiones de Spring.

OrderService.java (draft y create)

```
    // Se asume que los campos del servicio se llaman products, orders y
    // machine; usa los de tu clase si difieren. Imports: java.util.UUID.

    /** Construye el pedido sin guardarlo. */
    public Order draft(CreateOrderRequest request) {
        Order.OrderBuilder builder = Order.builder()
                .id(UUID.randomUUID().toString())
                .shippingAddress(request.shippingAddress())
                .region(request.region())
                .paymentType(request.paymentType());
        for (OrderItemRequest item : request.items()) {
            Product product = products.findById(item.productId())
                    .orElseThrow(() -> new ProductNotFoundException(
                            item.productId()));
            builder.line(new OrderLine(item.productId(), item.quantity(),
                    product.price()));
        }
        return builder.build();
    }

    public Order create(CreateOrderRequest request) {
        return orders.save(draft(request));
    }
```

Si tu `create` del H3 ya construía el pedido de otra manera, copia ese cuerpo en `draft` y deja que `create` lo llame y lo guarde: el resultado de `create` no cambia. El pedido nace en `Created` (valor por defecto del H7).

CheckoutFacade.java

```
package shoplab;

import java.util.Map;
import org.springframework.stereotype.Service;

@Service
public class CheckoutFacade {

    private final CheckoutMediator mediator;
    private final Map<String, NotificationChannel> channels;
    private final PriceCalculator calculator;
    private final DiscountService discounts;
    private final ValidationChain validation;

    public CheckoutFacade(CheckoutMediator mediator,
                          Map<String, NotificationChannel> channels,
                          PriceCalculator calculator,
                          DiscountService discounts,
                          ValidationChain validation) {
        this.mediator = mediator;
        this.channels = channels;
        this.calculator = calculator;
        this.discounts = discounts;
        this.validation = validation;
    }

    public Money amountDue(Order order) {
        return amountDue(order, calculator);
    }

    public Money amountDue(Order order, PriceCalculator custom) {
        Money price = custom.price(order);
        return price.minus(discounts.best(order, price));
    }

    public CheckoutResult checkout(Order order, String channel) {
        if (!channels.containsKey(channel)) {
            throw new IllegalArgumentException(
                    "unknown channel: " + channel);
        }
        validation.validate(order);
        return mediator.place(order, amountDue(order), channel);
    }
}
```

OrderController.java (constructor y método nuevo)

```
    // Campos y constructor: se añade ValidationChain.
    private final OrderService service;
    private final CheckoutFacade facade;
    private final ValidationChain validation;

    public OrderController(OrderService service, CheckoutFacade facade,
                           ValidationChain validation) {
        this.service = service;
        this.facade = facade;
        this.validation = validation;
    }

    // Imports: java.util.Map. Lo demás (create, find, repeat, checkout,
    // ship, cancel) no cambia.
    @PostMapping("/validate")
    public Map<String, Object> validate(
            @RequestBody CreateOrderRequest request) {
        Order draft = service.draft(request);
        validation.validate(draft);
        return Map.of("valid", true);
    }
```

`POST /orders/validate` no choca con `POST /orders/{id}/repeat`: tienen distinto número de segmentos. Como `validate` no llama a `save`, no queda nada guardado.

CatalogService.java (con products)

```
package shoplab;

public interface CatalogService {

    CatalogNode tree();

    /** Productos del catálogo, recorridos en profundidad y sin lista. */
    default Iterable<Product> products() {
        return () -> new CatalogIterator(tree());
    }
}
```

CatalogIterator.java

```java
package shoplab;

import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Iterator;
import java.util.List;
import java.util.NoSuchElementException;

public class CatalogIterator implements Iterator<Product> {

    private final Deque<CatalogNode> pending = new ArrayDeque<>();
    private Product ready;

    public CatalogIterator(CatalogNode root) {
        pending.push(root);
        advance();
    }

    /** Busca el siguiente producto y lo deja en 'ready'. */
    private void advance() {
        ready = null;
        while (ready == null && !pending.isEmpty()) {
            CatalogNode node = pending.pop();
            switch (node) {
                case Product p -> ready = p;
                case Category c -> pushInOrder(c.children());
                case Pack k -> pushInOrder(List.copyOf(k.products()));
            }
        }
    }

    /** Apila al revés para que el primer hijo sea el primero en salir. */
    private void pushInOrder(List<? extends CatalogNode> nodes) {
        for (int i = nodes.size() - 1; i >= 0; i--) {
            pending.push(nodes.get(i));
        }
    }

    @Override
    public boolean hasNext() {
        return ready != null;
    }

    @Override
    public Product next() {
        if (ready == null) {
            throw new NoSuchElementException();
        }
        Product result = ready;
        advance();
        return result;
    }
}
```

CatalogController.java

```java
package shoplab;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/catalog")
public class CatalogController {

    private final CatalogService catalog;

    public CatalogController(CatalogService catalog) {
        this.catalog = catalog;
    }

    @GetMapping
    public CatalogNode tree() {
        return catalog.tree();
    }

    @GetMapping("/flat")
    public Iterable<Product> flat() {
        return catalog.products();
    }
}
```

Prueba, con ShopLab en marcha y `p1` con `stock` 12 (si una URL lleva `?`, va entre comillas):

```
curl -i -H "Content-Type: application/json" -d '{"items":[{"productId":"p1","
    ↪ quantity":50}],"shippingAddress":{"recipient":"Ana","street":"Mayor
    ↪ 1","city":"Madrid","postalCode":"28001","country":"ES"},"region":"ES","
    ↪ paymentType":"CARD"}' localhost:8080/orders/validate
curl -i -H "Content-Type: application/json" -d '{"items":[{"productId":"p1","
    ↪ quantity":1}],"shippingAddress":{"recipient":"Ana","street":"Mayor 1","
    ↪ city":"Madrid","postalCode":"28001","country":"XX"},"region":"ES","
    ↪ paymentType":"CARD"}' localhost:8080/orders/validate
curl -s localhost:8080/catalog/flat
```

La primera responde 422 con `{"error":"not enough stock for p1"}`; la segunda, 422 con `unknown country: XX`; la tercera, un array con todos los productos, en el orden del catálogo. Con un pedido válido, `validate` responde 200 con `{"valid":true}`. Al hacer checkout de un pedido de 50 unidades, ahora responde 422 (antes 400) y no se cobra nada. **Errores típicos:** importar `org.springframework.core.annotation.Order` en un validador (tapa a `shoplab.Order`); llamar a `service.create` en `validate` (guardaría el pedido); olvidar `@Order` en un validador (el orden de la lista deja de estar garantizado); y mover la validación al controlador en lugar de la fachada, de modo que otros caminos de cobro la saltarían.

CAPÍTULO 12

Visitor e Interpreter

Objetivos del capítulo

- Reconocer el olor de una cadena de `instanceof` que genera informes sobre una jerarquía.
- Implementar Visitor clásico y explicar el doble despacho.
- Implementar la misma idea con `sealed` y `switch`, y conocer su límite.
- Reconocer el olor de las reglas de negocio escritas en `if`.
- Implementar Interpreter: gramática, AST con `sealed` y records, analizador recursivo descendente mínimo y evaluador.
- Decidir cuándo no usar Visitor ni Interpreter.
- Dar a ShopLab informes sobre el catálogo y un mini-DSL de promociones.

El catálogo del capítulo 6 es un árbol. Hoy quieres un informe de existencias; mañana, uno de precios; pasado, una exportación. Si cada informe se escribe dentro de los nodos, los nodos engordan sin parar. Visitor saca la operación fuera del árbol[7, 21]. Interpreter resuelve un problema vecino: unas reglas («descuento si el precio supera 100 y es un libro») que el negocio cambia sin avisar y que no quieres recompilar. Se representan como un árbol de datos y se evalúan[7, 21].

Java sin compilar y qué Java usamos

Como en los capítulos 3 a 11, el código Java está escrito con cuidado contra la documentación, pero no lo ha compilado ni ejecutado su autor. Todo se ciñe a Java 21: `sealed`, records y `switch` con patrones de tipo, que son definitivos en esa versión[16, 2]. No usamos patrones ni variables sin nombre (`_`), que en Java 21 siguen en vista previa. Los ejemplos de este capítulo son propios; el libro de Kiehl[12] (edición en preparación) trata en profundidad el estilo de datos con tipos sellados, pero ningún ejemplo de aquí depende de él.

12.1 El olor: `instanceof` en cadena para generar informes

Para contar existencias en el árbol del catálogo, el camino más corto es preguntar el tipo de cada nodo (clase de ejemplo en `shoplab.ejemplos.visitor`; no es de ShopLab):

StockOlor.java (olor)

```
1  package shoplab.ejemplos.visitor;
2
3  import shoplab.CatalogNode;
4  import shoplab.Category;
5  import shoplab.Pack;
6  import shoplab.Product;
7
```

```
public final class StockOlor {

    private StockOlor() {
    }

    public static int units(CatalogNode node) {
        if (node instanceof Product product) {
            return product.stock();
        } else if (node instanceof Category category) {
            int sum = 0;
            for (CatalogNode child : category.children()) {
                sum += units(child);
            }
            return sum;
        } else if (node instanceof Pack pack) {
            int sum = 0;
            for (Product product : pack.products()) {
                sum += product.stock();
            }
            return sum;
        }
        throw new IllegalArgumentException("unknown node " + node.name());
    }
}
```

Funciona, pero: hay que repetir esta escalera por cada informe; si aparece un nodo nuevo, el compilador no avisa de las escaleras que se quedan sin su rama; y el `throw` final es una puerta abierta a errores en ejecución.

12.2 Visitor clásico y doble despacho

Visitor define una interfaz con *un método por tipo de nodo* y obliga a cada nodo a tener un único método, `accept`, que se limita a llamar al método de su tipo. Las operaciones nuevas son clases nuevas que implementan la interfaz[7, 20].

CatalogVisitor.java

```
package shoplab;

public interface CatalogVisitor<R> {

    R visitProduct(Product product);

    R visitCategory(Category category);

    R visitPack(Pack pack);
}
```

Cada nodo de `CatalogNode` gana el método `accept`:

Los nodos aceptan visitantes

```
// En CatalogNode (interfaz sellada), añade:
<R> R accept(CatalogVisitor<R> visitor);
```

```
3
4  // En el record Product, añade dentro del cuerpo:
5  @Override
6  public <R> R accept(CatalogVisitor<R> visitor) {
7      return visitor.visitProduct(this);
8  }
9
10 // En el record Category, añade dentro del cuerpo:
11 @Override
12 public <R> R accept(CatalogVisitor<R> visitor) {
13     return visitor.visitCategory(this);
14 }
15
16 // En el record Pack, añade dentro del cuerpo:
17 @Override
18 public <R> R accept(CatalogVisitor<R> visitor) {
19     return visitor.visitPack(this);
20 }
```

Esas cuatro líneas son lo único que cambian los nodos, una vez, para siempre. Un visitante concreto (aquí, uno de ejemplo que solo cuenta productos) recorre el árbol llamando a `accept` sobre los hijos:

ProductCounter.java

```
1  package shoplab.ejemplos.visitor;
2
3  import shoplab.CatalogNode;
4  import shoplab.CatalogVisitor;
5  import shoplab.Category;
6  import shoplab.Pack;
7  import shoplab.Product;
8
9  public class ProductCounter implements CatalogVisitor<Integer> {
10
11     @Override
12     public Integer visitProduct(Product product) {
13         return 1;
14     }
15
16     @Override
17     public Integer visitCategory(Category category) {
18         int total = 0;
19         for (CatalogNode child : category.children()) {
20             total += child.accept(this);
21         }
22         return total;
23     }
24
25     @Override
26     public Integer visitPack(Pack pack) {
27         return pack.products().size();
28     }
29 }
```

El doble despacho

Una llamada `node.accept(visitor)` elige código dos veces:

1. **Por el tipo del nodo.** `node` se declara como `CatalogNode`, pero en ejecución es un `Category`; Java escoge el `accept` de `Category` (despacho dinámico normal).
2. **Por el tipo del visitante.** Dentro de ese `accept`, la llamada `visitor.visitCategory(this)` sabe ya que el nodo es una `Category` (el tipo de `this`), y Java escoge la implementación de `visitCategory` de la clase concreta del visitante.

El resultado depende de *dos* tipos a la vez, algo que un solo `obj.metodo()` no hace. Por eso los métodos se llaman `visitProduct`, `visitCategory`... y no `visit` sobrecargado: así el despacho por el tipo del nodo queda explícito en un sitio visible.

Error típico

Olvidar la recursión: en `visitCategory`, llamar a `visitProduct(child)` a mano exige saber el tipo del hijo y vuelve a la escalera de `instanceof`. Llama siempre a `child.accept(this)`.

12.3 Visitor con `sealed` y `switch`

Como `CatalogNode` es sellada, el compilador conoce todos sus subtipos. Un `switch` con patrones de tipo sobre ella tiene que cubrirlos todos o no compila[2, 16]. La misma operación, sin interfaz de visitante ni `accept`:

StockSwitch.java

```
package shoplab.ejemplos.visitor;

import shoplab.CatalogNode;
import shoplab.Category;
import shoplab.Pack;
import shoplab.Product;

public final class StockSwitch {

    private StockSwitch() {
    }

    public static int units(CatalogNode node) {
        return switch (node) {
            case Product product -> product.stock();
            case Category category -> category.children().stream()
                    .mapToInt(StockSwitch::units).sum();
            case Pack pack -> pack.products().stream()
                    .mapToInt(Product::stock).sum();
        };
    }
}
```

Sin `default`: si añades un cuarto subtipo a `CatalogNode`, este `switch` deja de compilar y el compilador te dice dónde. Es el mismo servicio que presta el Visitor clásico (una nueva interfaz de métodos hace fallar a todos los visitantes), con menos ceremonia.

El problema de la expresión

Las dos formas comparten un límite: la estructura es un eje y las operaciones, otro.

	Operación nueva	Tipo de nodo nuevo
Método en cada nodo	Tocas todos los nodos	Escribes un nodo
Visitor / `switch` sobre `sealed`	Escribes un visitante o un `switch`	Tocas todos los visitantes

Visitor y `switch` sellado ganan cuando las operaciones cambian a menudo y los tipos casi nunca; pierden cuando ocurre lo contrario. Diferencia práctica: el Visitor clásico permite cambiar de visitante en ejecución (se pasa como objeto, por ejemplo desde un `Map`); el `switch` es una función cerrada, más corta y más fácil de leer. Ni uno ni otro es mejor en general: el valor de un patrón depende del contexto[30].

12.4 Y en Spring: visitantes como beans seleccionados por nombre

Spring no aporta un Visitor propio. Lo único que hace es lo de siempre: guardar los visitantes como beans y entregártelos todos en un `Map<String, T>`, con el nombre del bean como clave, igual que hicimos con las fábricas y los canales (capítulo 3):

Selección por nombre (esquema)

```
@Component("stock")
public class StockReportVisitor implements CatalogVisitor<StockReport> {
    // ...
}

@RestController
@RequestMapping("/reports")
public class ReportController {

    private final Map<String, CatalogVisitor<?>> visitors;

    public ReportController(Map<String, CatalogVisitor<?>> visitors) {
        this.visitors = visitors;
    }
}
```

Añadir un informe nuevo es añadir una clase `@Component("nombre")`: el controlador no cambia. Lo que Spring pone es la inyección; el patrón sigue siendo el de arriba. El código completo está en el hito.

12.5 El olor: reglas de negocio escritas en `if`

Marketing quiere promociones: «15 % en libros caros», «envío gratis si hay poco stock y es música». Lo primero que sale:

PromoOlor.java (olor)

```
package shoplab.ejemplos.promo;

```

```
import shoplab.Product;

public final class PromoOlor {

    private PromoOlor() {
    }

    public static boolean applies(Product product) {
        return product.price().amount().intValue() > 100
                && product.category().equals("books");
    }
}
```

Cada regla nueva, o cada umbral que cambia, es una edición del código, una recompilación y un despliegue. Si las reglas deben poder venir de un fichero de configuración o de una petición, hay que tratarlas como *datos*.

12.6 Interpreter: gramática, AST y analizador mínimo

Interpreter representa las frases de un lenguaje pequeño como un árbol y le añade un evaluador[7, 21]. Para promociones usamos esta gramática (en notación informal; `|` es «o», `*` «cero o más veces»):

Gramática del mini-DSL

```
rule       := or
or         := and ( "OR" and )*
and        := not ( "AND" not )*
not        := "NOT" not | primary
primary    := "(" rule ")" | comparison
comparison := variable operator value
variable   := "price" | "stock" | "category"
operator   := "==" | "!=" | "<" | "<=" | ">" | ">="
```

La precedencia sale de la gramática: `NOT` liga más que `AND`, y este más que `OR`. Un ejemplo: `price > 100 AND category == books`.

El AST con `sealed` y records

Expr.java

```
package shoplab;

import java.math.BigDecimal;
import java.util.Set;

public sealed interface Expr
        permits Expr.Compare, Expr.And, Expr.Or, Expr.Not {

    enum Op { EQ, NE, LT, LE, GT, GE }

    record Compare(String variable, Op op, String value)
```

```
            implements Expr {

        private static final Set<String> VARIABLES =
                Set.of("price", "stock", "category");

        public Compare {
            if (!VARIABLES.contains(variable)) {
                throw new IllegalArgumentException(
                        "unknown variable: " + variable);
            }
            if (variable.equals("category")) {
                if (op != Op.EQ && op != Op.NE) {
                    throw new IllegalArgumentException(
                            "category only accepts == and !=");
                }
            } else {
                try {
                    new BigDecimal(value);
                } catch (NumberFormatException e) {
                    throw new IllegalArgumentException(
                            "not a number: " + value);
                }
            }
        }
    }

    record And(Expr left, Expr right) implements Expr {
    }

    record Or(Expr left, Expr right) implements Expr {
    }

    record Not(Expr operand) implements Expr {
    }
}
```

El constructor compacto de `Compare` valida la comparación al construirla: un AST con variables inexistentes no puede existir.

El analizador recursivo descendente

Un analizador recursivo descendente tiene un método por cada regla de la gramática; cada método consume los símbolos que le tocan y llama a los de las reglas que usa. Primero se parte el texto en símbolos (*tokens*); luego los métodos `or`, `and`, `not`, `primary` y `compare` copian la gramática.

PromoParser.java

```
package shoplab;

import java.util.ArrayList;
import java.util.List;
import java.util.Objects;
import java.util.regex.Matcher;
import java.util.regex.Pattern;
```

```
8
9  import org.springframework.stereotype.Component;
10
11 @Component
12 public class PromoParser {
13
14     private static final Pattern TOKEN = Pattern.compile(
15             "\\s*(<=|>=|==|!=|<|>|\\(|\\)|[A-Za-z0-9_.]+)");
16
17     public Expr parse(String text) {
18         Objects.requireNonNull(text, "text");
19         return new Run(tokenize(text.strip())).parseAll();
20     }
21
22     private static List<String> tokenize(String text) {
23         List<String> tokens = new ArrayList<>();
24         Matcher matcher = TOKEN.matcher(text);
25         int position = 0;
26         while (position < text.length()) {
27             matcher.region(position, text.length());
28             if (!matcher.lookingAt()) {
29                 throw new IllegalArgumentException(
30                         "unexpected text at: "
31                                 + text.substring(position));
32             }
33             tokens.add(matcher.group(1));
34             position = matcher.end();
35         }
36         return tokens;
37     }
38
39     private static final class Run {
40
41         private final List<String> tokens;
42         private int position;
43
44         Run(List<String> tokens) {
45             this.tokens = tokens;
46         }
47
48         Expr parseAll() {
49             Expr result = or();
50             if (position < tokens.size()) {
51                 throw new IllegalArgumentException(
52                         "unexpected token: " + tokens.get(position));
53             }
54             return result;
55         }
56
57         private Expr or() {
58             Expr left = and();
59             while (accept("OR")) {
60                 left = new Expr.Or(left, and());
61             }
62             return left;
63         }
64
```

```
65          private Expr and() {
66              Expr left = not();
67              while (accept("AND")) {
68                  left = new Expr.And(left, not());
69              }
70              return left;
71          }
72
73          private Expr not() {
74              if (accept("NOT")) {
75                  return new Expr.Not(not());
76              }
77              return primary();
78          }
79
80          private Expr primary() {
81              if (accept("(")) {
82                  Expr inner = or();
83                  expect(")");
84                  return inner;
85              }
86              return compare();
87          }
88
89          private Expr compare() {
90              String variable = next();
91              Expr.Op op = operator(next());
92              String value = next();
93              return new Expr.Compare(variable, op, value);
94          }
95
96          private static Expr.Op operator(String token) {
97              return switch (token) {
98                  case "==" -> Expr.Op.EQ;
99                  case "!=" -> Expr.Op.NE;
100                 case "<" -> Expr.Op.LT;
101                 case "<=" -> Expr.Op.LE;
102                 case ">" -> Expr.Op.GT;
103                 case ">=" -> Expr.Op.GE;
104                 default -> throw new IllegalArgumentException(
105                         "unknown operator: " + token);
106             };
107         }
108
109         private boolean accept(String token) {
110             if (position < tokens.size()
111                     && tokens.get(position).equals(token)) {
112                 position++;
113                 return true;
114             }
115             return false;
116         }
117
118         private void expect(String token) {
119             if (!accept(token)) {
120                 throw new IllegalArgumentException(
121                         "expected " + token);
```

```
            }
        }

        private String next() {
            if (position >= tokens.size()) {
                throw new IllegalArgumentException(
                        "unexpected end of rule");
            }
            return tokens.get(position++);
        }
    }
}
```

Dos detalles. `or` y `and` usan un bucle porque la gramática repite (`( "OR" and )*`); el resultado es asociativo por la izquierda. Y el estado mutable (`position`) vive en una clase interna que se crea en cada llamada a `parse`: así el bean `PromoParser` es seguro entre hilos.

El evaluador

El evaluador es un `switch` sobre el AST sellado, con una rama por cada tipo de nodo. En la versión de GoF cada nodo se evalúa a sí mismo; aquí el `switch` cumple ese papel con menos código[7, 2].

PromoEvaluator.java

```
package shoplab;

import java.math.BigDecimal;

import org.springframework.stereotype.Component;

@Component
public class PromoEvaluator {

    public boolean evaluate(Expr expr, Product product) {
        return switch (expr) {
            case Expr.Compare compare -> compare(compare, product);
            case Expr.And and ->
                    evaluate(and.left(), product)
                            && evaluate(and.right(), product);
            case Expr.Or or ->
                    evaluate(or.left(), product)
                            || evaluate(or.right(), product);
            case Expr.Not not -> !evaluate(not.operand(), product);
        };
    }

    private boolean compare(Expr.Compare compare, Product product) {
        if (compare.variable().equals("category")) {
            boolean same =
                    product.category().equalsIgnoreCase(compare.value());
            return compare.op() == Expr.Op.EQ ? same : !same;
        }
        BigDecimal actual = compare.variable().equals("price")
                ? product.price().amount()
```

```
                : BigDecimal.valueOf(product.stock());
        int order = actual.compareTo(new BigDecimal(compare.value()));
        return switch (compare.op()) {
            case EQ -> order == 0;
            case NE -> order != 0;
            case LT -> order < 0;
            case LE -> order <= 0;
            case GT -> order > 0;
            case GE -> order >= 0;
        };
    }
}
```

12.7 Y en Spring: `PromoService` con reglas de la configuración o de la petición

Spring no trae un Interpreter para este lenguaje (tiene el suyo, SpEL, pero no lo usamos aquí). Lo que aporta es dónde guardar y de dónde leer las reglas. `PromoParser` y `PromoEvaluator` ya son beans; un servicio los junta y acepta la regla de la petición o, si no viene ninguna, una por defecto leída de `application.properties`. El código completo está en el hito.

12.8 Cuándo no usar Visitor ni Interpreter

- **Visitor** no compensa si la jerarquía cambia más que las operaciones, o si solo hay una operación: un método en el nodo es más sencillo. Si la jerarquía es sellada y las operaciones son pocas, el `switch` suele bastar.
- **Interpreter** no compensa si las reglas se pueden expresar con unos pocos parámetros de configuración (un umbral y una categoría); un lenguaje propio es una superficie que mantener, documentar y proteger. Si la gramática crece mucho, el analizador a mano se vuelve difícil: ahí se usan generadores de analizadores o un lenguaje de expresiones existente.

12.9 En el proyecto: H9

Hito 9: informes con Visitor y promociones con el mini-DSL

Partes del H8b. En el H9:

- `CatalogVisitor<R>` y `accept` en los cuatro tipos de `CatalogNode`.
- `StockReportVisitor` (existencias totales y productos agotados) y `PriceReportVisitor` (número de productos, suma de precios y el más caro), como beans `stock` y `price`.
- `GET /reports/{name}`: el controlador recibe un `Map<String, CatalogVisitor<?>>`; 404 si el informe no existe.
- `Expr`, `PromoParser`, `PromoEvaluator` y `PromoService`, con la regla por defecto en `shop.promo.default-rule`.

- `POST /promos/evaluate`: evalúa una regla sobre un producto. No cambia pedidos ni precios.

Tres decisiones: los productos de un `Pack` se cuentan una sola vez, en el pack (el catálogo ya los quita de la categoría); los precios se suman tal cual, sin convertir monedas (el catálogo de ShopLab es todo en euros); y una regla mal escrita lanza `IllegalArgumentException`, que `ApiExceptionHandler` ya traduce a 400. El código completo está en la solución del ejercicio 12.6.

12.10 Ejercicios

Ejercicio 12.1: Tamaño de un sistema de ficheros, de dos maneras ★★★☆☆

En clases propias (paquete `shoplab.ejercicios.fs`), crea la interfaz sellada `FsNode` con los records `FileNode(String name, long bytes)` y `FolderNode(String name, List<FsNode> children)`. Calcula el tamaño total de una carpeta (suma de los `bytes` de todos los ficheros que contiene) de dos maneras: (1) con un Visitor clásico (`FsVisitor<R>`, `accept`, `SizeVisitor`) y (2) con un `switch` sobre la interfaz sellada. Escribe una clase con `main` que construya una carpeta `docs` con ficheros de 100 y 250 bytes y una subcarpeta con otro de 50 bytes, e imprima ambos resultados. Está bien si: las dos versiones imprimen `400`, y el `switch` no tiene `default`.

Pista: *No uses `CatalogNode`: esta jerarquía es otra. En el visitante de carpeta, llama a `child.accept(this)` sobre cada hijo.*

Práctica ▪ solución en la pág. 376

Ejercicio 12.2: Sigue la llamada: ¿quién decide qué? ★★★☆☆

Lee este código. Todas sus piezas están explicadas en el capítulo.

ShapeDemo.java

```java
public class ShapeDemo {

    interface ShapeVisitor<R> {
        R visitCircle(Circle circle);
        R visitSquare(Square square);
    }

    interface Shape {
        <R> R accept(ShapeVisitor<R> visitor);
    }

    record Circle(double radius) implements Shape {
        public <R> R accept(ShapeVisitor<R> visitor) {
            System.out.println("Circle.accept");
            return visitor.visitCircle(this);
        }
    }

    record Square(double side) implements Shape {
        public <R> R accept(ShapeVisitor<R> visitor) {
            System.out.println("Square.accept");
            return visitor.visitSquare(this);
        }
    }

    static class AreaVisitor implements ShapeVisitor<Double> {
        public Double visitCircle(Circle circle) {
            System.out.println("AreaVisitor.visitCircle");
            return Math.PI * circle.radius() * circle.radius();
        }
        public Double visitSquare(Square square) {
            System.out.println("AreaVisitor.visitSquare");
            return square.side() * square.side();
        }
    }

    public static void main(String[] args) {
        Shape shape = new Square(3);
        double area = shape.accept(new AreaVisitor());
        System.out.println(area);
    }
}
```

Responde: (a) ¿qué imprime el programa, línea a línea? (b) ¿Qué tipo tiene `shape` para el compilador y cuál en ejecución? ¿Qué decide cada uno de los dos despachos? (c) Si añades `record Triangle(double base, double height) implements Shape`, ¿qué tienes que tocar y qué te avisa el compilador? Está bien si: tu traza nombra los dos despachos en el orden en que ocurren.

Lectura de código ▪ solución en la pág. 378

Ejercicio 12.3: Pruebas para el evaluador de promociones ★★★☆☆

Escribe una clase de prueba JUnit (capítulo 3) para `PromoParser` y `PromoEvaluator` con tres métodos `@Test`, uno por regla: (1) `price > 100 AND category == books`; (2) `NOT stock == 0 OR category == music`; (3) `price <= 20.00 OR stock < 3`. Cada prueba comprueba la regla con al menos un producto que la cumple y otro que no. Está bien si: las tres pruebas pasan con `./gradlew test -tests shoplab.PromoRulesTest`.

Pista: *Un método auxiliar `matches(String rule, Product product)` que use `new PromoParser().parse(...)` y `new PromoEvaluator().evaluate(...)` evita repetir código. Los productos se construyen con el constructor de `Product`.*

Test ▪ solución en la pág. 378

Ejercicio 12.4: El operador IN en la gramática ★★★★☆

Haz una copia del proyecto (`cp -r`); este ejercicio no forma parte del estado de ShopLab. Añade a la gramática el operador `IN` para categorías: `category IN (books, music)`. Necesitas: un nuevo record `Expr.In(String variable, List<String> values)` (que solo acepte la variable `category`), que el analizador reconozca la coma y la lista, y que el evaluador tenga su rama. Está bien si: `category IN (books, music) AND price < 30` es verdadera para un libro de 20,00; es falsa para un producto de la categoría `hardware`; y el compilador te marca el `switch` del evaluador como incompleto hasta que añades la rama.

Pista: *Añade la coma al patrón de los símbolos y, en `compare()`, mira si el siguiente símbolo es `IN` antes de leer un operador.*

Práctica ▪ solución en la pág. 383

Ejercicio 12.5: ¿Visitor, switch o método en el nodo? ★★★★★

El equipo de ShopLab te pide tres cosas para el catálogo: (a) una exportación a CSV que cambia de columnas cada trimestre; (b) un método `displayName()` que todos los nodos deben ofrecer y que casi nunca cambia; (c) que el catálogo admita pronto un cuarto tipo de nodo, `Bundle`, y dentro de un año un quinto. Decide, para cada caso, entre un Visitor clásico, un `switch` sobre la jerarquía sellada y un método en cada nodo, y defiende la decisión por escrito (una página como máximo). Está bien si: usas el eje «operaciones nuevas frente a tipos nuevos», valoras la ayuda del compilador y el acoplamiento, y dices qué cambiaría tu decisión.

Diseño ▪ solución en la pág. 387

Ejercicio 12.6: Hito H9: informes con Visitor y promociones con el mini-DSL ★★★★☆

Implementa el hito H9 sobre ShopLab tal como lo dejó el capítulo anterior: `CatalogVisitor`, `accept`, `StockReport`/`StockReportVisitor`, `PriceReport`/`PriceReportVisitor`, `ReportController`, `Expr`, `PromoParser`, `PromoEvaluator`, `PromoService`, `PromoRequest`, `PromoResult`, `PromoController` y la propiedad `shop.promo.default-rule`. Está bien si: con `p1` (stock 12, 49,90 EUR) y `b1` (stock 5, 20,00 EUR), `GET /reports/stock` responde `units` 17; `GET /reports/price` responde `products` 2; `GET /reports/zz` responde 404; y `POST /promos/evaluate` con `price > 100 AND category == books` sobre un libro de 120,00 responde `matches true`, y sobre `b1`, `false`; una regla mal escrita responde 400.

Pista: *Escribe primero `Expr`, `PromoParser` y `PromoEvaluator` tal como aparecen en el capítulo; el resto del hito se apoya en ellos.*

Proyecto ▪ solución en la pág. 387

12.11 Resumen

Resumen

- Una escalera de `instanceof` para cada informe se repite y no avisa cuando aparece un tipo nuevo.
- Visitor saca las operaciones de la jerarquía: cada nodo tiene un `accept` y cada operación es un visitante.
- El doble despacho elige código por el tipo del nodo (`accept`) y por el tipo del visitante (`visitX`).
- Con `sealed`, un `switch` sin `default` hace lo mismo con menos código, y el compilador exige cubrir todos los casos.
- El problema de la expresión: Visitor y `switch` facilitan operaciones nuevas y dificultan tipos nuevos; los métodos en el nodo, al revés.
- En Spring, los visitantes son beans elegidos por nombre con un `Map<String, T>`; Spring no aporta un Visitor propio.
- Reglas de negocio en `if` obligan a recompilar; Interpreter las convierte en un AST evaluable.
- Interpreter: gramática, AST sellado con records, analizador recursivo descendente (un método por regla) y evaluador con `switch`.
- No uses Visitor si la jerarquía cambia mucho, ni Interpreter si unos parámetros de configuración bastan.
- En ShopLab, `GET /reports/{name}` elige el visitante por nombre y `POST /promos/evaluate` evalúa una regla sobre un producto.

Soluciones de los ejercicios

Solución del ejercicio 12.1

enunciado en la pág. 372

FsNode.java, FileNode.java, FolderNode.java, FsVisitor.java

```
package shoplab.ejercicios.fs;

public sealed interface FsNode permits FileNode, FolderNode {

    String name();

    <R> R accept(FsVisitor<R> visitor);
}

// ----- archivo aparte: FileNode.java -----
package shoplab.ejercicios.fs;

public record FileNode(String name, long bytes) implements FsNode {

    @Override
    public <R> R accept(FsVisitor<R> visitor) {
        return visitor.visitFile(this);
    }
}

// ----- archivo aparte: FolderNode.java -----
package shoplab.ejercicios.fs;

import java.util.List;

public record FolderNode(String name, List<FsNode> children)
        implements FsNode {

    @Override
    public <R> R accept(FsVisitor<R> visitor) {
        return visitor.visitFolder(this);
    }
}

// ----- archivo aparte: FsVisitor.java -----
package shoplab.ejercicios.fs;

public interface FsVisitor<R> {

    R visitFile(FileNode file);

    R visitFolder(FolderNode folder);
}
```

SizeVisitor.java, FsSizes.java, FsCheck.java

```java
package shoplab.ejercicios.fs;

public class SizeVisitor implements FsVisitor<Long> {

    @Override
    public Long visitFile(FileNode file) {
        return file.bytes();
    }

    @Override
    public Long visitFolder(FolderNode folder) {
        long total = 0;
        for (FsNode child : folder.children()) {
            total += child.accept(this);
        }
        return total;
    }
}

// ----- archivo aparte: FsSizes.java -----
package shoplab.ejercicios.fs;

public final class FsSizes {

    private FsSizes() {
    }

    public static long size(FsNode node) {
        return switch (node) {
            case FileNode file -> file.bytes();
            case FolderNode folder -> folder.children().stream()
                    .mapToLong(FsSizes::size).sum();
        };
    }
}

// ----- archivo aparte: FsCheck.java -----
package shoplab.ejercicios.fs;

import java.util.List;

public class FsCheck {

    public static void main(String[] args) {
        FsNode docs = new FolderNode("docs", List.of(
                new FileNode("a.txt", 100),
                new FileNode("b.txt", 250),
                new FolderNode("old", List.of(
                        new FileNode("c.txt", 50)))));
        System.out.println(docs.accept(new SizeVisitor()));
        System.out.println(FsSizes.size(docs));
    }
}
```

Ejecuta con `./gradlew compileJava` y `java -cp build/classes/java/main shoplab.ejercicios.fs.FsCheck`: imprime `400` dos veces. El visitante necesita la interfaz y los `accept`; el `switch` cabe en una función y, sin `default`, deja de compilar si añades un tercer subtipo. Como `SizeVisitor` devuelve `Long`, la línea `total += child.accept(this)` desempaqueta sola. Error típico: poner `default` en el `switch`; se pierde justo el aviso del compilador.

Solución del ejercicio 12.2

enunciado en la pág. 372

(a) Imprime, en este orden:

Salida

```
Square.accept
AreaVisitor.visitSquare
9.0
```

(b) Para el compilador, `shape` es un `Shape`; en ejecución es un `Square`. Primer despacho: `shape.accept(...)` elige el `accept` de `Square` por el tipo en ejecución del nodo (imprime `Square.accept`). Dentro de él, `this` tiene tipo `Square`, así que la llamada es `visitor.visitSquare(this)`; segundo despacho: el visitante recibido es un `AreaVisitor`, y se ejecuta su `visitSquare` (imprime `AreaVisitor.visitSquare`). El nodo decide *qué método de visita se invoca*; el visitante decide *qué se hace* en esa visita.
(c) Hay que añadir el `record Triangle` con su `accept`, que llamará a un nuevo método `visitTriangle`, y ese método a la interfaz. El compilador marca entonces como erróneas todas las implementaciones de `ShapeVisitor` (`AreaVisitor` no implementa `visitTriangle`): hay que tocar todos los visitantes. Es el coste del patrón: añadir tipos es caro. Error típico: creer que `AreaVisitor` sigue compilando porque `Triangle` lo ha heredado de algún sitio.

Solución del ejercicio 12.3

enunciado en la pág. 374

El código que se prueba es el de la teoría, repetido aquí entero para que las pruebas compilen solas: `Expr`, `PromoParser` y `PromoEvaluator`.

Expr.java

```
package shoplab;

import java.math.BigDecimal;
import java.util.Set;

public sealed interface Expr
        permits Expr.Compare, Expr.And, Expr.Or, Expr.Not {

    enum Op { EQ, NE, LT, LE, GT, GE }

    record Compare(String variable, Op op, String value)
            implements Expr {

        private static final Set<String> VARIABLES =
                Set.of("price", "stock", "category");

        public Compare {
            if (!VARIABLES.contains(variable)) {
                throw new IllegalArgumentException(
                        "unknown variable: " + variable);
            }
            if (variable.equals("category")) {
                if (op != Op.EQ && op != Op.NE) {
                    throw new IllegalArgumentException(
                            "category only accepts == and !=");
                }
            } else {
                try {
                    new BigDecimal(value);
                } catch (NumberFormatException e) {
                    throw new IllegalArgumentException(
                            "not a number: " + value);
                }
            }
        }
    }

    record And(Expr left, Expr right) implements Expr {
    }

    record Or(Expr left, Expr right) implements Expr {
    }

    record Not(Expr operand) implements Expr {
    }
}
```

PromoParser.java

```
package shoplab;

import java.util.ArrayList;
import java.util.List;
import java.util.Objects;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

import org.springframework.stereotype.Component;

@Component
public class PromoParser {

    private static final Pattern TOKEN = Pattern.compile(
            "\\s*(<=|>=|==|!=|<|>|\\(|\\)|[A-Za-z0-9_.]+)");

    public Expr parse(String text) {
        Objects.requireNonNull(text, "text");
        return new Run(tokenize(text.strip())).parseAll();
    }

    private static List<String> tokenize(String text) {
        List<String> tokens = new ArrayList<>();
        Matcher matcher = TOKEN.matcher(text);
        int position = 0;
        while (position < text.length()) {
            matcher.region(position, text.length());
            if (!matcher.lookingAt()) {
                throw new IllegalArgumentException(
                        "unexpected text at: "
                                + text.substring(position));
            }
            tokens.add(matcher.group(1));
            position = matcher.end();
        }
        return tokens;
    }

    private static final class Run {

        private final List<String> tokens;
        private int position;

        Run(List<String> tokens) {
            this.tokens = tokens;
        }

        Expr parseAll() {
            Expr result = or();
            if (position < tokens.size()) {
                throw new IllegalArgumentException(
                        "unexpected token: " + tokens.get(position));
            }
            return result;
        }

        private Expr or() {
            Expr left = and();
            while (accept("OR")) {
                left = new Expr.Or(left, and());
            }
```

PromoEvaluator.java

```java
package shoplab;

import java.math.BigDecimal;

import org.springframework.stereotype.Component;

@Component
public class PromoEvaluator {

    public boolean evaluate(Expr expr, Product product) {
        return switch (expr) {
            case Expr.Compare compare -> compare(compare, product);
            case Expr.And and ->
                    evaluate(and.left(), product)
                            && evaluate(and.right(), product);
            case Expr.Or or ->
                    evaluate(or.left(), product)
                            || evaluate(or.right(), product);
            case Expr.Not not -> !evaluate(not.operand(), product);
        };
    }

    private boolean compare(Expr.Compare compare, Product product) {
        if (compare.variable().equals("category")) {
            boolean same =
                    product.category().equalsIgnoreCase(compare.value());
            return compare.op() == Expr.Op.EQ ? same : !same;
        }
        BigDecimal actual = compare.variable().equals("price")
                ? product.price().amount()
                : BigDecimal.valueOf(product.stock());
        int order = actual.compareTo(new BigDecimal(compare.value()));
        return switch (compare.op()) {
            case EQ -> order == 0;
            case NE -> order != 0;
            case LT -> order < 0;
            case LE -> order <= 0;
            case GT -> order > 0;
            case GE -> order >= 0;
        };
    }
}
```

Y las pruebas:

src/test/java/shoplab/PromoRulesTest.java

```java
package shoplab;

import static org.junit.jupiter.api.Assertions.assertFalse;
import static org.junit.jupiter.api.Assertions.assertTrue;

import java.math.BigDecimal;

import org.junit.jupiter.api.Test;

class PromoRulesTest {

    private final PromoParser parser = new PromoParser();
    private final PromoEvaluator evaluator = new PromoEvaluator();

    private boolean matches(String rule, Product product) {
        return evaluator.evaluate(parser.parse(rule), product);
    }

    private Product product(String price, int stock, String category) {
        return new Product("x1", "Item",
                new Money(new BigDecimal(price), "EUR"),
                new ProductKind.Digital(), stock, category);
    }

    @Test
    void expensiveBooks() {
        String rule = "price > 100 AND category == books";
        assertTrue(matches(rule, product("120.00", 5, "books")));
        assertFalse(matches(rule, product("20.00", 5, "books")));
        assertFalse(matches(rule, product("120.00", 5, "music")));
    }

    @Test
    void notSoldOutOrMusic() {
        String rule = "NOT stock == 0 OR category == music";
        assertTrue(matches(rule, product("10.00", 3, "books")));
        assertTrue(matches(rule, product("10.00", 0, "music")));
        assertFalse(matches(rule, product("10.00", 0, "books")));
    }

    @Test
    void cheapOrAlmostGone() {
        String rule = "price <= 20.00 OR stock < 3";
        assertTrue(matches(rule, product("20.00", 50, "books")));
        assertTrue(matches(rule, product("99.00", 2, "books")));
        assertFalse(matches(rule, product("99.00", 50, "books")));
    }
}
```

Cada prueba cubre una regla con entradas que la cumplen y que no. La segunda comprueba la precedencia: `NOT` se aplica solo a `stock == 0`, y luego va el `OR`. La tercera comprueba el límite (`<=` incluye 20,00). Error típico: usar `new Money(new BigDecimal(20), ...)` con un `double` en lugar de una cadena; con cadenas el valor es exacto.

Solución del ejercicio 12.4 enunciado en la pág. 374

Trabaja en la copia. Cambian tres archivos.

Expr.java (con In)

```java
package shoplab;

import java.math.BigDecimal;
import java.util.List;
import java.util.Set;

public sealed interface Expr
        permits Expr.Compare, Expr.In, Expr.And, Expr.Or, Expr.Not {

    enum Op { EQ, NE, LT, LE, GT, GE }

    record Compare(String variable, Op op, String value)
            implements Expr {

        private static final Set<String> VARIABLES =
                Set.of("price", "stock", "category");

        public Compare {
            if (!VARIABLES.contains(variable)) {
                throw new IllegalArgumentException(
                        "unknown variable: " + variable);
            }
            if (variable.equals("category")) {
                if (op != Op.EQ && op != Op.NE) {
                    throw new IllegalArgumentException(
                            "category only accepts == and !=");
                }
            } else {
                try {
                    new BigDecimal(value);
                } catch (NumberFormatException e) {
                    throw new IllegalArgumentException(
                            "not a number: " + value);
                }
            }
        }
    }

    record In(String variable, List<String> values) implements Expr {

        public In {
            if (!"category".equals(variable)) {
                throw new IllegalArgumentException(
                        "IN only works with category");
            }
            if (values.isEmpty()) {
                throw new IllegalArgumentException("empty IN list");
            }
            values = List.copyOf(values);
        }
    }

    record And(Expr left, Expr right) implements Expr {
    }

    record Or(Expr left, Expr right) implements Expr {
    }

    record Not(Expr operand) implements Expr {
    }
}
```

PromoParser.java (con IN)

```
package shoplab;

import java.util.ArrayList;
import java.util.List;
import java.util.Objects;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

import org.springframework.stereotype.Component;

@Component
public class PromoParser {

    private static final Pattern TOKEN = Pattern.compile(
            "\\s*(<=|>=|==|!=|<|>|\\(|\\)|,|[A-Za-z0-9_.]+)");

    public Expr parse(String text) {
        Objects.requireNonNull(text, "text");
        return new Run(tokenize(text.strip())).parseAll();
    }

    private static List<String> tokenize(String text) {
        List<String> tokens = new ArrayList<>();
        Matcher matcher = TOKEN.matcher(text);
        int position = 0;
        while (position < text.length()) {
            matcher.region(position, text.length());
            if (!matcher.lookingAt()) {
                throw new IllegalArgumentException(
                        "unexpected text at: "
                                + text.substring(position));
            }
            tokens.add(matcher.group(1));
            position = matcher.end();
        }
        return tokens;
    }

    private static final class Run {

        private final List<String> tokens;
        private int position;

        Run(List<String> tokens) {
            this.tokens = tokens;
        }

        Expr parseAll() {
            Expr result = or();
            if (position < tokens.size()) {
                throw new IllegalArgumentException(
                        "unexpected token: " + tokens.get(position));
            }
            return result;
        }

        private Expr or() {
            Expr left = and();
            while (accept("OR")) {
                left = new Expr.Or(left, and());
            }
```

PromoEvaluator.java (con IN)

```
package shoplab;

import java.math.BigDecimal;

import org.springframework.stereotype.Component;

@Component
public class PromoEvaluator {

    public boolean evaluate(Expr expr, Product product) {
        return switch (expr) {
            case Expr.Compare compare -> compare(compare, product);
            case Expr.In in -> in.values().stream().anyMatch(
                    value -> product.category()
                            .equalsIgnoreCase(value));
            case Expr.And and ->
                    evaluate(and.left(), product)
                            && evaluate(and.right(), product);
            case Expr.Or or ->
                    evaluate(or.left(), product)
                            || evaluate(or.right(), product);
            case Expr.Not not -> !evaluate(not.operand(), product);
        };
    }

    private boolean compare(Expr.Compare compare, Product product) {
        if (compare.variable().equals("category")) {
            boolean same =
                    product.category().equalsIgnoreCase(compare.value());
            return compare.op() == Expr.Op.EQ ? same : !same;
        }
        BigDecimal actual = compare.variable().equals("price")
                ? product.price().amount()
                : BigDecimal.valueOf(product.stock());
        int order = actual.compareTo(new BigDecimal(compare.value()));
        return switch (compare.op()) {
            case EQ -> order == 0;
            case NE -> order != 0;
            case LT -> order < 0;
            case LE -> order <= 0;
            case GT -> order > 0;
            case GE -> order >= 0;
        };
    }
}
```

Al añadir `In` a `permits` y antes de tocar el evaluador, el `switch` de `evaluate` deja de compilar: es el aviso del sellado. El analizador mira `IN` justo después de la variable y lee la lista con `list()`. Errores típicos: olvidar la coma en el patrón de símbolos (el texto se rechaza con `unexpected text at: ,`) y olvidar `expect("(")` en la lista.

Solución del ejercicio 12.5 enunciado en la pág. 374

Solución de referencia.

Caso	Qué cambia más	Decisión
(a) Exportación CSV	Operaciones (columnas cada trimestre)	Visitor
(b) `displayName()`	Casi nada; lo necesitan todos los nodos	Método en cada nodo
(c) Más tipos de nodo	Tipos (`Bundle`, luego otro)	Método en cada nodo

(a) Es una operación que cambia con frecuencia y que no pertenece al modelo del catálogo: va fuera de los nodos. Entre Visitor y `switch`: como la jerarquía es sellada y la exportación es una sola función, el `switch` es la opción simple y suficiente. Un Visitor se justifica si se quieren varias exportaciones intercambiables por nombre (como los informes del hito), porque el visitante es un objeto y se inyecta. **(b)** Una operación que todos los nodos tienen y que no cambia es una responsabilidad del nodo: un método en cada uno es lo más claro, sin visitante ni `switch`. **(c)** Si llegan tipos nuevos, cada tipo nuevo obliga a tocar todos los visitantes. Con métodos en el nodo, un tipo nuevo es una clase nueva, que lleva su propia implementación.

Criterios para valorar otras soluciones. Razona sobre el eje «operaciones nuevas frente a tipos nuevos»; menciona que el compilador avisa en las dos formas (Visitor: el método nuevo de la interfaz; `switch`: la exhaustividad); y contempla el acoplamiento (el Visitor exige que los nodos conozcan la interfaz `CatalogVisitor`; el `switch` los deja sin saber nada de los informes).

Alternativa válida. En (a), un Visitor si tienes que elegir la exportación en ejecución a partir de un nombre. En (c), un `switch` sellado si el equipo prefiere ver cada operación en un único sitio y acepta tocar cada `switch` al llegar un tipo; lo que no vale es no mencionar el coste.

Qué cambiaría la decisión. Si en (b) aparecieran muchas operaciones parecidas a `displayName()`, el modelo se llenaría de ellas y sería mejor sacar algunas a un visitante; si en (c) los tipos dejaran de crecer, el Visitor volvería a ser cómodo.

Solución del ejercicio 12.6 enunciado en la pág. 374

Parte del estado tras el H8b. **1) Los nodos aceptan visitantes.** Crea `CatalogVisitor` (primer bloque de código de más abajo) y sustituye estos cuatro archivos por las versiones siguientes, que son los de tu proyecto más el método `accept`.

CatalogNode.java, Product.java, Category.java, Pack.java

```java
package shoplab;

public sealed interface CatalogNode permits Product, Category, Pack {

    String name();

    Money totalPrice(String currency);

    <R> R accept(CatalogVisitor<R> visitor);
}

// ----- archivo aparte: Product.java -----
package shoplab;

import java.util.Objects;

public record Product(String id, String name, Money price,
                      ProductKind kind, int stock, String category)
        implements CatalogNode {

    public Product {
        if (id == null || id.isBlank() || name == null || name.isBlank()
                || category == null || category.isBlank()) {
            throw new IllegalArgumentException(
                    "id, name and category are required");
        }
        if (stock < 0) {
            throw new IllegalArgumentException("stock must be >= 0");
        }
        Objects.requireNonNull(price);
        Objects.requireNonNull(kind);
    }

    @Override
    public Money totalPrice(String currency) {
        return price;
    }

    @Override
    public <R> R accept(CatalogVisitor<R> visitor) {
        return visitor.visitProduct(this);
    }
}

// ----- archivo aparte: Category.java -----
package shoplab;

import java.util.List;

public record Category(String name, List<CatalogNode> children)
        implements CatalogNode {

    public Category {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("name is required");
        }
        children = List.copyOf(children);
    }

    @Override
    public Money totalPrice(String currency) {
```

CatalogVisitor.java, StockReport.java, StockReportVisitor.java

```java
package shoplab;

public interface CatalogVisitor<R> {

    R visitProduct(Product product);

    R visitCategory(Category category);

    R visitPack(Pack pack);
}

// ----- archivo aparte: StockReport.java -----
package shoplab;

import java.util.ArrayList;
import java.util.List;

public record StockReport(int units, List<String> outOfStock) {

    public static final StockReport EMPTY =
            new StockReport(0, List.of());

    public StockReport plus(StockReport other) {
        List<String> all = new ArrayList<>(outOfStock);
        all.addAll(other.outOfStock);
        return new StockReport(units + other.units, List.copyOf(all));
    }
}

// ----- archivo aparte: StockReportVisitor.java -----
package shoplab;

import java.util.List;

import org.springframework.stereotype.Component;

@Component("stock")
public class StockReportVisitor implements CatalogVisitor<StockReport> {

    @Override
    public StockReport visitProduct(Product product) {
        List<String> out = product.stock() == 0
                ? List.of(product.id())
                : List.of();
        return new StockReport(product.stock(), out);
    }

    @Override
    public StockReport visitCategory(Category category) {
        StockReport total = StockReport.EMPTY;
        for (CatalogNode child : category.children()) {
            total = total.plus(child.accept(this));
        }
        return total;
    }

    @Override
    public StockReport visitPack(Pack pack) {
        StockReport total = StockReport.EMPTY;
        for (Product product : pack.products()) {
            total = total.plus(visitProduct(product));
        }
```

PriceReport.java, PriceReportVisitor.java

```java
package shoplab;

import java.math.BigDecimal;

public record PriceReport(int products, BigDecimal total,
        String mostExpensive, BigDecimal highest) {

    public static final PriceReport EMPTY =
            new PriceReport(0, BigDecimal.ZERO, "", BigDecimal.ZERO);

    public PriceReport plus(PriceReport other) {
        boolean otherHigher = other.highest.compareTo(highest) > 0;
        return new PriceReport(
                products + other.products,
                total.add(other.total),
                otherHigher ? other.mostExpensive : mostExpensive,
                otherHigher ? other.highest : highest);
    }
}

// ----- archivo aparte: PriceReportVisitor.java -----
package shoplab;

import java.math.BigDecimal;

import org.springframework.stereotype.Component;

@Component("price")
public class PriceReportVisitor implements CatalogVisitor<PriceReport> {

    @Override
    public PriceReport visitProduct(Product product) {
        BigDecimal amount = product.price().amount();
        return new PriceReport(1, amount, product.name(), amount);
    }

    @Override
    public PriceReport visitCategory(Category category) {
        PriceReport total = PriceReport.EMPTY;
        for (CatalogNode child : category.children()) {
            total = total.plus(child.accept(this));
        }
        return total;
    }

    @Override
    public PriceReport visitPack(Pack pack) {
        PriceReport total = PriceReport.EMPTY;
        for (Product product : pack.products()) {
            total = total.plus(visitProduct(product));
        }
        return total;
    }
}
```

ReportController.java

```java
package shoplab;

import java.util.Map;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.server.ResponseStatusException;

@RestController
@RequestMapping("/reports")
public class ReportController {

    private final Map<String, CatalogVisitor<?>> visitors;
    private final CatalogService catalog;

    public ReportController(Map<String, CatalogVisitor<?>> visitors,
            CatalogService catalog) {
        this.visitors = visitors;
        this.catalog = catalog;
    }

    @GetMapping("/{name}")
    public Object report(@PathVariable String name) {
        CatalogVisitor<?> visitor = visitors.get(name);
        if (visitor == null) {
            throw new ResponseStatusException(HttpStatus.NOT_FOUND,
                    "unknown report: " + name);
        }
        return catalog.tree().accept(visitor);
    }
}
```

2) El mini-DSL. Crea `Expr`, `PromoParser` y `PromoEvaluator` tal como están en la sección «Interpreter» del capítulo (los listados completos de esa sección son el código del hito; no cambian). Añade a `application.properties`:

application.properties (línea nueva)

```
shop.promo.default-rule=stock > 0 AND price >= 100
```

PromoRequest.java, PromoResult.java, PromoService.java, PromoController.java

```
package shoplab;

public record PromoRequest(String productId, String rule) {
}

// ----- archivo aparte: PromoResult.java -----
package shoplab;

public record PromoResult(String productId, String rule,
        boolean matches) {
}

// ----- archivo aparte: PromoService.java -----
package shoplab;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

@Service
public class PromoService {

    private final PromoParser parser;
    private final PromoEvaluator evaluator;
    private final ProductRepository products;
    private final String defaultRule;

    public PromoService(PromoParser parser, PromoEvaluator evaluator,
            ProductRepository products,
            @Value("${shop.promo.default-rule}") String defaultRule) {
        this.parser = parser;
        this.evaluator = evaluator;
        this.products = products;
        this.defaultRule = defaultRule;
        parser.parse(defaultRule);
    }

    public PromoResult evaluate(String productId, String rule) {
        String text = (rule == null || rule.isBlank())
                ? defaultRule
                : rule;
        Product product = products.findById(productId)
                .orElseThrow(() -> new ProductNotFoundException(productId));
        Expr expr = parser.parse(text);
        return new PromoResult(productId, text,
                evaluator.evaluate(expr, product));
    }
}

// ----- archivo aparte: PromoController.java -----
package shoplab;

import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/promos")
public class PromoController {
```

3) Prueba. Con la aplicación arrancada y los productos `p1` y `b1` creados:

Informes y promociones

```
curl -i http://localhost:8080/reports/stock
curl -i http://localhost:8080/reports/price
curl -i http://localhost:8080/reports/zz
curl -i -H "Content-Type: application/json" \
  -d '{"id":"b2","name":"Atlas","price":{"amount":120,"currency":"EUR"},"kind
    ↪ ":{"type":"digital"},"stock":3,"category":"books"}' \
  http://localhost:8080/products
curl -i -H "Content-Type: application/json" \
  -d '{"productId":"b2","rule":"price > 100 AND category == books"}' \
  http://localhost:8080/promos/evaluate
curl -i -H "Content-Type: application/json" \
  -d '{"productId":"b1","rule":"price > 100 AND category == books"}' \
  http://localhost:8080/promos/evaluate
curl -i -H "Content-Type: application/json" \
  -d '{"productId":"b1","rule":"price >"}' \
  http://localhost:8080/promos/evaluate
```

Resultados esperados: 200 con `"units":17,"outOfStock":[]`, 200 con `"products":2` (y `"mostExpensive":"Teclado"` si `p1` se llama así), 404, 201, 200 con `"matches":true`, 200 con `"matches":false` y 400 por `unexpected end of rule`. Los cuatro tipos de nodo ganan un único método; los informes se añaden con una clase `@Component("nombre")` y el controlador no cambia. Errores típicos: olvidar el nombre del bean (el mapa tendría claves como `stockReportVisitor` y `/reports/stock` daría 404); olvidar la propiedad `shop.promo.default-rule` (la aplicación no arranca); y no ordenar la recursión con `child.accept(this)`.

CAPÍTULO 13

Elegir y defender el patrón

Objetivos del capítulo

- Pasar del olor al patrón con una checklist de cinco preguntas.
- Distinguir patrones que se parecen: Strategy y State, Decorator y Proxy, Facade y Mediator, Adapter y Bridge, y Decorator, Strategy y Chain of Responsibility.
- Reconocer el sobrediseño y decidir no usar un patrón.
- Defender (o criticar) un patrón en una code review con argumentos, incluido el principio de sustitución de Liskov.
- Cerrar ShopLab: un único `POST /checkout` desde un carrito y un `DECISIONES.md` que justifica cada pieza.

Has visto los patrones uno a uno. En un equipo, nadie te pregunta «¿sabes qué es un Decorator?». Te preguntan «¿por qué has metido un Decorator aquí?». Este capítulo no trae patrones nuevos: trae criterio. Kerievsky lo resume con la idea de refactorizar *hacia* los patrones, solo cuando el código lo pide[11]. Y la evidencia empírica sobre si los patrones mejoran la calidad es mixta: depende de cuál y de cómo[30].

Java sin compilar

Como en los capítulos 3 a 12, el código Java está escrito con cuidado contra la documentación, pero su autor no lo ha compilado ni ejecutado. Si algo no compila, lee el error: suele ser un `import` o un nombre.

13.1 Del olor al patrón: una checklist

Un patrón es la respuesta a un problema concreto[1, 7]. Antes de escribir una interfaz nueva, contesta por escrito:

1. **¿Cuál es el olor?** Un `if` que crece, una clase que cambia por dos razones, un `new` que no puedes sustituir, un cambio que obliga a tocar cinco sitios. Si no puedes nombrarlo, no hay problema.
2. **¿Qué varía?** Un patrón aísla algo que cambia (el algoritmo, el formato, el proveedor). Si nada va a variar, no aísles nada.
3. **¿Qué tengo ya?** Un `record`, una lambda, un `Map` de beans o un método extra pueden bastar. Los lenguajes modernos absorben parte de los patrones[9].
4. **¿Qué cuesta?** Más clases, más saltos al leer, más cosas que explicar.
5. **¿Quién lo verá?** Si el equipo no lo reconoce, ponle el nombre del patrón en el nombre de la clase y justifícalo en el pull request.

Tabla de bolsillo (olor, patrón), toda ya vista:

- `switch` sobre un tipo para elegir un `new`: Factory Method (cap. 3).
- Constructor con muchos parámetros: Builder (cap. 4).
- API ajena incompatible: Adapter. Muchas llamadas para una tarea: Facade (cap. 5).
- Árbol de objetos tratado igual que una hoja: Composite. Añadir comportamiento sin heredar: Decorator (cap. 6).
- Algoritmo intercambiable: Strategy. Esqueleto fijo con pasos variables: Template Method (cap. 8).
- Comportamiento que depende del estado: State. Acciones deshacibles: Command (cap. 9).
- Varios objetos que reaccionan a un hecho: Observer; que se coordinan entre sí: Mediator (cap. 10).
- Validaciones en secuencia: Chain of Responsibility (cap. 11).

13.2 Patrones que se parecen

Los autores del catálogo ya los presentaron como una familia con relaciones entre sí[6], y el libro de 1994 dedica a cada uno un apartado de patrones relacionados[7]. Muchos comparten la misma forma (una interfaz y varias implementaciones) y se distinguen por la *intención*.

- **Strategy y State.** Ambos delegan en un objeto intercambiable. En Strategy elige el *cliente* y la estrategia no cambia sola (`DiscountStrategy`). En State cambia el *propio objeto* al recibir eventos y las transiciones son parte del diseño (`OrderState`).
- **Decorator y Proxy.** Ambos envuelven un objeto con su misma interfaz. Decorator *añade* comportamiento y se apila (`SurchargePrice`). Proxy *controla el acceso* (caché, permisos) y suele ser uno solo (`CachingCatalogService`).
- **Facade y Mediator.** Ambos reúnen varias piezas. Facade simplifica la entrada para quien llama y las piezas no la conocen. Mediator coordina piezas que antes se hablaban entre sí (`CheckoutMediator` ordena stock, cobro y envío).
- **Adapter y Bridge.** Adapter arregla una incompatibilidad que ya existe (`LegacyCardAdapter`). Bridge separa dos jerarquías desde el diseño para que evolucionen aparte (`Notification` y `NotificationChannel`).

Decorator, Strategy y Chain of Responsibility

La comparación que quedó pendiente desde el capítulo 6. Los tres componen objetos que implementan una interfaz común, y por eso se confunden.

- **Strategy**: *una* pieza elegida entre varias. Nadie más interviene. Ejemplo: un solo descuento por pedido.
- **Decorator**: *todas* las piezas de la pila actúan, cada una sobre el resultado de la siguiente y siempre delegan. Ejemplo: recargos que se suman.
- **Chain of Responsibility**: las piezas se miran en orden y cada una puede *cortar* la cadena (en ShopLab, el primer validador que falla gana).

Pregunta útil: ¿quién decide si el siguiente se ejecuta? En Strategy no hay siguiente; en Decorator siempre se ejecuta; en Chain lo decide cada eslabón.

13.3 Sobrediseño: cuándo no usarlos

El sobrediseño es pagar hoy por flexibilidad que nadie ha pedido. Señales:

- Una interfaz con una única implementación y sin segunda a la vista.
- Una fábrica que devuelve siempre el mismo tipo.
- Un Singleton escrito a mano en una aplicación Spring, donde el contenedor ya da una instancia única[24].
- Un Visitor sobre una jerarquía de dos tipos que no crece, o un Observer con un solo suscriptor que siempre estará ahí.
- Nombres como `AbstractManagerFactoryProvider`.

Regla práctica: espera a la segunda variante (o al segundo cambio doloroso) antes de abstraer. Antes, una clase y un método bastan. Quitar un patrón no es un fracaso: si el olor desaparece, el patrón sobraba.

13.4 Cómo argumentar en una code review

Un buen comentario de revisión tiene tres partes: *qué ves* (el olor), *qué propones* (el patrón o su ausencia) y *por qué* (qué cambio futuro se vuelve fácil o qué complejidad se evita). «Esto no me gusta» no sirve; «este `if` por tipo de pago se tocará con cada método nuevo; propongo un `Map<String, PaymentProcessor>`» sí.

El argumento más frecuente contra un diseño es el principio de sustitución de Liskov[13, 14]: un subtipo debe poder usarse donde se espera el tipo base sin que el programa se comporte distinto de forma inesperada. En una revisión suena así: «`LoggingOrderService` hereda de `OrderService` pero `find` devuelve vacío para ciertos ids; quien reciba un `OrderService` no lo espera. Usa un Decorator que delegue siempre». Señales de violación: la subclase lanza excepciones nuevas, exige más precondiciones o devuelve resultados que el contrato base no admite.

Y al defender el tuyo: cita el olor, enseña el cambio que ahora es fácil y admite el coste. Si el revisor propone algo más simple que resuelve el mismo olor, acepta.

13.5 En el proyecto: H10

Hito 10: un único checkout desde un carrito y DECISIONES.md

Partes del H9. En el H10:

- `CheckoutRequest(cartId, shippingAddress, region, paymentType, channel)`; `channel` es opcional y vale `email` por defecto.
- `OrderService.createFrom(Cart, ShippingAddress, String, PaymentType)` convierte cada `CartLine` en una `OrderLine` con el precio actual del `ProductRepository`, construye el `Order` con el Builder y lo guarda.
- `POST /checkout`: crea el pedido desde el carrito y llama al mismo `CheckoutFacade.checkout` de siempre. No hay otro flujo de cobro.
- `DECISIONES.md` en la raíz del proyecto, con el patrón, el olor y la justificación de cada

pieza.

Decisiones: el carrito no se vacía tras cobrar (otro checkout con el mismo carrito crea otro pedido); un pedido que falla en validación o cobro queda guardado en estado `Created`; el canal va en el cuerpo, no en la URL. El código completo está en la solución del ejercicio 13.5.

13.6 Ejercicios

Ejercicio 13.1: Pull request con cuatro cambios ★★★☆☆

Eres revisor de este pull request de ShopLab. Para cada cambio, escribe un comentario de revisión de tres partes (olor, patrón o ausencia de patrón, justificación) y di si lo aprobarías.

Cambio 1: LabelPrinterFactory.java

```
public interface LabelPrinter { String print(Order order); }

public class PdfLabelPrinter implements LabelPrinter {
    public String print(Order order) { return "PDF " + order.getId(); }
}

public class LabelPrinterFactory {
    public LabelPrinter create() { return new PdfLabelPrinter(); }
}
```

Cambio 2: PaymentDispatcher.java

```
@Component
public class PaymentDispatcher {
    private final Map<String, PaymentProcessor> processors;

    public PaymentDispatcher(Map<String, PaymentProcessor> processors) {
        this.processors = processors;
    }

    public PaymentResult pay(String type, Money amount) {
        PaymentProcessor p = processors.get(type);
        if (p == null) {
            throw new IllegalArgumentException("unknown: " + type);
        }
        return p.charge(amount);
    }
}
```

Cambio 3: OrderIdGenerator.java

```
public final class OrderIdGenerator {
    private static final OrderIdGenerator INSTANCE =
            new OrderIdGenerator();
    private OrderIdGenerator() { }
    public static OrderIdGenerator getInstance() { return INSTANCE; }
    public String next() { return UUID.randomUUID().toString(); }
}
```

Cambio 4: LoggingOrderService.java

```
@Service
public class LoggingOrderService extends OrderService {

    @Override
    public Optional<Order> find(String id) {
        if (id.startsWith("tmp-")) {
            return Optional.empty();
        }
        log.info("find {}", id);
        return super.find(id);
    }
}
```

Está bien si: das un veredicto razonado de los cuatro cambios y, en el último, citas el principio de sustitución de Liskov. (Supón que `PaymentProcessor` tiene `charge(Money)` y que el resto del código necesario existe.)

Pista: *Pregúntate en cada cambio: ¿qué varía? ¿Hay ya una segunda variante?*

Lectura de código ▪ solución en la pág. 401

Ejercicio 13.2: Refactorizar un CheckoutService largo ★★★★☆

En clases propias (paquete `shoplab.ejercicios.checkout`) tienes este servicio. Es la versión corta de uno de 120 líneas: tiene tres olores (validaciones en `if`, descuento por cupón en `if/else` y método de pago en `if/else`).

CheckoutService.java

```java
package shoplab.ejercicios.checkout;

import java.util.List;

public class CheckoutService {

    public record Sale(List<Long> lineCents, String country,
                       String coupon, String method) { }

    public String checkout(Sale sale) {
        if (sale.lineCents().isEmpty()) {
            throw new IllegalArgumentException("empty sale");
        }
        if (sale.country() == null || sale.country().isBlank()) {
            throw new IllegalArgumentException("country missing");
        }
        long total = 0;
        for (long cents : sale.lineCents()) {
            total += cents;
        }
        if (total <= 0) {
            throw new IllegalArgumentException("total must be > 0");
        }
        if ("WELCOME10".equals(sale.coupon())) {
            total = total * 90 / 100;
        } else if ("BULK".equals(sale.coupon())
                && sale.lineCents().size() >= 5) {
            total = total * 85 / 100;
        } else if (sale.coupon() != null && !sale.coupon().isEmpty()) {
            throw new IllegalArgumentException("bad coupon");
        }
        if ("CARD".equals(sale.method())) {
            return "CARD-" + total;
        } else if ("TRANSFER".equals(sale.method())) {
            return "TR-" + total;
        }
        throw new IllegalArgumentException("unknown method");
    }
}
```

Refactoriza aplicando *solo* los patrones que se justifiquen y explica en un comentario de Javadoc de la clase qué olor resuelve cada uno y qué patrón descartaste. Escribe una clase `SaleCheck` con `main` que pruebe tres ventas. Está bien si: el comportamiento es idéntico (mismos resultados y mismos mensajes de error), añadir un cupón o un método de pago no obliga a tocar el método `checkout`, y cada patrón usado resuelve un olor concreto.

Práctica ▪ solución en la pág. 402

Ejercicio 13.3: Dos patrones que sobran ★★★★☆

Este código de un módulo de tarifas usa dos patrones sin necesitarlos. Identifícalos, explica por qué sobran con los criterios de la sección de sobrediseño y propón la versión más simple, con el mismo comportamiento.

Tarifas.java

```java
package shoplab.ejercicios.tarifas;

public final class AppClock {
    private static final AppClock INSTANCE = new AppClock();
    private AppClock() { }
    public static AppClock getInstance() { return INSTANCE; }
    public long now() { return System.currentTimeMillis(); }
}

sealed interface Fee permits FlatFee, PercentFee {
    <R> R accept(FeeVisitor<R> visitor);
}

record FlatFee(long cents) implements Fee {
    public <R> R accept(FeeVisitor<R> v) { return v.visitFlat(this); }
}

record PercentFee(int percent) implements Fee {
    public <R> R accept(FeeVisitor<R> v) { return v.visitPercent(this); }
}

interface FeeVisitor<R> {
    R visitFlat(FlatFee fee);
    R visitPercent(PercentFee fee);
}

class FeeCalculator implements FeeVisitor<Long> {
    private final long base;
    FeeCalculator(long base) { this.base = base; }
    public Long visitFlat(FlatFee fee) { return fee.cents(); }
    public Long visitPercent(PercentFee fee) {
        return base * fee.percent() / 100;
    }
}

public class TarifaService {
    public String invoice(Fee fee, long base) {
        long charge = fee.accept(new FeeCalculator(base));
        return AppClock.getInstance().now() + ":" + charge;
    }
}
```

Está bien si: nombras Singleton y Visitor, das al menos dos razones para cada uno, tu versión tiene menos clases, sigue siendo fácil de probar (no depende de la hora real) y mantiene la comprobación de que todos los tipos de `Fee` están cubiertos.

Diseño ▪ solución en la pág. 404

Ejercicio 13.4: Diseña la función de reembolsos ★★★★★

El negocio quiere reembolsos en ShopLab: el cliente solicita el reembolso de un pedido pagado; un empleado lo aprueba o lo rechaza; si se aprueba, se devuelve el dinero por la pasarela y se avisa a otras partes (fidelización, contabilidad). Los reembolsos deben poder auditarse. Diseña la función eligiendo y defendiendo los patrones (o su ausencia). Entrega:

(1) una tabla de decisiones (pieza, patrón, olor, justificación), (2) el código de las piezas centrales y (3) las alternativas que descartaste. Criterios de valoración (no hay pasos): cada patrón responde a un olor nombrado; las transiciones ilegales no compilan o fallan con un error claro; el reembolso no duplica el flujo de cobro ni toca la lógica de `CheckoutFacade`; añadir un oyente nuevo no obliga a modificar el servicio; se justifica lo que *no* se usa.

Diseño ▪ solución en la pág. 405

Ejercicio 13.5: Hito 10: POST /checkout y DECISIONES.md ★★★★☆

Partes del H9. Implementa el hito H10 como se describe en la sección anterior: `CheckoutRequest`, `OrderService.createFrom`, `POST /checkout` (que llama al mismo `CheckoutFacade.checkout`) y el archivo `DECISIONES.md`. Está bien si: con el producto `p1` (stock 12), un carrito con 2 unidades y un `POST /checkout` responde 200; el pedido queda en `paid`; el stock de `p1` baja a 10; un carrito inexistente da 404 y uno vacío da 400; y `DECISIONES.md` tiene una entrada por cada patrón del proyecto.

Pista: *No hay controlador nuevo de pedidos: crea `CheckoutController` y reutiliza `CartService.find`, `OrderService` y `CheckoutFacade`.*

Proyecto ▪ solución en la pág. 409

13.7 Resumen

Resumen

- Un patrón responde a un olor concreto; sin olor nombrado y sin algo que varíe, no hay patrón.
- Checklist: olor, qué varía, qué tengo ya, qué cuesta, quién lo verá.
- Strategy elige una pieza; Decorator las apila todas; Chain deja que cada eslabón corte.
- Strategy elige el cliente, State cambia solo; Decorator añade, Proxy controla; Facade simplifica, Mediator coordina; Adapter arregla, Bridge separa.
- Sobrediseño: interfaces de una sola implementación, Singleton a mano en Spring, Visitor sobre jerarquías cerradas. Espera a la segunda variante.
- Un comentario de revisión dice qué ves, qué propones y por qué. Liskov es un argumento frecuente contra la herencia que cambia el contrato.
- En ShopLab, `POST /checkout` crea el pedido desde el carrito y pasa por el mismo `CheckoutFacade`; `DECISIONES.md` deja por escrito cada elección.

Soluciones de los ejercicios

Solución del ejercicio 13.1 enunciado en la pág. 397

- **Cambio 1 (rechazar).** Olor: ninguno todavía; hay una interfaz, una implementación y una fábrica que devuelve siempre `new PdfLabelPrinter()`. Es una fábrica sin variación (sobrediseño). Comentario: «Hoy solo imprimimos PDF; borra la fábrica y la interfaz, o pide una segunda variante antes de abstraer. Cuando aparezca, un `Map<String, LabelPrinter>` de beans basta».

- **Cambio 2 (aprobar).** Olor resuelto: el `if` por tipo de pago que crecía con cada método nuevo. Es Strategy con un `Map` inyectado por Spring, que además valida el tipo desconocido. Comentario: «Bien aplicado: añadir un procesador es añadir un bean».
- **Cambio 3 (rechazar).** Singleton a mano en una aplicación Spring: no se puede sustituir en pruebas y oculta una dependencia. Comentario: «Haz de `OrderIdGenerator` un `@Component` e inyéctalo por constructor; el contenedor ya garantiza una instancia[24]». Un generador de ids ni siquiera necesita estado.
- **Cambio 4 (rechazar).** Viola Liskov: la subclase devuelve vacío para ids que el tipo base sí devolvería, así que quien reciba un `OrderService` se encuentra un comportamiento que el contrato no admite. Además hereda de una clase concreta y no tiene constructor válido. Comentario: «Usa un Decorator (o un aspecto) que registre y delegue siempre; el filtro de ids `tmp-` es otra responsabilidad y va aparte».

Errores típicos: aprobar el 1 «por si acaso»; rechazar el 2 por ser «demasiado patrón»; criticar el 4 solo por heredar, sin nombrar el contrato roto.

Solución del ejercicio 13.2 enunciado en la pág. 398

Olores y patrones: las validaciones crecerán y cada una puede cortar el proceso: cadena de validadores (Chain of Responsibility, el primer error gana); el descuento por cupón es un algoritmo intercambiable: Strategy; el método de pago, igual: Strategy. Descarté Builder (`Sale` es un `record` de cuatro campos), Facade (no hay un subsistema que ocultar) y State (no hay estado).

CheckoutService.java

```java
package shoplab.ejercicios.checkout;

import java.util.List;
import java.util.Map;
import java.util.Optional;

/**
 * Olores y patrones:
 * - validaciones en if: cadena de Validator (el primer error corta).
 * - cupones en if/else: Strategy (Discount) elegida por cupon.
 * - metodo de pago en if/else: Strategy (Payment) elegida por metodo.
 * Descartados: Builder (Sale es un record pequeno), Facade (no hay
 * subsistema que ocultar) y State (no hay estados).
 */
public class CheckoutService {

    public record Sale(List<Long> lineCents, String country,
                       String coupon, String method) { }

    interface Validator {
        Optional<String> validate(Sale sale);
    }

    interface Discount {
        long apply(Sale sale, long total);
    }

    interface Payment {
        String pay(long total);
    }

    private final List<Validator> validators = List.of(
            sale -> sale.lineCents().isEmpty()
                    ? Optional.of("empty sale") : Optional.empty(),
            sale -> sale.country() == null || sale.country().isBlank()
                    ? Optional.of("country missing") : Optional.empty());

    private final Map<String, Discount> discounts = Map.of(
            "WELCOME10", (sale, total) -> total * 90 / 100,
            "BULK", (sale, total) -> sale.lineCents().size() >= 5
                    ? total * 85 / 100 : total);

    private final Map<String, Payment> payments = Map.of(
            "CARD", total -> "CARD-" + total,
            "TRANSFER", total -> "TR-" + total);

    public String checkout(Sale sale) {
        for (Validator validator : validators) {
            Optional<String> error = validator.validate(sale);
            if (error.isPresent()) {
                throw new IllegalArgumentException(error.get());
            }
        }
        long total = 0;
        for (long cents : sale.lineCents()) {
            total += cents;
        }
        if (total <= 0) {
            throw new IllegalArgumentException("total must be > 0");
        }
        total = applyCoupon(sale, total);
```

SaleCheck.java

```java
package shoplab.ejercicios.checkout;

import java.util.List;

public class SaleCheck {

    public static void main(String[] args) {
        CheckoutService service = new CheckoutService();
        System.out.println(service.checkout(
                new CheckoutService.Sale(List.of(1000L, 500L), "ES",
                        "WELCOME10", "CARD")));
        System.out.println(service.checkout(
                new CheckoutService.Sale(List.of(2000L), "ES",
                        null, "TRANSFER")));
        try {
            service.checkout(new CheckoutService.Sale(
                    List.of(), "ES", null, "CARD"));
        } catch (IllegalArgumentException e) {
            System.out.println("error: " + e.getMessage());
        }
    }
}
```

Imprime `CARD-1350`, `TR-2000` y `error: empty sale`. Funciona porque cada `if/else` se ha convertido en una búsqueda en un `Map` o en una lista; para un cupón nuevo se añade una entrada y `checkout` no cambia.
Criterios para valorar otras soluciones: mismo comportamiento y mensajes; cada patrón nombra su olor; no hay patrones decorativos (una fábrica, un Builder, una fachada sin subsistema). Alternativa válida: una única lista de validadores como beans de Spring con `@Order`, como en el cap. 11. Errores típicos: olvidar que `Map.of` no admite claves nulas (por eso se comprueba antes que el cupón no es nulo) y cambiar el orden: el cupón se valida después del total, como antes.

Solución del ejercicio 13.3

enunciado en la pág. 399

Sobran el **Singleton** `AppClock` y el **Visitor** `FeeVisitor`.

- Singleton: es un envoltorio de `System.currentTimeMillis()`, sin estado que compartir. Fija la hora real y no se puede sustituir en pruebas; el acceso global oculta la dependencia. Basta un `java.time.Clock` recibido por constructor (en Spring, un bean).
- Visitor: solo hay un visitante, la jerarquía tiene dos tipos que no crecen y el visitante se crea en un único sitio. Un `switch` sobre la interfaz sellada da lo mismo con menos código y el compilador sigue exigiendo cubrir todos los casos.

Tarifas.java

```java
package shoplab.ejercicios.tarifas;

import java.time.Clock;
import java.time.Instant;

sealed interface Fee permits FlatFee, PercentFee { }

record FlatFee(long cents) implements Fee { }

record PercentFee(int percent) implements Fee { }

public class TarifaService {

    private final Clock clock;

    public TarifaService(Clock clock) {
        this.clock = clock;
    }

    public String invoice(Fee fee, long base) {
        long charge = switch (fee) {
            case FlatFee flat -> flat.cents();
            case PercentFee percent -> base * percent.percent() / 100;
        };
        return clock.millis() + ":" + charge;
    }

    public static void main(String[] args) {
        Clock fixed = Clock.fixed(Instant.ofEpochMilli(1000L),
                java.time.ZoneOffset.UTC);
        TarifaService service = new TarifaService(fixed);
        System.out.println(service.invoice(new FlatFee(250), 10000));
        System.out.println(service.invoice(new PercentFee(10), 10000));
    }
}
```

Imprime `1000:250` y `1000:1000`. De cinco tipos y una clase se pasa a tres tipos y una clase, y la hora es controlable. Criterios: menos clases sin perder claridad; `switch` sin `default`; reloj inyectado. Alternativa válida: dejar un método `long charge(long base)` en cada record. Error típico: quitar el Visitor pero dejar un `if instanceof` que no avisa de los tipos nuevos. Cuándo volvería el Visitor: si aparecen muchas operaciones sobre `Fee` (informes, exportaciones) y los tipos son estables.

Solución del ejercicio 13.4

enunciado en la pág. 400

Solución de referencia: State para el ciclo de vida, Command para las solicitudes (auditables) y eventos de Spring para avisar a quien tenga interés.

- **State** (`RefundState` sellada): hay un ciclo de vida con transiciones legales (olor: banderas `approved`/`rejected` y `if` por todas partes). Como en el cap. 9.
- **Command** (`RefundCommand`): las acciones son datos que se guardan en el historial del reembolso. Olor: sin ellas la auditoría exige registrar a mano cada cambio.

- **Eventos** (`RefundIssued`): fidelización y contabilidad reaccionan sin que el servicio los conozca (cap. 10).
- **Puerto** (`RefundGateway`): la devolución usa su propia interfaz; una implementación puede apoyarse en el adaptador de pago que ya exista.
- Descartados: Mediator (el servicio solo llama a la pasarela y publica, no coordina varias piezas), Strategy para el tipo de reembolso (todavía una sola variante) y reutilizar `OrderStateMachine` (el reembolso tiene su propio ciclo).

RefundState.java, Refund.java, RefundCommand.java

```
package shoplab.ejercicios.refund;

public sealed interface RefundState {
    record Requested() implements RefundState { }
    record Approved() implements RefundState { }
    record Rejected(String reason) implements RefundState { }
    record Refunded(String transactionId) implements RefundState { }
}

// ----- archivo aparte: Refund.java -----
package shoplab.ejercicios.refund;

import java.util.List;

public record Refund(String id, String orderId, long cents,
                     RefundState state, List<RefundCommand> history) {

    public Refund next(RefundState newState, RefundCommand command) {
        java.util.ArrayList<RefundCommand> copy =
                new java.util.ArrayList<>(history);
        copy.add(command);
        return new Refund(id, orderId, cents, newState,
                List.copyOf(copy));
    }
}

// ----- archivo aparte: RefundCommand.java -----
package shoplab.ejercicios.refund;

public sealed interface RefundCommand {
    record Approve() implements RefundCommand { }
    record Reject(String reason) implements RefundCommand { }
    record Execute() implements RefundCommand { }
}

// ----- archivo aparte: RefundGateway.java -----
package shoplab.ejercicios.refund;

public interface RefundGateway {
    String refund(String orderId, long cents);
}

// ----- archivo aparte: RefundIssued.java -----
package shoplab.ejercicios.refund;

public record RefundIssued(String refundId, String orderId, long cents) { }
```

RefundService.java, RefundDemo.java

```
package shoplab.ejercicios.refund;

import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import org.springframework.context.ApplicationEventPublisher;

public class RefundService {

    private final Map<String, Refund> refunds = new ConcurrentHashMap<>();
    private final RefundGateway gateway;
    private final ApplicationEventPublisher events;
    private int sequence = 0;

    public RefundService(RefundGateway gateway,
                         ApplicationEventPublisher events) {
        this.gateway = gateway;
        this.events = events;
    }

    public synchronized Refund request(String orderId, long cents) {
        String id = "R" + (++sequence);
        Refund refund = new Refund(id, orderId, cents,
                new RefundState.Requested(), List.of());
        refunds.put(id, refund);
        return refund;
    }

    public synchronized Refund handle(String id, RefundCommand command) {
        Refund refund = refunds.get(id);
        if (refund == null) {
            throw new IllegalArgumentException("unknown refund: " + id);
        }
        RefundState next = switch (refund.state()) {
            case RefundState.Requested requested -> switch (command) {
                case RefundCommand.Approve approve ->
                        new RefundState.Approved();
                case RefundCommand.Reject reject ->
                        new RefundState.Rejected(reject.reason());
                default -> throw illegal(refund, command);
            };
            case RefundState.Approved approved -> switch (command) {
                case RefundCommand.Execute execute ->
                        new RefundState.Refunded(gateway.refund(
                                refund.orderId(), refund.cents()));
                default -> throw illegal(refund, command);
            };
            default -> throw illegal(refund, command);
        };
        Refund updated = refund.next(next, command);
        refunds.put(id, updated);
        if (next instanceof RefundState.Refunded) {
            events.publishEvent(new RefundIssued(
                    id, refund.orderId(), refund.cents()));
        }
        return updated;
    }

    private IllegalStateException illegal(Refund refund,
                                          RefundCommand command) {
        return new IllegalStateException("cannot apply "
```

Imprime el evento, el estado `Refunded[transactionId=TX-o1]` con dos comandos en el historial y `error: cannot apply Approve in state Refunded`. Funciona porque cada transición ilegal cae en un `default` que lanza, el cobro externo solo se llama desde un estado `Approved`, y el evento se publica después de guardar.
Criterios para valorar otras soluciones: cada patrón con su olor; transiciones ilegales con error claro; el reembolso no toca `CheckoutFacade`; oyentes nuevos sin tocar el servicio; se explica qué se descartó. Alternativas válidas: una tabla de transiciones en lugar de State si solo hubiera dos estados; un Mediator si el reembolso tuviera que coordinar stock, pasarela y contabilidad en orden. Errores típicos: reutilizar `CheckoutFacade` para devolver dinero, publicar el evento antes de la llamada a la pasarela y usar Strategy sin segunda variante.

Solución del ejercicio 13.5

enunciado en la pág. 401

Parte del H9. Los cambios son tres clases/métodos y un documento.

`CheckoutRequest.java`

```
package shoplab;

public record CheckoutRequest(String cartId,
                              ShippingAddress shippingAddress,
                              String region,
                              PaymentType paymentType,
                              String channel) {
}
```

Añade este método dentro de `OrderService` (usa los campos `products`, de tipo `ProductRepository`, y `orders`, de tipo `OrderRepository`, que su constructor ya recibe; si en tu clase se llaman distinto, cambia los dos nombres). Necesita `import java.util.UUID;`, que ya tiene por `create`.

`OrderService.java (método nuevo)`

```
public Order createFrom(Cart cart, ShippingAddress shippingAddress,
                        String region, PaymentType paymentType) {
    Order.OrderBuilder builder = Order.builder()
            .id(UUID.randomUUID().toString())
            .shippingAddress(shippingAddress)
            .region(region)
            .paymentType(paymentType);
    for (CartLine line : cart.lines()) {
        Product product = products.findById(line.productId())
                .orElseThrow(() ->
                        new ProductNotFoundException(line.productId()));
        builder.line(new OrderLine(product.id(), line.quantity(),
                product.price()));
    }
    return orders.save(builder.build());
}
```

CheckoutController.java

```java
package shoplab;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.server.ResponseStatusException;

@RestController
@RequestMapping("/checkout")
public class CheckoutController {

    private final CartService carts;
    private final OrderService orders;
    private final CheckoutFacade facade;

    public CheckoutController(CartService carts, OrderService orders,
                              CheckoutFacade facade) {
        this.carts = carts;
        this.orders = orders;
        this.facade = facade;
    }

    @PostMapping
    public CheckoutResult checkout(@RequestBody CheckoutRequest request) {
        Cart cart = carts.find(request.cartId())
                .orElseThrow(() -> new ResponseStatusException(
                        HttpStatus.NOT_FOUND,
                        "cart not found: " + request.cartId()));
        Order order = orders.createFrom(cart, request.shippingAddress(),
                request.region(), request.paymentType());
        String channel = request.channel() == null
                || request.channel().isBlank()
                ? "email" : request.channel();
        return facade.checkout(order, channel);
    }
}
```

DECISIONES.md

```
# Decisiones de diseno de ShopLab

Cada entrada: pieza, patron, olor que resuelve y justificacion.

## Creacionales
- Singleton -- ShopSettings (@Bean). Olor: ajustes leidos en cada
  clase. El contenedor da una instancia unica sin getInstance().
- Factory Method -- PaymentProcessorFactory. Olor: switch por tipo
  de pago con new. Un Map de beans lo reemplaza.
- Abstract Factory -- RegionFactory (Spain/France). Olor: impuesto,
  envio y moneda que deben ser coherentes por region.
- Builder -- Order y ShippingAddress (Lombok). Olor: constructor con
  muchos parametros y listas. Inmutables y legibles.
- Prototype -- repeat de pedido (withId). Olor: copiar a mano un
  pedido para repetirlo. Es trivial por ser inmutable.

## Estructurales
- Adapter -- LegacyCardAdapter, FastPayAdapter. Olor: APIs ajenas
  incompatibles con PaymentProcessor.
- Facade -- CheckoutFacade. Olor: el controlador conocia validar,
  cobrar y avisar. Un solo punto de entrada.
- Bridge -- Notification y NotificationChannel. Olor: tipo de aviso
  por canal en una jerarquia que se multiplica.
- Composite -- CatalogNode (Product, Category, Pack). Olor: tratar
  distinto hojas y grupos.
- Decorator -- SurchargePrice sobre PriceCalculator. Olor: recargos
  opcionales y apilables en un if gigante.
- Proxy -- CachingCatalogService. Olor: catalogo consultado muchas
  veces. Controla el acceso sin tocar el servicio.
- Flyweight -- TaxRateFactory. Olor: miles de TaxRate iguales.

## De comportamiento
- Strategy -- DiscountStrategy. Olor: descuentos en if/else.
- Template Method -- AbstractPaymentFlow. Olor: dos flujos de pago
  con los mismos pasos duplicados.
- State -- OrderState y OrderStateMachine. Olor: banderas booleanas
  y transiciones ilegales sin control.
- Command -- CartCommand y CommandHistory. Olor: deshacer sin modelo.
- Memento -- CartSnapshot y SnapshotStore. Olor: restaurar el
  carrito sin romper su encapsulacion.
- Observer -- OrderPlaced y listeners. Olor: el flujo conocia aviso
  y puntos de fidelidad.
- Mediator -- CheckoutMediator. Olor: stock, cobro y envio se
  llamaban entre si. Una sola pieza fija el orden.
- Chain of Responsibility -- ValidationChain y OrderValidator.
  Olor: validaciones en if; el primer error corta.
- Iterator -- CatalogIterator. Olor: recorrer el arbol exponiendo
  su estructura.
- Visitor -- CatalogVisitor (stock, price). Olor: informes dentro
  de los nodos. Sirve mientras los tipos del catalogo no cambien.
- Interpreter -- Expr, PromoParser, PromoEvaluator. Olor: reglas de
  promocion cambiables sin recompilar.

## Donde NO usamos patron
- CartService y OrderService: clases de servicio normales.
- Un segundo flujo de cobro para el checkout desde carrito: se
  rechazo; POST /checkout reutiliza CheckoutFacade.
```

Prueba de extremo a extremo (con la aplicación arrancada con `./gradlew bootRun`). Sustituye `CART_ID` y `ORDER_ID` por los ids de las respuestas.

```
curl -i -H "Content-Type: application/json" -X POST localhost:8080/products -d
    ↪  '{"id":"p1","name":"Teclado","price":{"amount":49.9,"currency":"EUR
    ↪ "},"kind":{"type":"physical","weightGrams":800},"stock":12,"category":"
    ↪ hardware"}'
curl -i -X POST localhost:8080/carts
curl -i -H "Content-Type: application/json" -X POST localhost:8080/carts/
    ↪ CART_ID/items -d '{"productId":"p1","quantity":2}'
curl -i -H "Content-Type: application/json" -X POST localhost:8080/checkout -d
    ↪  '{"cartId":"CART_ID","shippingAddress":{"recipient":"Ana","street":"
    ↪ Calle Sol 1","city":"Madrid"},"region":"ES","paymentType":"CARD"}'
curl -i localhost:8080/orders/ORDER_ID
curl -i localhost:8080/products/p1
```

El checkout responde 200 con `CheckoutResult`; el pedido tiene `"state":"type":"paid"` y `p1` muestra `"stock":10`. Con un `cartId` que no existe: 404. Con un carrito creado pero vacío: el `Order` no admite cero líneas y `ApiExceptionHandler` responde 400. Con 50 unidades: 422 de la cadena de validación. El cuerpo con `"channel":"sms"` avisa por SMS; con un canal desconocido, 400.

Por qué funciona: `POST /checkout` solo traduce un carrito en un `Order` y delega en `CheckoutFacade.checkout`, que ya valida, cobra con el flujo de pago, guarda `Paid` y descuenta stock. Errores típicos: crear un segundo camino de cobro, olvidar guardar el pedido antes de llamar al mediador (que busca el pedido para aplicar `Pay`) e importar `org.springframework.core.annotation.Order`, que choca con `shoplab.Order`.

Estado de partida del capítulo siguiente (el cierre): el de este hito.

Referencias

[1] Kent Beck y Ward Cunningham. «Using Pattern Languages for Object-Oriented Programs». En: *OOPSLA-87 Workshop on Specification and Design for Object-Oriented Programming*. Sep. de 1987. URL: https://c2.com/doc/oopsla87.html (visitado 08-10-2026).

[2] Gavin Bierman. *JEP 441: Pattern Matching for switch*. OpenJDK. 2023. URL: https://openjdk.org/jeps/441 (visitado 08-10-2026).

[3] Joshua Bloch. *Effective Java*. 3.ª ed. Addison-Wesley, 2018. ISBN: 978-0-13-468599-1.

[4] Martin Fowler. *Inversion of Control Containers and the Dependency Injection pattern*. 23 de ene. de 2004. URL: https://martinfowler.com/articles/injection.html (visitado 08-10-2026).

[5] Eric Freeman y Elisabeth Robson. *Head First Design Patterns. Building Extensible and Maintainable Object-Oriented Software*. 2.ª ed. O'Reilly Media, 2020. ISBN: 978-1-4920-7800-5.

[6] Erich Gamma et al. «Design Patterns: Abstraction and Reuse of Object-Oriented Design». En: *ECOOP'93 — Object-Oriented Programming*. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 1993, págs. 406-431. DOI: 10.1007/3-540-47910-4_21.

[7] Erich Gamma et al. *Design Patterns. Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994. ISBN: 978-0-201-63361-0.

[8] Gradle. *Gradle User Manual (9.8.1)*. URL: https://docs.gradle.org/current/userguide/userguide.html (visitado 08-10-2026).

[9] Jan Hannemann y Gregor Kiczales. «Design Pattern Implementation in Java and AspectJ». En: *Proceedings of the 17th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA '02)*. Seattle, WA: ACM, nov. de 2002, págs. 161-173. DOI: 10.1145/582419.582436.

[10] Cay S. Horstmann. *Core Java, Volume I. Fundamentals*. 13.ª ed. Oracle Press, 2024. ISBN: 978-0-13-532837-8.

[11] Joshua Kerievsky. *Refactoring to Patterns*. Addison-Wesley, 2004. ISBN: 978-0-321-21335-8.

[12] Chris Kiehl. *Data-Oriented Programming in Java*. Manning Early Access Program (MEAP); publicación prevista en octubre de 2026. Manning, 2026. ISBN: 978-1-63343-693-0.

[13] Barbara H. Liskov y Jeannette M. Wing. «A Behavioral Notion of Subtyping». En: *ACM Transactions on Programming Languages and Systems* 16.6 (nov. de 1994), págs. 1811-1841. DOI: 10.1145/197320.197383.

[14] Robert C. Martin. *Agile Software Development, Principles, Patterns, and Practices*. Pearson, 2002. ISBN: 978-0-13-597444-5.

[15] nvim-lspconfig contributors. *jdtls: Java language server configuration for Neovim (nvim-lspconfig)*. URL: https://github.com/neovim/nvim-lspconfig/blob/master/lsp/jdtls.lua (visitado 08-10-2026).

[16] Oracle. *Java Platform, Standard Edition: Java Language Updates, Release 21*. Oracle. Ene. de 2025. URL: https://docs.oracle.com/en/java/javase/21/language/ (visitado 08-10-2026).

[17] Oracle. *Java SE 21 & JDK 21 API Specification*. Oracle. URL: https://docs.oracle.com/en/java/javase/21/docs/api/index.html (visitado 08-10-2026).

[18] Project Lombok. *Lombok Features*. URL: https://projectlombok.org/features/ (visitado 08-10-2026).

[19] Project Lombok. *Setting up Lombok with Gradle*. URL: https://projectlombok.org/setup/gradle (visitado 08-10-2026).

[20] Refactoring.Guru. *Catalog of Design Patterns*. URL: https://refactoring.guru/design-patterns/catalog (visitado 08-10-2026).

[21] Vaskaran Sarcar. *Java Design Patterns. A Hands-On Experience with Real-World Examples*. 3.ª ed. Apress, 2022. ISBN: 978-1-4842-7970-0.

[22] Spring Boot project. *Spring Boot 4.0 Migration Guide*. URL: https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-4.0-Migration-Guide (visitado 08-10-2026).

[23] Spring Boot project. *Spring Boot Gradle Plugin Reference Guide (4.1.1)*. URL: https://docs.spring.io/spring-boot/gradle-plugin/index.html (visitado 08-10-2026).

[24] Spring Boot project. *Spring Boot Reference Documentation (4.1.1)*. URL: https://docs.spring.io/spring-boot/index.html (visitado 08-10-2026).

[25] Spring Framework project. *Additional Capabilities of the ApplicationContext: Standard and Custom Events (Spring Framework 7.0.9)*. URL: https://docs.spring.io/spring-framework/reference/core/beans/context-introduction.html (visitado 08-10-2026).

[26] Spring Framework project. *Aspect Oriented Programming with Spring (Spring Framework 7.0.9)*. URL: https://docs.spring.io/spring-framework/reference/core/aop.html (visitado 08-10-2026).

[27] Spring Framework project. *The IoC Container (Spring Framework 7.0.9)*. URL: https://docs.spring.io/spring-framework/reference/core/beans.html (visitado 08-10-2026).

[28] Spring Framework project. *Using the JDBC Core Classes to Control Basic JDBC Processing and Error Handling (Spring Framework 7.0.9)*. URL: https://docs.spring.io/spring-framework/reference/data-access/jdbc/core.html (visitado 08-10-2026).

[29] Spring project. *Building a RESTful Web Service*. URL: https://spring.io/guides/gs/rest-service (visitado 08-10-2026).

[30] Cheng Zhang y David Budgen. «What Do We Know about the Effectiveness of Software Design Patterns?» En: *IEEE Transactions on Software Engineering* 38.5 (sep. de 2012), págs. 1213-1231. DOI: 10.1109/TSE.2011.79.